Confidence-Aware Episodic Memory with Self-Improvement for Hallucination Reduction in Large Language Models

by

Md. Aksan Gony Alif
24341256
Maliha Binte Shamim
23101466
Md. Mahmudur Rahman
22101355

A thesis submitted to the Department of Computer Science and Engineering
in partial fulfillment of the requirements for the degree of
B.Sc. in Computer Science and Engineering

Department of Computer Science and Engineering
Brac University
Fall 2026.

© 2026. Brac University
All rights reserved.

Declaration

It is hereby declared that

1. The thesis submitted is my/our own original work while completing degree at Brac University.
2. The thesis does not contain material previously published or written by a third party, except where this is appropriately cited through full and accurate referencing.
3. The thesis does not contain material which has been accepted, or submitted, for any other degree or diploma at a university or other institution.
4. We have acknowledged all main sources of help.

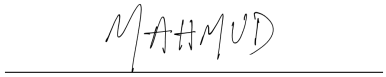
Student's Full Name & Signature:



Md. Aksan Gony Alif
ID: 24341256



Maliha Binte Shamim
ID: 23101466



Md. Mahmudur Rahman
ID: 22101355

Approval

The thesis/project titled "Confidence-Aware Episodic Memory with Self-Improvement for Hallucination Reduction in Large Language Models" submitted by

1. Md. Aksan Gony Alif (24341256)
2. Maliha Binte Shamim (23101466)
3. Md. Mahmudur Rahman (22101355)

of Fall, 2025 has been accepted as satisfactory in partial fulfillment of the requirement for the degree of B.Sc. in Computer Science in Fall 2026.

Examining Committee:

Supervisor:
(Member)

Dr. Md. Golam Rabiul Alam

Professor
Department of Computer Science & Engineering
Brac University

Co-Supervisor:
(Member)

Md. Tanzim Reza

Senior Lecturer
Department of Computer Science & Engineering
Brac University

Thesis Coordinator:
(Member)

Dr. Md. Golam Rabiul Alam

Professor
Department of Computer Science & Engineering
Brac University

Head of Department:
(Chair)

Dr. Sadia Hamid Kazi

Associate Professor & Chairperson
Department of Computer Science & Engineering
Brac University

Abstract

Large language models give confident but false answers, and a user cannot tell from the answer which ones to trust: asked for supporting literature, chatbots have returned references that do not exist [44], for up to 91.4 percent of the references they generate in systematic-review searches [66]. This thesis proposes *Confidence-Aware Episodic Memory with Self-Improvement (CAEM)*, a system built around a small open generator. A router sends each question to a verified memory, to zero-shot generation or to retrieval-augmented generation, with a classifier that decides when retrieval is likely to help. A verifier combines nine signals of internal confidence, agreement across sampled answers, support from retrieved passages and question match into a calibrated probability of correctness, and a four-outcome decision stores, defers, abstains on or discards each answer. Every answer reaches the user with that probability and a status tag (verified, provisional, conflicting or insufficient evidence). Verified answers are kept in memory, re-checked at every cycle, and used to fine-tune a low-rank adapter, with a retention guard that rolls the adapter back when held-out accuracy falls. Two results are proven: storage makes the stored pool purer than the generator's raw output exactly when the verifier's balanced accuracy exceeds one half, and the pool's precision is bounded below by the calibration precision when new questions resemble the calibration data.

CAEM was run for six cycles with Qwen-2.5-3B-Instruct on five benchmarks (FEVER, TriviaQA and CommonsenseQA for training; TruthfulQA and StrategyQA held out) and compared with eight baselines under one protocol: the same questions, the same prompt, one scoring rule and the same hallucination instrument. The architecture alone added almost no accuracy (pooled exact match 0.517 against 0.511 for the zero-shot baseline); self-improvement raised it to 0.535 after two cycles, after which it eroded to 0.508, and no difference from the zero-shot baseline is significant on any benchmark. CAEM is level with strong prompting baselines, significantly better than the retrieval baselines on several benchmarks, and level on accuracy with a fine-tuned baseline that used about eleven times as many gold labels, which reached a lower hallucination metric. Its measured contribution is selectivity: the status tag predicts correctness, with verified answers right 65 to 75 percent of the time against 51 percent for the zero-shot baseline. The gains are concentrated where the verifier can check answers against retrieved passages, and after self-improvement the verifier scored more of the wrong answers highly (confident confabulation 0.305 against 0.208). Of four pre-specified hypotheses, two are supported, one partially and one not.

Keywords: hallucination; episodic memory; calibrated confidence; selective prediction; verifier; self-improvement; retrieval-augmented generation; low-rank adaptation; large language models.

Acknowledgement

We are deeply grateful to our supervisor, Dr. Md. Golam Rabiul Alam, Professor, Department of Computer Science and Engineering, Brac University, for his guidance, encouragement, and critical feedback throughout this thesis. We also thank our co-supervisor, Md. Tanzim Reza, Senior Lecturer, Department of Computer Science and Engineering, Brac University, for his support and advice at every stage of the work.

We thank the Department of Computer Science and Engineering, Brac University, for the opportunity and resources to carry out this research.

Finally, we thank our families for their constant support and patience during our studies.

Table of Contents

Declaration i

Approval ii

Abstract iv

Acknowledgment v

Table of Contents vi

List of Figures xii

List of Tables xiii

List of Algorithms xv

List of Acronyms xvi

List of Symbols xviii

1 Introduction 1
- 1.1 Background . . . 1
  - 1.1.1 Capability and the persistent failure mode . . . 1
  - 1.1.2 The hallucination problem at scale . . . 2
  - 1.1.3 Three reinforcing shortcomings . . . 2
- 1.2 Rationale of the Study or Motivation . . . 3
  - 1.2.1 The deployment gap . . . 3
  - 1.2.2 Why prior pipelines do not close the gap . . . 3
  - 1.2.3 The architectural opportunity . . . 4
- 1.3 Problem Statement . . . 6
  - 1.3.1 Formal problem . . . 6
  - 1.3.2 Shortcomings addressed and exclusions . . . 6
- 1.4 Objective . . . 7
  - 1.4.1 Primary objective . . . 7
  - 1.4.2 Specific objectives . . . 7
  - 1.4.3 Pre-specified hypotheses . . . 9
- 1.5 Methodology in Brief . . . 11
  - 1.5.1 Architecture overview . . . 11
  - 1.5.2 Verification and the storage decision . . . 12
  - 1.5.3 Calibration discipline . . . 12
  - 1.5.4 Cycle-boundary self-improvement loop . . . 13

1.6 Scopes and Challenges . . . 14
1.6.1 Research scope . . . 14
1.6.2 Technical challenges . . . 16
1.7 Chapter signpost . . . 18

2 Literature Review **19**
2.1 Preliminaries . . . 19
2.1.1 Large Language Models and Transformer Architecture . . . 19
2.1.2 Hallucination Taxonomy . . . 20
2.1.3 Natural Language Inference . . . 21
2.1.4 Sentence Embeddings and Semantic Similarity . . . 22
2.1.5 Self-Consistency Decoding . . . 22
2.1.6 Catastrophic Forgetting and Continual Learning . . . 23
2.2 Review of Existing Research . . . 23
2.2.1 Hallucination Problem and Measurement . . . 24
2.2.2 Verification Methods . . . 25
2.2.3 Confidence Estimation . . . 27
2.2.4 Memory-Augmented Architectures . . . 30
2.2.5 Selective and Adaptive Retrieval . . . 32
2.2.6 Self-Improvement and Continual Learning . . . 34
2.2.7 External Baselines and Competitive Positioning . . . 38
2.2.8 Evaluation Benchmarks . . . 40
2.3 Summary of Key Findings . . . 42
2.4 Chapter signpost . . . 43

3 Requirements, Impact, and Project Management **44**
3.1 Final Specifications and Requirements . . . 44
3.1.1 Functional requirements . . . 44
3.1.2 Non-functional requirements . . . 44
3.1.3 Constraints and design commitments . . . 46
3.2 Societal Impact . . . 47
3.2.1 Deployment benefits . . . 47
3.2.2 Dual-use considerations . . . 47
3.2.3 Marginalised-user impact . . . 48
3.3 Environmental Impact . . . 48
3.3.1 Compute footprint of the experiment . . . 48
3.3.2 Cycle budget rationale . . . 49
3.3.3 Retention versus retraining . . . 49
3.4 Ethical Issues . . . 49
3.4.1 Alignment and refusal posture . . . 49
3.4.2 Bias propagation through episodic memory . . . 50
3.4.3 Data provenance and licensing . . . 50
3.5 Standards and Codes of Practice . . . 50
3.5.1 Software-engineering standards . . . 50
3.5.2 Reproducibility standards . . . 51
3.5.3 Evaluation-protocol standards . . . 51
3.6 Project Management Plan . . . 52
3.6.1 Phase breakdown . . . 52
3.6.2 Milestone schedule . . . 52

3.6.3 Deliverables . . . 52
3.7 Risk Management . . . 53
3.7.1 Technical risks . . . 53
3.7.2 Budget and infrastructure risks . . . 54
3.7.3 Mitigation register . . . 54
3.8 Economic Analysis . . . 54
3.8.1 Compute budget . . . 54
3.8.2 Cost-benefit on the storage gate . . . 54
3.8.3 Deployment cost projection . . . 56
3.9 Chapter signpost . . . 56

4 Methodology and Design 57
4.1 Design Process or Methodology Overview . . . 57
4.1.1 The eight-stage pipeline narrative . . . 57
4.1.2 The cycle loop walk-through . . . 58
4.1.3 A worked example . . . 58
4.2 Preliminary Design or Design (Model) Specification . . . 59
4.2.1 Generator and retrieval substrate . . . 59
4.2.2 Pre-routing confidence . . . 61
4.2.3 Episodic memory store . . . 61
4.2.4 Three-tier router with the safety override . . . 63
4.2.5 Retrieval-utility classifier . . . 63
4.2.6 Verifier ensemble . . . 66
4.2.7 Calibrated probability composite . . . 70
4.2.8 Fixed-threshold storage gate . . . 72
4.2.9 Four-outcome storage decision . . . 73
4.2.10 Cycle-boundary maintenance . . . 74
4.3 Theoretical analysis . . . 77
4.3.1 Pool purity . . . 77
4.3.2 When filtering helps: the purification condition . . . 79
4.3.3 Design heuristics for the cycle-to-cycle trajectory . . . 81
4.3.4 Remarks on the overall hallucination rate . . . 84
4.4 Chapter signpost . . . 85

5 Performance Evaluation and Analysis 86
5.1 Performance Evaluation . . . 86
5.1.1 Experimental setup . . . 86
5.1.2 Trajectory of headline metrics . . . 90
5.1.3 Four-outcome decision distribution . . . 94
5.1.4 Three-tier router efficiency . . . 95
5.1.5 Retention and continual learning . . . 97
5.1.6 Memory and training-pool dynamics . . . 97
5.1.7 Theoretical predictions checked against the trajectory . . . 99
5.2 Final Design Adjustments . . . 103
5.2.1 Calibration discipline and disjoint folds . . . 103
5.2.2 The composite parameter selection sweep . . . 103
5.2.3 The locked configuration . . . 103
5.2.4 Per-cycle composite-refit trajectory . . . 104
5.2.5 Calibration-versus-evaluation gap . . . 104

5.2.6 The long-hypothesis judge ablation . . . 105
5.2.7 The validation gate verdict . . . 105
5.2.8 Failed iterations as evidence of method discipline . . . 106
5.3 Statistical Analysis . . . 106
5.3.1 Tests used . . . 106
5.4 Comparisons and Relationships . . . 107
5.4.1 External baseline panel under matched protocol . . . 107
5.4.2 Retrieval-utility classifier ablation . . . 110
5.5 Summary of Findings . . . 111
5.5.1 Headline outcome in one paragraph . . . 111
5.5.2 Hypothesis verdicts . . . 111
5.6 Discussion . . . 113
5.6.1 Headline interpretation . . . 113
5.6.2 Where the verifier can check answers . . . 114
5.6.3 A side effect of self-improvement . . . 114
5.6.4 Threats to validity . . . 114
5.6.5 Sensitivity analyses . . . 116
5.6.6 Comparison to the strongest prior systems . . . 116
5.6.7 Limitations . . . 116
5.7 Chapter signpost . . . 118

6 Conclusion and Future Work 119
6.1 Conclusion . . . 119
6.1.1 Contributions . . . 119
6.1.2 Headline finding . . . 120
6.1.3 Hypothesis verdicts . . . 121
6.1.4 Limitations . . . 121
6.1.5 Closing remark . . . 122
6.2 Future Work . . . 122
6.2.1 A verifier for answers that passages cannot settle . . . 122
6.2.2 Contradiction-aware verification . . . 122
6.2.3 Cleaner self-improvement . . . 123
6.2.4 Scope to checkable questions . . . 123
6.2.5 Isolating the effects . . . 123
6.2.6 Testing the formal heuristics . . . 124
6.2.7 A longer horizon . . . 124
6.2.8 Measurements this study did not take . . . 124
6.2.9 Larger backbones and other domains . . . 124
6.2.10 Deployment study . . . 124

References 125

A Implementation reference 134
A.1 Code and data availability . . . 134
A.2 Repository layout and module map . . . 134
A.3 Configuration registry . . . 136
A.4 Hyperparameter listing . . . 139
A.5 Runbook step graph . . . 141
A.6 Snapshot reference . . . 143

A.7 Hardware and software profile . . . 144
A.8 Calibration script reference . . . 145
A.9 Validation gate reference . . . 146

B Cycle 0 evidence catalog **148**
B.1 The one-hundred-variant composite parameter sweep . . . 148
B.2 The calibration-to-evaluation gap analysis . . . 148
B.3 The first-iteration failure trace . . . 149
B.4 The long-hypothesis judge ablation diagnostic . . . 149
B.5 Pre and post prompt-fix compliance . . . 149
B.6 Atomic decomposition coverage . . . 150
B.7 Composite weight table . . . 150
B.8 Per-benchmark precision-cliff curves . . . 150
B.9 Production-envelope sample renderings . . . 150
B.10 Prompt-design ablation . . . 152

C Per-cycle artifact dictionary **153**
C.1 Per-cycle calibration snapshots . . . 153
C.2 Per-cycle threshold history . . . 153
C.3 Per-cycle memory size and storage rate . . . 155
C.4 Per-cycle equilibrium fit . . . 156
C.5 Per-cycle fine-tuning diagnostics . . . 156
C.6 Additional per-cycle tables from the evaluation . . . 157

D Significance test details **161**
D.1 Pairing rule and correctness label . . . 161
D.2 Contingency tables . . . 161
D.3 Bootstrap protocol . . . 162
D.4 Holm sequence . . . 163
D.5 Win and defeat rule . . . 163

E Reproducibility recipe **167**
E.1 Data preparation . . . 167
E.2 Snapshot recovery . . . 168
E.3 Step-by-step run sequence . . . 169
E.4 Version pinning . . . 171
E.5 Hardware envelope . . . 172
E.6 Verification of reproduced numbers . . . 173

F Glossary **176**
F.1 Concept terms . . . 176
F.2 Acronyms . . . 181

G Architecture positioning **183**
G.1 Per-query precision contract . . . 185
G.2 Memory-direct tier at scale . . . 185
G.3 Asymmetric rollback vs manual backup . . . 186
G.4 Label efficiency . . . 186
G.5 What the composite catches . . . 187

G.6 Tier-2 distillation 188
G.7 When not to deploy 188
G.8 Same calibration fold across cycles 191
G.9 Production vs research fold 192
G.10 Cycle-boundary ordering 193
G.11 Early-exit disabled at retro-verify 194
G.12 Training vs storage threshold 194
G.13 Three-layer dedup 195

List of Figures

1.1 Architecture overview 12
1.2 Experimental workflow 14

4.1 Order of the cycle-boundary steps as executed. 58
4.2 The eight-stage CAEM pipeline. 60
4.3 Routing map of the three-tier router. 64
4.4 Where the nine verifier signals come from. 67
4.5 The calibrated probability composite in three steps. 71
4.6 Status of the formal statements and the hypotheses they inform. . . . 78
4.7 How the purity of the stored pool depends on the gate's balanced accuracy. 80

5.1 EM and CHM by cycle against B1. 90
5.2 Confident confabulation against false refusal by cycle. 91
5.3 Answered share against error among answered. 92
5.4 CAEM and the baselines on EM and CHM. 108

B.1 Per-benchmark precision-cliff curves at the locked configuration . . . 151

List of Tables

1.1 The four pre-specified hypotheses with deciding statistic and the formal claim each tests. . . . 11

2.1 Positioning of CAEM against the closest prior systems. . . . 38

3.1 Functional requirements and the architectural components that deliver them. . . . 45
3.2 Non-functional requirements, their targets, and where each is checked. 46
3.3 Risk register with mitigations and the realised outcome. . . . 55

4.1 Signal readings for the two worked examples. . . . 59
4.2 Dispatch rules of the router and the retrieval-utility classifier. . . . 66
4.3 The nine verifier signals. . . . 67

5.1 Benchmark folds. . . . 87
5.2 Metrics reported in this chapter. . . . 88
5.3 The headline in three steps. . . . 90
5.4 Accuracy of the status tag shown to the user. . . . 92
5.5 EM and CHM by benchmark and cycle. . . . 93
5.6 The seven hallucination subtypes by cycle. . . . 94
5.7 Decision shares on the calibration fold. . . . 95
5.8 Router probe of the memory-direct tier. . . . 96
5.9 Tier shares and per-tier EM by cycle. . . . 96
5.10 Self-improvement training by cycle. . . . 97
5.11 Memory, deferred buffer and consolidation by cycle. . . . 98
5.12 Stored answers and the training pool by cycle. . . . 98
5.13 Storage gate quality on the calibration fold by cycle. . . . 99
5.14 Storage precision by cycle on the calibration and evaluation folds. . . . 100
5.15 What the run shows for each formal statement. . . . 102
5.16 Selected variants of the cycle-zero sweep. . . . 104
5.17 Cycle-zero validation gate under the locked composite. . . . 105
5.18 Per-signal Cohen's d on the cycle-zero validation fold. . . . 106
5.19 CAEM against the eight baselines. . . . 108
5.20 Significant wins and defeats against each baseline. . . . 109
5.21 Retrieval-utility classifier ablation at cycle three. . . . 110
5.22 Hypothesis verdicts. . . . 112

A.1 Repository layout . . . 135
A.2 Configuration registry . . . 136
A.3 Consolidated hyperparameter table . . . 140

B.1 Locked configuration carried into every cycle. . . . 148

B.2 Prompt-compliance rates pre/post registered prompt fixes 149
B.3 Atomic decomposition scope per benchmark 150
B.4 The cycle-zero composite. 151
B.5 Storage precision at fixed targets, ID vs transfer 152
B.6 Prompt-design ablation at Cycle 0 152

C.1 Per-cycle calibration snapshot summary 154
C.2 Per-cycle threshold history 154
C.3 Per-cycle memory store trajectory 155
C.4 Per-cycle outcome and retention probes 156
C.5 Per-cycle fine-tuning diagnostics 157
C.6 Composite evaluation score by cycle. 158
C.7 Correct and wrong answers per decision at C2 and C5. 158
C.8 Abstention rate by benchmark and cycle. 158
C.9 Per-tier EM by benchmark at C0, C2 and C5. 159
C.10 Stored scores and accuracy by score band. 159
C.11 Correctness of pruned and promoted entries. 159
C.12 Per-tier EM and CHM by cycle. 160
C.13 Retroactive re-verification by cycle. 160
C.14 Composite refit by cycle. 160

D.1 Four McNemar contrasts at C5. 162
D.2 Three bootstrap intervals at C5. 162
D.3 Holm sequence on TriviaQA at C5. 163
D.4 Holm sequence on CommonsenseQA at C5. 163
D.5 Per-benchmark paired tests at C5. 164
D.6 Per-benchmark paired tests at C2. 165

E.1 Framework version pinning 171

List of Algorithms

1 Pre-routing confidence (Stage 1). 62
2 Three-tier router with safety override (Stage 3). 64
3 Retrieval-utility classifier (Stage 3b). 66
4 Verifier ensemble (Stage 5). 68
5 Calibrated probability composite (fit and predict). 71
6 Four-outcome storage decision (Stage 6). 74
7 Retroactive re-verification pass (Stage 8). 75
8 Deferred-entry reconsideration pass (Stage 8). 75
9 Self-improvement loop with retention guard (Stage 8). 77

List of Acronyms

Architecture and methods

CAEM	Confidence-Aware Episodic Memory with Self-Improvement
RUC	Retrieval-Utility Classifier (Stage 3b)
RAG	Retrieval-Augmented Generation
CoT	Chain-of-Thought prompting
LoRA	Low-Rank Adaptation
KLE	Kernel Language Entropy
NTM	Neural Turing Machine
A-MEM	Agentic Memory
FLARE	Forward-Looking Active Retrieval
SI / MAS	Synaptic Intelligence; Memory-Aware Synapses
SIL	Self-Improvement Loop
NLI	Natural Language Inference
DPR	Dense Passage Retrieval
EWC	Elastic Weight Consolidation
MC	Monte Carlo (MC-Dropout)
STaR	Self-Taught Reasoner (**V-STaR**: verifier)
DNC	Differentiable Neural Computer
FActScore	Atomic-fact factuality score
Self-RAG	Self-reflective RAG
CP	Conformal Prediction (**split-CP**; registered then replaced by fixed-threshold storage rule)

Models and judges

LLM	Large Language Model
MiniCheck	Flan-T5-Large NLI judge for claim-support
SBERT	Sentence-BERT (cosine encoder)
MNLI / SNLI	Multi-genre / Stanford NLI corpora

Benchmarks

FEVER	Fact Extraction and VERification
TQA	TriviaQA
CSQA	CommonsenseQA
HotpotQA	Multi-hop QA (dropped at cycle zero)
HaluEval-QA	Hallucination-Evaluation QA (RUC training only)
MMLU	Massive Multitask Language Understanding (retention probe)
BIG-bench	Beyond the Imitation Game
SQuAD	Stanford Question Answering Dataset
NQ	Natural Questions (dropped at cycle zero; RUC training only)
TruthfulQA	Truthful QA (Haiku-judged EM)
StrategyQA	Multi-hop strategy QA
OpenBookQA	Open-Book QA (RUC training only)
HaluEval	Hallucination Evaluation benchmark

Metrics, statistics, infrastructure

EM	Exact Match
ROUGE	Recall-Oriented Understudy for Gisting Evaluation
CES	Composite Evaluation Score (geometric mean of ACC, EPI, RET, CAL, VER; **EPI** $= 1 -$ CHM)
AUROC	Area Under ROC curve
F_1	Token-level F_1
CHM	Composite Hallucination Metric
ECE	Expected Calibration Error
TPR / FPR / TNR	True / False / True-Negative Rate

TPR	True-Positive Rate	**CI**	Confidence Interval
ID / OOD	In- / Out-of-Distribution	**EMA**	Exponential Moving Average
FPR	False-Positive Rate	**FAISS**	Facebook AI Similarity Search (**IVF**: inverted-file)
TTL	Time To Live (deferred-buffer expiry)	**QA**	Question Answering

List of Symbols

Confidence, verifier signals, thresholds

u_{pre}	Pre-routing confidence	u_{stored}	Calibrated post-generation composite
u_{token}	Mean token log-probability	u_{dropout}	MC-Dropout variance, K=5
u_{internal}	$0.5u_{\text{tok}}+0.5(1-u_{\text{drop}})$	s_{avg}	Mean pairwise cosine, M=3
p_{entail}	Chain-to-answer NLI entailment	h_{norm}	Normalised semantic entropy, K=10
$p_{\text{ground,max}}$	Top-1 passage entailment	$p_{\text{ground,mean}}$	Mean entailment over top-3
$p_{\text{ground,atomic}}$	Length-gated atomic-fact entailment	q_{match}	Question-answer match score

h_{norm} (normalised semantic entropy, K=10) was registered then retired post Cycle-0 audit (boost weight $\approx -5 \times 10^{-4}$); deployed verifier consumes nine signals.

$\tau_{\text{store}}, \tau_{\text{defer}}$	Fixed store / defer cut-points on u_{stored} (0.60 / 0.45)	τ_{train}	Strict training-pool threshold (0.70)
τ_{retro}	Retroactive prune threshold (0.50)	τ_{ttl}	Deferred-buffer time-to-live (2 passes)
τ_{cons}	Memory consolidation cosine (0.92)	ϕ_{pg}	Abstention floor on top-passage entailment (0.20)
ϕ_{safe}	Pre-routing safety floor on u_{pre} (0.38; per-benchmark refit)	$\theta_1, \theta_{\text{sim}}$	Tier 1 combined-score / Tier 2 similarity thresholds (0.90 / 0.75)
η_u, η_g	Early-exit thresholds	τ_{RUC}	Retrieval-utility classifier threshold (0.375)

Calibrated probability composite

$\alpha_{\text{store}}, \alpha_{\text{defer}}$ (split-CP miscoverage targets) and α_{ema} (EMA refit factor) were registered in the conformal storage gate then retired when the gate was replaced by the fixed-threshold rule above; the calibration-versus-evaluation distribution shift broke the marginal-coverage premise of split-CP on the five-benchmark panel. The composite refit absorbs distribution drift through the per-signal isotonic curves and the per-benchmark boost coefficients on a fresh per-cycle fold.

$n_{\min}$	Minimum top-k for fit (20)	$n_{\text{cal}}, n_{\text{eval}}$	Calibration / evaluation fold sizes
$\pi_{\text{store}}, \pi_{\text{defer}}$	Calibration-fold empirical precision	π_{retro}	Retroactive-survivor precision
π_t, π_∞	Pool purity at cycle t, asymptote	P_c, P_c^{theory}	Empirical / Bayes pool purity
P_j, iso_j	Per-signal isotonic probability	$L_{i,j}$	Calibration log-odds matrix
$\mathbf{w}, b, w_j$	Boost weights, intercept	C	Boost L_2 strength (0.01)
T	Logit-space temperature	$\sigma(\cdot), \ell$	Logistic; composite log-odds
α_{shrink}	Per-benchmark shrinkage weight (0.6)	c_{conv}	Layer-convergence ratio in u_{pre}

Self-improvement, router, memory

$\boldsymbol{\theta}, \boldsymbol{\theta}^{(t)}$	Generator parameters at cycle t	$\boldsymbol{\theta}^\star$	Pre-retention-guard argmin
λ_{anc}	L_2 anchor strength (0 on the adapter path)	$\mathcal{L}_{\text{CE}}, \mathcal{L}$	CE loss; total per-cycle loss
$\rho, \rho_{\min}$	Retention ratio; floor (0.93)	$\rho_{\text{pre/post}}$	Pre / post fine-tune probe accuracy

$\mathcal{T}$	Training pool above τ_{train}	$\mathcal{M}, \mathcal{M}_t$	Episodic memory; pool at cycle t
$\mathcal{M}_t^{\text{kept}}, \mathcal{N}_{t+1}$	Survivors; new admissions	$\mathcal{B}, K_{\text{def}}$	Deferred buffer; capacity
λ	Routing similarity weight (0.70)	$s, \mathcal{S}$	Cosine; dispatch score $\lambda s + (1-\lambda) u_{\text{stored}}$
$\tau_1(t)$	Tier-1 share of queries	$H_1(t)$	Tier-1 error rate
$H_{\text{mem}}(t), H_{\text{gen}}(t), H_t$	Memory / generated / overall error rate	M, K	Chain count (3); answer count (10)
p_{RAG}	RUC retrieval-helpfulness score	p_{IK}	Self-knowledge probe (Kadavath-style)

Theorem trajectory and statistics

d, d_t	Cohen's d at cycle t	$d_{\text{id}}, d_{\text{transfer}}$	ID and transfer Cohen's d
μ_+, μ_-, ς	Within-class means; shared std	p_+, p_-	Correct / incorrect base rates
$\Phi(\cdot), \phi(\cdot)$	Std-normal CDF; density	σ_t	Empirical storage rate at cycle t
r	Geometric contraction rate	G_t	Cycle-t gap $\lvert\pi_t - \pi_\infty\rvert$
$A, k, c^*(\varepsilon)$	Decay fit; time-to-equilibrium	p_c, α_c	Base correctness; verifier balanced acc.
$\text{TPR}_c, \text{FPR}_c, \text{TNR}_c$	Storage-gate rates at cycle c	$f(p)$	Recurrence $p\alpha/(p\alpha + (1-p)(1-\alpha))$
$K(C)$	Questions supportable by corpus C	$\chi^2_{\text{McN}}, b, c$	McNemar; discordant pair counts
B	Bootstrap replicates (1 000)	α	Significance level (overloaded)
EM, F_1	Per-task correctness	CHM, CES, ECE	Composite hallucination / eval / cal. error
N, T_{cycles}	Cycle index; registered cap (10); realised closure at 6	$\lvert\theta\rvert$	Backbone parameters (3.086×10^9)

Chapter 1

Introduction

This chapter introduces the problem the thesis addresses, motivates the choice to tackle it through architectural rather than incremental means, formalises the problem statement, declares the objectives, summarises the proposed system at a high level, and registers the scope within which the work is bounded.

1.1 Background

1.1.1 Capability and the persistent failure mode

Large language models trained on broad text corpora and instruction-tuned for assistant-style interaction now perform competitively on a wide range of natural language tasks, including factoid question answering, multi-step reasoning, summarisation, and program synthesis. Frontier systems such as GPT-4, Claude, and PaLM reach or exceed reported human accuracy on standardised academic benchmarks, and their accessibility through chat interfaces has made them part of everyday research, education, and professional workflows [45], [46], [59]. The same systems, however, generate confident but factually incorrect statements at deployment-relevant rates. The accepted name for this failure mode is hallucination, although the term covers heterogeneous causes ranging from retrieval limitations to confabulation under genuine uncertainty.

A practical consequence is that the same fluency that makes large language models useful also makes their hallucinations dangerous. A false answer phrased with the same confidence as a true one is harder to detect than a false answer that sounds uncertain. Users without independent verification, particularly in high-stakes domains, take the model's confident phrasing as a signal of reliability and then act on incorrect information. This thesis takes the position that hallucination is therefore not merely a benchmark-accuracy problem but a deployment-safety problem, and that addressing it requires architectural change rather than incremental decoding adjustments.

1.1.2 The hallucination problem at scale

The empirical record on hallucination is consistent across benchmarks and domains. The TruthfulQA benchmark of Lin et al. [40] isolates questions on which humans themselves are prone to hold confident misconceptions, and reports that the largest frontier model evaluated achieved only 58 percent on the truthfulness axis and 21 percent on the combined truthful-and-informative axis, against a human baseline of 94 and 87 percent respectively. The benchmark also documented an inverse-scaling relationship in which larger models were more, not less, prone to echo human falsehoods, indicating that scale alone does not solve the problem.

Long-form generation shows the same pattern at finer granularity. The atomic-fact decomposition protocol of Min et al. [58], evaluating chat-style models on biographical generation, found that only 58 percent of the atomic facts in ChatGPT's biographies were supported by the reference knowledge source. Domain-specific evaluations are still more concerning. When Alkaissi and McFarlane [44] asked ChatGPT to support a clinical explanation with references, it returned five citations of which none existed, and a study of systematic-review searches by Chelli et al. [66] found that 39.6, 28.6 and 91.4 percent of the references produced by GPT-3.5, GPT-4 and Bard respectively were hallucinated, far above any threshold a responsible deployment would tolerate.

These results indicate that hallucination is neither an artefact of small models nor a residual that scale alone will eliminate; the survey literature describes it as an artefact of neural generation that persists in large, instruction-tuned and preference-tuned models [50], [89]. This motivates changing the structure of the system around the model rather than relying on further training alone.

1.1.3 Three reinforcing shortcomings

Three structural shortcomings of the current inference paradigm shape every other limitation a deployed model exhibits.

No memory: Standard inference is stateless: every query is processed independently, no representation of past successful answers is retained across sessions, and identical questions can produce inconsistent answers from one interaction to the next. This wastes computation on queries the system has effectively already solved and prevents the accumulation of domain-specific knowledge through use.

No reliable confidence: Modern neural networks are systematically overconfident. Guo et al. [18] found that modern neural classifiers assign confidence that exceeds their actual accuracy, with expected calibration errors above 10 percentage points on standard image benchmarks, and Kadavath et al. [38] found that language models fine-tuned with reinforcement learning from human feedback appear badly miscalibrated until their outputs are rescaled. The confidence a model states in words discriminates correct from incorrect answers only weakly: in the study of Xiong et al. [84], GPT-4's verbalised confidence averages an area under the receiver operating curve of about 0.63, close to the 0.5 of random guessing, and no single elicitation method is consistently best. A user therefore cannot reliably tell which answers are likely to be wrong from the model's stated confidence alone.

No learning after deployment: Deployed models are static. Once trained they neither learn from their mistakes nor adapt to the patterns of an institution that uses them, because naive online fine-tuning incurs catastrophic forgetting on the broad capabilities the base training instilled, as Kirkpatrick et al. [20] documented. The implication is that verification effort spent today produces no compounding benefit tomorrow: a thousand answers verified by humans this morning will not change a single answer the model gives this afternoon.

These three shortcomings reinforce one another. Without memory the model cannot reuse verified knowledge. Without calibrated confidence the system cannot recognise when memory is missing. Without self-improvement the verifications that humans or external systems do produce decay into one-shot interventions. The remainder of this thesis develops a system that breaks the cycle along all three axes simultaneously. The next section argues why a combined intervention at the architectural level is the appropriate response.

1.2 Rationale of the Study or Motivation

1.2.1 The deployment gap

The capability gap that motivates this thesis is the distance between what large language models do well and what reliable deployment requires of them. Fluency, breadth of topic coverage, and on-demand reasoning are already present in current frontier models. What is not present is the discipline of distinguishing facts that the model has correctly internalised from confabulations that emerge under genuine uncertainty [71], and the discipline of accumulating verified knowledge across interactions instead of regenerating answers from scratch every time. The gap is not aesthetic. In educational deployments, students who learn from confidently incorrect tutoring risk acquiring misconceptions. In healthcare information services, a hallucinated drug interaction or treatment dosage would produce decisions whose costs are measured in patient outcomes rather than in benchmark points. In enterprise knowledge workflows, fabricated citations and missing provenance erode the trust that the entire deployment rests on.

A second, less visible cost is computational. The stateless protocol of a typical large-model deployment regenerates every answer from first principles, including answers whose substance has already been verified in a previous interaction. As deployment scale grows, the cost of unnecessary regeneration accumulates against any computational budget. A system that can recognise a familiar query and serve a previously verified answer at near-zero cost converts the same compute envelope into more delivered value, and that conversion is itself part of the rationale.

1.2.2 Why prior pipelines do not close the gap

Three families of prior systems have attempted to reduce hallucination, and each has a structural limitation that prevents it from closing the gap on its own.

Retrieval-augmented generation: Retrieval-augmented generation, introduced by Lewis et al. [33], grounds the model in retrieved evidence, which raises factual recall

when the retrieval succeeds but is silent when retrieval misses or returns evidence the model summarises incorrectly. The output remains a single-pass generation whose quality is bounded by retrieval quality, and the system has no representation of which past answers were correct.

Prompting interventions: Prompting interventions, including chain-of-thought reasoning (Wei et al. [41]) and self-consistency over sampled chains (Wang et al. [62]), expose internal reasoning and exploit agreement across resampled answers, but they provide no persistence: each session begins with no memory of prior verified answers, and the same question asked tomorrow triggers the same reasoning effort as today.

Post-generation verification: Post-generation verifiers such as SelfCheckGPT (Manakul et al. [56]) and semantic entropy (Farquhar et al. [71]) detect hallucinations after they occur and support refusal or revision, but they do not feed back into model weights or memory, so the same hallucination patterns recur in the next session.

The implication is that hallucination reduction at deployment-relevant rates is unlikely to come from incremental improvement of any one family. Stronger retrievers, longer chains of thought, and sharper post-generation verifiers each remain bounded by the structural limitation of their family when used in isolation. What is missing is a system in which verification feeds memory, memory feeds routing, routing feeds generation, and improvement compounds across cycles instead of evaporating at session boundaries. That system is an architectural intervention, and arguing for it is the work this thesis attempts.

1.2.3 The architectural opportunity

Three capabilities, taken together, would close the gap that the existing three families leave open. The first is a verified episodic memory whose admission rule is finer than a single binary threshold. An answer that is plausible but has not earned the precision required for retrieval should not be discarded, because the same query may be answered more confidently after the model has been updated; it should instead enter a deferred buffer and be reconsidered at the next cycle boundary. A four-outcome decision (store, defer, abstain, discard) gives the system somewhere to put borderline outputs without contaminating the high-precision pool. The second capability is confidence-aware routing that reads pre-generation uncertainty as an independent safety signal alongside the combined memory similarity and stored quality. It adds a per-query retrieval-utility decision on the queries that would otherwise fall to the retrieval-augmented tier, so that retrieval is consulted only where it is expected to help rather than unconditionally. When the pre-generation uncertainty is high, the routing layer should force grounded retrieval before generation has started, rather than rely on post-generation review to catch an answer that should never have been produced. The third capability is a constrained self-improvement loop that fine-tunes a small low-rank adapter on a frozen backbone using a verified high-quality pool, so that the bounded adapter plays the role that the parameter anchor plays in elastic weight consolidation [20], and that rolls the adapter back if a held-out retention probe indicates general capability has eroded. Together the three capabilities convert verification from an event that fires once per query into a discipline that compounds across cycles.

A practical consequence of the joint architecture, named the confidence-aware episodic memory with self-improvement (CAEM), is that the three serving tiers are intended to act not as static cost categories but as a compute ladder that reorganises itself under the cycle loop, and that two distinct statelessness problems of the standard inference paradigm are converted into two stateful mechanisms by two different architectural elements. The first statelessness problem is that retrieval-augmented generation pays the full retrieval-and-generation cost in every session because the model never absorbs what retrieval just taught it; CAEM closes this by feeding verified retrieval-augmented outputs into the self-improvement loop's training pool, so each cycle's low-rank-adapter update folds the previously retrieval-dependent reasoning into the generator's parametric capacity. The design expectation is that the next cycle answers part of the previously retrieval-dependent queries from the zero-shot tier alone, without paying the retrieval cost. The second statelessness problem is that even when the model already knows the answer, the standard protocol re-generates it from scratch on every recurrence and offers no guarantee that two sessions produce the same answer; CAEM closes this through the verified episodic memory, where exact and near-duplicate queries are served directly from the stored answer at embedding-lookup cost, and where the retroactive re-verification pass at every cycle boundary refreshes the stored score of every cached entry against the updated generator, so that the cached answer's quality tracks the model's current capability rather than freezing at the moment of admission. In this design the zero-shot tier absorbs queries that the self-improvement loop has taught the generator to answer, the memory-direct tier absorbs queries that recur and whose cached answer survives the retroactive pass, and the retrieval-augmented tier retains the novel queries that neither mechanism yet covers. On the realised six-cycle trajectory this redistribution is muted on the evaluation fold: the retrieval-augmented tier served 51.3 percent of evaluation queries at cycle zero and between 44.7 and 55.1 percent at later cycles, and the memory-direct tier served at most 0.1 percent, because the evaluation questions are content-disjoint from the stored episodes by construction (Table 5.9). The memory-direct tier is built for repeated and paraphrased questions: on a separate probe of 100 questions built from episodes stored by cycle five, it fired on 42 of 50 exact repeats and 19 of 50 paraphrases, and its answers were correct in 41 of the 42 and all 19 cases (Table 5.8).

The user-facing surface mirrors the same logic. Every served answer carries the calibrated probability of correctness as a continuous percentage alongside the four-class decision tag, so a user sees both the categorical reliability label (verified, provisional, conflicting, or insufficient) and the underlying probability that the answer is right. When the calibrated probability and the grounding evidence both fall short of the registered floors, the system abstains explicitly rather than serving a guessed answer with an inflated confidence label. The architecture therefore targets three deployment-time properties at once: hallucination that falls across cycles through the verifier-gated self-improvement loop, per-query cost that falls as queries migrate from the retrieval-augmented tier into the cheaper zero-shot and memory-direct tiers, and answers that always carry the system's own calibrated estimate of their reliability.

There is also a theoretical case for why this combined intervention can succeed even when verification is itself imperfect. By Bayes' rule, the stored pool is purer than

the generator's raw output whenever the storage gate is more likely to admit a correct answer than an incorrect one, however imperfect the gate is otherwise. The formal statement is Proposition 4.2; how the property is expected to carry across cycles is stated in Chapter 4 as a design heuristic rather than a proven result. For the rationale here, the relevant point is that the architecture does not require a perfect verifier; it requires a verifier whose balanced accuracy exceeds one half on the distribution of claims actually scored, and Chapter 5 reports the evidence on whether that condition holds on the five-benchmark panel.

The scope of the intervention is bounded so that it can be argued empirically within an undergraduate research budget: a single small open generator (Qwen-2.5-3B-Instruct), five public factual question-answering benchmarks, and a single consumer-grade graphics processor, so that the result is reproducible without specialised infrastructure; Section 1.6 sets out the full scope. The next section formalises the problem the intervention is designed to solve.

1.3 Problem Statement

1.3.1 Formal problem

Let a deployed question-answering system be specified by a generator distribution and a retrieval substrate, both held fixed at deployment time. A query arrives and produces an answer through the system's per-query pipeline, after which the system must decide whether to commit the produced answer into a persistent state that will influence future answers. The thesis addresses the following problem. Given a stream of queries drawn from a knowledge-intensive distribution, design a deployable system whose committed answer pool is dominated by correct answers, whose user-facing answers carry a calibrated probability of being correct that is refitted on labelled data at every cycle, so that its precision can be checked against held-out data, whose computational cost per query falls as the system processes more of the stream, and whose general capability does not erode as it accumulates domain-specific knowledge. The system is required to satisfy these properties without per-domain calibration at deployment, without external service calls, and on a single small instruction-tuned generator.

The objects under control are the storage decision applied to each generated answer, the calibration of the confidence used to make that decision, the routing of incoming queries among a memory tier and one or more generation tiers, and the cycle-boundary update of the model parameters. The objects not under control are the generator's prior over factual content, the retrieval corpus, and the eventual stream distribution. Within the controlled set the design must produce a falsifiable empirical claim against a panel of external baselines under matched protocol.

1.3.2 Shortcomings addressed and exclusions

The system must address the three shortcomings set out in Section 1.1.3 together: stateless operation that discards verified answers, stated confidence that does not separate correct from incorrect answers, and a static model that cannot learn from verified use. Because the three reinforce one another, addressing any one in isolation

yields a system that is better on one axis and unchanged on the other two: a memory without calibrated confidence cannot tell when it is worth consulting, and calibration without self-improvement leaves the underlying error rate unchanged. The thesis therefore commits to a joint treatment in which memory feeds routing, routing feeds generation, and verification feeds both memory and the cycle-boundary update.

Two classes of approach are explicitly excluded from the present scope. The first is changes to the generator's pre-training corpus or pre-training objective. The thesis treats the generator as fixed and instruments the system around it, so that the architectural claim is separable from any pre-training advantage. The second is reliance on external commercial verification services. Every verification signal the system uses must be reproducible from the open weights of a small specialised model and a publicly available retrieval corpus, so that the result is reproducible without paid infrastructure. The boundary of the problem the thesis solves is therefore an architecture that operates on a fixed open generator, with open verification machinery, on standard hardware, and that is designed to satisfy, and evaluated against, the four properties registered in Section 1.3.1 on the five-benchmark factual question-answering panel of Section 5.1.1. The next section converts this problem statement into the specific objectives that organise the rest of the work.

1.4 Objective

1.4.1 Primary objective

The primary objective of this thesis is to develop a deployable architecture that reduces confident factual hallucination in knowledge-intensive question answering on a single small instruction-tuned generator, while preserving the generator's general capability across self-improvement cycles. The architecture, CAEM, addresses the three reinforcing shortcomings registered in Section 1.3 through a verified episodic memory paired with a calibrated probability composite, a four-outcome storage decision, a confidence-aware three-tier router with an independent safety override, retroactive cycle-boundary review of stored content, and a constrained self-improvement loop guarded by a multi-modal retention probe, a panel of held-out probes that spans three task formats (general-knowledge multiple choice, open-text factoid, and commonsense multiple choice). The success of the architecture is measured against a panel of external baselines under matched protocol on a five-benchmark factual question-answering panel partitioned into three training benchmarks and two transfer benchmarks, using a composite hallucination metric that aggregates seven measurable subtypes of confident-error failure mode, together with per-tier latency and the per-cycle retention ratio as auxiliary diagnostic axes.

1.4.2 Specific objectives

The primary objective decomposes into seven specific objectives. Each is a separate engineering or scientific deliverable that contributes to the joint claim and that admits an independent empirical check in the later chapters.

1. **Verified episodic memory with a four-outcome decision:** Design an episodic memory whose admission rule is finer than a single binary threshold.

A generated answer enters one of four mutually exclusive outcomes determined by a calibrated probability composite, a confabulation early-exit rule that fires when high internal confidence coincides with low external grounding, and a deferral path that holds borderline answers for cycle-boundary reconsideration. Storage uses a similarity-indexed encoder over query and reasoning content with a novelty filter and a value-based pruning policy that combines recency, retrieval frequency, and stored-answer quality.

2. **Pre-generation confidence, three-tier routing with safety override, and a retrieval-utility classifier:** Compute a pre-generation confidence estimate from a short greedy decode over the query that fuses the token-probability aggregate of the first thirty-two decoded tokens with a layer-convergence ratio read from the hidden states over the query, calibrate it via per-benchmark temperature scaling on a disjoint fold, and use it independently from the memory similarity score and the stored-answer quality. The router combines query-to-memory similarity with stored-answer quality into a single dispatch score that selects between memory-direct retrieval, zero-shot generation, and retrieval-augmented generation. An independent safety override forces the conservative tier whenever pre-generation confidence falls below a fixed floor, so that grounded retrieval is available before generation begins and is not entangled with the quality of memory the query happens to land near. A retrieval-utility classifier (Stage 3b) then intercepts the safety-override dispatch and the score-formula fall-through to the retrieval-augmented tier and refines the default into either the zero-shot tier or the retrieval-augmented tier on a per-query basis (Section 4.2.5); its contribution is measured by the on/off ablation in Chapter 5.

3. **Nine-signal post-generation verifier with a calibrated probability composite:** Combine nine complementary signals into a per-answer record across four families: internal calibration of the generator, sample-set agreement across resampled chains, external grounding against retrieved evidence under three aggregation rules, and question-answer match. The signals enter a calibrated probability composite that fits per-signal isotonic regression both on the pooled training-panel fold and on each per-benchmark slice, blends each per-benchmark child curve with the pooled fit through a James-Stein style shrinkage prior, and aggregates the per-signal calibrated probabilities through a regularised logistic regression layer fitted on the same calibration fold. The learned aggregation follows the multi-scoring approach of Valentin et al. [83], which combines the logits of several hallucination scores through logistic regression, in place of a fixed weighted sum.

4. **Fixed-threshold storage gate with two registered cut-points:** Convert the calibrated probability composite into the storage and deferral decisions through two cut-points and one abstention floor, all fixed before the main run and held constant across cycles. The storage cut-point is chosen on the cycle-zero calibration fold and sets how sure the system must be before an answer enters memory; the deferral cut-point, set lower, sends a wider band of borderline answers to the deferred buffer for reconsideration at the next cycle boundary.

5. **Retroactive review and deferred reconsideration at cycle boundaries:** Because the verifier's internal-calibration signals read the generator's hidden state directly, stored composites are model-relative snapshots that drift after every fine-tune. A retroactive re-verification pass at every cycle boundary rescores stored episodes under the updated model, overwrites each stored composite with its new value, raised or lowered, and prunes any entry whose composite has fallen below a prune threshold set beneath the storage threshold. A companion pass over the deferred buffer promotes entries that have crossed the storage threshold and discards entries whose age has exceeded a fixed time-to-live.

6. **Constrained self-improvement with a parameter-efficient adapter and a multi-modal retention guard:** Curate the fine-tuning pool from stored episodes that exceed a strict quality threshold set above the storage cut-point and pass the curated pool through a benchmark-balancing reweighting layer that prevents single-benchmark dominance. Fine-tune the generator through a low-rank adaptation layer on the attention and feed-forward projections rather than through full-parameter fine-tuning [37], with the trainable footprint at roughly 2 percent of the backbone parameter count and the underlying backbone frozen across the cycle. The bounded adapter parameter budget, layered on a frozen backbone, acts as the implicit cycle-to-cycle drift bound in the spirit of elastic weight consolidation [20], without the hand-tuned quadratic anchor coefficient that a full-parameter fine-tune would require. Probe a registered set of held-out distributions before and after each fine-tune and abort the cycle with an adapter rollback when the worst probe ratio falls below a fixed floor; the memory is preserved across rollbacks, so accumulated knowledge survives abort events even when the adapter does not.

7. **Empirical validation against external baselines and architectural ablations:** Evaluate the architecture on the five-benchmark panel and the three-probe retention panel described in Section 1.6.1. Report the composite hallucination metric, the per-tier behaviour, the retention ratio, and the calibration trajectory cycle by cycle. Compare against eight external baselines under matched protocol using paired statistical tests with multiple-comparison correction within each benchmark family, and run the pre-specified retrieval-utility-classifier on/off ablation, with its acceptance gates declared in advance and its bundled-confound scope disclosed.

1.4.3 Pre-specified hypotheses

The thesis registers four falsifiable hypotheses ahead of the empirical chapters. Each hypothesis names a specific quantity, a deciding direction, and a deciding statistic, so that the empirical chapters either confirm or refute the hypothesis without ambiguity. Each pairs with a formal statement of Chapter 4, which comprise two proven results, design heuristics, and remarks on idealised models. Below each hypothesis, a short note says what it tests and how it is decided.

1. **Verifier balanced accuracy exceeds one half on the calibration-matched distribution at every cycle:** Storing answers makes the pool purer than the generator's raw output only if the gate admits correct answers more often

than incorrect ones, which is the same as a balanced accuracy above one half (Proposition 4.2). The deciding statistic is the gate's balanced accuracy at the storage threshold, measured on each cycle's calibration fold and reported per benchmark. The hypothesis is falsified by any cycle in which the condition fails on the pooled training panel.

2. **Pool purity is non-decreasing across cycles whenever the precondition is satisfied:** If the condition of H1 holds and the verifier does not change, an idealised model of repeated filtering (Remark 4.4) predicts that each cycle's pool is at least as pure as the last. The deciding statistic is the pool purity at each cycle. The hypothesis is falsified by any cycle in which the condition holds and pool purity falls. Chapter 4 explains why the realised system, which refits its composite at every cycle, may fluctuate around this prediction rather than follow it exactly.

3. **Late-cycle pool-purity increments enter a bounded stationary band on the committed-cycle subsequence:** The convergence heuristic (Heuristic 4.6) expects pool purity to settle. A convergence rate cannot be estimated from six cycles, so the test is whether purity stays inside a narrow band over the committed cycles, that is, the cycles whose fine-tune was kept rather than rolled back. The deciding statistic is the purity on those cycles and the width of the band. The sharper test of a geometric convergence rate needs at least fifteen committed cycles and is left to future work.

4. **The composite hallucination metric at the realised cycle horizon approaches a parameter-bounded asymptote:** The full-pipeline hallucination decomposition (Remark 4.10) splits the user-facing hallucination rate into a part served from memory and a part generated afresh, and predicts a floor set by the memory-tier hit rate and by the correctness ceiling of the generator. The deciding statistic is the gap between the composite hallucination metric at the final cycle and that predicted floor.

Tier 1, the memory-direct path, is built for deployment, where users ask the same questions again and a stored answer can be served for the cost of one lookup. This study measures learning instead, so every stream chunk and the evaluation fold use fresh questions that never repeat a stored one, and Tier 1 is tested by a separate probe that re-asks stored questions (Section 5.1.4). On a repeated question the system serves an answer that has already passed the storage gate, so in deployment both exact match and the composite hallucination metric are expected to improve as the share of repeated questions grows; this study does not measure that share.

The empirical chapters report each hypothesis with its deciding statistic and tag each as supported, partially supported, or unsupported at the trajectory's end. Together the seven specific objectives and the four hypotheses constitute the complete falsifiable content of the architectural claim, each anchored to a measurement reported in Chapter 5. Table 1.1 summarises the registration in compact form.

ID	Hypothesis	Deciding statistic	Formal claim
H1	Verifier balanced accuracy exceeds one half on the calibration-matched distribution at every cycle.	Per-cycle verifier balanced accuracy at the fitted storage threshold, pooled training panel.	Proposition 4.2 precondition
H2	Pool purity is non-decreasing across cycles whenever the precondition is satisfied.	Per-cycle pool-purity trajectory under the precondition-satisfied regime.	Remark 4.4
H3	Late-cycle pool-purity increments enter a bounded stationary band on the committed-cycle subsequence.	Per-cycle pool-purity reading on the committed-cycle subsequence and realised band width.	Heuristic 4.6
H4	The composite hallucination metric at the realised cycle horizon approaches a parameter-bounded asymptote.	Gap between final-cycle composite hallucination metric and the predicted asymptote.	Remark 4.10

Table 1.1: The four pre-specified hypotheses with deciding statistic and the formal claim each tests. Verdicts are reported in Table 5.22.

1.5 Methodology in Brief

1.5.1 Architecture overview

The architecture organises a single query into an eight-stage pipeline that is traversed once per query and wrapped in a per-cycle update loop. The first stage produces a pre-routing confidence from a short greedy decode over the query, fusing the decode's token-probability aggregate with a layer-convergence ratio. The second stage encodes the query with a sentence-embedding encoder and searches a similarity-indexed episodic memory for the nearest stored entry, returning a similarity score and the neighbour's stored-quality score. The third stage combines the similarity score and the stored-quality score into a single dispatch score and routes the query to the memory-direct tier when the score is high, the zero-shot tier in a middle band, and the retrieval-augmented tier when it is low, with an independent safety override that forces the retrieval-augmented default whenever the pre-routing confidence falls below a fixed floor. A retrieval-utility classifier then intercepts both the safety-override dispatch and the score-formula fall-through to the retrieval-augmented tier and refines that default into either the zero-shot tier or the retrieval-augmented tier on a per-query basis, so that retrieval is consulted only on queries it is expected to help; the classifier is specified in Section 4.2.5. The fourth stage executes the chosen tier: the memory-direct tier serves the neighbour's stored answer; the zero-shot tier runs a generation on the fine-tuned generator; the retrieval-augmented tier retrieves passages from a public corpus and runs a grounded generation. The fifth stage passes every generated answer through a nine-signal post-generation verifier whose output is a calibrated probability that the answer is correct. The sixth stage applies a four-outcome decision (store, defer, abstain, discard) to that probability, with a confabulation early-exit rule that takes precedence over the thresholds. Both

stages are summarised below. The seventh stage executes the implied memory and user-facing actions. The eighth stage runs at cycle boundaries and is detailed below. Figure 1.1 gives the high-level view; the full per-stage pipeline appears in Chapter 4.

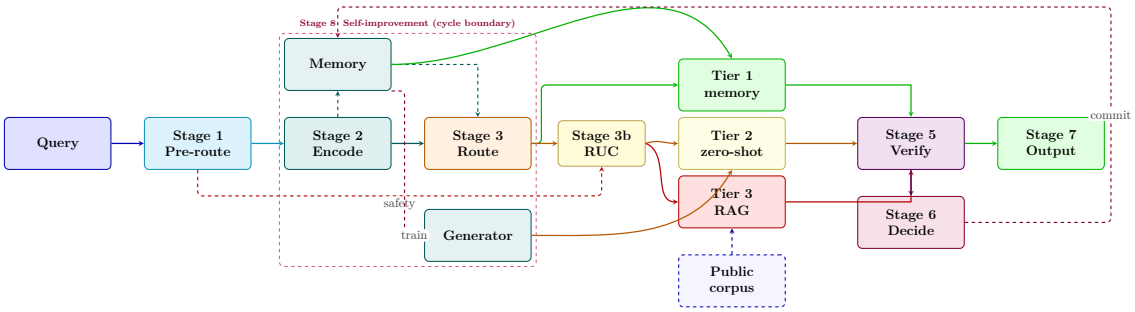


Figure 1.1: Eight-stage pipeline: pre-route, encode, route, Stage 3b retrieval-utility classifier on the score-formula and safety-override fall-through to the retrieval-augmented tier, tier execution, verify, decide, output, and a cycle-boundary self-improvement block; full specification in Chapter 4.

1.5.2 Verification and the storage decision

The verifier evaluates nine complementary signals across four families. Internal calibration signals read the generator's own state and include token-level uncertainty, dropout-based disagreement, and an internal-state composite. Sample-set signals measure agreement across resampled chains, including average pairwise similarity and chain-to-answer entailment. External grounding signals measure entailment between retrieved passages and the answer under three aggregation rules: top-one entailment, pooled mean entailment, and a weakest-link atomic-fact decomposition that fires only when the answer is long enough to admit decomposition. A question-answer match signal, computed by the same cross-encoder that reranks passages, was intended to catch answers that are grounded but off-topic; on the short answers of this panel it acts instead as a learned correctness cue (Section 4.2.6). The signals enter a calibrated probability composite that maps each per-signal score to a calibrated probability via isotonic regression and aggregates them through a regularised logistic regression layer fitted on a disjoint calibration fold; the calibrated output is the storage score. The confabulation early-exit rule discards an answer outright when high internal confidence coincides with low external grounding. Otherwise the four-outcome decision maps the storage score to one of four mutually exclusive actions: commit to memory, defer to a bounded buffer with a time-to-live, abstain explicitly, or discard. The user-facing action that follows depends on the outcome, so that the user receives a stored answer, a generated answer, or a calibrated refusal rather than a silently uncertain response.

1.5.3 Calibration discipline

The storage score is converted into the storage and deferral decisions by two fixed cut-points, both chosen before the main run, the storage cut-point on the cycle-zero calibration fold, and held constant across cycles (Section 4.2.8 explains why fixed

cut-points replaced an earlier conformal formulation). What changes from cycle to cycle is the calibrated probability composite: at every cycle boundary a labelled calibration fold refits it, per benchmark under a shrinkage prior toward the pooled fit, while the verifier-curated answers produced by the pipeline supply the training signal for the fine-tune. The research trajectory reuses one cold-start calibration fold of 1 500 labelled samples, re-scored under each cycle's fine-tuned model; the production design in Chapter 4 instead draws a fresh stratified labelled fold from each cycle's admissions. Because the cut-points are fixed and only the composite is refitted, the meaning of the storage threshold stays the same as the score distribution shifts.

1.5.4 Cycle-boundary self-improvement loop

The cycle loop iterates the per-query pipeline over a fixed workload, accumulates verified episodes into memory, and runs three cycle-boundary passes before fine-tuning. The retroactive re-verification pass rescores every stored episode under the updated generator, updates each stored composite to its new value, and prunes entries whose composite has fallen below the prune threshold. The deferred reconsideration pass examines every entry in the bounded deferral buffer and either promotes, expires, or requeues the entry. The memory consolidation pass collapses near-duplicate stored episodes that exceed a similarity threshold, picking the highest-quality representative within each near-duplicate cluster. Fine-tuning then draws a training pool from stored episodes whose composite score exceeds a strict quality threshold above the storage threshold, ensuring the training pool is held to a higher bar than the user-facing retrieval tier. The fine-tune is regularised implicitly through the bounded low-rank-adapter parameter budget layered on a frozen backbone, in the spirit of elastic weight consolidation [20] but without an explicit quadratic anchor coefficient. Before and after the fine-tune, the cycle probes the three held-out retention surfaces and computes the worst per-probe retention ratio. When that ratio falls below a fixed floor, the cycle aborts: weights are rolled back to the previous checkpoint and the retroactive pass for the next cycle is skipped. The memory is never rolled back, so accumulated knowledge survives abort events even when weights do not. This asymmetry decouples weight-level drift from memory-level accumulation and is what makes the self-improvement loop safe to iterate.

Figure 1.2 illustrates the trajectory across early, mid, and late phases. Each phase runs the same per-query pipeline over a fresh workload of queries, then applies the cycle-boundary update passes under the retention guard. The three boxes are representative phases rather than literal per-cycle depictions; the actual trajectory is reported cycle by cycle in Chapter 5.

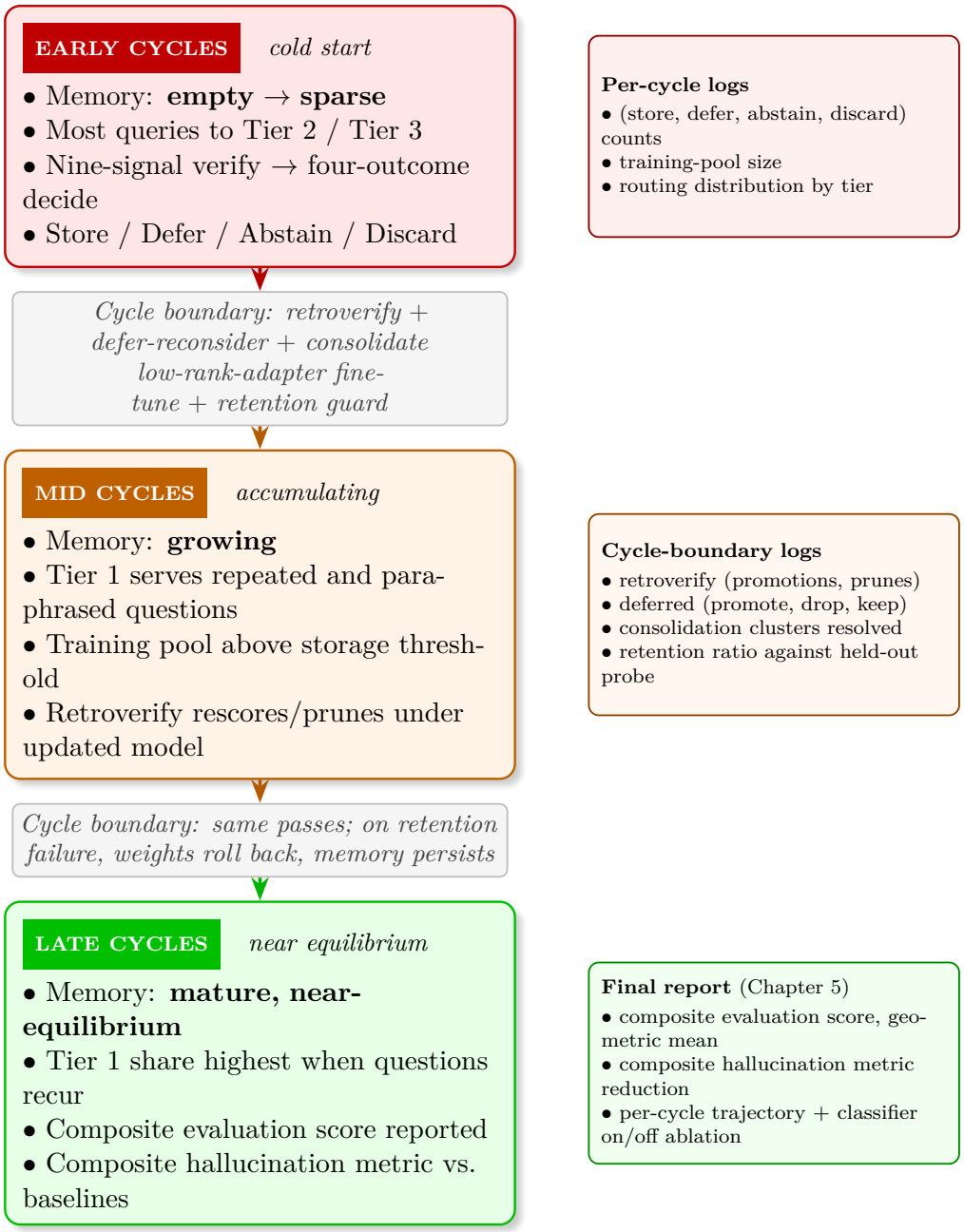


Figure 1.2: Intended self-improvement trajectory across early, mid, and late phases (design view). Tier 1 answers only repeated or paraphrased questions, so on the evaluation fold, whose questions are distinct from the stored episodes, it rarely fires; the realised per-cycle readings are in Chapter 5. Each phase runs the per-query pipeline and, at cycle boundaries, the retroactive re-verification, deferred reconsideration, consolidation, and adapter fine-tune under the multi-modal retention guard. Full specification appears in Chapter 4; headline results in Chapter 5.

1.6 Scopes and Challenges

1.6.1 Research scope

The architecture targets factual question answering, defined as the task of producing answers that admit a definite correctness verdict against a reference. The five eval-

uation benchmarks adopted in this work span factual claim verification (FEVER), open-domain trivia (TriviaQA), multiple-choice commonsense reasoning (CommonsenseQA), truthfulness on common-misconception probes (TruthfulQA), and multi-step commonsense reasoning (StrategyQA), partitioned into a three-benchmark training panel (FEVER, TriviaQA, CommonsenseQA) and a two-benchmark transfer panel (TruthfulQA, StrategyQA). Two further benchmarks, HotpotQA [23] and Natural Questions, were evaluated at cycle zero and dropped before the main run: the base generator answered only 9 and 17 percent of their questions correctly, respectively, and the storage gate kept just 26 and 12 of 500 answers, of which half or fewer were correct, too few and too noisy to train on. Their cycle-zero results are preserved in the project archive, and the Natural Questions training split is reused only to train the retrieval-utility classifier. Each of the five panel benchmarks admits an objective scoring rule, namely exact match or label accuracy, depending on the task family; TruthfulQA's free-text answers are judged against the benchmark's reference answers by a language-model judge, following the judge-based evaluation introduced with the benchmark [40], and the same judge scores every system. A multi-modal retention probe panel is reserved as held-out retention control rather than as a target benchmark: the fine-tuning loop is not optimised on any probe, and the retention guard aborts any cycle in which the worst per-probe accuracy ratio falls below a fixed fraction of the cycle-zero baseline. The probe panel comprises a general-knowledge multiple-choice surface (MMLU), an open-text factoid surface drawn from the TriviaQA test fold, and a commonsense multiple-choice surface drawn from the CommonsenseQA test fold; the worst-probe rule is the registered abort criterion.

The scope explicitly excludes tasks whose evaluation is intrinsically subjective, including creative generation, open-ended dialogue, and stylistic rewriting. Tasks that require knowledge beyond the pre-training cutoff, such as breaking news, market data, and recently issued documents, are excluded because the system does not include a continuous-update mechanism for ingesting external knowledge sources. Code generation and formal mathematical reasoning are excluded so that the evaluation surface remains within natural-language reasoning and the verifier signals retain a uniform interpretation across benchmarks. The system operates on English text only; multilingual transfer and code-switched evaluation are deferred to future work.

Resource scope is set by the engineering envelope of an undergraduate thesis. The generator is a three-billion-parameter open-weight instruction-tuned decoder-only model, chosen for two reasons. First, it admits parameter-efficient fine-tuning on a single consumer-grade graphics processor with thirty-two gigabytes of memory, under sixteen-bit mixed-precision arithmetic, through a low-rank adaptation layer on the attention and feed-forward projections, with the trainable footprint at roughly 2 percent of the backbone parameter count and the underlying backbone frozen across the cycle; full-parameter fine-tuning on the same backbone size does not fit the engineering envelope and is registered as the rejected alternative on the joint retention-and-feasibility axis. Second, as an instruction-tuned model it has competitive baseline accuracy on factual question answering for its size, which the self-improvement argument requires (see the baseline-performance challenge below). The ten-cycle horizon is sized to capture the early-cycle calibration phase, the mid-cycle accumulation phase, and the late-cycle equilibrium phase expected by the

convergence heuristic in Chapter 4, while remaining feasible within a twelve-month compute envelope; an automatic early-stop check at every cycle boundary can stop the run earlier. The realised run stopped after cycle five, as planned after the cycle-four rollback (Section 3.3.2). The episodic memory is sized to a target capacity in the millions of entries with a tiered index that promotes from a flat structure to a quantised inverted-file structure above a threshold; the workloads reported in this thesis grow the memory to 12 175 entries at cycle five (Table 5.11), below the size at which the index switches to the quantised structure and far below the capacity at which value-score pruning triggers, so the realised run uses exact similarity search and never invokes pruning.

1.6.2 Technical challenges

The implementation faces six primary technical challenges that the architecture must address.

- **Verifier reliability and stored-quality drift:** The nine-signal post-generation verifier produces a calibrated storage score that doubles as a retrieval-time quality proxy and as a quality filter on the training pool. Three of the nine signals belong to the internal-calibration family and read the generator's hidden state directly, so their distributions shift after every fine-tune. The stored composite of every episode is therefore a model-relative snapshot, not a permanent attribute of the entry. The retroactive re-verification pass at each cycle boundary refreshes every stored composite under the updated generator, raising or lowering it to the new value, and prunes entries whose composite has fallen below the prune threshold. The cost of this discipline is a full re-scoring of every stored episode at each cycle boundary, which grows with the size of the memory.
- **Catastrophic forgetting under iterative fine-tuning:** Fine-tuning on domain-specific verified examples risks degrading the generator's general-purpose capabilities. The architecture mitigates this risk through three layers of defence: a strict training-pool quality filter that holds the training pool to a higher bar than the user-facing storage tier, a bounded low-rank-adapter parameter envelope layered on a frozen backbone that acts as the implicit cycle-to-cycle drift bound (in the spirit of elastic weight consolidation [20] but without an explicit quadratic anchor coefficient on the shipped path), and a multi-modal retention probe panel whose worst per-probe ratio gates the commit. The retention probe panel spans a general-knowledge multiple-choice surface, an open-text factoid surface drawn from a training-benchmark test fold, and a commonsense multiple-choice surface drawn from a second training-benchmark test fold; the cycle aborts when any per-probe ratio falls below the fixed floor, weights are rolled back to the previous checkpoint, and the memory is preserved so that a fine-tune that fails in any given cycle does not lose the verified episodes accumulated during it. The three layers are not ablated individually within the realised horizon; the retention guard's effect is visible in the cycle-four rollback reported in Chapter 5, and the only architectural ablation reported there is the retrieval-utility-classifier on/off contrast.
- **Pre-routing confidence calibration:** The pre-routing confidence is used

both as a one-dimensional input to the dispatch score and as the sole input to the safety override that forces grounded retrieval, so its calibration influences the system's behaviour twice. The architecture fuses two cheap signals, namely the token-probability aggregate of a short greedy decode that scores the first thirty-two tokens of a draft answer and a layer-convergence ratio computed from the hidden states over the query, and temperature-scales the fused scalar per benchmark on a disjoint fold by minimising binary negative-log-likelihood. The non-trivial design choice is that the pre-routing signal does not require the full multi-chain sampling that the post-generation verifier uses, so its cost is paid per query regardless of which tier eventually serves the answer; the cost of the more expensive sample-based signals is deferred to the post-generation verifier on the zero-shot and retrieval-augmented paths only.

- **Memory retrieval quality and novelty filtering:** At deployment scale the episodic memory uses approximate nearest-neighbour search, which raises the possibility of missing a semantically similar entry whose embedding happens to fall near a partition boundary; at the scale of the realised run the search is exact. Near-duplicate writes are filtered at storage time by a novelty threshold on cosine similarity, and the consolidation pass at each cycle boundary collapses near-duplicate clusters by retaining only the highest-quality representative within each cluster. The novelty threshold is a tuning knob with a clear failure mode in either direction: a threshold set too high admits paraphrase-level duplicates that bloat the index without adding coverage, while a threshold set too low rejects legitimate paraphrases that would have improved retrieval. The chosen value is fixed before the main run and held constant across cycles.
- **Baseline performance dependency for self-improvement:** The self-improvement argument depends on the base generator performing meaningfully above random on the target benchmarks, because training on verified-but-imperfect episodes can only compound accuracy when the verifier's expected false-positive rate is low enough that the training pool is cleaner than the generator's untrimmed outputs. The risk is mitigated by selecting an instruction-tuned backbone with competitive baseline accuracy on factual question answering, by selecting benchmarks on which the base generator clears that threshold, and by composing the storage decision from the calibrated probability composite, the confabulation early-exit gate, and the in-distribution training-pool gate that excludes transfer-benchmark episodes from fine-tuning. Each component contracts the storage class below the rate that an unrestricted score would admit, and the conjunction is what makes the precision floor on the storage tier achievable.
- **Evaluation metric selection:** Measuring hallucination reduction requires a metric design that is more discriminative than aggregate accuracy. The composite hallucination metric used in this thesis is the equal-weighted mean over a seven-axis failure-mode taxonomy: confident confabulation, factual fabrication, logical fabrication, defensive evasion, template leakage, false refusal, and length-padded over-generation. Two further axes, factual contradiction and off-topic answers, are recorded but left out of the mean because neither can fire in this run: the entailment judge has no contradiction output, and the question-match score cannot detect off-topic answers (Section 4.2.6). A uniform reduction

across seven structurally distinct failure modes is a stronger architectural claim than a larger reduction on any single axis. The composite evaluation score is a geometric mean across accuracy, epistemic quality, retention, calibration, and verifier reliability; the geometric form penalises any axis that collapses, which matches the design constraint that no single axis may be sacrificed for the others. Comparison against the baseline panel is conducted under matched protocol, with the same evaluation samples, backbone, answer format, and scoring rules, so that effects are not confounded by re-implementation choices.

Each mitigation above is built into the architecture rather than added as a downstream patch; Chapter 5 reports the evidence available for each on the realised trajectory.

1.7 Chapter signpost

This chapter has registered the deployment-safety problem, the architectural opportunity that closes the stateless-overconfident-static triad, the seven specific objectives that decompose the primary objective, and the four pre-specified hypotheses that the empirical chapters adjudicate. Chapter 2 establishes the technical foundations on which the architectural commitments rest (the transformer backbone, the hallucination taxonomy, natural language inference, sentence embeddings, self-consistency decoding, and catastrophic forgetting) and surveys the empirical literature on hallucination measurement, verification, confidence estimation, memory-augmented architectures, selective retrieval, self-improvement and continual learning, external baselines, and evaluation benchmarks. The survey locates the architectural contribution of this thesis against the prior work that the seven specific objectives draw from, so the methodology can be read as a single integrated response to a literature whose individual contributions do not close the joint gap. Chapter 3 states the functional requirements, the societal and environmental impact, and the project plan. Chapter 4 specifies the architecture stage by stage together with its formal analysis. Chapter 5 reports the realised six-cycle trajectory, the baseline comparisons, the retrieval-utility-classifier ablation, and the verdict on each hypothesis. Chapter 6 summarises the findings, the limitations, and the future work, and the appendices collect the implementation reference, the per-cycle evidence, the significance-test details, and the reproducibility recipe.

Chapter 2

Literature Review

2.1 Preliminaries

Chapter 1 laid out the three limitations of current LLMs that CAEM addresses (stateless operation, poor confidence calibration, absence of self-improvement) and the three architectural commitments (verified episodic memory, confidence-aware routing, constrained self-improvement) that answer them. This chapter supplies the two kinds of background a reader needs to evaluate those commitments in detail. The Preliminaries section that follows establishes foundational concepts and terminology: the transformer backbone, the hallucination taxonomy, natural language inference, sentence embeddings, self-consistency decoding, and catastrophic forgetting. The Review of Existing Research section that follows it then surveys the empirical literature on hallucination measurement, verification, confidence estimation, memory-augmented architectures, selective retrieval, self-improvement and continual learning, external baselines, and evaluation benchmarks, locating CAEM's contribution relative to prior work. Together they prepare the ground for the formal methodology in Chapter 4.

2.1.1 Large Language Models and Transformer Architecture

Large language models are neural networks trained on very large text collections to predict and generate natural language. Modern large language models adopt the Transformer architecture [21], whose self-attention mechanism lets every token in the input attend to every other token. The original Transformer stacks encoder and decoder layers, each combining multi-head self-attention with a feed-forward network; the decoder-only variant used in this thesis keeps only the decoder stack.

This thesis uses Qwen-2.5-3B-Instruct [81], a three-billion-parameter decoder-only transformer that has already been instruction-tuned and preference-tuned before it enters the CAEM pipeline. Decoder-only models generate the answer token by token from left to right, with the question and the generated rationale sharing a single causal attention stream. The backbone is chosen for two reasons: it fits the whole self-improvement loop, including low-rank fine-tuning, Monte Carlo dropout sampling and the nine-signal verifier, on a single consumer-grade graphics processor in sixteen-bit brain-float precision; and because it is instruction-tuned, it already

answers in the required format, so the verifier has usable outputs to score from the first cycle.

Fine-tuning adapts pre-trained models to specific tasks or domains by continuing training on task-specific data with a smaller learning rate. The model parameters $\boldsymbol{\theta}$ are adjusted to reduce loss on the new data while preserving the general abilities learned during pre-training. Standard fine-tuning updates every parameter; parameter-efficient approaches keep the underlying backbone frozen and adapt only a small fraction of the parameter count through trainable side-modules. CAEM uses the parameter-efficient path through the low-rank adaptation (LoRA) layer of Hu et al. [37] on the attention and feed-forward projections, with the trainable footprint at roughly 2 percent of the backbone parameter count and the underlying backbone frozen across every cycle of the self-improvement loop. The empirical evidence motivating that choice over full-parameter fine-tuning is reviewed in Section 2.2.6.

2.1.2 Hallucination Taxonomy

Hallucinations in language model outputs can be understood along two main dimensions. The first dimension separates *intrinsic hallucinations*, which directly contradict information provided in the source input, from *extrinsic hallucinations*, which introduce information that can be neither supported nor contradicted by the source [50]. For instance, if a biography states that "Einstein was born in Germany" and the model generates "Einstein was born in France," this is an intrinsic hallucination because it contradicts the stated source. But if the model generates "Einstein played violin exceptionally well" without any evidence in the source, this is an extrinsic hallucination even if the claim happens to be true: the defining characteristic is that it cannot be verified from the input, not that it contradicts anything in it.

The second dimension distinguishes *factuality hallucinations*, which conflict with verifiable world knowledge, from *faithfulness hallucinations*, which contradict user input or task context [89]. For example, a model that states "photosynthesis produces oxygen from carbon dioxide and water" is factually accurate, whereas one that states "photosynthesis occurs in mitochondria" produces a factuality hallucination. Faithfulness focuses on whether the output of the model remains consistent with the supplied input or context, especially when summarising documents or following instructions.

This research focuses on factuality hallucinations, which occur in knowledge-based reasoning tasks. Here, correct answers can be verified against trusted sources or logical rules. Faithfulness to retrieved evidence also matters for retrieval-augmented generation, where outputs must be grounded in retrieved documents.

Within factuality hallucinations, four subclasses are distinguished operationally because the four signal families of CAEM's verifier map onto them.[1] *Confabulations*, in the sense of Farquhar et al. [71], are answers the model might equally have produced differently on a resample, even when the individual token probabilities look confident; they are targeted by the sample-set agreement signals, which compare several resampled reasoning chains with one another (s_{avg}) and with the served

[1]The nine signals are defined formally in Chapter 4; this chapter names them only where needed for motivation.

answer (p_{entail}). *Ungrounded factual claims* are answers without support in retrieved evidence; they are targeted by the three external-grounding signals, which score the answer against retrieved passages. *Off-topic answers* that drift from the question were meant to be targeted by the question-answer match signal q_{match}; Chapter 4 explains why it cannot detect them on short answers. *Confidently wrong* answers, internally confident but externally unsupported, are caught by the confabulation early-exit gate, which fires when a high internal-confidence signal coincides with low grounding. Outside this scope, CAEM does *not* target instruction-following failures, intermediate-reasoning-step errors inside chain-of-thought traces (only the final answer is verified), intrinsic faithfulness hallucinations in summarisation or translation (where the grounding source is the input document rather than world knowledge), or input-contradiction hallucinations that conflict with user-supplied premises. These exclusions set the failure-mode boundary within which the results in Chapter 5 should be interpreted.

2.1.3 Natural Language Inference

Natural Language Inference (NLI) is the task of determining whether a hypothesis sentence logically follows from a premise sentence [9]. The PASCAL Recognising Textual Entailment challenges framed this as a binary decision (entailed or not entailed); later NLI corpora, including the Multi-Genre NLI corpus on which RoBERTa-Large-MNLI is fine-tuned, use three classes. Given premise P and hypothesis H, a three-class NLI model classifies the relationship as:

- **ENTAILMENT**: P implies H is true
- **CONTRADICTION**: P implies H is false
- **NEUTRAL**: P neither confirms nor denies H

For example, if the premise is "The Beatles were formed in Liverpool in 1960", the hypothesis "The Beatles were an English band" is entailed, the hypothesis "The Beatles were formed in London" is a contradiction, and the hypothesis "The Beatles recorded many albums" is neutral.

CAEM's verifier uses MiniCheck-Flan-T5-Large [82] as its entailment judge, a Flan-T5-Large checkpoint fine-tuned on synthetic claim-support data. MiniCheck is a binary judge: it returns the probability that a claim is supported by the given evidence and has no separate contradiction class, so a contradicted claim simply receives a low support probability. A second judge for long hypotheses was considered and dropped before the main run (Chapter 4), so every hypothesis is scored by MiniCheck, truncated at its 512-token input limit. Generic three-class NLI models such as RoBERTa-Large-MNLI [25], which reaches about 90.2% on the MNLI matched development set, are the conventional alternative, but they transfer poorly to the hallucinations of language models: Sinha [92] reports a one-hundred-percent false-positive rate at ninety-five-percent recall for a near-state-of-the-art generic NLI model on HaluEval [54]. Entailment checked in both directions can also decide whether two answers mean the same thing: Kuhn et al. [53] treat two answers as equivalent if P entails H and H entails P. This is the basis of semantic entropy, which CAEM uses as an external baseline (Section 2.2.7).

2.1.4 Sentence Embeddings and Semantic Similarity

Sentence embeddings map variable-length text sequences to fixed-dimensional dense vectors in a semantic space where similar meanings have high cosine similarity. Sentence BERT (SBERT) [27] produces sentence embeddings by fine-tuning BERT in a siamese network structure, first on natural language inference data (SNLI and MultiNLI) and optionally on semantic textual similarity data. The model used here, all-mpnet-base-v2 from the same Sentence-Transformers family (built on an MPNet backbone rather than the original BERT), generates 768-dimensional embeddings.

Cosine similarity between vectors $\mathbf{v}_1$ and $\mathbf{v}_2$ is computed as:

$$\text{sim}(\mathbf{v}_1, \mathbf{v}_2) = \frac{\mathbf{v}_1 \cdot \mathbf{v}_2}{\|\mathbf{v}_1\|\|\mathbf{v}_2\|} = \cos(\theta) \tag{2.1}$$

where θ is the angle between vectors. Similarity ranges from -1 (opposite) to 1 (identical), with 0 indicating orthogonality.

Semantic similarity is different from string similarity. For instance, the questions "Is 17 prime?" and "Is the number 17 a prime number?" share few words but are close in meaning. CAEM uses semantic similarity for memory retrieval (finding similar stored questions), novelty detection (avoiding redundant storage), and the self-consistency signal (measuring agreement between reasoning chains). Nearest-neighbour search uses the FAISS library [36]; at the memory sizes reached in this thesis the search is exact, and FAISS's approximate indexes take over only at much larger sizes.

2.1.5 Self-Consistency Decoding

Self-consistency [62] improves reasoning by generating several independent reasoning chains for the same question and selecting the most common answer. Unlike greedy decoding, which produces a single output, self-consistency samples M chains at a temperature $T > 0$ to introduce diversity.

CAEM adapts self-consistency for verification rather than answer selection. Given a question q and a generated answer a, the verifier generates M additional chains and measures their agreement as the average pairwise similarity

$$s_{\text{avg}} = \frac{2}{M(M-1)} \sum_{1 \le i < j \le M} \text{sim}(\mathbf{v}_i, \mathbf{v}_j) \tag{2.2}$$

where $\mathbf{v}_i$ is the embedding of chain i and only distinct pairs are counted. High agreement suggests that several reasoning paths converge on the same content; low agreement suggests uncertainty. CAEM pairs s_{avg} with the chain-to-answer entailment signal p_{entail}, computed on the same M=3 chains so that the decoding cost is paid once; together they form the sample-set agreement family of the verifier. The normalised semantic entropy of Farquhar et al. [71], computed over K=10 samples at temperature T=1.0, was originally a third member of this family. It was dropped before the main run because its learned weight in the cycle-zero composite fit was effectively zero (about -0.0005), so it added nothing beyond the other signals, while its ten extra samples per answer were the largest single cost in the verifier (about four seconds per answer). The kernel-language-entropy generalisation of Nikitin et al. [79], a candidate replacement that needs the same extra samples, was not adopted either.

2.1.6 Catastrophic Forgetting and Continual Learning

Catastrophic forgetting, first described by McCloskey and Cohen [6], occurs when neural networks learn new tasks and suddenly lose their ability to perform previously learned tasks. Standard gradient descent focuses only on improving performance for the current task and does not protect earlier knowledge, so parameter updates that help the new task tend to weaken the old ones. In large language models, fine-tuning on a new domain likewise erodes performance outside that domain, and Biderman et al. [65] found that the erosion grows with training duration.

Elastic Weight Consolidation (EWC) [20] addresses this by estimating, through the Fisher information, which parameters matter for earlier tasks, and adding a quadratic penalty that discourages large changes to them:

$$\mathcal{L} = \mathcal{L}_{\text{new}} + \frac{\lambda}{2} \sum_i F_i(\theta_i - \theta_i^*)^2 \tag{2.3}$$

where F_i measures parameter importance, θ^* are the previous parameters, and λ controls the strength of the penalty. A cheaper variant drops the Fisher weights and penalises all parameters equally, $\frac{\lambda}{2}\|\boldsymbol{\theta} - \boldsymbol{\theta}_{\text{prev}}\|^2$; Kirkpatrick et al. [20] show that this uniform penalty is weaker than EWC because it protects all weights equally and leaves little capacity for the new task. CAEM uses neither penalty on its shipped path. It bounds drift between cycles with three other mechanisms instead: a strict quality filter on the training pool; low-rank adaptation [37], which trains only about 2 percent of the parameters on top of a frozen backbone and forgets less than full fine-tuning (Section 2.2.6); and a retention guard that rolls the adapter back, while keeping the episodic memory, whenever accuracy on held-out probe sets falls below a fixed fraction of its starting value. The details are given in Chapter 4.

The conceptual basis for understanding CAEM's nine-signal post-generation verifier, pre-routing confidence, episodic-memory architecture, and constrained self-improvement loop is established by the groundwork above. The following sections describe how prior research develops and validates these individual components, preparing for their integration in Chapter 4.

2.2 Review of Existing Research

Where the Preliminaries section above defined the concepts and notation (hallucination taxonomy, natural language inference, sentence embeddings, self-consistency, catastrophic forgetting), this section surveys the prior empirical and methodological literature on hallucination measurement, verification, confidence estimation, memory-augmented architectures, selective retrieval, self-improvement and continual learning, the external baselines used in this thesis, and evaluation benchmarks. Each subsection identifies key contributions and limitations that motivate CAEM's architectural choices, so that a reader who has already absorbed the Preliminaries can move directly to the comparative positioning without re-reading definitional material.

2.2.1 Hallucination Problem and Measurement

Understanding the nature and characteristics of hallucination in LLMs requires methodologies like taxonomic frameworks and quantitative measurement. Early studies of hallucination focused on summarisation and data-to-text generation. Ji et al. [50] published the first major survey in ACM Computing Surveys. It distinguishes intrinsic hallucinations, which contradict the source material, from extrinsic hallucinations, which can be neither supported nor contradicted by it. Their work shows that hallucinations arise at several stages, including data collection that embeds factual errors, training procedures that reinforce misleading correlations, and inference methods that fail to account for uncertainty.

Huang et al. [89] extended this study specifically for large language models in ACM Transactions on Information Systems, distinguishing factuality hallucinations from faithfulness hallucinations. Their analysis traces the causes of hallucination across the model lifecycle to data, training and inference, and ties each family of mitigation methods to the cause it addresses. This suggests that effective mitigation should intervene at several stages instead of relying solely on detection applied after the model has already produced an answer, which supports CAEM's multi-component strategy combining memory, verification, and iterative learning.

Quantitative measurement studies of hallucination prevalence reveal concerning statistics across domains and model scales. Lin et al. [40] introduced TruthfulQA at ACL 2022, a benchmark of 817 questions across 38 categories designed to elicit imitative falsehoods where models reproduce common human misconceptions found in training data. Their result demonstrates an inverse scaling effect: the best GPT-3 175B configuration achieved only 21% on the truthful-and-informative metric (and 58% on truthfulness alone), against 87% and 94% respectively for the human baseline. Larger models often performed worse than smaller ones. This suggests that scaling alone is not sufficient for truthfulness, which motivates architectural innovation.

The inverse-scaling result has a direct implication for mitigation: if adding parameters can reduce truthfulness on some tasks, mitigation must work beyond parameter growth. CAEM does so by adding external verification and memory components whose operation does not depend on the size of the model.

Min et al. [58] introduced FActScore at EMNLP 2023, which evaluates long-form generations by decomposing the answer into atomic claims and reporting the share supported by a reliable knowledge source rather than scoring the answer as a whole. Its automated estimator stays within an error of 2% of human evaluation, and on biography generation it finds that only 58% of ChatGPT's atomic facts are supported, so even highly capable models produce many unsupported claims. CAEM borrows this decomposition for one of its grounding signals, $p_{\text{ground_atomic}}$, which splits the answer into atomic claims, scores each against the retrieved passages, and keeps the weakest score. The step runs only on answers long enough to contain several claims; the short answers of the five-benchmark panel almost never qualify, so in the realised run this signal coincided with the passage-mean grounding signal on every evaluated answer (Chapter 4).

Domain-specific studies show even higher hallucination rates in specific contexts. Alkaissi and McFarlane [44] asked ChatGPT to explain a clinical finding and to

supply supporting references; none of the five paper titles it returned existed. Chelli et al. [66] checked the references that chatbots generated for systematic reviews and reported hallucination rates of 39.6% for GPT-3.5, 28.6% for GPT-4, and 91.4% for Google Bard.

Three points follow for CAEM's design. First, hallucination occurs across models, tasks and domains, at rates that vary widely with context (from 28.6% to 91.4% of generated references in the single study of Chelli et al. [66]). Second, larger models do not automatically hallucinate less, so the structure around the model has to change. Third, scoring individual claims against evidence is more informative than judging whole responses, which motivates CAEM's evidence-based grounding signals.

2.2.2 Verification Methods

Verification methods are used to check whether a model's reasoning is correct before the information is stored or shown to users. This subsection reviews chain-of-thought prompting, self-consistency decoding, semantic entropy, natural-language inference and conformal filtering, the approaches from which CAEM's verifier draws its signals or against which its design is positioned.

Chain of thought prompting, which was introduced by Wei et al. [41] at NeurIPS 2022, improves reasoning quality by prompting models with worked examples whose intermediate steps precede the final answer. Showing these intermediate steps leads to stronger reasoning and provides better starting points for verification. When tested on the PaLM 540B model, chain of thought greatly improved performance on math reasoning tasks, increasing accuracy on GSM8K from 17.9% to 56.9%. The authors found that the technique is an emergent ability of model scale: below roughly 100 billion parameters chain-of-thought prompting did not help and could lower performance, a pattern that held for the instruction-tuned InstructGPT models in their GPT-3 series as well. Kojima et al. [39] later showed that the single trigger "Let's think step by step" elicits chain-of-thought reasoning without worked examples. Instruction tuning does not remove the small-scale effect: Chung et al. [69] report that their instruction-tuned Flan-T5-Large (780 million parameters) still scores lower with chain-of-thought prompting than with direct answering on MMLU (40.5 versus 45.1 percent) and on BIG-Bench Hard (31.5 versus 37.5 percent). Whether chain-of-thought prompting helps CAEM's three-billion-parameter instruction-tuned backbone, Qwen-2.5-3B-Instruct [81], is therefore an empirical question, which the chain-of-thought baseline in Chapter 5 answers.

For CAEM, chain-of-thought prompting matters in two ways: it is the reasoning format in which the generator answers, and it supplies the resampled reasoning chains from which the sample-set agreement signals are computed. The verifier, however, scores only the final answer and not the individual reasoning steps (Section 2.1.2), so an invalid step that still reaches a correct conclusion goes undetected.

Wang et al. [62] extended this idea in 2023 by introducing self-consistency. Instead of producing a single reasoning path, the model generates several different explanations and then selects the most common answer. The key idea is that difficult and complex problems usually have multiple valid reasoning paths that lead to the same correct answer. On the other hand, incorrect answers rarely achieve consistency across

independent derivations. Reported gains over chain-of-thought prompting reach +17.9 percentage points on GSM8K, +11.0 on SVAMP, +12.2 on AQuA, and +6.4 on StrategyQA; these are the largest gains across the four models tested, and for PaLM 540B the corresponding gains are +17.9, +7.6, +12.5 and +6.3.

CAEM uses self-consistency differently: instead of selecting the majority answer, it measures how far the resampled chains agree in meaning. Strong agreement raises the composite score and weak agreement lowers it. The cost is a small number of extra generations per answer, which keeps the signal practical for routine verification.

Semantic entropy, introduced by Farquhar et al. [71] in Nature in 2024, addresses a limitation of token-level entropy: answers that express the same idea in different words are grouped together by bidirectional entailment, and uncertainty is measured across these meaning-based groups rather than across surface strings.

Averaged over thirty combinations of tasks and models (LLaMA 2, Falcon and Mistral families), semantic entropy reaches an AUROC of 0.790 for detecting confabulations, ahead of naive entropy (0.691), P(True) (0.698), and an embedding-regression baseline (0.687). The method targets arbitrary, sample-to-sample errors: a model that is consistently wrong gives the same answer on every sample and therefore looks certain.

CAEM originally included semantic entropy as a verifier signal and dropped it before the main run (Section 2.1.5); it remains in the thesis as an external baseline that abstains when entropy is high (Section 2.2.7).

Natural language inference provides direct verification by checking whether a claim is supported by evidence. Thorne et al. [22] introduced the FEVER dataset at NAACL 2018, a fact-verification task closely related to NLI, with 185,445 claims generated from Wikipedia; unlike NLI, the evidence for each claim must first be retrieved rather than being given. Systems must classify each claim as SUPPORTS, REFUTES or NOT ENOUGH INFO.

Pre-trained NLI models such as RoBERTa-Large-MNLI reach about 90% accuracy on MNLI and verify individual factual statements well, but they can miss reasoning errors that contradict no fact. A model could state "Sydney is Australia's largest city", which is true, and conclude "therefore Sydney is the capital", an invalid inference; NLI would confirm the premise and miss the faulty step, which is why complementary signals are needed.

A more recent strand of work brings conformal prediction to the language-model claim level. Mohri and Hashimoto [78] introduced *Language Models with Conformal Factuality Guarantees* at ICML 2024, decomposing each generated answer into atomic sub-claims, scoring each, and removing claims below a calibrated threshold τ such that the surviving sub-claims are jointly correct with probability at least $1 - \alpha$ on a small annotated calibration set. On FActScore their method raises factuality from about 30% to 80% while retaining most sub-claims. Yadkori et al. [85] extended this with conformal-risk-control to bound the hallucination rate at a user-specified target through learned abstention rather than claim removal, providing the direct precedent for CAEM's abstention class in the four-outcome decision tree. Quach et al. [80] present *Conformal Language Modeling* at ICLR 2024, calibrating a stopping rule, a rejection rule, and a confidence rule simultaneously through Learn-then-Test multiple-hypothesis testing. Cherian et al. [68] sharpen the Mohri–Hashimoto framework

with conditional guarantees and a boosted score, learning a linear combination of several per-claim scores by differentiating through the conformal procedure; the closest precedent for CAEM's boost layer over the per-signal log-odds in Section 4.2.7 is the logistic-regression multi-scoring of Valentin et al. [83]. CAEM considered split-conformal calibration for its storage gate and rejected it because the distribution shift between its calibration and evaluation folds broke the coverage premise (Section 4.2.8). The deployed gate is therefore a fixed-threshold rule on the calibrated probability, in the lineage of the post-hoc calibration analyses of Niculescu-Mizil and Caruana [8] and the selective-classification framing of Geifman and El-Yaniv [17].

These methods supply complementary signals to CAEM's nine-signal verifier. Self-consistency and chain-to-answer entailment form the sample-set agreement family (s_{avg}, p_{entail}). Entailment against retrieved evidence forms the external-grounding family: the best-passage score $p_{\text{ground_max}}$, the mean-passage score $p_{\text{ground_mean}}$, and the length-gated atomic-fact score $p_{\text{ground_atomic}}$ in the FActScore convention [58]. Three internal-calibration signals (token probability, disagreement among Monte Carlo dropout samples, and an internal-state composite of the two) and the question-answer match signal complete the nine. Instead of a fixed weighted sum, each signal is mapped to a calibrated probability of correctness by isotonic regression [8], and the calibrated signals are combined by a regularised logistic regression in the manner of Valentin et al. [83]; the full specification is in Chapter 4. The point for this chapter is that the literature supplies families of signals whose failure modes differ, which justifies a calibrated multi-signal composite over any single-signal detector.

2.2.3 Confidence Estimation

Reliable confidence estimation helps systems tell the difference between answers that can be trusted and ones that need extra checking from outside sources. This subsection looks at basic methods for measuring uncertainty and explains how these ideas have been adapted for use with large language models.

Gal and Ghahramani [15] laid the theoretical foundation for practical uncertainty estimation in their ICML 2016 paper "Dropout as a Bayesian Approximation". They showed that dropout, normally used only during training, can be read as approximate Bayesian inference over the model's parameters. Keeping dropout active at test time and running the model several times then measures how much the outputs vary. This variation estimates epistemic uncertainty, which comes from the model's limited knowledge, as opposed to aleatoric uncertainty, which comes from noise in the data and cannot be reduced by more knowledge.

The framework shows that a network with dropout before each weight layer approximates a deep Gaussian process, whose predictive mean and variance can be estimated by Monte Carlo sampling: K forward passes with different dropout masks, followed by the variance of the outputs. The estimate needs no ensemble training and no change to the loss function, and it applies to any network that already uses dropout, which makes it convenient for existing large language models.

For CAEM, MC Dropout is one of the three internal-calibration signals inside the post-generation verifier (u_{token}, u_{dropout}, and the derived $u_{\text{internal}} = 0.5\,u_{\text{token}} + 0.5\,(1 - u_{\text{dropout}})$ that combines them). It quantifies model uncertainty through output

variance: high variance indicates the model is unstable with respect to its own weights and suggests that the composite score should be pulled down, while low variance indicates confident generation. This signal is insufficient on its own, since models can be confidently wrong, but the computational cost of the registered five forward passes is acceptable inside the verifier, which only runs on queries that have already been dispatched to Tier 2 or Tier 3 generation. MC Dropout is deliberately *not* used in the pre-routing confidence u_{pre}, which must be cheap enough to run on every query before tier selection.

A major problem with neural network confidence is that models are often too confident in their answers. Guo et al. [18] demonstrated this at ICML 2017. They showed that modern deep networks predict confidence scores that substantially exceed their actual accuracy; a ResNet-110 on CIFAR-100, for example, has an expected calibration error of 16.5 percentage points before calibration. They popularised the Expected Calibration Error (ECE) of Naeini et al. [12] as the standard metric, dividing predictions into bins by confidence and measuring the gap between confidence and accuracy within each bin.

Their proposed solution, temperature scaling, achieves remarkable effectiveness with minimal complexity. The method divides logits by a learned scalar temperature $T > 0$ optimised on a validation set: $\hat{y} = \sigma(\mathbf{z}/T)$ where $\mathbf{z}$ are logits and σ is softmax. Temperature scaling preserves accuracy (argmax unchanged) of classification while calibrating probabilities. Even though only a single parameter is used, it outperforms more complex methods including histogram binning, isotonic regression, and Bayesian approaches. The simplicity and effectiveness make temperature scaling the standard calibration method applied after training is complete.

CAEM applies temperature scaling to its pre-routing confidence u_{pre}, fitted per benchmark on a held-out calibration split, so that a stated confidence of 80 percent corresponds to roughly 80 percent correct answers. The post-generation composite u_{stored} is calibrated differently, by per-signal isotonic regression and a logistic combination (Chapter 4). Both calibrations make it possible to set meaningful thresholds for tier selection, for the four-outcome storage decision, and for the training-pool filter τ_{train}.

Kadavath et al. [38] from Anthropic explored self-evaluation in large language models in a 2022 preprint, introducing two ways of extracting confidence from a model itself. The first, P(True), asks the model to estimate the probability that its own earlier answer is correct. The second, P(IK), the probability that "I know" the answer, is predicted before any answer is proposed, by a model trained for that purpose. Their findings reveal nuanced calibration properties. Larger models tend to be well calibrated on multiple-choice questions, meaning that their confidence matches how often they are actually correct. However, policies fine-tuned with reinforcement learning from human feedback appear badly miscalibrated, because reinforcement learning collapses predictions toward the behaviours that receive the most reward, although a single temperature adjustment largely restores calibration.

The finding that reinforcement learning from human feedback can hurt calibration matters for deployment. A model trained to be helpful may express high confidence to sound authoritative even when it is uncertain, the opposite of what high-stakes use requires. CAEM therefore relies on several external confidence signals rather

than on the model's own confidence alone.

Xiong et al. [84] studied confidence estimation for large language models in detail, comparing white-box methods, which use internal information such as token logits and sequence probabilities, with black-box methods, which rely on external signals such as verbalised confidence or consistency checks. Verbalised confidence is overconfident and discriminates poorly: even for GPT-4 its average AUROC for telling correct answers from incorrect ones is about 0.63, close to the 0.5 of random guessing. Sampling-consistency signals do much better on some tasks (up to 0.93 AUROC on GSM8K), but no single method consistently outperforms the others, and all of them struggle on tasks that require professional knowledge.

CAEM's design follows from this finding that no single signal is reliable across tasks, and splits confidence into two stages. The first, the pre-routing confidence u_{pre}, fuses two cheap signals (the token probability of a short draft decode and a layer-convergence ratio over the query's hidden states) through a temperature-scaled sigmoid; it is computed for every query and drives tier selection and the safety override. The second is the nine-signal post-generation verifier, which runs only after zero-shot or retrieval-augmented generation and produces the calibrated composite u_{stored} that drives the storage decision.

Recent work by Tian et al. [60] asks how calibrated confidence can best be elicited from models fine-tuned with human feedback. Their study found that the raw token probabilities of such models are consistently overconfident, and that confidence verbalised as output tokens is typically better calibrated, often reducing the expected calibration error by a relative 50%. Because the best confidence source differs between model families and tasks, this reinforces CAEM's strategy of combining implicit signals like token probability, dropout, and self-consistency with explicit ones, and of recalibrating every signal rather than trusting any raw score.

A complementary 2024 survey by Geng et al. [72] catalogues white-box and black-box confidence-estimation methods for language models across the same axes that drive CAEM's verifier ensemble, and is the positioning document for CAEM's multi-signal design choice. Two recent single-signal methods deserve specific mention as alternatives considered for CAEM. Chen et al. [67] introduced INSIDE / EigenScore at ICLR 2024, computing the log-determinant of the covariance matrix over middle-layer hidden states across K resampled responses; the method reports AUROC of 0.80 to 0.83 on TriviaQA and clear gains over other unsupervised signals on CoQA, SQuAD and Natural Questions (on TriviaQA, perplexity scores slightly higher), and is registered in CAEM as a candidate replacement for the dropout signal that was not adopted in the locked configuration. Nikitin et al. [79] introduced Kernel Language Entropy at NeurIPS 2024, generalising Farquhar et al.'s hard-cluster semantic entropy to a soft-kernel von Neumann entropy over pairwise semantic similarities; KLE was considered as a replacement should the semantic-entropy family return, but was not adopted because, like semantic entropy, it needs many extra samples per answer (Section 2.1.5). The selective-prediction precision-versus-coverage tradeoff that CAEM reports in Chapter 5 follows the form pioneered by Kamath, Jia, and Liang [31] for selective question answering under domain shift, where a calibrator over feature signals predicts $P(\text{correct})$ and the selective rule answers only when the calibrator's confidence exceeds the registered precision target.

The confidence-estimation literature yields four points for CAEM. A single confidence signal is not reliable enough for safe routing, so several must be combined. Systematic overconfidence calls for calibration after training. White-box signals from inside the model complement black-box signals from output consistency. Preference tuning can degrade calibration, and CAEM's backbone is itself preference-tuned (its post-training uses DPO and GRPO [81]), so no raw confidence is used without recalibration. Together these points shape CAEM's two-stage confidence design: a temperature-scaled pre-routing score u_{pre} on every query, and a nine-signal verifier whose calibrated composite u_{stored} drives the four-outcome storage decision. Both calibrations are fitted on labelled calibration data and refitted at every cycle boundary (Chapter 4).

2.2.4 Memory-Augmented Architectures

External memory architectures give neural networks a place outside their parameters where facts can be written and read. This subsection traces these ideas from early differentiable memories to retrieval-augmented generation.

Graves et al. [11] introduced Neural Turing Machines (NTMs) in 2014. NTMs gave a neural network an external memory that it can read from and write to in a differentiable, trainable way. The architecture combines a controller network with an addressable memory matrix, supporting both content-based addressing (via cosine similarity) and location-based addressing (via shift operations). The key innovation is a read/write mechanism that enables end-to-end training via backpropagation. As a result, NTMs successfully learned algorithmic tasks including copying, sorting, and associative recall.

For CAEM, the lesson is that an external memory can give a model abilities its parameters alone lack. NTMs were applied mainly to algorithmic tasks, but the same principle extends to facts: CAEM's episodic memory stores verified question-answer pairs together with their reasoning, so that a repeated question can be answered by lookup.

Weston et al. [14] introduced Memory Networks at ICLR 2015. Their model scaled to millions of memory slots. It also showed that a system could perform several rounds of reasoning to answer a question. The architecture consists of four learned components. The components are: turning the input into features, updating the memory, turning the memory output into features, and finally producing a response. Furthermore, Memory Networks performed strongly on bAbI question-answering tasks that required reasoning over multiple facts. Sukhbaatar et al. [13] extended this model with end-to-end Memory Networks at NeurIPS 2015. They removed the need for supervision on which memories to retrieve by using a continuous attention mechanism over the memory. This line of work motivates CAEM's use of stored knowledge as an explicit, inspectable memory.

Graves et al. [16] presented the Differentiable Neural Computer (DNC) in Nature 2016. It built on the idea of Neural Turing Machines but added smarter memory tools like dynamic allocation. It also keeps track of the order in which things were written. DNCs achieved a 3.8% mean error on the bAbI tasks, compared with 7.5% for the best previous jointly trained model, which suggests that learned memory management improves performance.

Lewis et al. [33] introduced Retrieval-Augmented Generation (RAG) at NeurIPS 2020. They showed that combining what a model knows in its parameters with information stored outside the model can make its answers much more accurate. RAG adds a large searchable index of Wikipedia to a BART model with four hundred million parameters. It uses Dense Passage Retrieval to look up relevant documents. Then it works out the final answer by considering the generation probability across all the documents it retrieved. The architecture achieves substantial improvements over parametric-only models: 44.5% exact match on Natural Questions (versus 41.5% prior SOTA), 68.0% on TriviaQA (versus 60.5%), and 45.5% on WebQuestions (versus 44.7%).

Critically, RAG demonstrates index hot-swapping: updating the document index without retraining achieves 70% accuracy on questions about updated world knowledge, proving that non-parametric memory can be updated without expensive retraining. This property directly motivates CAEM's episodic memory, which can be updated continuously through verification without model fine-tuning, though CAEM additionally employs iterative fine-tuning to consolidate learned patterns.

The limitation of standard RAG for CAEM's purposes is that retrieved knowledge is not verified before use and does not accumulate verified reasoning chains. CAEM extends RAG by adding verification layers and storing verified outputs for future retrieval, creating a growing knowledge base rather than relying solely on static external corpora.

Johnson et al. [36] developed FAISS in 2019 for similarity search over billions of items, enabling efficient memory-augmented systems. The library achieves an 8.5× speedup over prior GPU methods through optimised k-selection, and supports index types such as an inverted file with product quantisation, which searches in sub-linear time. For CAEM, FAISS provides fast similarity search over the stored episodes, of which there were 12 175 at the end of the realised trajectory.

Reimers and Gurevych [27] introduced Sentence-BERT (SBERT) at EMNLP 2019. SBERT produces semantically meaningful sentence embeddings through siamese-network fine-tuning. The speed improvement over standard BERT is substantial: finding the most similar sentence pair in a group of ten thousand sentences takes about sixty-five hours with standard BERT, but only about five seconds with SBERT, because one can compare the precomputed embeddings directly instead of running a heavy cross-encoder over every pair. This efficiency gain is what makes CAEM's memory search possible: each query is turned into a 768-dimensional embedding, and the system can look through the entire memory index in real time.

More recent work focuses on writeable episodic stores and on insertion-time memory augmentation, both of which are direct architectural neighbours of CAEM's memory module. Das et al. [70] introduced Larimar at ICML 2024, an associative key-value episodic memory with one-shot edits, selective forgetting, and an order-of-magnitude faster knowledge-editing path than direct fine-tuning; Larimar establishes that a writeable episodic store is a feasible alternative to parametric storage but does not associate a quality scalar with each entry. Xu et al. [95] introduced A-MEM, a Zettelkasten-inspired agentic memory that writes structured textual notes at insertion and evolves links between memories over time; A-MEM is the closest architectural neighbour to CAEM in the sense that both write structured metadata at insertion,

but A-MEM's metadata is textual (tags, links) while CAEM's is a single calibrated scalar storage score that drives a tiered router rather than evolving link structure. Adaptive-RAG [75], reviewed in Section 2.2.5, shares CAEM's count of three tiers but routes on query complexity at inference time rather than on the confidence stored with each memory entry, which is the distinction CAEM's router draws (Section 4.2.4).

These developments establish the feasibility and benefit of external knowledge storage. NTMs show that differentiable memory supports structured reasoning; Memory Networks scale to millions of facts with multi-hop reasoning; RAG shows that non-parametric knowledge improves factual accuracy and can be updated; FAISS and Sentence-BERT provide the infrastructure for practical deployment. CAEM builds on these advances in three ways. First, a query is routed by a combined score $\mathcal{S} = \lambda\, s + (1-\lambda)\, u_{\text{stored}}$ of its similarity s to the nearest stored entry and that entry's stored confidence, so a close match stored with low confidence is not served from memory. Second, an independent safety override sends any query whose pre-routing confidence u_{pre} falls below a fixed floor to the retrieval-augmented default, however close its memory match, after which the retrieval-utility classifier makes the final choice between retrieval and zero-shot generation. Third, the memory is filled only through the four-outcome storage decision (Store, Defer, Abstain, Discard), with a bounded buffer for borderline answers, so the knowledge base grows only from answers that clear the storage threshold. These extensions distinguish CAEM from static-corpus RAG.

2.2.5 Selective and Adaptive Retrieval

The retrieval-augmented architectures of the previous subsection retrieve unconditionally on every query: every incoming question pays the retrieval cost, and the retrieved passages enter the generator's context whether the query benefits from external evidence or not. A complementary line of work argues that retrieval is not uniformly helpful and proposes architectures that decide whether to retrieve on a per-query basis. The selective-retrieval literature anchors the architectural design of Section 4.2.5, which adds a binary retrieval-utility classifier between CAEM's three-tier router and its retrieval-augmented fallback so that queries on which retrieval would amplify the question's misconception framing route to the zero-shot tier instead.

Mallen et al. [55] established the empirical anchor for the selective-retrieval position at ACL 2023. They introduced PopQA, a fourteen-thousand-question benchmark of entity-centric factoid queries, and showed that large language models handle popular entities through their parametric memory more reliably than through retrieval, while the same models fail systematically on long-tail entities that retrieval can rescue. The architectural lesson is that retrieval utility is a function of the subject entity's popularity rather than a global hyperparameter, and the corresponding routing rule is a per-relation popularity threshold derived from Wikipedia pageview ranks. With GPT-3 davinci-003, the adaptive retrieval rule reaches 46.5% accuracy on PopQA, 5.3 percentage points above the best non-adaptive method. The lesson for CAEM is that retrieval utility can be predicted in part from cheap question-side features such as entity popularity, which CAEM's retrieval-utility classifier includes among

its inputs (Section 4.2.5).

Wang et al. [63] extended this line to a learned classifier at EMNLP Findings 2023. Their self-knowledge routing system collects training labels by running each candidate question through both a no-retrieval pipeline and a retrieval-augmented pipeline, recording the per-pipeline exact-match score, and labelling the question as *retrieval-helpful* when the retrieval-augmented exact match exceeds the no-retrieval exact match and as *retrieval-hurt* otherwise. A classifier is then built on the labelled set (the paper compares direct prompting, in-context learning, a BERT-base classifier and a k-nearest-neighbour rule, of which the last performs best), and at inference the classifier dispatches each query to retrieval or to the zero-shot generator according to its prediction. The training-label rule (compare exact match across the two routings) is the direct precedent for the empirical-delta labelling rule that CAEM's retrieval-utility classifier inherits in Section 4.2.5; the architectural difference is that CAEM's classifier sits downstream of the existing tier-1 and tier-2 dispatch outcomes and gates only the retrieval-augmented fallback, while Wang's classifier replaces the dispatch entirely.

Jeong et al. [75] introduced Adaptive-RAG at NAACL 2024 as a three-class extension of the binary routing decision. Their classifier (a T5-large language model fine-tuned on automatically-collected complexity labels) dispatches each query to one of three classes (no retrieval, single-step retrieval, multi-step retrieval), trading routing accuracy for the cost of running a large model at routing time. The reported cost-accuracy frontier on a six-dataset open-domain question-answering suite (SQuAD, Natural Questions, TriviaQA, HotpotQA, 2WikiMultiHopQA, MuSiQue) beats single-strategy baselines and the unconditional retrieval-augmented baseline; for CAEM's purposes the T5-large routing cost is the design's main weakness, while the authors themselves single out the quality of the automatically assigned complexity labels. The architectural lesson for CAEM is that a learned classifier on automatically-collected outcome labels can decide the routing question, but that putting a large model at routing time costs more than the architecture gains; the follow-up of Marina et al. [90] extends Adaptive-RAG to the regime where the classifier carries no language-model forward pass and matches the routing accuracy of the language-model-based variant on the same suite. CAEM's retrieval-utility classifier sits at the lightweight end of this design space: the classifier itself is a logistic regression head, and its features reuse quantities the pipeline already computes, including a self-knowledge probe read from one forward pass of the base generator.

Asai et al. [64] introduced Self-RAG at ICLR 2024 as the end-to-end fine-tuned alternative to the external-classifier approach. Self-RAG trains a single language model to emit four kinds of reflection tokens during generation (a `Retrieve` token that triggers retrieval, an `IsREL` token that judges passage relevance, an `IsSUP` token that judges support, and an `IsUSE` token that judges usefulness), so the routing decision is made by the same model that generates the answer. The deployed signal for retrieval routing is the probability the model assigns to the `Retrieve` token at decoding time; removing retrieval at inference time drops PopQA accuracy by roughly 45 percent relative in the authors' ablation, the empirical receipt for the joint design's load-bearing role. The architectural cost is that the reflection tokens are tied to the LLaMA-family backbone Self-RAG fine-tunes on, and retargeting them to a different backbone changes enough of the system to invalidate the architectural comparison;

this is the same constraint that prevents Self-RAG from entering the external baseline panel of Chapter 5 as a numerical baseline rather than a citation-level reference.

Baek et al. [86] introduced Probing-RAG at NAACL Findings 2025 as a midpoint between the heavy classifier and the in-generator reflection tokens. They train a linear probe on the intermediate hidden states of the generator (rather than on its output tokens), with the probe trained under a balanced positive-negative protocol that explicitly enforces a fifty-fifty class ratio. The probe is trained to predict, from the hidden states, whether the model's current answer is correct and hence whether additional retrieval is needed; the reported accuracy across five open-domain question-answering datasets beats single-step retrieval, FLARE, DRAGIN, and Adaptive-RAG on the same suite while retrieving roughly half as often. The architectural lesson for CAEM is the most directly relevant of the five: the closest cousin of CAEM's retrieval-utility classifier on the feature axis is the hidden-state-as-routing-signal pattern that Probing-RAG establishes, with the methodological difference that Probing-RAG probes intermediate layers, whereas CAEM reads the base generator's last-token hidden state, without the adapter, as a single self-knowledge feature alongside thirty-five other question-side and passage-side features.

CAEM's retrieval-utility classifier reaches a pooled out-of-fold AUROC of 0.850 under nested cross-validation on 3 208 labelled rows (Section 4.2.5). The systems reviewed above are evaluated on different question suites and with different metrics, so this figure is not directly comparable with theirs.

2.2.6 Self-Improvement and Continual Learning

Self-improvement mechanisms enable models to learn from interaction and feedback, while continual learning addresses the challenge of acquiring new knowledge without forgetting previous capabilities. This subsection reviews methods for iterative refinement and catastrophic forgetting mitigation that inform CAEM's self-improvement loop.

Kirkpatrick et al. [20] introduced Elastic Weight Consolidation (EWC), defined in the Preliminaries, in PNAS in 2017. Its Fisher-weighted penalty lets parameters that matter little for earlier tasks adapt freely while protecting those that matter. On ten Atari games learned in sequence, EWC let a single agent learn many games without catastrophic forgetting, whereas plain stochastic gradient descent forgot earlier games, although EWC did not reach the score of ten separately trained agents. For CAEM, EWC supplies the motivation for bounding drift between cycles; as noted in the Preliminaries, CAEM achieves that bound with a low-rank adapter on a frozen backbone, a strict training-pool filter and a retention guard rather than with an explicit penalty. The low-rank choice is reviewed next.

The choice to fine-tune through a low-rank adapter rather than the full backbone is a separate and equally load-bearing decision in the self-improvement loop. Hu et al. [37] introduced low-rank adaptation as a parameter-efficient alternative to full fine-tuning that injects trainable low-rank matrices into the weight matrices of a frozen backbone; the original paper adapts only the attention projections and reports quality on par with full fine-tuning at less than 1 percent of the trainable parameter count on GPT-3 175B, and later work extends the adapters to the feed-forward projections. The

choice between low-rank adaptation and full fine-tuning has empirical consequences for continual learning that the recent literature has measured directly. Biderman et al. [65] compare low-rank adaptation with full fine-tuning for Llama-2-7B on code and mathematics, sweeping ranks from 16 to 256, and report two findings relevant to CAEM's setting: low-rank adaptation learns less of the target domain than full fine-tuning, but it forgets less of the base model's general capability, measured as average HellaSwag, ARC-Challenge, and WinoGrande accuracy. In code instruction tuning on Magicoder data, for example, full fine-tuning falls to 0.414 against 0.509 for low-rank adaptation at rank sixty-four. Forgetting grows with training duration in both cases, and in mathematics low-rank adaptation at rank 256 forgets nearly as much as full fine-tuning, so higher rank trades retention for fit. The QLoRA work of Dettmers et al. [47] establishes the second decisive practical point: applying low-rank adaptation to all linear transformer-block layers, both attention and feed-forward, rather than to the attention projections alone is the most critical configuration choice for matching full fine-tuning, whereas the rank itself has little effect once all layers are adapted. CAEM's adapter target set spans every linear projection inside the transformer block (the four attention projections together with the three feed-forward projections of the gated multi-layer perceptron), which is the all-linear-layer target set that this literature converges on, fitted at rank thirty-two with $\alpha = 2r$; with this choice the effective scale α/r stays at two, so the rank-driven gradient collapse that motivates the rank-stabilised alternative for fixed-α adapters does not arise, and rank thirty-two is far below the large ranks at which the theory of Kalajdzievski [52] predicts instability for a constant scale. Low-rank adaptation is also what lets the self-improvement loop fit on a single consumer-grade graphics processor; full fine-tuning at the same backbone size would not fit. The within-adapter analogue of catastrophic forgetting under sequential fine-tuning, documented by Wang et al. [61] and orthogonal-decomposition fixes for it, is registered as future work; the present thesis combines the bounded low-rank-adapter parameter envelope on a frozen backbone, the multi-modal retention guard with adapter rollback, and the strict training threshold above the storage cut-point as the three-mechanism defence against iterative-adapter drift across the realised cycle trajectory.

Natarajan et al. [10] provided theoretical foundations for learning with noisy labels at NeurIPS 2013. Their unbiased estimator method constructs proxy loss functions recovering optimal classifiers despite label corruption. The key insight is that if the noise process is known or can be estimated (e.g., what fraction of positive labels are actually negative), modified loss functions achieve consistent parameter estimation. On a synthetic non-separable dataset their methods achieve more than 88% accuracy even when 40% of the labels are corrupted, and they are competitive with other methods for learning under label noise.

For CAEM, this framework explains why training on an imperfect pool can still help: no verifier is perfect, so some stored episodes carry wrong answers. Natarajan et al.'s guarantees, however, assume a loss corrected for the noise rates, which CAEM does not apply (Proposition 4.2). CAEM instead lowers the noise rate directly, by admitting to the training pool only episodes above a stricter threshold $\tau_{\text{train}} > \tau_{\text{store}}$, so that the training pool is purer than the memory as a whole. The realised purity of the training pool is reported in Chapter 5.

When Zelikman et al. [43] introduced STaR (Self Taught Reasoner) at NeurIPS 2022,

they showed successful bootstrapping using iterative self improvement. STaR loop generates rationales for problems, filters for correct answers, fine tunes on correct rationale-answer pairs and then repeats. A key innovation is rationalisation. For problems the model fails to answer correctly, it is given the correct answer as a hint and generates a rationale that leads to it. This data augmentation adds training signal from problems the model could not yet solve, which the basic loop discards.

On CommonsenseQA, STaR improved over a few-shot baseline by 35.9 percentage points (72.5% against 36.6%), showing that bootstrapping from a model's own outputs works when coupled with appropriate filtering. The parallel to CAEM is direct: generate predictions, verify them, fine-tune on verified examples, and repeat. CAEM extends STaR in two ways. First, simple correctness filtering is replaced by the nine-signal verifier, whose calibrated composite u_{stored} drives the four-outcome storage decision. Second, verified items persist in episodic memory for future retrieval rather than being consumed only as training data.

Recent work on intrinsic self-correction reveals important limitations. Huang et al. [74] found at ICLR 2024 that prompting language models to review and correct their own reasoning does not improve accuracy without external feedback, and at times degrades it. Kamoi et al. [76] analysed at TACL 2024 when language models can correct their mistakes and found that intrinsic self-correction often fails. The bottleneck is reliable feedback: a model that produces an incorrect first answer usually cannot generate a reliable correction signal without external validation.

This limitation motivates CAEM's design. Instead of asking the model to review itself, CAEM uses external verification: an entailment judge against retrieved evidence, agreement across resampled chains, and recalibrated internal signals. These supply the external feedback that the self-correction literature identifies as a precondition for useful self-improvement, and because their failure modes differ, a calibrated combination of them is expected to be more reliable than any one of them or than the model's own self-evaluation. Chapter 5 reports how well the verifier separates correct from incorrect answers.

The self-improvement literature also reports successes: when external feedback is available, for example code execution results, formal verification or retrieved sources, models can use it to correct themselves. CAEM's retrieval-augmented tier supplies feedback of this kind, since the retrieved evidence both grounds the generation and gives the verifier something to check the answer against.

Domain adaptation research provides additional relevant findings. Gururangan et al. [30] demonstrated at ACL 2020 that continued pre-training on domain-specific unlabelled data (domain-adaptive pre-training) followed by task-specific fine-tuning outperforms direct fine-tuning on limited labelled data. While CAEM does not perform domain-adaptive pre-training due to computational constraints, the finding supports iterative refinement: progressive training on accumulated verified examples enables gradual domain adaptation.

The distinction between CAEM's approach and standard continual learning deserves emphasis. Traditional continual learning addresses learning sequential tasks A, B, C without forgetting previous tasks. CAEM's scenario differs: the task remains constant (question answering), but the model is meant to improve at that task

through accumulated verified examples. This setting is closer to curriculum learning and iterative refinement than task-sequential continual learning, though catastrophic forgetting concerns still apply since fine-tuning on domain-specific verified examples risks degrading general capabilities.

Several 2024–2025 contributions tighten the theoretical and architectural framing of self-improvement in ways directly relevant to CAEM's theoretical analysis and to its retention guard. Hosseini et al. [73] introduced V-STaR at COLM 2024, training a DPO verifier over correct and incorrect self-generated solutions and using it to rank test-time candidates; CAEM generalises V-STaR by using the verifier for training-data admission rather than only for test-time ranking, and by adding the episodic memory and the retention guard. Song et al. [93] formalise the *generation–verification gap*, the improvement obtained by reweighting a model's generations with its own verification, and show empirically that self-improvement requires a positive gap, that small models fail to achieve one, and that iterative self-improvement saturates within two or three rounds; this is the conceptual motivation for CAEM's $\alpha > 1/2$ verifier-purity precondition in Section 4.3. Yi et al. [96] prove theoretically, in linear regression, that a verifier filter over synthetic data prevents model collapse in self-consuming training loops and yields near-term improvement, while in the long run the model converges to the verifier's knowledge centre, so that gains plateau or reverse unless the verifier is unbiased; this is the strongest published safety anchor for CAEM's storage-gate-filtered SIL, and also its main caution. Das et al. [87] prove at ICML 2025 that retraining a model on its own predicted hard labels increases population accuracy in a binary linearly-separable classification setting with randomly flipped labels, and show empirically that retraining only on samples where the prediction agrees with the given label helps further; this is the closest published analogue of CAEM's pool-purity inequality (Theorem 4.1), and CAEM extends the idea from binary classification with self-predicted labels to open-ended generation with an external multi-signal verifier. Huang, Block, Foster et al. [88] formalise self-improvement as a sharpening mechanism at ICLR 2025 and prove convergence-style amortisation-via-self-training bounds that are the closest published analogue of CAEM's bounded-band stationarity heuristic on the committed-cycle subsequence (Heuristic 4.6). Lang, Sontag, and Vijayaraghavan [77] provide a complementary expansion-based bound at NeurIPS 2024 for when a strong student trained on weak-teacher pseudolabels outperforms the teacher; their bound requires expansion assumptions rather than a verifier-accuracy threshold and is therefore complementary to, rather than redundant with, CAEM's data-purity formulation.

These methods inform CAEM's self-improvement loop as follows. EWC motivates bounding drift between cycles, which CAEM achieves with a low-rank adapter on a frozen backbone, an approach the parameter-efficient fine-tuning literature shows forgets less than full fine-tuning. Noisy-label learning shows that learning from imperfect labels remains possible when the noise is bounded, which motivates the stricter training threshold. STaR supplies the generate, filter, fine-tune and repeat cycle. The self-correction literature motivates external verification rather than self-review. Finally, the retention guard, which rolls the adapter back but never the memory, turns forgetting into a detected and rejected event rather than a silent degradation. The trajectory actually achieved is reported in Chapter 5.

Table 2.1 positions CAEM against the closest published systems on the architectural

axes that distinguish them: whether the system stores a per-entry confidence at insertion, what signal the router uses, what filter the training-data admission applies, what regulariser preserves general capability under self-improvement, and whether a formal precision or convergence guarantee is supplied. No single prior system shares CAEM on every axis simultaneously; the table makes the differentiation explicit.

System	Stored conf.	Routing signal	Training filter	CL guard	Formal guarantee
CAEM (this work)	yes (calibrated u_{stored})	combined sim + u_{stored}, u_{pre} override	9-signal cal-prob composite + fixed-threshold gate (per-bench shrinkage; LoRA SIL)	bounded LoRA adapter envelope + multi-modal retention rollback	calibration-consistent precision floor; purification condition; trajectory heuristics
A-MEM [95]	no (textual tags)	similarity + link evolution	–	–	–
Larimar [70]	no (key-value addr.)	key match	one-shot edit	–	–
Adaptive-RAG [75]	no	query complexity classifier	–	–	–
V-STaR [73]	no	–	DPO verifier (test-time rank)	–	–
Mohri-Hashimoto [78]	no	–	split-conformal sub-claim filter	–	conformal factuality
MiniCheck [82]	no	–	NLI judge (fact verification)	–	–

Table 2.1: Positioning of CAEM against the closest prior systems. Insertion-time stored confidence, combined-score routing on a shared memory, the calibrated-probability composite at a fixed-threshold gate, the LoRA self-improvement primitive with an adapter-rollback guard against a multi-modal retention probe panel, and the calibration-consistent precision floor with its purification condition together constitute CAEM's distinguishing combination.

2.2.7 External Baselines and Competitive Positioning

The related work reviewed above clusters into three families relevant to CAEM's evaluation: prompt-level reasoning methods, retrieval-augmented generation methods, and training-time self-improvement methods, to which a confidence-calibration method is added. Chapter 5 runs representatives of each as external baselines, so that every mechanism CAEM adds to a plain language model is compared with at least one prior system that uses that mechanism alone. This subsection states what each baseline is and what it isolates; the numerical results appear in Chapter 5.

The prompt-level family contributes three baselines. The first, **zero-shot Qwen-2.5-3B-Instruct**, uses the same decoder-only backbone CAEM fine-tunes, with no CoT prefix, no retrieval, and no verification. This is the empirical floor: any gain CAEM reports must exceed the zero-shot score on the same backbone, otherwise the gain is attributable to model capacity rather than to the CAEM mechanism. The second,

Chain-of-Thought prompting [41], adds the standard zero-shot "Let's think step by step" trigger [39] without any further modification. CoT isolates the contribution of intermediate reasoning on the same backbone and controls for the possibility that CAEM's gains reflect better rationale generation rather than the verifier-memory architecture. The third, **Five-shot Chain-of-Thought**, prepends five in-context (question, reasoning, answer) demonstrations from a benchmark training split before the CoT trigger and is the strongest prompting-only comparison against CAEM's retrieval-free Tier-2.

The retrieval family contributes three baselines. **DPR-RAG** [32], [33] retrieves the top $k = 5$ passages from the same Wikipedia index that CAEM's retrieval tier uses (passages of about 100 words, split in the style of DPR) and generates from the same Qwen-2.5-3B-Instruct backbone through CAEM's retrieval prompt; this isolates the contribution of external grounding without any verification or memory. **CoT+DPR-RAG** combines the CoT trigger with DPR retrieval and isolates the contribution of reasoning over retrieved evidence. **FLARE** [51] extends DPR-RAG with active retrieval: rather than retrieving once at the start of generation, FLARE generates one sentence at a time and triggers a retrieval whenever the token-level confidence on the upcoming sentence falls below a registered threshold, with retrieved passages prepended to the generation context for the next sentence; this isolates the contribution of selective and step-conditioned retrieval over single-pass retrieval. A fourth retrieval system, **Self-RAG** [64], is acknowledged as the most ambitious retrieve-and-critique pipeline in the literature but is included only as a citation-level reference rather than as a replicated numerical baseline, because Self-RAG's critic tokens are tied to a LLaMA-family backbone and retargeting them to Qwen-2.5-3B-Instruct would change enough of the system to invalidate the architectural comparison.

The training-time family contributes one baseline. **Vanilla fine-tuning** fine-tunes all weights of Qwen-2.5-3B-Instruct on the gold answers to the same training questions CAEM saw, for the same number of cycles, with no verifier filter, no retention guard and no episodic memory. It isolates the contribution of CAEM's curation machinery against the simplest alternative, supervised training on labelled answers, and it is the only baseline that uses a gold label for every training question. The confidence-calibration family contributes one baseline. **Semantic entropy** [71] samples several generations at a raised temperature, clusters them into groups of equivalent meaning, and uses the entropy of the cluster distribution as the per-query uncertainty; the baseline runs the same backbone as the zero-shot system and abstains when the entropy is high, which makes it the most direct external comparator to CAEM's calibrated composite. **STaR** [43], discussed above as the closest prior method in spirit, is not run: its rationalisation step and binary correctness filter differ enough from CAEM's verifier and four-outcome decision that a comparison would mix architectural contribution with rationalisation quality.

Taken together, the eight external baselines (B1 to B8 in Section 5.4.1) span the prior systems that share at least one mechanism with CAEM: three isolate the prompt-level component (B1 zero-shot, B2 CoT, B5 five-shot CoT), three the retrieval component (B3 DPR-RAG, B4 CoT+DPR-RAG, B7 FLARE), one the training-time component (B6 vanilla fine-tuning), and one a sampling-based uncertainty estimate on the same backbone (B8 semantic entropy). Neither Self-RAG nor STaR is run, so neither

enters the significance tests. No single baseline combines all four families; CAEM's contribution is their combination, which the comparison in Chapter 5 tests against each family in isolation.

2.2.8 Evaluation Benchmarks

Rigorous evaluation requires benchmarks testing diverse reasoning capabilities with objective correctness criteria. This subsection reviews the five primary factual-QA benchmarks that constitute CAEM's locked evaluation panel (FEVER, TriviaQA, CommonsenseQA training, and TruthfulQA, StrategyQA transfer) and the MMLU general-knowledge probe that sits inside the multi-modal retention guard, explaining how each tests a different aspect of hallucination reduction. Natural Questions is also described for completeness: it and HotpotQA were evaluated at cycle zero and dropped before the main run because the base generator answered too few of their questions correctly for the storage gate to build a usable training pool (Section 1.6.1).

Thorne et al. [22] created the FEVER dataset at NAACL 2018 for fact verification, providing 185,445 claims generated from Wikipedia requiring classification as SUPPORTS, REFUTES, or NOT ENOUGH INFO relative to Wikipedia evidence. Claims are designed to test factual knowledge; an illustrative pair (not taken from the dataset) is "Barack Obama was the 44th President of the United States" (SUPPORTS) and "The Beatles formed in Manchester" (REFUTES, since they formed in Liverpool). The NOT ENOUGH INFO category is particularly valuable, testing whether systems recognise insufficient evidence rather than hallucinating conclusions. FEVER directly evaluates the external-grounding family of CAEM's verifier, because the task structure matches the premise-hypothesis entailment check that produces p_{ground} and p_{entail}.

Joshi et al. [19] introduced TriviaQA at ACL 2017, providing about 95,000 trivia question-answer pairs authored by trivia enthusiasts, which with on average six supporting documents each from the web and Wikipedia give over 650,000 question-answer-evidence triples. Questions span an unusually wide topical range (history, science, geography, entertainment) and the provided evidence is noisy (only a subset of the retrieved documents actually answers the question), making TriviaQA a realistic test of retrieval-augmented generation under distractor documents. For CAEM, TriviaQA tests whether the Tier-3 retrieval-augmented path can ground answers against noisy evidence and whether the mean-match grounding signal $p_{\text{ground_mean}}$ remains calibrated when retrieval quality varies across queries.

Kwiatkowski et al. [24] introduced Natural Questions (NQ) at TACL 2019, providing real anonymised queries to the Google search engine paired with Wikipedia-page long answers and span-level short answers. Unlike synthetic benchmarks, NQ captures the distribution of questions users actually ask, with the accompanying variability in specificity, ambiguity, and presupposition. NQ is the dominant open-domain QA benchmark in recent retrieval-augmented-generation literature [32], [33]. CAEM evaluated NQ at cycle zero and dropped it before the main run: the base generator answered only 17 percent of its questions correctly, too few for the storage gate to build a usable training pool. Its training split is reused only to train the retrieval-utility classifier.

Lin et al. [40] designed TruthfulQA specifically to measure how models mimic human falsehoods, providing 817 questions across 38 categories in health, law, finance, politics, and conspiracies. Questions are adversarially constructed to elicit common misconceptions: "What happens if you crack your knuckles?" where the false but common answer is "you get arthritis" and the true answer is "nothing harmful happens; no proven link to arthritis." The inverse scaling finding (larger models often less truthful) makes TruthfulQA the most challenging benchmark for parametric scaling approaches. Human performance reaches 94% truthful and 87% truthful-and-informative, while the best GPT-3 175B configuration achieves 58% and 21% respectively. For CAEM, TruthfulQA was registered as the benchmark where the sample-set family of signals was expected to pay off, on the reasoning that a misconception would make the model alternate between the correct answer and the misconception across stochastic decodes, which the sample-set agreement signals s_{avg} and p_{entail} would detect. The literature predicts otherwise: semantic entropy targets arbitrary, sample-to-sample errors and explicitly does not cover models that are consistently wrong because of common misconceptions, as Farquhar et al. [71] state, since such a model gives the same wrong answer on every decode and agreement-based signals then read it as confident. The realised results agree with the literature rather than with the registered expectation: TruthfulQA answers the gate stored were correct only 31 to 38 percent of the time at cycles one to five, against 72 to 75 percent on the training panel (Table C.7), and the semantic-entropy baseline is among the highest-scoring baselines on TruthfulQA exact match rather than being penalised for heavy abstention (Chapter 5). TruthfulQA is therefore the benchmark that exposes the blind spot of agreement-based verification.

Geva et al. [34] introduced StrategyQA at TACL 2021, presenting 2,780 questions requiring implicit reasoning where decomposition steps are not mentioned in questions. For example, the dataset question "Could a llama birth twice during the War in Vietnam?" requires knowing Vietnam War duration (implicit: 1955–1975) and llama gestation period (implicit: approximately 11 months), then computing whether two births fit the timeframe. Human accuracy reaches 87% while best models achieve approximately 66%. The implicit reasoning requirement tests whether models genuinely understand underlying concepts or merely perform shallow pattern matching. For CAEM, StrategyQA evaluates the sample-set agreement signal s_{avg} because implicit reasoning should converge across multiple chains if the underlying world-knowledge is consistent; divergence across chains is a strong hallucination signal specifically on this benchmark.

Hendrycks et al. [35] introduced MMLU in 2021, a 57-subject multiple-choice benchmark covering humanities, STEM, social sciences, and professional domains. MMLU is used in CAEM as one of three components of the *multi-modal retention probe panel*, not as a target benchmark. The fine-tune is not optimised for MMLU; instead, MMLU is probed before the first cycle (the pristine baseline) and after every cycle's fine-tune, alongside two further probes drawn from held-out test slices of the training panel: an open-text factoid probe on the held-out TriviaQA test fold, and a commonsense multiple-choice probe on the held-out CommonsenseQA test fold. The cycle aborts and rolls the adapter back to the previous snapshot whenever the worst per-probe ratio against the pristine baseline falls below the fixed floor $\rho_{\min} = 0.93$; the worst-probe rule replaces an earlier single-probe rule because a general-knowledge

multiple-choice probe alone missed the failure mode where MCQ accuracy stayed high while open-text generation degraded silently. The episodic memory is preserved across rollbacks. The choice of MMLU as one of the three probes is deliberate: catastrophic forgetting manifests most visibly as erosion of capability on tasks the model was not trained on this cycle, so a broad out-of-distribution probe is a more sensitive trigger than in-distribution QA, and the held-out training-bench test folds add the asymmetric-drift sensitivity that the broad probe alone would miss.

The benchmarks are complementary. FEVER tests entailment-based grounding, including the NOT ENOUGH INFO case where the right behaviour is to recognise missing evidence. TriviaQA tests retrieval-augmented generation under noisy evidence. CommonsenseQA tests commonsense reasoning over short multiple-choice options. TruthfulQA tests agreement-based and relevance signals on imitative falsehoods, as a transfer-only benchmark held out from the SIL training pool. StrategyQA tests sample-set agreement on implicit multi-hop reasoning, also as a transfer-only benchmark. MMLU, the open-text TriviaQA test fold and the multiple-choice CommonsenseQA test fold close the catastrophic-forgetting loop through the retention probe panel. Together they cover a range of factual reasoning tasks, so that results on the panel show whether CAEM's effects carry across reasoning types rather than fitting one task.

2.3 Summary of Key Findings

The literature reviewed in this chapter supports five findings that shape CAEM.

First, hallucination is persistent and is not solved by scale. Rates vary widely by domain, up to 91.4% of chatbot-generated references in one systematic-review study [66]; on TruthfulQA the best GPT-3 configuration was truthful and informative on only 21% of questions, with larger models often less truthful [40]; and FActScore finds only 58% of ChatGPT's atomic facts supported [58]. This motivates changing the structure around the model rather than relying on scale.

Second, no single verification or confidence signal is reliable across tasks. Verbalised confidence discriminates weakly [84], modern networks are systematically overconfident [18], preference-tuned models are miscalibrated until rescaled [38], [60], and uncertainty methods that do well on open generation do poorly on constrained outputs and vice versa [94]. Combining calibrated signals has published precedent in the logistic multi-scoring of Valentin et al. [83], and generic NLI models transfer poorly to language-model hallucinations [92], which favours a judge trained on claim-support data [82]. CAEM therefore combines nine recalibrated signals into one composite.

Third, external memory and retrieval improve factual accuracy and can be updated without retraining [16], [33], but standard retrieval-augmented generation neither verifies what it retrieves nor accumulates verified answers, and retrieval does not help every query [55], [63]. CAEM therefore stores only verified answers, routes on similarity combined with stored confidence, and decides per query whether retrieval is worth using.

Fourth, self-improvement works when it receives reliable external feedback and fails when a model only reviews itself [43], [74], [76]; verifier-filtered retraining can help

but saturates within a few rounds [93], [96]. CAEM therefore filters its training data through an external verifier, with a stricter threshold than its memory uses.

Fifth, fine-tuning causes forgetting, which low-rank adaptation reduces but does not remove [37], [65], and weight-regularisation penalties add little once the backbone is pre-trained [42], [57]. CAEM therefore relies on a low-rank adapter on a frozen backbone and on a retention guard that rolls back the adapter, but never the memory, when held-out accuracy falls.

The five-benchmark panel and the three-probe retention panel exercise these mechanisms on different failure modes, and the baselines of Section 2.2.7 each isolate one family of prior methods. No prior system combines all of these elements (Table 2.1), which is the gap CAEM addresses.

2.4 Chapter signpost

The preliminaries section and the literature review have established the technical foundations on which the architecture rests and have located CAEM's joint intervention against the verification, retrieval, memory-augmented, and continual-learning prior work whose individual contributions do not close the deployment-safety gap registered in Chapter 1. Chapter 3 translates the architectural opportunity surfaced by the literature into a registered specification: seven functional requirements paired with named architectural components, five non-functional requirements with empirical anchors, six production-parity design commitments, and the societal, environmental, ethical, standards, project-management, risk, and economic readings that bound the engineering envelope. The specification is then discharged end-to-end by the methodology chapter that follows.

Chapter 3

Requirements, Impact, and Project Management

3.1 Final Specifications and Requirements

3.1.1 Functional requirements

The architecture must perform seven functions at deployment, each derived from the problem statement in Chapter 1; Table 3.1 lists them with the component that delivers each. Three of them need a word of explanation. Abstention (FR2) means that, when the four-outcome decision is to abstain, the system returns an explicit insufficient-evidence message instead of a hedged or speculative answer. Recall (FR4) answers from memory only when a stored episode is close enough to the new question; otherwise the router sends the query to zero-shot generation or to retrieval-augmented generation, and the retrieval-utility classifier decides between those two for the queries that the router cannot settle on confidence alone. Rendering (FR7) shows the user the calibrated probability of correctness as a percentage, together with one of four reliability tags, on every answer; under abstention the answer text is hidden but the probability is still shown.

3.1.2 Non-functional requirements

Five non-functional requirements bound the operating envelope of the architecture (Table 3.2).

Latency: The cost of a query depends on its tier: the memory-direct tier costs an embedding lookup and a similarity search, the zero-shot tier adds one decode, and the retrieval-augmented tier adds passage retrieval and a grounded decode. Per-tier latencies are reported in Chapter 5.

Storage precision: Among the answers that the gate stores, the fraction that are correct should be high. The requirement is stated on the calibration fold at the locked cut-point $\tau_{\text{store}} = 0.60$, where the storage precision π_{store} is

$$\Pr\Big(\text{EM} = 1 \,\Big|\, u_{\text{stored}} \geq \tau_{\text{store}}\Big)_{\text{cal}} \;=\; \pi_{\text{store}}. \tag{3.1}$$

ID	Requirement	Architectural component (specified in Chapter 4)
FR1	Calibrated answer per query	Nine-signal verifier feeding the per-benchmark calibrated probability composite (with shrinkage prior)
FR2	Explicit abstention under uncertainty	Fixed-threshold storage gate + confabulation early-exit rule
FR3	Commit verified episodes to memory as one record (question, answer, supporting context, storage score)	Four-outcome decision tree (store branch)
FR4	Recall on subsequent queries	Three-tier router with combined-score dispatch and safety override; the retrieval-utility classifier re-decides, at threshold $\tau_{\text{RUC}} = 0.375$, the queries that reach the safety override or fall through the score formula
FR5	Cycle-boundary fine-tune on verified episodes only	Self-improvement loop with low-rank adapter, pool-reweighting layer, and strict training threshold
FR6	Retention guard with adapter rollback that keeps the memory	Multi-modal retention probe panel (worst-probe rule) and asymmetric memory-versus-adapter rollback
FR7	Calibrated probability rendered to the user with every answer	Production response envelope: four-class decision tag + continuous u_{stored} percentage; insufficient-evidence message replaces the answer text under abstain

Table 3.1: Functional requirements and the architectural components that deliver them.

The cut-point was chosen on the cycle-zero calibration fold before the main run, and the realised precision at every cycle is reported in Chapter 5.

Retention: After every fine-tune, accuracy on each probe of the multi-modal probe panel must stay at or above 93 percent of its value before any fine-tuning; otherwise the cycle is aborted and the adapter is rolled back:

$$\min_{p \in \mathcal{P}} \rho_p(M_t) \;:=\; \min_{p \in \mathcal{P}} \frac{\mathrm{EM}(M_t,\, p)}{\mathrm{EM}(M_0,\, p)} \;\geq\; \rho_{\min} = 0.93, \tag{3.2}$$

where M_t is the generator after the fine-tune at cycle t and $\mathcal{P}$ is the probe panel: a general-knowledge multiple-choice probe (MMLU), an open-text factoid probe drawn from the TriviaQA test fold, and a commonsense multiple-choice probe drawn from the CommonsenseQA test fold.

Reproducibility: Every released artefact, including the locked cut-points, the calibrated composite, the seed-pool composition and the per-cycle snapshots, must be regenerable from one recorded random seed and a published code commit.

Integrity: No sample in the evaluation fold may appear in the calibration fold, the seed pool or the per-cycle stream chunks, so that calibration-fold and evaluation-fold results are measured on disjoint samples.

ID	Requirement	Target	Where it is checked
NFR1	Per-query latency	Tier-dependent: embedding lookup (Tier 1), single decode (Tier 2), retrieval + grounded decode (Tier 3)	Per-tier latency in Chapter 5
NFR2	Storage precision at $\tau_{\text{store}} = 0.60$	Calibration-fold precision of the stored class, with the cut-point fixed before the main run	Per-cycle precision in Chapter 5
NFR3	Retention floor (multi-modal probe panel)	≥ 0.93 of the pre-fine-tune accuracy on every probe; the worst probe decides	Per-cycle probe ratios in Chapter 5; one breach (cycle four) rolled back
NFR4	Reproducibility	Every artefact regenerable from one seed and a published commit	Locked configuration files and archived snapshots (Section A.1)
NFR5	Integrity	Evaluation, calibration, seed and stream-chunk folds disjoint	Disjointness verified per benchmark

Table 3.2: Non-functional requirements, their targets, and where each is checked.

3.1.3 Constraints and design commitments

The requirements imply six design commitments, all fixed before the main run.

- **Single backbone:** The generator backbone is the same in every cycle and every tier; only the low-rank adapter is trained, so reported gains come from memory, calibration and adapter fine-tuning rather than from a change of backbone.

- **Frozen cut-points:** The storage and deferral cut-points, $\tau_{\text{store}} = 0.60$ and $\tau_{\text{defer}} = 0.45$ on the calibrated probability scale, are fixed once before the main run and never changed, so the operating point of the storage decision does not drift. Only the calibrated composite is refitted at each cycle boundary, on the held-out calibration fold re-scored with the current adapter, with each benchmark's curve shrunk toward the pooled fit so that benchmarks with little data do not overfit.
- **Local execution:** No external model service sits in the inference path: the generator, the verifier ensemble, the embedding encoder and the passage index all run from open weights on one rented graphics processor. The language-model judge that scores TruthfulQA belongs to the evaluation, not to the system.
- **Bounded cycles:** The self-improvement loop is capped at ten cycles and may stop earlier on an automatic early-stop check (Chapter 4).
- **Matched protocol:** Every external baseline is evaluated on the same samples, prompts and scoring rules as the architecture, so that the comparison in Chapter 5 is not confounded by re-implementation choices.
- **Single seed:** All randomness derives from one recorded seed, and the per-cycle artefacts are archived so that a third party can regenerate the trajectory.

3.2 Societal Impact

3.2.1 Deployment benefits

The intended societal benefit of the architecture is less confident misinformation from deployed language models on factual question answering. On the panel of this thesis the overall hallucination rate did not fall below that of strong prompting baselines (Chapter 5); the benefit the run does show is that every answer carries a calibrated probability and a reliability tag that predicts whether it is right, and that the answer is withheld when the evidence is insufficient. This explicit refusal channel is itself a benefit, because it turns otherwise hidden uncertainty into a signal that users such as educators, journalists and information specialists can read and act on. Fluent but ungrounded answers have caused documented harm in health information: Alkaissi and McFarlane [44] report fabricated clinical references, and Chelli et al. [66] report high rates of hallucinated references in systematic-review searches. In such settings, and in others such as legal triage and educational tutoring, a system that abstains is preferable to one that fabricates. The episodic memory can also share verification cost across users: once an answer has been verified, it can be served directly when the same or a paraphrased question is asked again, so the verification done for an early user benefits later users who ask the same thing.

3.2.2 Dual-use considerations

A verified-memory system that learns from its own interactions could memorise harmful content if the verifier is misconfigured or if adversarial inputs reach its training pool. Three mechanisms limit this risk.

Retroactive re-verification: At each cycle boundary every stored episode is re-scored with the updated generator and the refitted composite; its score can go down as well as up, and entries whose score falls below 0.50 are removed, so a bad entry does not persist unchallenged.

Strict training threshold: The training pool admits only episodes whose storage score is at least 0.70, above the storage cut-point of 0.60, so marginal episodes stay out of the fine-tune even when they are admissible for retrieval.

Retention guard: If a fine-tune degrades any probe in the multi-modal panel (general-knowledge multiple choice plus held-out test slices of two training benchmarks) below the floor, the cycle is aborted and the adapter is rolled back, so one damaging cycle cannot silently carry forward.

These mechanisms reduce the risk but do not remove it; how far they reach depends on the verifier's coverage, which is reported empirically and discussed as a threat to validity in Chapter 5.

3.2.3 Marginalised-user impact

The harm caused by confident hallucination is not evenly distributed. Users who cannot independently check a confident-sounding answer, such as students using language-model tutors, patients using language-model health agents, and applicants using language-model administrative agents, are the most exposed, because they have no domain expert to fall back on. The calibrated abstention class is a partial response: by returning an explicit refusal rather than a hedged guess when confidence is low and little supporting evidence is found, the system reduces the most dangerous failure mode for exactly these users. The abstention is governed by a fixed cut-point on a calibrated probability rather than by a soft heuristic, and the abstention rate is reported in the four-outcome decision distribution in Chapter 5. The architecture does not solve unequal access to verification, but it narrows one channel through which fluency is mistaken for reliability.

3.3 Environmental Impact

3.3.1 Compute footprint of the experiment

The compute of the empirical programme falls into three phases: the cycle-zero calibration phase, the main run, and the external baseline panel. The retrieval-utility-classifier ablation needed no run of its own: it compares the main run, at the cycle-three close, with an earlier trajectory that ran without the classifier (the main run itself continued to cycle five, and all its other results cover cycles zero to five). Everything ran on a single rented graphics processor of the consumer 32-gigabyte class (an RTX 5090). The main run was the largest: according to its run log, cycles zero to five took 6.0, 29.0, 35.4, 43.2, 24.3 and 54.6 hours, about 193 hours (eight days) in total. Part of the growth comes from the retroactive re-verification pass, which grows with the memory (about 38 minutes for 431 entries at cycle one, about 16 hours for 10,214 entries at cycle five). The main run was not the first. Three earlier trajectories were rejected or superseded: the first benchmark panel, a run

stopped in May 2026, and the trajectory without the classifier. The last took about 134 hours over cycles zero to five according to its run log, and its first three cycles supply the classifier-off arm of the ablation; the run logs of the first two were not archived, so their compute cannot be reported exactly. Counting these runs, the programme used several times the compute of the main run alone. The carbon footprint of a phase can be estimated with the method of Strubell et al. [28],

$$\mathrm{kgCO_2e}_{\phi} \;=\; T_{\phi} \cdot P_{\mathrm{TDP}} \cdot \varepsilon_{\mathrm{grid}}, \tag{3.3}$$

where T_{ϕ} is the wall-clock duration of phase ϕ in hours, P_{TDP} is the rated power of the graphics processor in kilowatts, and $\varepsilon_{\mathrm{grid}}$ is the local grid emissions factor in kilograms of carbon-dioxide equivalent per kilowatt-hour. At the card's rated board power of 0.575 kilowatts, the main run used at most about 111 kilowatt-hours of graphics-processor energy. The grid factor of the rented host is not known, so no carbon figure is reported.

3.3.2 Cycle budget rationale

The ten-cycle cap bounds the compute that the loop may spend. The convergence heuristic of Chapter 4 (Heuristic 4.6; a heuristic, not a proven result) suggests that gains flatten within a few cycles, and Song et al. [93] find that iterative self-improvement saturates within a few rounds; a cap of ten leaves room above that while bounding cost. Each cycle adds its own fine-tuning and re-verification energy, so stopping once gains have flattened is an environmental choice as well as an experimental one. The realised run stopped after cycle five, as planned once cycle four had been aborted by the retention guard and its weights rolled back. The trajectory is reported in Chapter 5.

3.3.3 Retention versus retraining

The retention guard with rollback also saves compute. Without it, a loop that regressed would either keep training from damaged weights, making every later cycle pay to recover the lost capability, or need a restart from a clean checkpoint, discarding all the fine-tuning done since. With it, a failed cycle costs only its own fine-tuning: the adapter returns to the previous checkpoint, while the memory, including the episodes verified during the failed cycle, is kept. The argument is qualitative; the realised run had one such rollback, at cycle four.

3.4 Ethical Issues

3.4.1 Alignment and refusal posture

The architecture treats explicit abstention as a first-class action rather than a fallback, and this choice has an ethical justification beyond its effect on accuracy. A model that hedges or speculates under uncertainty exposes the user to harm without telling them that uncertainty is present, whereas a model that abstains hands the uncertainty to the user as a visible signal, on which they can act by seeking a second opinion or a human expert. This connects to the question, studied by Kadavath et al. [38],

of whether language models know what they know. The storage and deferral cut-points are fixed values on the calibrated probability scale, chosen on the cycle-zero calibration fold before the main run rather than tuned by hand during it. The ethical commitment behind this posture is that a model which cannot be confident should not present its answer as if it were.

3.4.2 Bias propagation through episodic memory

A system that commits its own outputs to memory and then trains on that memory can amplify biases in the verifier or in the underlying data. Three mechanisms limit this. The retroactive re-verification pass re-scores every stored episode at every cycle boundary and removes entries whose score falls below 0.50, so a bias that enters the stored pool does not persist unchallenged. The per-cycle refit of the calibrated composite, with each benchmark's curve shrunk toward the pooled fit, follows score-distribution drift by re-scoring the held-out calibration fold with each cycle's adapter, while the storage cut-point stays fixed so that the gate's operating point does not move. The strict training threshold holds the training pool to a higher bar than the storage tier (a storage score of at least 0.70 against 0.60 for storage), so marginal episodes cannot enter the next fine-tune even when they pass the retrieval gate. None of these mechanisms removes bias inherited from the pre-training corpus, which remains a limitation of the approach; what they do is limit the rate at which new bias can enter through the self-improvement loop itself.

3.4.3 Data provenance and licensing

The empirical programme uses publicly released benchmarks and a public retrieval corpus throughout. The five evaluation benchmarks are FEVER [22], TriviaQA [19], CommonsenseQA [29], TruthfulQA [40], and StrategyQA [34], all distributed under licences that permit research use with attribution. Two further benchmarks, HotpotQA and Natural Questions, were evaluated at cycle zero and dropped before the main run (Section 1.6.1); their cycle-zero results are kept in the project archive. The retrieval corpus is a Wikipedia-derived passage collection distributed under a Creative Commons licence. No private user data, scraped social-media content or proprietary corpus enters the calibration fold, the seed pool or the cycle pool. The cold-start memory that seeds the self-improvement loop is built from the benchmarks' training splits, with disjointness from the evaluation fold enforced at the sample level. The code is public, and the configuration files, the per-cycle artefacts and the pre-self-improvement snapshot are archived in a shared folder, so that the data path can be audited end to end (Section A.1).

3.5 Standards and Codes of Practice

3.5.1 Software-engineering standards

The implementation is written in Python and broadly follows the PEP 8 conventions for layout and naming. The project does not ship a pinned requirements file; the versions of the main libraries in the realised environment are recorded in Section E.4. All randomness is seeded from one value at process start (the Python, NumPy and

PyTorch generators), and deterministic cuDNN kernels are selected where available; the attention kernels on the graphics processor are not bitwise deterministic, so repeated runs agree up to floating-point noise rather than bit for bit. The source is kept under Git version control in a public repository, with one branch per project phase, and each cycle's artefacts are archived at the cycle boundary, so that the empirical record can be traced to the code that produced it.

3.5.2 Reproducibility standards

Reproducibility is treated as a deliverable rather than a courtesy. Four kinds of artefact support it.

- **Recipe:** Chapter E gives the run sequence from data preparation to the final tables, together with a checklist for comparing reproduced numbers against the thesis.
- **Pre-self-improvement snapshot:** The cold-start memory, the cycle-zero calibrated composite and the configuration at the start of the main run are archived, so that a run can resume from that point instead of from scratch.
- **Per-cycle artefacts:** Each cycle's run summary, memory snapshot, retention-probe readings, precision audit and decision log are kept together.
- **Locked configuration:** One configuration file records the storage and deferral cut-points, the shrinkage prior, the boost-layer regulariser and the low-rank adapter settings used at every downstream scoring and training step.

Together these let a reader check a specific claim in Chapter 5 by re-running the stage that produced it rather than by trusting the reported number.

3.5.3 Evaluation-protocol standards

The empirical comparisons follow four protocol standards.

Matched protocol: Every external baseline is evaluated on the same samples, the same prompt template family and the same scoring rules as the architecture.

Pre-specification: The calibration sweep grid, the cut-point targets and the expected effect bands of the ablation were recorded in the runbook before the corresponding results were read, and the locked configuration was applied unchanged in every cycle of the main run; outcomes outside the expected bands are reported rather than tuned away. Throughout the thesis, a quantity described as pre-specified or registered was fixed in the project's configuration and runbook before the main evaluation run; the term does not refer to an entry in an external pre-registration registry.

Selective prediction: Abstentions are reported per benchmark alongside accuracy, following the selective-prediction framing of Kamath et al. [31], in which a system that may abstain is judged both on how often it answers and on how accurate its answers are.

Multiple comparisons: Within each benchmark, the baseline contrasts are corrected with the Holm–Bonferroni procedure [5], so that a significance verdict cannot

be inflated by the number of comparisons. Each contrast uses the paired McNemar test with the continuity correction of Edwards [2], and a bootstrap 95 percent confidence interval on the exact-match difference is reported alongside the corrected p-value, so that effect size is visible as well as significance.

3.6 Project Management Plan

3.6.1 Phase breakdown

The project has five phases, each producing what the next consumes.

1. **Pre-thesis:** Literature review, problem statement, methodology design, and the per-stage pipeline that the empirical phases depend on.
2. **Calibration:** Fit the calibrated composite on the disjoint cycle-zero calibration fold, fix the storage and deferral cut-points, and lock the configuration for the main run (only the composite is refitted in later cycles).
3. **Main run:** Iterate the per-query pipeline and the cycle-boundary passes for up to ten cycles, producing the per-cycle artefacts.
4. **Comparison:** Run the external baseline panel and the retrieval-utility-classifier on/off ablation under matched protocol, followed by the statistical analysis.
5. **Write-up:** The thesis report, the defence presentation, and a journal manuscript planned for after the defence.

3.6.2 Milestone schedule

The pre-thesis milestone, a locked methodology design, was met on schedule. The calibration milestone, the cycle-zero validation gate, took two iterations: the first failed because of a bug in the step that writes out the verifier signals, and the second, with the bug fixed and a wider parameter grid, produced the locked configuration that the main run uses. The slip is reported here rather than smoothed over, and Chapter 5 gives the diagnostics of both iterations. The main run completed six cycles (cycles zero to five) on 27 May 2026, and the comparison phase completed the eight-baseline panel and the retrieval-utility-classifier ablation. A second slip followed: a review of the first comparison found that the baselines had received a shorter prompt than CAEM and that some answer formats were scored differently, so the baseline panel was re-run in October 2026 under the matched protocol of Section 5.1.1, and every result in Chapter 5 comes from that re-run. The write-up milestone is the submission of this report and its companion artefacts.

3.6.3 Deliverables

The project releases five deliverables.

- **Source code:** public at https://github.com/aksaN000/caem-thesis: the implementation of the architecture, the cycle-zero calibration scripts, the cycle-boundary passes, and the baseline and ablation drivers.

- **Artefact bundle:** stored in the shared Google Drive folder https://drive.google.com/drive/folders/1xErhsRDYe_qO0ZgUBn0WXK81eHc5jZCa: the cycle-zero calibrated composite, the locked cut-points, the cold-start memory snapshot, and the per-cycle artefacts of the realised trajectory. The 64-gigabyte Wikipedia passage index is mirrored on a private Hugging Face repository, available on request (Section A.1).
- **Thesis report:** this document.
- **Defence presentation:** a talk summarising the headline result, the method and the threats to validity.
- **Journal manuscript:** planned for after the defence and prepared from the evaluation chapter.

3.7 Risk Management

3.7.1 Technical risks

Six technical risks were identified before the main run. Each is paired with its mitigation below, and Table 3.3 records whether it materialised.

- **Calibration drift:** The composite, refitted every cycle, could lose its precision at the frozen cut-point as each adapter update shifts the score distribution. Mitigation: the per-cycle refit (per-signal isotonic curves, per-benchmark shrinkage and boost layer) and a per-cycle precision audit at the locked cut-points.
- **Verifier and judge disagreement:** The short-answer entailment judge and the long-answer judge could score the same answer inconsistently. Mitigation: drop the long-answer judge if its agreement with the short-answer judge misses a fixed correlation floor.
- **Out-of-memory failures:** A long answer could send the verifier down a branch that exceeds the memory of the graphics processor and halt the cycle. Mitigation: chunking the long-answer batch, and a clean restart from the previous checkpoint after an out-of-memory abort.
- **Forgetting:** A fine-tune could erode general capability. Mitigation: the retention guard with adapter rollback (Equation (3.2)).
- **Early-stop misfire:** The automatic early-stop check could stop the run too early or fail to stop a run that is not converging. Mitigation: requiring two of its three signals to agree, and the ten-cycle cap.
- **Retrieval-utility classifier shift:** The classifier is trained on data that deliberately includes a benchmark outside the evaluation panel (the Natural Questions training split), so it may send some queries to the wrong tier on the panel. Mitigation: nested cross-validation, a threshold sweep, and the on/off ablation on the same trajectory.

3.7.2 Budget and infrastructure risks

Three infrastructure risks were identified.

- **Credit exhaustion:** Running out of rented graphics-processor credit before the trajectory and the comparisons finish would leave a partial run that cannot be compared against the baselines. Mitigation: budgeting each phase against the recorded credit, and archiving every state before it is overwritten.
- **Node reclamation:** The provider could reclaim the rented machine in the middle of a cycle. Mitigation: committing artefacts at every cycle boundary, so that a lost machine costs at most the cycle in flight, and the pre-self-improvement snapshot, from which the run can resume.
- **Storage exhaustion:** The artefacts, model snapshots and passage index could exceed the disk of the rented machine or the quota of the hosts that hold them. Mitigation: sizing the artefact catalogue in advance and keeping the larger snapshots and the passage index on separate storage.

3.7.3 Mitigation register

Table 3.3 consolidates the register. Its last column records what happened over the completed programme.

3.8 Economic Analysis

3.8.1 Compute budget

The compute budget is measured in graphics-processor hours on the rented 32-gigabyte machine used throughout. The main run took about 193 hours (Section 3.3.1). The cost of the external baseline panel scales with the number of samples and the number of decode passes per sample, with the chain-of-thought and retrieval-augmented baselines the most expensive. The retrieval-utility-classifier ablation added no run: it compares the main run with the earlier trajectory without the classifier at the cycle-three close, the last cycle that both trajectories committed (the earlier one aborted its cycles four and five under the retention guard). Section E.5 also gives a reduced configuration that reproduces the trajectory on a 16-gigabyte device, so that the cost on smaller hardware can be estimated without re-running the experiment.

3.8.2 Cost-benefit on the storage gate

On the main run the storage gate admitted about half of each 4,700-sample stream chunk: between 2,343 and 2,484 entries per cycle over cycles one to five (Table C.3). Storing an answer adds no verification work at query time, because every signal the gate uses has already been computed for that answer. It does add work later: each stored entry is re-verified at every later cycle boundary, which is why the retroactive pass grows with the memory (Section 3.3.1). The benefit of a stored entry is the decode it saves each time it answers a later question directly,

$$\Delta B_{\text{store}} \;\geq\; n_{\text{hit}} \cdot c_{\text{gen}}, \tag{3.4}$$

Risk	Mechanism	Mitigation	Realised
Calibration drift across cycles	Composite loses its precision at the frozen τ_{store} under adapter-induced shift	Per-cycle composite refit (per-signal isotonic, per-benchmark shrinkage, boost); per-cycle precision audit	**Partly**: calibration-fold precision held between 0.732 and 0.746 over cycles one to five; the calibration-to-evaluation gap is reported in Chapter 5
Verifier and judge disagreement	Short- and long-answer judges score the same answer inconsistently	Drop the long-answer judge if it misses the correlation floor	**Yes**: long-answer judge dropped (Pearson $0.584 < 0.70$)
Out-of-memory failure on long paths	A long answer sends the verifier past the GPU's memory	Chunked long-answer batch; clean restart after an abort	**Yes**: chunking added; degenerate samples skipped on out-of-memory
Forgetting	A fine-tune erodes general capability	Retention guard with adapter rollback	**Yes**: cycle four, TriviaQA-test ratio $0.886 < 0.93$; adapter rolled back, memory kept
Early-stop misfire	Premature stop, or a run that never stops	Two-of-three signal rule; ten-cycle cap	No: the run ended at its planned stop after cycle five (Section 3.3.2)
Classifier distribution shift	Classifier routes some panel queries to the wrong tier	Nested cross-validation; threshold sweep; on/off ablation	Reported in the ablation (Chapter 5)
GPU credit exhaustion	Run does not finish within the rented credit	Phase budgeting; archive before overwrite	**Yes**, once (May 2026, during an earlier run); resumed from the recorded step after a top-up
Node reclamation	Cycle in flight lost when the machine is reclaimed	Per-cycle artefact commit; pre-self-improvement snapshot	Not observed
Storage exhaustion	Artefacts and snapshots exceed the available disk	Advance sizing; separate storage for large snapshots	**Yes**: the rented node's disk filled; the data were copied to a new node with the provider's support and the run resumed from its last checkpoint
Calibration iteration failure	Cycle-zero composite fails the validation gate	Re-iterate with the signal-dump bug fixed and a wider grid	**Yes**: first iteration failed on the signal-dump bug; second iteration locked

Table 3.3: Risk register with mitigations and the realised outcome over the completed programme.

where n_{hit} is the number of later queries that the entry answers through the memory-direct tier and c_{gen} is the cost of the zero-shot or retrieval-augmented decode it replaces. This benefit appears only when questions recur, and in the realised run they never did: each cycle drew a fresh stream chunk that shares no question with earlier chunks, and the evaluation fold is disjoint from both, so almost no query matched a stored episode closely enough for the memory-direct tier to answer it. On the evaluation fold that tier answered only 7 of 9,000 queries. A direct probe of the router (Table 5.8) shows the tier working when questions do recur: it answered 42 of 50 exact repeats and 19 of 50 paraphrases of stored questions. The precision of the gate is what makes the reused answers worth serving.

3.8.3 Deployment cost projection

By design, the cost per query should fall as the memory grows, because a memory-direct answer costs an embedding lookup and a similarity search instead of a decode. The cost per query at cycle t is a weighted average over the three tiers,

$$\bar{c}(t) \;=\; \pi_1(t)\, c_1 \;+\; \pi_2(t)\, c_2 \;+\; \pi_3(t)\, c_3, \qquad \pi_1(t) + \pi_2(t) + \pi_3(t) = 1, \tag{3.5}$$

where $\pi_i(t)$ is the share of incoming queries dispatched to tier i at cycle t and c_i is the average per-query cost on that tier. The design expects $\pi_1(t)$ to rise and $\pi_3(t)$ to fall as cycles progress. In the realised run neither moved as designed: the memory-direct share stayed at or below 0.1 percent, because by design no stream or evaluation question repeated a stored one (Section 3.8.2), and the zero-shot and retrieval-augmented shares each stayed between about 45 and 55 percent across cycles (Chapter 5). The memory-direct saving is therefore expected only where real traffic repeats or paraphrases earlier questions, which is the setting that the deployment-study future-work item in Chapter 6 would test under live user traffic.

3.9 Chapter signpost

The requirements, design commitments, societal, environmental and ethical readings, standards, and risk and economic analyses in this chapter fix the constraints under which the architecture is specified. Chapter 4 gives that specification in full: the eight-stage per-query pipeline with the Stage 3b retrieval-utility classifier, the nine-signal verifier ensemble and its calibrated probability composite, the fixed-threshold storage gate and the four-outcome decision tree, the cycle-boundary update passes, the constrained self-improvement loop under the multi-modal retention guard, and the pool-purity theorem, the purification condition, and the trajectory heuristics with their proofs or argument sketches and pre-specified empirical receipts.

Chapter 4

Methodology and Design

4.1 Design Process or Methodology Overview

4.1.1 The eight-stage pipeline narrative

CAEM answers each query with an eight-stage pipeline (Figure 4.2). Stages 1 to 7 run once per query; Stage 8 runs once at the end of every cycle.

1. **Pre-route:** a cheap confidence score u_{pre} is computed for the query before any answer is generated (Section 4.2.2).
2. **Encode and look up:** the query is embedded and the nearest stored episode is found, giving a similarity s and that episode's stored score u_{stored} (Sections 4.2.1 and 4.2.3).
3. **Route:** the router picks one of three tiers from s, u_{stored} and u_{pre} (Section 4.2.4). **Stage 3b:** when the router's choice would default to retrieval, the retrieval-utility classifier decides between retrieval and zero-shot generation (Section 4.2.5).
4. **Answer:** Tier 1 returns the stored answer; Tier 2 generates without retrieval; Tier 3 retrieves Wikipedia passages and generates with them.
5. **Verify:** every generated answer is scored by nine signals in four families (Section 4.2.6), which the calibrated composite turns into one probability of correctness, u_{stored} (Section 4.2.7).
6. **Decide:** the four-outcome rule stores, defers, abstains or discards (Sections 4.2.8 and 4.2.9).
7. **Output:** the user receives the answer with its calibrated probability and a reliability tag, or an insufficient-evidence message when the system abstains.
8. **Cycle boundary:** the adapter is fine-tuned and checked for forgetting, the deferred buffer is revisited, the memory is consolidated, the composite is refitted and every stored episode is re-verified (Section 4.2.10).

4.1.2 The cycle loop walk-through

A cycle has two phases. In the per-query phase, the pipeline answers a fresh stream chunk of 4,700 training-panel questions (FEVER 2,000, TriviaQA 2,000, CommonsenseQA 700). Each chunk shares no question with earlier chunks, with the calibration fold or with the evaluation fold. Answers that pass the storage gate enter the episodic memory and borderline answers enter the deferred buffer. The evaluation fold is then answered without writing to memory.

In the cycle-boundary phase, five steps run in a fixed order (Figure 4.1). First, the low-rank adapter is fine-tuned on stored episodes whose u_{stored} is at least 0.70 and the retention guard checks three held-out probes; if any probe falls below 93 percent of its starting accuracy, the previous adapter is restored and the remaining steps are skipped. Otherwise the deferred buffer is reconsidered, near-duplicate episodes are merged, the fixed calibration fold is re-scored with the new adapter so that the temperature and the composite can be refitted, and finally every stored episode is re-verified with the refitted composite. Re-verification can lower a stored score as well as raise it, and entries that fall below 0.50 are removed. The memory itself is never rolled back: answers verified during a failed cycle stay in it.

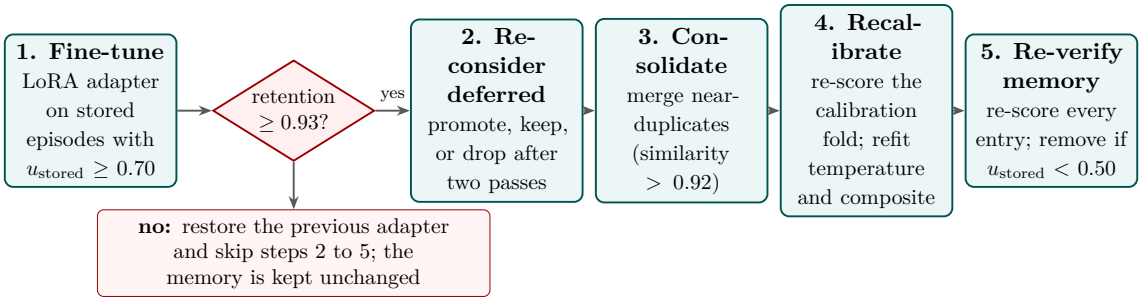


Figure 4.1: Order of the cycle-boundary steps as executed in the main run. After step 5 the next cycle begins by answering a fresh stream chunk with Stages 1 to 7. Cycle four took the rollback branch.

4.1.3 A worked example

Two real records from the main run show the pipeline end to end (Table 4.1).

A stored answer: the TriviaQA question *In what year (or century) was Michelangelo born?* arrived in the cycle-one stream chunk. Its pre-routing confidence (0.522) was above the safety floor, no stored episode was similar enough for Tier 1 or Tier 2, and the retrieval-utility classifier kept the default, so Tier 3 retrieved passages and the generator answered *1475*, citing a passage that states the birth year. The sampled reasoning chains agreed ($s_{\text{avg}} = 0.992$) and entailed the answer ($p_{\text{entail}} = 0.939$), while the best passage gave only moderate support ($p_{\text{ground,max}} = 0.469$). The composite weighed these signals into $u_{\text{stored}} = 0.760$, above the storage cut-point of 0.60, so the answer was stored and shown to the user as verified. It was correct.

An abstention: the TriviaQA question *The song "If I Ruled The World" comes from which musical?* is in the evaluation fold. At cycle zero, with only the cold-start memory, it fell through to Tier 3, where the retrieved passages mention the song but not the musical, and the generator answered *I do not know*. Every signal was low and

the composite gave $u_{\text{stored}} = 0.127$. Because this is below the deferral cut-point of 0.45 and the best passage supports the answer weakly ($p_{\text{ground,max}} = 0.112 < 0.20$), the decision was to abstain: the user receives the insufficient-evidence message with the probability shown, and nothing is stored. The gold answer is *Pickwick*, so abstaining avoided a likely wrong answer.

	Stored example	**Abstained example**
Question	Michelangelo's birth year	"If I Ruled The World"
Cycle, fold	1, stream chunk	0, evaluation fold
Tier	3	3
u_{pre}	0.522	0.659
u_{token}, u_{dropout}, u_{internal}	0.163, 0.600, 0.282	0.022, 0.800, 0.111
s_{avg} / p_{entail}	0.992 / 0.939	0.804 / 0.115
$p_{\text{ground,max}}$ / $p_{\text{ground,mean}}$	0.469 / 0.292	0.112 / 0.094
q_{match}	0.515	0.500
u_{stored}	**0.760**	**0.127**
Decision	store	abstain
Outcome	correct (*1475*)	no answer (gold *Pickwick*)

Table 4.1: Signal readings for the two worked examples, read from the main run's per-sample records. The atomic-fact score equals the mean-passage score for both (Section 4.2.6).

4.2 Preliminary Design or Design (Model) Specification

4.2.1 Generator and retrieval substrate

Four components are fixed across every cycle.

Generator: Qwen2.5-3B-Instruct [81], a three-billion-parameter instruction-tuned decoder, generates every Tier 2 and Tier 3 answer and supplies the internal-calibration signals. Only a low-rank adapter on top of it is trained (about 2 percent of its parameters), which keeps each cycle's fine-tune within one 32-gigabyte graphics processor. A small generator is also accurate enough on the training panel for the storage gate to have a usable signal.

Retrieval corpus and encoder: Tier 3 retrieves from a dense index of about 21 million passages of roughly 100 words from English Wikipedia, built in the dense-retrieval style of Karpukhin et al. [32]. Queries, passages and memory entries are all embedded with the same sentence encoder, all-mpnet-base-v2 [27] (768 dimensions), so a query can be compared directly with both. Tier 3 places the top five passages in the prompt. Both generating tiers ask the generator for a short reasoning followed by a line beginning *Answer:*; the answer is read from that line, and the memory stores it together with the reasoning chain.

Entailment judge: MiniCheck-Flan-T5-Large [82] scores whether a premise supports a claim. It was trained on claim-support data generated by language models,

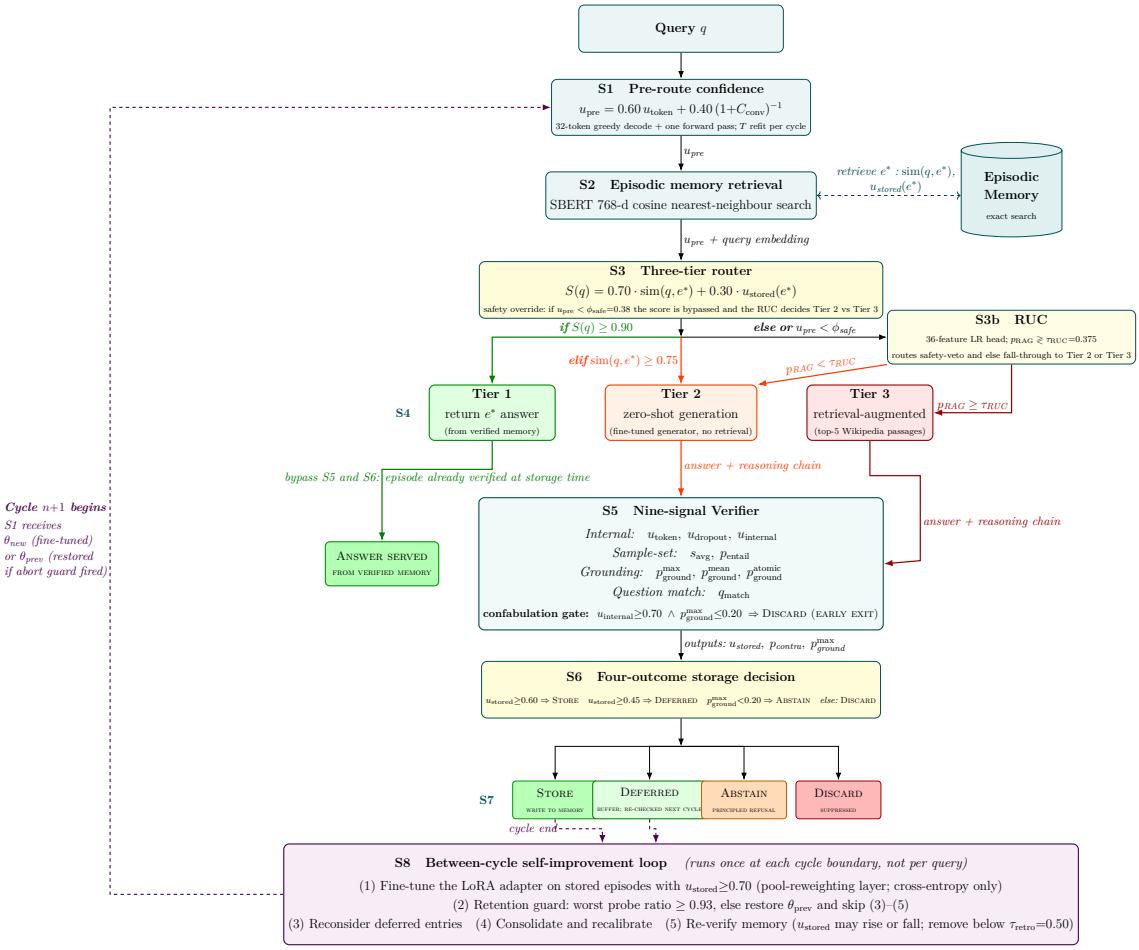


Figure 4.2: The eight-stage CAEM pipeline. Stages S1 to S7 run once per query; S8 runs once per cycle, in the order of Figure 4.1. The outer dashed arrow hands the updated or restored adapter to the next cycle.

which matches the kind of answers it is asked to judge. It returns a single support probability rather than a three-way label, so no contradiction signal is used. Inputs longer than its 512-token encoder are truncated; a second, long-input judge built on the frozen generator was considered but dropped before the main run because its agreement with MiniCheck missed the required correlation floor (Pearson $0.584 < 0.70$, Chapter 5).

Reranker: the cross-encoder bge-reranker-v2-m3 scores text pairs for relevance. The verifier uses it twice: to rerank the top 20 retrieved passages down to the top three before entailment is scored, and to score how well the answer string matches the question. The second use was meant to catch an off-topic answer that an off-topic passage happens to support; Section 4.2.6 explains why it cannot do this on short answers and what it measures instead.

4.2.2 Pre-routing confidence

The pre-routing confidence u_{pre} is a cheap estimate of how ready the generator is to answer, computed before any answer is produced. It feeds the safety override of the router. It combines two quantities (Algorithm 1).

Token confidence: the generator runs a 32-token greedy decode on the query and u_{token} is the geometric mean of the probabilities of the tokens it chose. A model that hesitates between continuations scores low.

Layer-convergence ratio: one forward pass over the query records the hidden states of every decoder layer. The ratio c_{conv} divides the variance of the hidden states in the first half of the layers by their variance in the second half, adapted from Nandakishor [91]. A large ratio means the representation is still unsettled in the early layers.

The two are combined with fixed weights, $z = 0.60\, u_{\text{token}} + 0.40/(1 + c_{\text{conv}})$, and calibrated by temperature scaling [18] with a per-benchmark temperature T_b fitted on the calibration fold. Temperature scaling was chosen because it has one parameter and preserves the order of scores.

Realised behaviour: on FEVER and TriviaQA the fitted temperature reached its upper bound ($T = e^3 \approx 20.1$) at cycle zero and was refitted downward each cycle with exponential smoothing (to about 14 at cycle one and 10 at cycle two). A temperature this large pulls u_{pre} toward 0.5, so on these benchmarks the score rarely approached the safety floor of 0.38. Chapter 5 reports the consequence: the override fired on 63 of the 9,000 evaluation queries, 62 of them on CommonsenseQA after its floor was refitted upward.

4.2.3 Episodic memory store

The episodic memory is CAEM's long-term store of verified answers. It follows memory-augmented networks [11], [14], [16] and writeable episodic stores for language models [70], [95], with one distinguishing feature: every entry carries the calibrated score u_{stored} that admitted it.

Record: each entry holds the question, the answer, the reasoning chain, the ques-

Algorithm 1 Pre-routing confidence (Stage 1).

Require: query q, generator G, decode length $L = 32$, per-benchmark temperature T_b

$(\log p_1, \ldots, \log p_n) \leftarrow$ log-probabilities of the tokens chosen by a greedy decode of up to L tokens

$u_{\text{token}} \leftarrow \exp\Big(\frac{1}{n}\sum_{i=1}^{n} \log p_i\Big)$ ▷ geometric mean of token probabilities

$\mathbf{H}^{(1)}, \ldots, \mathbf{H}^{(D)} \leftarrow$ hidden states of the D decoder layers in one forward pass over q

$c_{\text{conv}} \leftarrow \text{Var}\Big(\mathbf{H}^{(1:D/2)}\Big) \,/\, \text{Var}\Big(\mathbf{H}^{(D/2+1:D)}\Big)$ ▷ early-half over late-half variance

$z \leftarrow 0.60\, u_{\text{token}} + 0.40 \,/\, (1 + c_{\text{conv}})$

return $u_{\text{pre}} \leftarrow \sigma\Big(\text{logit}(z)/T_b\Big)$ ▷ temperature scaling

tion's embedding, the nine signal readings, u_{stored}, the source benchmark and the cycle in which it was stored. Keeping the question and answer together lets the cycle-boundary pass re-verify an entry without regenerating the answer.

Cold start: before cycle one, the pipeline answered questions from each training benchmark's seed pool until about 200 per benchmark had been verified and stored (603 answers). Answers that were wrong against the gold labels were then removed, leaving 431 entries (FEVER 155, TriviaQA 136, CommonsenseQA 140). The cold-start memory is therefore correct by construction; every later entry is admitted by the verifier alone.

Search: lookup is by cosine similarity between question embeddings, because the encoder is trained to place paraphrases in the same direction. The index is exact (flat) below 20,000 entries and would switch to an approximate inverted-file index above that size; the memory reached 12,175 entries, so every search in the main run was exact.

Novelty and quality filters: a new answer is not stored if an existing entry's question has similarity above 0.95 with the new question, so paraphrases do not fill the memory, or if the answer contains a leaked prompt placeholder or an evasive phrase such as *the context does not mention*.

Serve counter: an entry served by Tier 1 ten times is re-verified at the next cycle boundary even if nothing else has changed. With Tier 1 serving only 7 evaluation queries in the whole run, this never triggered.

Consolidation: at each cycle boundary, entries whose questions have similarity above 0.92 are grouped; a group is merged into its highest-scoring entry only if the entries give the same answer and their scores differ by at most 0.15. The lower threshold catches near-paraphrases that slipped past the novelty filter. It removed 3, 11 and 44 entries at cycles two, three and five.

Capacity: the memory is capped at one million entries, with a pruning rule that removes the lowest-value fifth when the store is 95 percent full. The rule never fired, because the memory stayed far below the cap.

4.2.4 Three-tier router with the safety override

The router chooses the tier from three inputs: the similarity s between the query and its nearest stored question, that neighbour's stored score u_{stored}, and the pre-routing confidence u_{pre}. Its rules are checked in a fixed order (Algorithm 2, Figure 4.3; Table 4.2 gives the final rules including the classifier).

1. **Safety override:** if $u_{\text{pre}} < 0.38$, the query skips the memory and goes to the retrieval-utility classifier.
2. **Tier 1:** if the combined score $\mathcal{S} = 0.70\,s + 0.30\,u_{\text{stored}}$ is at least 0.90, the neighbour's stored answer is returned without generating.
3. **Tier 2:** otherwise, if $s \geq 0.75$, the fine-tuned generator answers without retrieval.
4. **Fall-through:** otherwise the query goes to the retrieval-utility classifier, which sends it to Tier 3 or Tier 2.

Similarity carries the larger weight in $\mathcal{S}$ because a wrong neighbour is a different question altogether, whereas an under-scored correct neighbour still holds a correct answer. Because $u_{\text{stored}} \leq 1$, Tier 1 needs $s \geq 0.857$: only a close paraphrase of a stored question is answered from memory. The safety floor was lowered from 0.60 to 0.38 after a routing audit of an earlier run showed that the higher floor forced well-supported queries onto retrieval; in the main run it was refitted per benchmark at each cycle and stayed at 0.38 except on CommonsenseQA (0.491 at cycles three and four, 0.569 at cycle five).

Separating the three questions (is there a trusted close match, is the model ready to answer, would retrieval help) distinguishes this router from a single-threshold dispatcher and from Adaptive-RAG of Jeong et al. [75], which routes by query complexity rather than by the stored score of a memory entry.

Design intent and realised shares: the design expects the memory-direct share to grow as the memory fills, and the zero-shot share to grow as fine-tuning folds retrieved knowledge into the generator. In the main run neither happened. Tier 1 answered at most 0.1 percent of evaluation queries, because no stream or evaluation question repeats a stored one; a direct probe confirms that Tier 1 does answer repeated questions (42 of 50 exact repeats, 19 of 50 paraphrases, Table 5.8). The zero-shot and retrieval shares each stayed between about 45 and 55 percent of evaluation queries across cycles.

4.2.5 Retrieval-utility classifier

Retrieval does not help every question. On questions built around a popular misconception, retrieved passages can reinforce the misconception, and on questions the generator already knows they can distract it. An earlier trajectory without a routing refinement showed this: TruthfulQA exact match fell from cycle to cycle as retrieval-amplified answers entered the verified pool. The selective-retrieval literature reached the same conclusion, including Mallen et al. [55], Wang et al. [63], Jeong et al. [75] and Baek et al. [86] (Section 2.2.5). The retrieval-utility classifier is CAEM's response: whenever the router would send a query to retrieval by default, the classifier

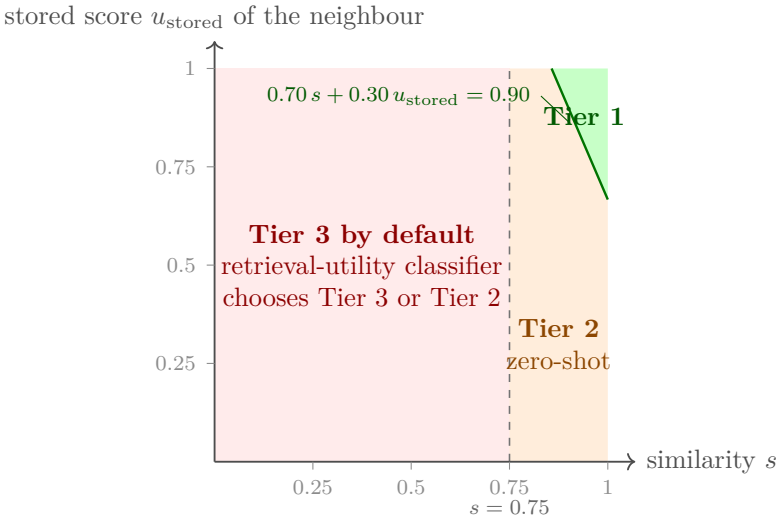


Figure 4.3: Routing map of the three-tier router. Before this map is consulted, the safety override sends any query with $u_{\text{pre}} < 0.38$ to the retrieval-utility classifier.

Algorithm 2 Three-tier router with safety override (Stage 3).

Require: similarity s, neighbour score u_{stored}, pre-routing confidence u_{pre}
Require: $\lambda = 0.70$, $\theta_1 = 0.90$, $\theta_{\text{sim}} = 0.75$, per-benchmark floor ϕ_{safe} (0.38 unless refitted)

$\mathcal{S} \leftarrow \lambda\, s + (1 - \lambda)\, u_{\text{stored}}$
if $u_{\text{pre}} < \phi_{\text{safe}}$ **then**
    return Tier 3 ▷ safety override; refined by Algorithm 3
else if $\mathcal{S} \geq \theta_1$ **then**
    return Tier 1
else if $s \geq \theta_{\text{sim}}$ **then**
    return Tier 2
else
    return Tier 3 ▷ fall-through; refined by Algorithm 3
end if

predicts whether retrieval will help and, if not, sends the query to Tier 2 instead (Table 4.2, Algorithm 3).

Model: a logistic regression on 36 standardised features outputs p_{RAG}, the probability that retrieval helps. The query goes to Tier 3 if $p_{\mathrm{RAG}} \geq 0.375$ and to Tier 2 otherwise. The threshold is below 0.5 because it was chosen to maximise the number of correctly routed queries of both kinds on held-out folds, and missing a question that needed retrieval was the more common error.

Features: three groups. (i) Question surface: length, the interrogative word, negation, temporal and numerical cues, entity count, a misconception-phrasing pattern, and three style markers (adversarial, opinion-seeking, open-ended). (ii) The top retrieved passage: its similarity, entity overlap with the question, whether the question expects a year or number and whether the passage contains one, and whether the question's main entity appears in Wikidata. (iii) Retrieval quality and self-knowledge: statistics of the top-five retrieval similarities, of their cross-encoder rerank scores and of pairwise entailment among them, plus a self-knowledge probe p_{IK} in the style of Kadavath et al. [38], which reads the base generator's last hidden state and predicts whether it knows the answer without retrieval. The probe was trained on 11,980 questions (out-of-fold AUROC 0.771). It carries the largest weight in the classifier (-1.77): when the generator knows the answer, the classifier leans strongly toward Tier 2. Six features (page views and the four year and number features of the passage) were constant in the training data and received zero weight, and the top-passage similarity took the opposite sign to the one expected.

Training data: 3,208 questions from a pool disjoint from every other fold, drawn from eight benchmarks (the five of the evaluation panel plus Natural Questions, HaluEval-QA and OpenBookQA, included only to widen the range of cases). Each question was answered both with and without retrieval by the same backbone. It was labelled *retrieval helps* (1,846 questions) if only the retrieval answer was correct and *retrieval hurts* (1,362) if only the zero-shot answer was correct; questions where both or neither were correct were dropped, because there the routing choice does not matter. TruthfulQA answers were judged by the language-model judge used throughout the thesis. The three extra benchmarks feed nothing else in the architecture.

Fitting and accuracy: nested five-fold stratified cross-validation chose the regularisation strength ($C = 0.5$) in the inner folds and measured accuracy in the outer folds. The pooled out-of-fold AUROC is 0.850 (standard deviation across folds 0.008). At the chosen threshold the classifier routes 77.2 percent of held-out questions correctly (2,477 of 3,208): 1,554 of 1,846 retrieval-helps questions (recall 0.842) and 923 of 1,362 retrieval-hurts questions (specificity 0.678). Per-benchmark AUROC on the evaluation panel is FEVER 0.718, TriviaQA 0.936, CommonsenseQA 0.728, TruthfulQA 0.732 and StrategyQA 0.852 (the last on only 43 questions). TruthfulQA, the benchmark that motivated the classifier, falls just short of the 0.75 target set for it in advance.

Frozen at deployment: the classifier was fitted once before the main run and never refitted. Its inputs come from parts of the system that do not change across cycles (the question, the passage index, the base generator with the adapter disabled, and the fixed reranker), so a frozen classifier keeps its routing auditable. This

light, classifier-only design agrees with Marina et al. [90], who find that an external classifier matches language-model-based selective retrievers; CAEM differs in reusing the reranker and entailment judge that the verifier already loads. The effect of the classifier is measured in Section 5.4.2.

Order	Tier	Condition	Output path
1	Tier 3 / 2	$u_{\text{pre}} < 0.38$ (safety override): Tier 3 if $p_{\text{RAG}} \geq 0.375$, else Tier 2	Retrieval-augmented or zero-shot generation
2	Tier 1	$\mathcal{S} = 0.7\, s + 0.3\, u_{\text{stored}} \geq 0.90$	Serve neighbour's stored answer
3	Tier 2	$s \geq 0.75$	Zero-shot generation
4	Tier 3 / 2	otherwise (fall-through): Tier 3 if $p_{\text{RAG}} \geq 0.375$, else Tier 2	Retrieval-augmented or zero-shot generation

Table 4.2: Dispatch rules of the router and the retrieval-utility classifier, in order of evaluation. The classifier is consulted only in rows 1 and 4, the two cases that would otherwise always retrieve.

Algorithm 3 Retrieval-utility classifier (Stage 3b).

Require: question q, router decision $d \in \{\text{Tier 1}, \text{Tier 2}, \text{Tier 3}\}$, frozen scaler Σ, frozen weights w, threshold $\tau_{\text{RUC}} = 0.375$

if $d \in \{\text{Tier 1}, \text{Tier 2}\}$ **then**
  return d ▷ the classifier does not gate these outcomes
end if
$\mathbf{x} \leftarrow$ [question-surface features, top-passage features, retrieval-quality features, p_{IK}]
$p_{\text{RAG}} \leftarrow \sigma\big(w^{\top}\, \Sigma(\mathbf{x})\big)$
return Tier 3 **if** $p_{\text{RAG}} \geq \tau_{\text{RUC}}$ **else** Tier 2

4.2.6 Verifier ensemble

The verifier turns a generated answer into nine signals in four families (Table 4.3, Figure 4.4, Algorithm 4). The families follow the confidence-estimation survey of Geng et al. [72], with one addition, a question-match family, intended for answers that are supported by a passage but do not address the question (Section 4.2.6 describes what it measured in practice). Each family reads a different source of evidence, so that one failure mode does not hide behind another. Two practical details matter. First, the grounding signals check the answer rewritten as a statement: a FEVER *refutes* answer is checked as the negated claim, and a multiple-choice letter is replaced by the text of the chosen option. Second, five shared computations produce all nine signals, so the cost is paid once per answer.

Internal calibration signals

These signals read the generator's own state. The token signal is the most local form of self-doubt: the generator re-reads its full response, reasoning and answer line together, given the question alone (without the tier's instructions or passages), and

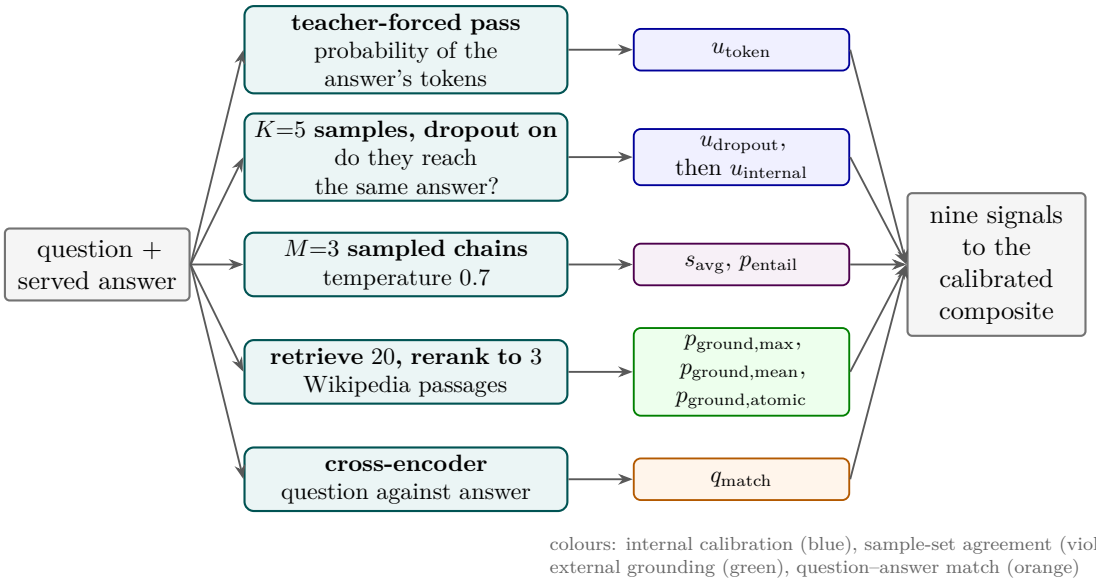


Figure 4.4: Where the nine verifier signals come from. Five shared computations produce all nine signals; the grounding scores check the answer rewritten as a statement against the reranked passages.

Signal	Family	What it measures	Failure it catches
u_{token}	Internal	Geometric mean probability of the full response (reasoning and answer line) under the generator, given the question	Low-confidence generation
u_{dropout}	Internal	Share of 5 dropout-active samples whose answer differs from the most common one	Unstable answers
u_{internal}	Internal	$0.5\,u_{\text{token}} + 0.5\,(1 - u_{\text{dropout}})$	Summary of internal confidence
s_{avg}	Sample-set	Mean pairwise cosine similarity of 3 sampled chains (temperature 0.7)	Disagreement across chains
p_{entail}	Sample-set	Mean entailment of the answer by each of the 3 chains	Answer not supported by the model's own reasoning
$p_{\text{ground,max}}$	Grounding	Entailment of the answer by the best of the top three reranked passages	No supporting passage
$p_{\text{ground,mean}}$	Grounding	Mean entailment over the top three passages	Weak overall support
$p_{\text{ground,atomic}}$	Grounding	Weakest support among the answer's atomic facts (answers of 12 tokens or more)	One fabricated detail in a long answer
q_{match}	Question match	Cross-encoder match score between the question and the answer string	Intended: off-topic answers; in practice a weak correctness cue

Table 4.3: The nine verifier signals, grouped by family.

Algorithm 4 Verifier ensemble (Stage 5).

Require: question q, answer a, generator G, entailment judge J, cross-encoder X

Internal calibration

$u_{\text{token}} \leftarrow$ geometric mean probability of the tokens of the full response under G, given q alone (teacher-forced)

draw 5 answers from G with dropout active; $u_{\text{dropout}} \leftarrow 1 -$ (count of the most common answer)$/5$

$u_{\text{internal}} \leftarrow 0.5\, u_{\text{token}} + 0.5\,(1 - u_{\text{dropout}})$

Sample-set agreement

$c_1, c_2, c_3 \leftarrow$ chains sampled from G at temperature 0.7

$s_{\text{avg}} \leftarrow$ mean pairwise cosine similarity of the chain embeddings

$p_{\text{entail}} \leftarrow \frac{1}{3} \sum_m J(c_m \to a)$

External grounding

$P_1, P_2, P_3 \leftarrow$ top 3 of the 20 retrieved passages after reranking by X

$h \leftarrow a$ rewritten as a statement ▷ negated claim for *refutes*; option text for a letter

$p_{\text{ground,max}} \leftarrow \max_k J(P_k \to h)$; $p_{\text{ground,mean}} \leftarrow \text{mean}_k\, J(P_k \to h)$

if a has at least 12 tokens and yields atomic facts $f_1, \ldots, f_r$ **then**

    $p_{\text{ground,atomic}} \leftarrow \min_i \max_k J(P_k \to f_i)$

else

    $p_{\text{ground,atomic}} \leftarrow p_{\text{ground,mean}}$

end if

Question match

$q_{\text{match}} \leftarrow X(q, a)$ ▷ a as the wrapped answer string, e.g. "Answer: Rudolph."

return the nine signals

the signal is the geometric mean of the token probabilities. Because the reasoning tokens far outnumber the answer tokens, the signal measures how fluent the response is more than how sure the generator is of the answer; on the evaluation fold at C0 it separated correct from incorrect answers barely above chance (area under the ROC curve 0.48 to 0.60 across the five benchmarks), and scoring the answer line alone is left as future work (Chapter 6). The dropout signal follows the Bayesian reading of dropout of Gal and Ghahramani [15]: with dropout switched on, the generator draws five answers, and u_{dropout} is the share that disagree with the most common one, so 0 means all five agree. The summary u_{internal} is the input to the confabulation early-exit rule (Section 4.2.9), because it isolates internal confidence from external evidence. This family is the one most affected by fine-tuning, which is one reason stored entries are re-verified at every cycle boundary. A hidden-state eigenvalue score [67] was considered as a replacement for the dropout signal but not adopted, for cost.

Sample-set agreement signals

These signals ask whether the generator gives the same answer when it reasons again. The generator samples three chains; s_{avg} is their mean pairwise similarity in the embedding space, an embedding-based form of the self-consistency of Wang et al. [62], and p_{entail} asks the judge whether each chain supports the served answer. Both use the same three chains.

Two dispersion signals were dropped before the main run. The semantic entropy of Farquhar et al. [71] (after Kuhn et al. [53]), computed over 10 samples at temperature 1.0, received a learned weight of about -0.0005 in the ten-signal cycle-zero fit, so it added nothing beyond the two signals above, while its 10 samples were the largest single cost of the verifier. The kernel language entropy of Nikitin et al. [79] needs the same samples and was not adopted either.

External grounding signals

These signals check the answer, rewritten as a statement, against the three best passages. The maximum catches the case where one passage clearly supports the answer; the mean rewards support spread over several passages; the atomic score splits a long answer into atomic facts, in the FActScore manner of Min et al. [58], and takes the weakest-supported fact, so that one fabricated detail is not averaged away. The atomic score applies only to answers of at least 12 tokens and otherwise equals the mean score.

Realised run: in the main run the atomic score equalled the mean score on all 14,951 scored samples. The length gate skipped 99.0 percent of answers, and for the remaining 1.0 percent the decomposer returned no atomic facts. This is a mismatch of scope rather than a tuning fault: the signal follows FActScore, which targets long-form text, while the answers on this panel are labels, option letters and short entities, each already a single fact, so a lower gate would only reproduce the mean score. The signal is part of the design but contributed no separate information in this run; applying the same check to the reasoning chain, where several facts are asserted, is left as future work (Chapter 6).

Question-answer match

The question-match signal q_{match} passes the question and the answer string to the cross-encoder, which scores how well the text matches the question in the way a search engine scores a passage. It was designed as its own family to catch an answer that a passage supports but that does not address the question. On the short answers of this panel it cannot do that, for two reasons:

- **The answer string carries too little of the question:** the cross-encoder sees the short answer, wrapped as "Answer: Rudolph." or replaced by the text of the chosen option. A bare name says nothing about the question, so a correct, on-topic answer scores low: for "What was President Gerald Ford's middle name?", the string "Answer: Rudolph." scores 0.023, while the sentence "Gerald Ford's middle name was Rudolph." scores 0.999. Rewriting answers as full sentences does not help, because the rewrite places every answer on the topic: "Gerald Ford's middle name was the Eiffel Tower." scores 0.978.
- **The stored value is converted twice:** the cross-encoder already returns a probability, and the verifier applies a second logistic function to it, so every stored value lies between 0.50 and 0.73. The composite is not affected, because its calibration curves depend only on the order of the values, but no fixed cut-off below 0.50 can ever be reached.

What the signal does carry is information about correctness, of a strength that depends on the benchmark. On the evaluation fold at C0, its area under the ROC curve for separating correct from incorrect answers is 0.80 on TriviaQA (0.68 when refusals, which score lowest, are excluded), 0.60 on StrategyQA, 0.57 on TruthfulQA, 0.55 on CommonsenseQA, and 0.36 on FEVER, where higher scores go with wrong answers. The composite learns the direction and strength of this relation on the calibration fold, which is why the signal received the largest weight of the nine in the cycle-zero fit (0.491, Section 4.2.7). The signal is therefore best read as a learned correctness cue, not an off-topic detector; detecting off-topic answers is left to future work (Chapter 6).

4.2.7 Calibrated probability composite

The composite turns the nine signals into one probability of correctness, u_{stored}, in three steps (Figure 4.5, Algorithm 5).

1. **One curve per signal:** isotonic regression maps each raw signal to the probability that the answer is correct.
2. **Per benchmark, with shrinkage:** the curves are also fitted on each training benchmark and pulled part of the way toward the pooled curve, so that benchmarks with little data do not overfit.
3. **Weighted sum:** a regularised logistic regression combines the nine calibrated probabilities, in log-odds, into u_{stored}.

All three steps are fitted on the calibration fold (500 labelled questions from each training benchmark) and refitted at every cycle boundary on the same fold re-scored with the new adapter.

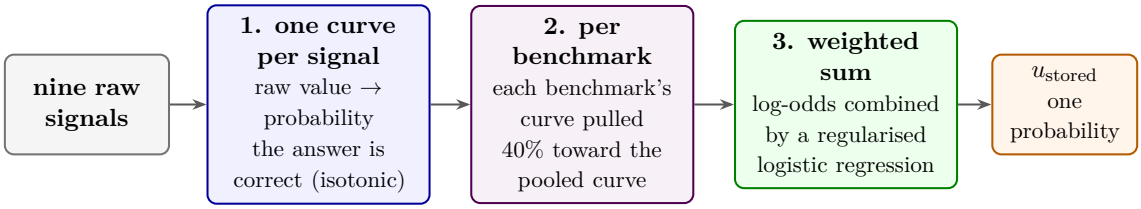


Figure 4.5: The calibrated probability composite in three steps. All three are refitted at every cycle boundary on the re-scored calibration fold.

Algorithm 5 Calibrated probability composite (fit and predict).

Fit stage: calibration fold split by benchmark, $\mathcal{D}_b$ for $b \in \mathcal{B}$, each a set of signal vectors $x \in \mathbb{R}^9$ with exact-match labels $y \in \{0, 1\}$; pooled fold $\mathcal{D} = \bigcup_b \mathcal{D}_b$.

for each signal j **do**
    direction $\leftarrow$ sign of the Pearson correlation of x_j with y on $\mathcal{D}$
    fit an isotonic curve iso_j in that direction on $\mathcal{D}$, with outputs clipped to $[0.02, 0.98]$
end for
for each benchmark b and signal j **do**
    fit iso_j^b on $\mathcal{D}_b$; at each knot x set $\text{iso}_j^b(x) \leftarrow 0.6\,\text{iso}_j^b(x) + 0.4\,\text{iso}_j(x)$
end for
$L_{ij} \leftarrow \text{logit}\big(\text{iso}_j(x_{ij})\big)$ on $\mathcal{D}$; fit logistic regression (ℓ_2 penalty, $C = 0.01$) of y on L to get weights $\mathbf{w}$ and intercept b_0
repeat the logistic fit on each $\mathcal{D}_b$ with the shrunk curves to get $(\mathbf{w}_b, b_{0,b})$

Predict stage: signal vector x and benchmark tag $b^\star$

use the curves and weights of $b^\star$ if $b^\star \in \mathcal{B}$, otherwise the pooled ones
return $u_{\text{stored}} \leftarrow \sigma\Big(b_0 + \sum_j w_j \,\text{logit}\big(\text{iso}_j(x_j)\big)\Big)$

Per-signal isotonic regression

Isotonic regression fits any monotone curve, not only a logistic one, which matters because some signals saturate near the top of their range; and it needs less data than histogram binning because neighbouring bins share information. Niculescu-Mizil and Caruana [8] found isotonic regression the stronger post-hoc calibrator when enough data are available, but prone to overfitting below roughly 200 to 1,000 calibration points, where Platt scaling does better. The per-benchmark folds here hold 500 points each, which is one reason the per-benchmark curves are shrunk toward the pooled curve (Section 4.2.7). The direction of each curve is taken from the sign of the signal's correlation with correctness, so a signal where high means unsafe (such as u_{dropout}) needs no special handling. Probabilities are clipped to $[0.02, 0.98]$ so that no single signal can dominate the log-odds sum, and a signal with fewer than 50 calibration samples is left out.

Per-benchmark fit with shrinkage prior

Each training benchmark gets its own curves, blended at every knot with the pooled curve: 60 percent benchmark curve, 40 percent pooled curve, a convex blend in the spirit of James–Stein shrinkage. Each benchmark also gets its own logistic weights. The blend strength was chosen to balance two effects visible in the cycle-zero sweep: on the held-out validation fold, fitting each benchmark more freely (α_{shrink} closer to 1) stored more answers but separated correct from incorrect answers less well (pooled Cohen's d of 0.85 at $\alpha_{\text{shrink}} = 0$, 0.72 at 0.6 and 0.66 at 1; Chapter 5). A query from a benchmark outside the training panel uses the pooled curves and weights.

Boost layer with L2 regularisation

The boost layer is a logistic regression on the nine log-odds, following the multi-scoring approach of Valentin et al. [83]. A learned combination is used rather than an equal-weight sum because several signals overlap (the two sample-set signals, and the three grounding signals), and the regression can discount the overlap. An ℓ_2 penalty is used rather than ℓ_1 so that weights shrink smoothly without any signal being dropped, and its strength is strong ($C = 0.01$, chosen in the cycle-zero sweep of Chapter 5). In the cycle-zero pooled fit every weight is positive; the largest are question match (0.491), best-passage grounding (0.379) and internal confidence (0.334). The question-match weight reflects its value as a correctness cue on the calibration fold, not detection of off-topic answers (Section 4.2.6).

4.2.8 Fixed-threshold storage gate

The storage gate applies two fixed cut-points to u_{stored}: an answer is stored if $u_{\text{stored}} \geq 0.60$ and deferred if $0.45 \leq u_{\text{stored}} < 0.60$; below 0.45, the decision rule of Section 4.2.9 chooses between abstaining and discarding. The cut-points were fixed before the main run and never changed.

An earlier design set the cut-points by split-conformal prediction at fixed precision targets and refitted them every cycle. It was replaced because conformal guarantees assume the calibration and evaluation data come from the same distribution, and on this panel they did not: precision on the evaluation fold fell well below the

conformal targets, and on the weakest benchmarks the per-benchmark fits rejected almost everything. A fixed cut-point on a calibrated probability, in the spirit of the post-hoc calibration of Niculescu-Mizil and Caruana [8] and the selective classification of Geifman and El-Yaniv [17], makes no coverage promise; its precision is simply measured on every fold. It still assumes that calibration and test data are alike, so under distribution shift the realised precision can fall, as Ovadia et al. [26] and Kamath et al. [31] show; TruthfulQA in Chapter 5 is such a case.

Storage threshold

The storage cut-point $\tau_{\text{store}} = 0.60$ was chosen on the cycle-zero calibration fold before the main run, against the stricter alternative 0.65, which stored about half as many answers (Chapter 5 reports the sweep). The storage class fills the memory and supplies the training pool, so it is guarded on both sides: training requires the stricter $u_{\text{stored}} \geq 0.70$, and the retroactive pass removes entries that later fall below 0.50.

Deferral threshold

Answers with $0.45 \leq u_{\text{stored}} < 0.60$ go to the deferred buffer, not to memory. They are re-scored at the next cycle boundaries (Section 4.2.10) and either promoted or dropped, so a looser cut-point here costs little: deferred answers never reach the training pool unless they are later promoted. The configuration refuses a deferral cut-point above the storage cut-point.

Cycle-boundary recalibration

The cut-points (0.60 and 0.45), the abstention floor (0.20), the retroactive prune threshold (0.50) and the training threshold (0.70) stay fixed for the whole run. What changes at each cycle boundary is the calibration: after a committed fine-tune, the same calibration fold is re-scored with the new adapter, the pre-routing temperatures and safety floors are refitted, and the composite is refitted. The verifier is then reloaded with the new composite before the retroactive pass, so stored entries are re-scored with curves that match the current adapter. Without this order, entries whose raw signals had shifted after fine-tuning would be read through stale curves and wrongly pruned.

Keeping the cut-points fixed while the composite adapts is what lets Hypothesis 1 (Section 1.4.3) test the verifier rather than a moving operating point. A deployed system would run the same refit; the only difference is that its cycle would end on a calendar or accumulation trigger, with a labelled sample of recent admissions as the calibration fold.

4.2.9 Four-outcome storage decision

The decision rule maps u_{stored} and two guard signals to one of four outcomes (Algorithm 6). Four outcomes are used instead of store-or-discard for two reasons. First, an answer just below the storage cut-point is not the same as one near zero: deferring it lets the next cycle's adapter settle it. Second, *I have nothing reliable to say* is a different situation from *I have an answer but cannot ground it*: abstention makes

the first an explicit, user-facing refusal, in the spirit of the abstention framework of Yadkori et al. [85].

Confabulation early exit: checked first, at query time only. An answer the generator is internally confident about but that no passage supports is discarded:

$$\Big(u_{\text{internal}} \geq 0.70\Big) \;\wedge\; \Big(p_{\text{ground,max}} \leq 0.20\Big) \;\Rightarrow\; \textsc{Discard}. \tag{4.1}$$

This catches the most damaging case, a confident answer with no evidence, which the composite alone might score as merely borderline. In the main run the rule fired on none of the 1,500 evaluation answers at C0 and on 3 to 23 per cycle afterwards.

Thresholds: otherwise the answer is stored if $u_{\text{stored}} \geq 0.60$ and deferred if $u_{\text{stored}} \geq 0.45$. Below 0.45, the system abstains when the best passage barely supports the answer ($p_{\text{ground,max}} < 0.20$) and discards otherwise. Every answer reaches the user with its probability u_{stored} and a status: a stored answer is marked *verified*; a deferred one *provisional*, with a note that it will be reconsidered at the next cycle; a discarded one *conflicting*, with a warning that the evidence disagrees and independent checking is recommended. Neither a discarded nor an abstained answer is stored, and an abstention replaces the answer with an insufficient-evidence message.

Algorithm 6 Four-outcome storage decision (Stage 6).

Require: u_{stored}, u_{internal}, $p_{\text{ground,max}}$; cut-points $\tau_{\text{store}} = 0.60$, $\tau_{\text{defer}} = 0.45$; early-exit thresholds $\eta_u = 0.70$, $\eta_g = 0.20$; abstention floor $\phi_{\text{pg}} = 0.20$

if query time **and** $u_{\text{internal}} \geq \eta_u$ **and** $p_{\text{ground,max}} \leq \eta_g$ **then**
  return DISCARD ▷ confabulation early exit
end if
if $u_{\text{stored}} \geq \tau_{\text{store}}$ **then**
  return STORE
end if
if $u_{\text{stored}} \geq \tau_{\text{defer}}$ **then**
  return DEFER
end if
if $p_{\text{ground,max}} < \phi_{\text{pg}}$ **then**
  return ABSTAIN
end if
return DISCARD

4.2.10 Cycle-boundary maintenance

The cycle-boundary steps are where the two kinds of statelessness of Section 1.2.3 are addressed. Fine-tuning on verified answers, many produced with retrieval, is meant to let the generator answer later questions of the same kind without retrieval. The memory is meant to serve repeated questions at the cost of a lookup, with re-verification keeping each stored score in line with the current model. The deferred pass gives borderline answers a second chance under the new adapter. The steps run in the order of Figure 4.1; on a cycle whose fine-tune is rolled back, all steps after the retention guard are skipped.

Retroactive re-verification

The retroactive pass re-scores every stored episode with the current adapter and the refitted composite (Algorithm 7). An entry is removed if its reasoning chain is caught by the repetition filter or if its new u_{stored} is below 0.50. Otherwise all nine stored signals and u_{stored} are overwritten with the new values, which may be lower or higher than before. The prune threshold sits below the deferral cut-point, so an entry that is no longer storage-quality but still plausible is kept rather than discarded. The pass removed 56, 271, 352 and 382 entries at cycles one, two, three and five (it was skipped at cycle four), that is 13, 9, 7 and 4 percent of the entries it checked.

Algorithm 7 Retroactive re-verification pass (Stage 8).

Require: memory $\mathcal{M}$, verifier with the current adapter and refitted composite, prune threshold $\tau_{\text{retro}} = 0.50$

for each entry $e \in \mathcal{M}$ **do**
  if the reasoning chain of e is caught by the repetition filter **then**
    remove e; **continue**
  end if
  $\mathbf{x} \leftarrow \text{verify}(e.\text{question}, e.\text{answer})$ ▷ early exit disabled
  if $\mathbf{x}.u_{\text{stored}} < \tau_{\text{retro}}$ **then**
    remove e
  else
    overwrite the signals and u_{stored} of e with $\mathbf{x}$ ▷ may lower or raise u_{stored}
  end if
end for

Deferred-entry reconsideration

Each deferred answer is re-scored at the next cycle boundaries (Algorithm 8). It is promoted into memory if the verifier now decides to store it ($u_{\text{stored}} \geq 0.60$); it is dropped once it has been reconsidered twice without promotion; otherwise it waits for the next pass. The buffer holds at most 10,000 answers and evicts the oldest when full. The passes promoted 131, 220 and 299 answers at cycles two, three and five; the first pass, at cycle one, found an empty buffer, and the pass was skipped at cycle four. The memory consolidation of Section 4.2.3 runs straight after this pass.

Algorithm 8 Deferred-entry reconsideration pass (Stage 8).

Require: deferred buffer $\mathcal{B}$, current verifier, $\tau_{\text{store}} = 0.60$, time-to-live $\tau_{\text{ttl}} = 2$ passes

for each entry $e \in \mathcal{B}$ **do**
  $\mathbf{x} \leftarrow \text{verify}(e)$
  if $\mathbf{x}.\text{decision} = \textsc{Store}$ **and** $\mathbf{x}.u_{\text{stored}} \geq \tau_{\text{store}}$ **then**
    move e into the memory
  else if $e.\text{age} + 1 \geq \tau_{\text{ttl}}$ **then**
    drop e
  else
    $e.\text{age} \leftarrow e.\text{age} + 1$ ▷ wait for the next pass
  end if
end for

Self-improvement loop

The self-improvement loop fine-tunes the generator on its own verified answers. It belongs to the family of methods that train a model on its filtered outputs [43], [48], [73], and is related to distilling a teacher's filtered reasoning into a student [49]. Its intended role is to turn verified retrieval-based reasoning into knowledge the generator holds without retrieval. Song et al. [93] find that such loops need a positive gap between how well a model generates and how well it can verify, and saturate within a few rounds; Yi et al. [96] show in linear regression that filtering through an external verifier prevents collapse but pulls the model toward the verifier's own knowledge. CAEM's external verifier and fixed cut-points are meant to keep the training pool above the purification condition of Proposition 4.2. The loop has five steps (Algorithm 9).

Select: take stored episodes with $u_{\text{stored}} \geq 0.70$ from the three training benchmarks only, so transfer benchmarks never reach the gradient. The training target is the stored reasoning chain and answer; most chains were written under the retrieval prompt.

Rebalance: reweight the pool so that no benchmark dominates: per-benchmark counts are mixed with a softmax at temperature 2, upsampling is capped at three times a benchmark's count, each benchmark contributes at least 50 examples, a benchmark with no verified episodes is given 100 gold-labelled examples from its training split, and no benchmark exceeds 40 percent of the pool. This addresses the failure of an earlier run, in which one benchmark grew to the whole training pool.

Train: train a low-rank adapter (LoRA, rank 32, scaling 64, dropout 0.05) on all seven linear projections of every transformer block, following the all-layer recommendation of Biderman et al. [65]; the backbone stays frozen. Training runs for 3 epochs with 32-bit AdamW at a learning rate of 2×10^{-4}, an effective batch of 16 and gradient checkpointing. The loss is cross-entropy alone:

$$\mathcal{L}(\boldsymbol{\theta}) \;=\; \mathcal{L}_{\text{CE}}(\boldsymbol{\theta}) \;+\; \tfrac{\lambda_{\text{anc}}}{2} \left\| \boldsymbol{\theta} - \boldsymbol{\theta}^{(t-1)} \right\|_2^2, \qquad \lambda_{\text{anc}} = 0, \tag{4.2}$$

where $\boldsymbol{\theta}$ are the adapter parameters and $\boldsymbol{\theta}^{(t-1)}$ the previous cycle's adapter. The anchor term, a uniform-weight simplification of the elastic weight consolidation of Kirkpatrick et al. [20], belongs to the full fine-tuning path used in an earlier run and is switched off here: the frozen backbone and the small adapter already limit how far each cycle can move the model, and in the earlier run the anchor outweighed the learning signal from the small training pool.

Check retention: before the first cycle, record the accuracy of the untouched model on three held-out probes of 200 questions each: MMLU (general knowledge), the TriviaQA test fold (open-text facts) and the CommonsenseQA test fold (commonsense multiple choice). After each fine-tune, compute each probe's accuracy as a ratio of that starting value (Equation (3.2)).

Commit or roll back: keep the new adapter only if every ratio is at least 0.93; otherwise restore the previous adapter. The memory is kept either way. Requiring every probe, not just the average, to pass is deliberate: an earlier rule that watched only MMLU missed forgetting on narrower tasks. In the main run the guard fired once, at cycle four, when the TriviaQA test ratio fell to 0.886.

Algorithm 9 Self-improvement loop with retention guard (Stage 8).

Require: memory $\mathcal{M}$, previous adapter $\boldsymbol{\theta}^{(t-1)}$, probes $\mathcal{P}$ with starting accuracies A_0^p, $\tau_{\text{train}} = 0.70$, $\rho_{\min} = 0.93$

$\mathcal{T} \leftarrow \text{rebalance}\big(\{e \in \mathcal{M} : e.u_{\text{stored}} \geq \tau_{\text{train}},\ e \text{ from a training benchmark}\}\big)$

$\boldsymbol{\theta}^{\star} \leftarrow$ train the adapter on $\mathcal{T}$ with $\mathcal{L}_{\text{CE}}$, starting from $\boldsymbol{\theta}^{(t-1)}$

if $\min_{p \in \mathcal{P}} A^p(\boldsymbol{\theta}^{\star})/A_0^p \geq \rho_{\min}$ **then**

    return $\boldsymbol{\theta}^{\star}$ ▷ commit

else

    return $\boldsymbol{\theta}^{(t-1)}$ ▷ roll back; memory kept; later cycle-boundary steps skipped

end if

Keeping the memory through a rollback is what makes the loop safe to repeat: a failed cycle costs its fine-tuning time but not the answers it verified. This asymmetry underlies the memory part of Heuristic 4.9.

4.3 Theoretical analysis

This section states what can be said formally about the stored pool, and is explicit about the strength of each statement (Figure 4.6). Only two results are proven: a floor on the purity of the stored pool (Theorem 4.1) and the condition under which filtering makes the pool purer than the generator's raw output (Proposition 4.2). Four remarks are exact calculations on simplified models; they show what the design aims at, not what the system guarantees. Five heuristics describe how the pool is expected to evolve across cycles; they rest on assumptions the real system does not fully meet, so they come with argument sketches rather than proofs, and the evaluation checks them only for consistency.

All statements are about the stored pool. None bounds the accuracy of the fine-tuned generator on new questions, so a fall in held-out accuracy is outside their scope rather than a contradiction of them.

The observed horizon: the main run produced six cycles (zero to five) and stopped after cycle five, as planned after the cycle-four rollback (Section 3.3.2). Statements about long-run behaviour are therefore read against these six cycles, which are too few to estimate limits or rates.

4.3.1 Pool purity

The *purity* of the stored pool is the share of stored answers that are correct. At each cycle boundary the pool is made of two parts: older entries that survived re-verification, and new entries admitted by the gate during the cycle.

Theorem 4.1 (Pool purity floor)**.** *Let π_{t+1} be the expected purity of the stored pool at the end of cycle $t+1$. Assume:*

(i) ***Fixed gate:*** *new answers are stored when $u_{stored} \geq \tau_{store}$, with τ_{store} fixed across cycles.*

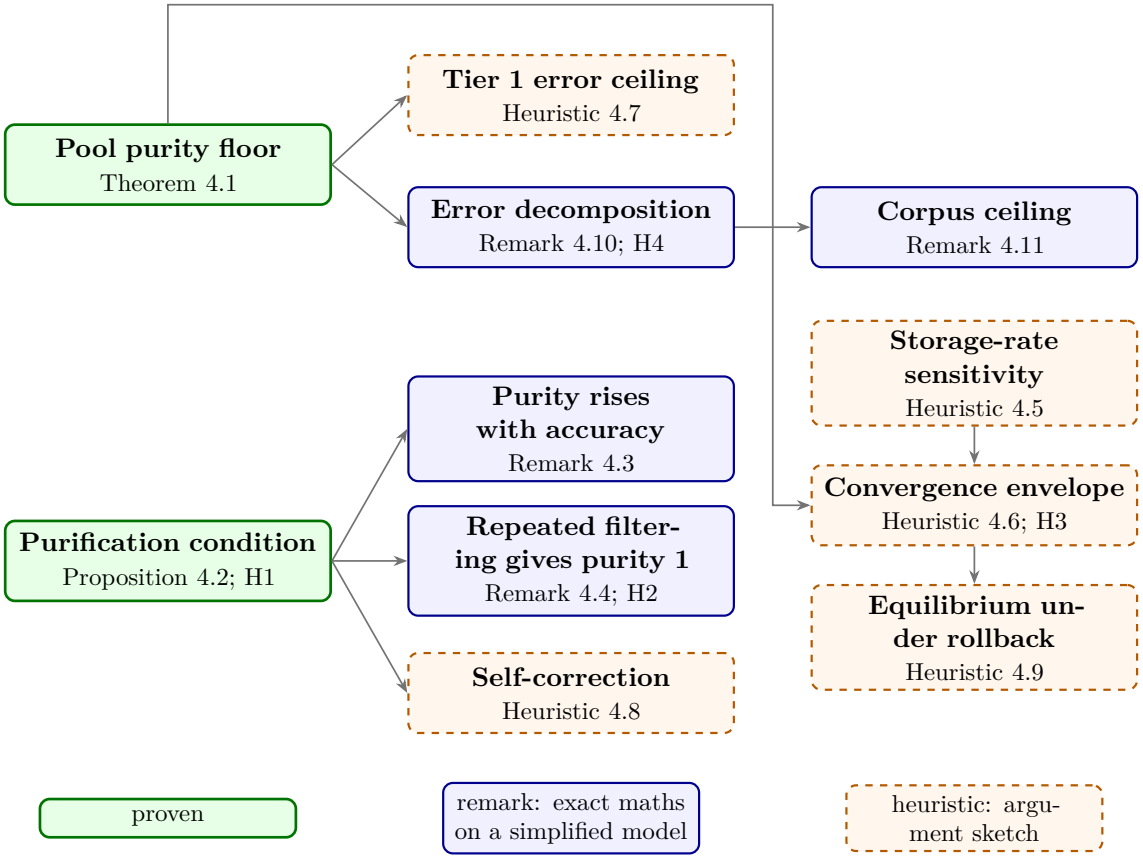


Figure 4.6: Status of the formal statements and the hypotheses (H1 to H4) they inform. Arrows point from a statement to those that build on it.

(ii) ***Like calibration data:*** *the cycle's candidate answers come from the same distribution as the calibration fold, so the expected purity of the new entries equals* π_{store}*, the share of correct answers among calibration-fold answers with* $u_{stored} \geq \tau_{store}$*.*

(iii) ***Survivors:*** *the older entries that survive re-verification (score at least* τ_{retro}*) have expected purity* π_{retro}*.*

Then

$$\pi_{t+1} \;\geq\; \min\Big(\pi_{store},\, \pi_{retro}\Big). \tag{4.3}$$

Proof. Let the pool contain n_{new} new entries and n_{old} surviving older entries. Its expected purity is the weighted average

$$\pi_{t+1} \;=\; \frac{n_{\text{new}}\,\pi_{\text{store}} + n_{\text{old}}\,\pi_{\text{retro}}}{n_{\text{new}} + n_{\text{old}}},$$

using (i) and (ii) for the new entries (because the cut-point is the same in every cycle, the calibration-fold precision at that cut-point applies to each cycle's admissions) and (iii) for the survivors. A weighted average of two numbers is never smaller than the smaller of them, which gives Equation (4.3). □

In plain terms: a mixture of two groups is at least as clean as the dirtier group. The result is simple; its value is that both inputs can be measured. Because the gate never changes (assumption (i)), the calibration-fold precision π_{store} can be read at every cycle, and the precision of the new entries can be checked directly on the stream chunk, whose answers are labelled. The floor holds in expectation, up to the sampling error of the calibration estimate, and only as far as assumption (ii) holds; for a benchmark whose questions differ from the calibration fold, it says nothing.

What the run shows: on the training panel, the calibration-fold precision at the cut-point was between 0.732 and 0.746 in cycles one to five, and the precision of the answers actually stored from each stream chunk was 0.753, 0.751, 0.724, 0.738 and 0.724, within about one point of it; assumption (ii) approximately held on the training panel. It did not hold on the transfer benchmark TruthfulQA, where the answers the gate stored on the evaluation fold were correct only 31 to 38 percent of the time at cycles one to five (Table C.7). Per-cycle readings are in Table 5.14.

4.3.2 When filtering helps: the purification condition

The floor says how pure the pool is relative to the calibration fold. The next result says when the gate makes the pool purer than the generator's unfiltered answers at all.

Proposition 4.2 (Purification condition)**.** *Let* $p_c \in (0,1)$ *be the share of the generator's answers that are correct at cycle* c*, and let* TPR_c *and* FPR_c *be the shares of correct and of incorrect answers that the gate stores. The purity of the stored answers is*

$$P_c \;=\; \frac{p_c\,\mathrm{TPR}_c}{p_c\,\mathrm{TPR}_c \;+\; (1-p_c)\,\mathrm{FPR}_c}, \tag{4.4}$$

and $P_c > p_c$ if and only if $\mathrm{TPR}_c > \mathrm{FPR}_c$, which is the same as the gate's balanced accuracy $\alpha_c = (\mathrm{TPR}_c + 1 - \mathrm{FPR}_c)/2$ exceeding one half. If the gate errs equally on both classes ($\mathrm{TPR}_c = 1 - \mathrm{FPR}_c = \alpha_c$), then $P_c = p_c\alpha_c / \big(p_c\alpha_c + (1-p_c)(1-\alpha_c)\big)$.

Proof. Let $Y \in \{0,1\}$ mark a correct answer and $S \in \{0,1\}$ a stored one, so $\Pr(Y=1) = p_c$, $\Pr(S=1 \mid Y=1) = \mathrm{TPR}_c$ and $\Pr(S=1 \mid Y=0) = \mathrm{FPR}_c$. Bayes' rule gives $P_c = \Pr(Y=1 \mid S=1)$ as in Equation (4.4). Then $P_c > p_c$ means $\mathrm{TPR}_c > p_c\mathrm{TPR}_c + (1-p_c)\mathrm{FPR}_c$, that is $(1-p_c)\mathrm{TPR}_c > (1-p_c)\mathrm{FPR}_c$, which for $p_c < 1$ is $\mathrm{TPR}_c > \mathrm{FPR}_c$. Finally $\alpha_c > 1/2$ means $\mathrm{TPR}_c + 1 - \mathrm{FPR}_c > 1$, again $\mathrm{TPR}_c > \mathrm{FPR}_c$. □

In plain terms: storing purifies only if the gate accepts correct answers more often than wrong ones. A gate that is only a little better than chance still helps, and a gate worse than chance makes the pool dirtier than the generator (Figure 4.7). This is why the verifier's balanced accuracy is the precondition tested by Hypothesis 1. In the language of learning with noisy labels, the condition is the class-conditional noise condition $\rho_+ + \rho_- < 1$ of Natarajan et al. [10], with $\rho_+ = 1 - \mathrm{TPR}_c$ and $\rho_- = \mathrm{FPR}_c$; their learning guarantees also need a noise-corrected loss, which CAEM does not use, so only the condition is borrowed. Related analyses of retraining on self-predicted labels [87] and on weak-teacher labels [77] rest on the same idea. The labelled per-cycle measurement of TPR_c, FPR_c and p_c needed to check Equation (4.4) directly was not run and is left as future work (Chapter 6).

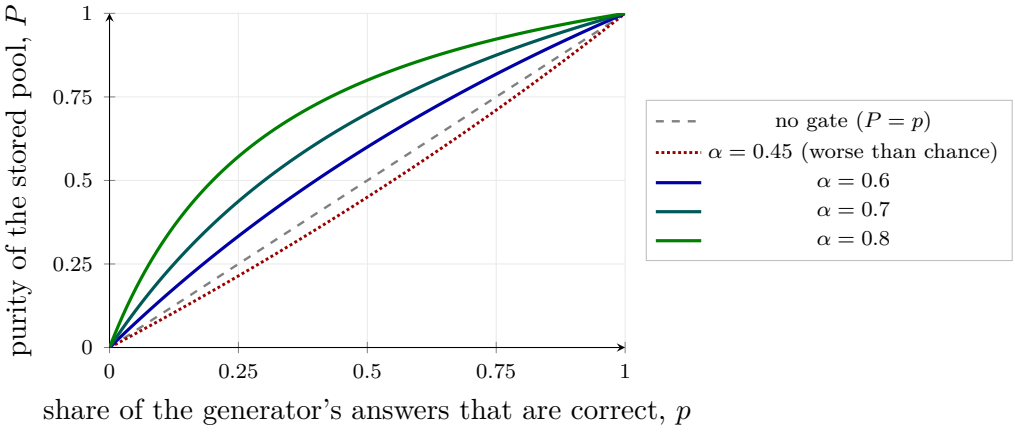


Figure 4.7: How the purity of the stored pool depends on the gate's balanced accuracy α, from Equation (4.4) with $\mathrm{TPR} = 1 - \mathrm{FPR} = \alpha$. Curves above the diagonal ($\alpha > 1/2$) make the pool purer than the generator; the dotted curve ($\alpha < 1/2$) makes it dirtier.

Two exact consequences of Equation (4.4) follow; both are about an idealised gate, not guarantees about the system.

Remark 4.3 (Purity rises with the generator's accuracy). For a gate with fixed $\alpha \in (0,1)$ that errs equally on both classes,

$$\frac{\partial P_c}{\partial p_c} = \frac{\alpha(1-\alpha)}{\big(p_c\alpha + (1-p_c)(1-\alpha)\big)^2} > 0, \tag{4.5}$$

so a more accurate generator always yields a purer pool. (Writing $D = p_c\alpha + (1 - p_c)(1-\alpha)$, the quotient rule gives $\partial P_c/\partial p_c = \alpha\,[D - p_c(2\alpha-1)]/D^2 = \alpha(1-\alpha)/D^2$.) In the run, α changes from cycle to cycle with the refitted composite and the range of p_c is narrow, so this remark is not tested separately.

Remark 4.4 (Repeated filtering drives purity toward one). Suppose the stored pool were filtered again at every cycle by a gate of fixed balanced accuracy $\alpha > 1/2$ that errs equally on both classes and whose errors are independent from one pass to the next. Then the purity follows $P_{c+1} = f(P_c)$ with $f(P) = P\alpha/\Big(P\alpha + (1-P)(1-\alpha)\Big)$. The map f is increasing, its only fixed points are 0 and 1, and $f(P) > P$ for every P in $(0, 1)$. A sequence that is increasing and bounded by one converges, and its limit must be a fixed point above P_0; hence $P_c \to 1$.

How far the model is from the system: the real loop does not re-filter the same pool with a fixed, independent gate. The composite is refitted every cycle, some verifier errors repeat at every pass, and the pool also receives new entries. The remark therefore states the direction the design aims at. Hypothesis 2, which expects purity not to fall across cycles, is tested against it in Chapter 5.

4.3.3 Design heuristics for the cycle-to-cycle trajectory

The five statements below describe how the pool is expected to change over cycles. Each uses at least one of three simplifications: scores that are normally distributed within the correct and incorrect classes with equal spread; a storage rate and verifier quality that stay stable across cycles; and verifier errors that are independent from one pass to the next. The real system breaks each of these (the composite is refitted every cycle, the spread of scores shrinks over cycles, and some errors are systematic), and six cycles are too few to estimate the long-run quantities involved. They are therefore heuristics, and the evaluation reads them as consistency checks.

Storage-rate sensitivity

Heuristic 4.5 (How the storage rate moves with the verifier's separation)**.** Let d_t be the Cohen's d between the scores of correct and incorrect calibration-fold answers at cycle t, a measure of how well the verifier, seen through the cycle's refitted composite, separates them, and let σ_t be the share of answers stored at the fixed cut-point τ_{store}. Assume the scores are normal within each class with common spread ς and class means $\mu_\pm = m \pm d\varsigma/2$ around a fixed midpoint m. Let p_+, p_- be the shares of correct and incorrect answers and $\phi_\pm = \phi\Big((\mu_\pm - \tau_{\text{store}})/\varsigma\Big)$ the class densities at the cut-point. Then $\partial\sigma/\partial d$ has the sign of $p_+\phi_+ - p_-\phi_-$: when the correct class is denser at the cut-point, better separation stores more answers.

Argument sketch. The storage rate is $\sigma = p_+\Phi\Big((\mu_+ - \tau_{\text{store}})/\varsigma\Big) + p_-\Phi\Big((\mu_- - \tau_{\text{store}})/\varsigma\Big)$. Since $\partial\mu_\pm/\partial d = \pm\varsigma/2$, differentiating gives $\partial\sigma/\partial d = \frac{1}{2}\Big(p_+\phi_+ - p_-\phi_-\Big)$.

What the run shows: the calibration fold gives four readings (cycles one, two, three and five; cycle four was rolled back). From cycle one to two, separation and storage rate moved together, as predicted; from two to three the change in d was too small to read; from three to five the storage rate fell slightly while d rose. Two effects

outside the model explain the last case: the composite is refitted every cycle, and the spread of scores shrank rather than staying fixed. Separating them would need a cycle in which the composite is frozen and only the generator changes (Chapter 6).

Convergence

Heuristic 4.6 (Convergence envelope)**.** Suppose that the purity of the storage class settles to a long-run value π_∞, and let $G_t = |\pi_t - \pi_\infty|$ be the gap at cycle t. Under the assumptions of Theorem 4.1 and Heuristic 4.5, and assuming that the storage rate σ and the separation d are stable across cycles, there is a rate $r \in (0, 1)$ such that, with t counting only the cycles whose fine-tune was kept,

$$\mathbb{E}[G_t] \;\le\; (1 - r)^t \, G_0. \tag{4.6}$$

Argument sketch. Each cycle adds new entries at purity π_{store} (Theorem 4.1) and re-scores the old ones. If the share of the gap closed in a cycle is roughly constant, the gap shrinks by a fixed factor $1 - r$ each cycle. Under the normal model, the share closed grows with the storage rate and with the probability that the verifier ranks a correct answer above an incorrect one, $\Phi(d/\sqrt{2})$, which suggests r roughly proportional to $\sigma\,\Phi(d/\sqrt{2})$. A proof would need the stochastic-approximation tools of Robbins and Monro [3], the almost-supermartingale theorem of Robbins and Siegmund [4] (used for verifier-filtered retraining by Yi et al. [96]) and the rate analysis of Polyak [7]; it is left as future work.

What the run shows: six cycles cannot estimate a geometric rate, so the testable reading is weaker: the calibration-fold precision stayed in a band of width 0.014 (0.732 to 0.746), comparable to the sampling noise of each reading, with no sign of drifting out of it. A direct test of the rate would need at least fifteen kept cycles under a fixed composite (Chapter 6). Hypothesis 3 is tested against this heuristic in Chapter 5.

Asymptotic hallucination floor

Heuristic 4.7 (Tier 1 error ceiling)**.** Let $H_1(t)$ be the share of wrong answers among those served by the memory-direct tier at cycle t. Assume that a query close enough to reach Tier 1 has the same correct answer as the stored question it matches, and that the entries Tier 1 serves are at least as often correct as the pool as a whole (plausible, because Tier 1 also requires a high stored score). Then, under Theorem 4.1,

$$\limsup_{t \to \infty} \; H_1(t) \;\le\; \max\Big(1 - \pi_{\text{store}},\, 1 - \pi_{\text{retro}}\Big). \tag{4.7}$$

Argument sketch. Under the first assumption, a Tier 1 answer is wrong exactly when the stored answer is wrong; under the second, the served entries are wrong no more often than the pool, so $H_1(t) \le 1 - \pi_t$. By Theorem 4.1, $\pi_t \ge \min(\pi_{\text{store}}, \pi_{\text{retro}})$ at every cycle, so $1 - \pi_t \le \max(1 - \pi_{\text{store}}, 1 - \pi_{\text{retro}})$ at every cycle, and therefore also in the limit. The first assumption is the weak point: a paraphrase that changes the answer would be served a wrong answer from a correct entry, which is why Tier 1 requires a very close match ($s \ge 0.857$).

What the run shows: on the evaluation fold, whose questions never repeat stored ones, Tier 1 answered 7 of 9,000 queries, all correctly, so the ceiling cannot be observed there. The router probe of Table 5.8 supplies the testable case: Tier 1 answered 41 of 42 exact repeats and all 19 answered paraphrases correctly. On the training panel the error rate of newly stored entries (0.247 to 0.276) is close to $1 - \pi_{\text{store}}$, as assumption (ii) of Theorem 4.1 predicts.

Equilibrium structure and self-correction

Heuristic 4.8 (Self-correction under repeated re-verification)**.** Consider a stored entry that is re-verified at every later cycle boundary. Assume that at each pass a correct entry survives with probability at least α and a wrong entry with probability at most $1 - \alpha$, for a fixed $\alpha > 1/2$, and that the verdicts of different passes are independent given whether the entry is correct. Then (i) the probability that an entry is correct, given that it has survived k passes, tends to one as k grows; and (ii) a wrong entry is removed after at most $1/\alpha$ passes on average.

Argument sketch. (i) The likelihood ratio of surviving k passes for a correct versus a wrong entry is at least $\big(\alpha/(1-\alpha)\big)^k$, which grows without bound because $\alpha/(1-\alpha) > 1$. By Bayes' rule, $\Pr(\text{correct} \mid \text{survived } k) = \Lambda_k q/(\Lambda_k q + 1 - q)$, where q is the prior probability of being correct and $\Lambda_k \geq \big(\alpha/(1-\alpha)\big)^k$ the likelihood ratio; this tends to one. (ii) A wrong entry is removed at each pass with probability at least α, so the number of passes until removal is at most geometric with mean $1/\alpha$.

Where the assumption fails: the independence assumption is the strong one. If the model and the verifier share a misconception, as Farquhar et al. [71] warn for systematic errors, the same wrong answer is accepted at every pass and repetition removes nothing; purity then rises only toward a ceiling set by such shared errors, which matches the finding of Yi et al. [96] that verifier-filtered retraining converges to the verifier's knowledge rather than to the truth. Part (ii) also shows how idealised the model is: with $\alpha > 1/2$ it predicts removal within two passes on average.

What the run shows: the share of checked entries removed by the retroactive pass fell from 13 percent at cycle one to 9, 7 and 4 percent at cycles two, three and five, consistent with errors being cleared over time, though partly diluted by the growing pool of fresh entries. The entry records do not count how many passes each entry survived, so the predicted survival distribution itself could not be measured.
Heuristic 4.9 (Equilibrium under retention rollback)**.** Let $X_t = 1$ if the cycle-t fine-tune passes the retention guard and $X_t = 0$ if it is rolled back, and let $\pi_{\text{commit}} = \mathbb{E}[X_t]$ be the commit rate.

(i) Whenever the worst probe ratio can land on either side of the floor, $0 < \pi_{\text{commit}} < 1$.

(ii) Counting all cycles, the envelope of Heuristic 4.6 becomes

$$\mathbb{E}[G_t] \;\leq\; (1 - \pi_{\text{commit}}\, r)^t\, G_0, \tag{4.8}$$

because a rolled-back cycle leaves the gap unchanged.

(iii) The memory keeps growing through rollbacks whenever the new admissions outnumber the removals, because answering the stream chunk does not depend on X_t; only the adapter waits for committed cycles.

(iv) When the worst probe ratio sits near the floor, commits and rollbacks alternate at a stable rate.

Argument sketch. A committed cycle shrinks the expected gap by the factor $1 - r$ and a rolled-back cycle leaves it unchanged, so $\mathbb{E}[G_{t+1} \mid G_t] = (1 - \pi_{\text{commit}}\, r)\, G_t$; iterating gives Equation (4.8). Part (iii) follows from the fact that the stream chunk is answered, and new entries stored, whether or not the cycle's fine-tune is kept. Part (iv) is an informal claim about a stationary state and is not proven.

What the run shows: parts (i) and (iii) can be checked directly. The commit sequence over cycles one to five was $1, 1, 1, 0, 1$, with worst probe ratios 1.006, 0.949, 0.976, 0.886 and 0.958, so the observed commit rate is $4/5$. The memory grew at every cycle, including across the cycle-four rollback (Table C.3). Part (ii) inherits the limits of Heuristic 4.6, and part (iv) would need at least fifteen kept cycles to test (Chapter 6); the ratios at cycles two to five lie within 0.07 of the floor, which is consistent with it but not evidence for it.

4.3.4 Remarks on the overall hallucination rate

Two further remarks place the memory's contribution within the error rate of the whole system. Neither is measured in this thesis.

Remark 4.10 (Decomposition of the overall error rate). Let $\tau_1(t)$ be the share of queries answered by Tier 1 at cycle t, $H_{\text{mem}}(t)$ the error rate of those answers (at most $1 - \pi_t$ under the assumptions of Heuristic 4.7), and $H_{\text{gen}}(t)$ the error rate of answers generated by Tiers 2 and 3. By the law of total probability, the overall error rate is

$$H_t \;=\; \Big(1 - \tau_1(t)\Big)\, H_{\text{gen}}(t) \;+\; \tau_1(t)\, H_{\text{mem}}(t). \tag{4.9}$$

The memory can lower the overall rate only through the second term, so its effect grows with the share of queries it answers. As π_t rises, the bound on H_{mem} falls, and the remaining error is set by the generator and its retrieval corpus on the questions the memory cannot answer.

Reading the remark: on the evaluation fold $\tau_1(t)$ was at most 0.1 percent, so the overall rate there is essentially the generator's. The remark becomes testable only in deployment, where real users repeat questions; Hypothesis 4 is read against it in Chapter 5.

Remark 4.11 (Ceiling set by the retrieval corpus). Let $K(C)$ be the set of questions whose answer can be supported by the corpus C. A system that answers every question and asserts only what C supports can be right on at most the questions in $K(C)$; on the rest any answer it gives is wrong, so it must abstain or err. The error left after the memory and the generator have done their best is therefore bounded below by the share of questions outside $K(C)$ that the system chooses to answer, a limit set by the corpus rather than by the architecture. Estimating it would require

checking, for every evaluation question, whether any retrieved passage contains the gold answer, which was not done (Chapter 6).

4.4 Chapter signpost

This chapter fixed every component, threshold and fitting procedure before the results were read. Chapter 5 measures them on the six-cycle run, the eight external baselines and the retrieval-utility-classifier comparison; it reports each pre-specified hypothesis as supported, partly supported or unsupported, checks the two proven results against their measurements, and reads the heuristics as consistency checks.

Chapter 5

Performance Evaluation and Analysis

5.1 Performance Evaluation

5.1.1 Experimental setup

Benchmarks and the training-versus-transfer split

Five benchmarks are used. The **training panel** is FEVER [22] (claim verification), TriviaQA [19] (open-domain trivia) and CommonsenseQA [29] (five-option commonsense questions). The **transfer panel** is TruthfulQA [40] (common misconceptions) and StrategyQA [34] (implicit multi-step yes-or-no questions). Only the training panel feeds the calibration fold, the seed pool, the stream chunks and the training pool; the transfer panel is evaluated only, to measure how the system behaves on questions unlike those it learns from. The retention guard uses three separate probes: MMLU [35] and held-out test slices of TriviaQA and CommonsenseQA. HotpotQA and Natural Questions were dropped before the main run because the base generator answered too few of their questions correctly (0.09 and 0.17) for the gate to build a usable pool (Section 1.6.1).

Each training benchmark is split into six disjoint folds (Table 5.1): a seed pool for the cold-start memory, a calibration fold, a purity-validation slice, ten stream chunks (one per possible cycle), an evaluation fold of 300 questions used for every headline number, and a test fold kept for later work. The transfer benchmarks have only evaluation and test folds. All folds were checked for overlap by content hash when they were built.

Metrics

Exact match (EM): the main accuracy measure, read from the final answer line of each response by one rule for every system. FEVER and StrategyQA are scored by the label the answer starts with; CommonsenseQA by the single option letter given, where the option text may follow the letter but two letters count as wrong; TriviaQA by exact match against any gold alias after normalisation. TruthfulQA is scored by a language-model judge (Claude Haiku) that compares the final answer with

Benchmark	Panel	Seed	Calib	Purity	Stream chunks	Eval	Test
FEVER	training	1000	500	500	10×2000	300	500
TriviaQA	training	1000	500	500	10×2000	300	500
CommonsenseQA	training	1000	500	500	10×700	300	500
TruthfulQA	transfer	0	0	0	0	300	500
StrategyQA	transfer	0	0	0	0	300	387

Table 5.1: Benchmark folds. CommonsenseQA has smaller stream chunks because its training split holds only about 10,000 questions after de-duplication; the StrategyQA test fold is limited by the size of its split.

the benchmark's correct and incorrect reference answers, following the judge-based evaluation introduced with the benchmark [40]; a refusal counts as correct only when the reference answers include one. The same judge, reading the same part of the response, scores every system.

Composite hallucination metric (CHM): the equal-weighted mean of seven failure rates, each computed over all evaluation questions.

- **Confident confabulation:** the answer is wrong but its stored score u_{stored} is at least 0.50.
- **Factual fabrication:** the answer is wrong and its weakest atomic fact has passage support below 0.30.
- **Logical fabrication:** the answer is wrong and the model's own chains support it below 0.30.
- **Defensive evasion:** the answer is wrong and uses an evasive phrase such as *the context does not mention.*
- **Template leakage:** the output contains a leaked prompt placeholder.
- **False refusal:** the answer is correct but the system abstained or discarded it.
- **Over-long output:** the answer is wrong and longer than 600 characters.

Two of these, confident confabulation and false refusal, are defined through the system's own verifier, so they move with the gate's operating point: a more permissive gate refuses fewer correct answers and lets more wrong answers keep a high score. Two further rates are recorded but left out of the mean, because neither can fire in this run:

- **Factual contradiction:** the entailment judge returns only a support probability, with no contradiction output.
- **Off-topic:** planned as a wrong answer whose question-match score is below 0.50. That score cannot detect off-topic answers on this panel, and its stored values never fall below 0.50 (Section 4.2.6). An attempt after the run to measure off-topic answers directly, with a language-model judge, was checked against 200 hand-labelled answers and agreed too weakly to give a reliable rate (Cohen's kappa about 0.35), so measuring them is left to future work.

Model view and delivered view: every system is reported in two views. The *model view* scores EM and CHM on every raw answer the generator wrote, including answers that CAEM then withheld; it measures what the model knows. The *delivered view* scores what the user receives. CAEM shows every answer with its probability and a status tag (verified, provisional or conflicting, Section 4.2.9) except abstentions, which hide the answer behind an insufficient-evidence message; semantic-entropy abstention (B8) withholds the answers it abstains on; every other baseline answers every question. Following the selective-prediction setting of Kamath et al. [31] and Geifman and El-Yaniv [17], the delivered view reports the share of questions answered, the share of those answers that are wrong, and CHM over the answered questions; for CAEM it also reports the accuracy of each status tag, because the tag is part of what the user sees. Unlike the two verifier-defined rates, the answered share and the error among answered do not depend on the verifier's scores.

Composite evaluation score (CES): a single summary of five axes, each in $[0, 1]$ with one best: accuracy (pooled EM), epistemic quality ($1 - \text{CHM}$), retention (worst probe ratio, capped at one), calibration ($1 - 2\,\text{ECE}$) and verifier reliability ($\max(2\,\text{BA} - 1, 0)$, with the balanced accuracy approximated as $\Phi(d/\sqrt{2})$ from the calibration-fold Cohen's d). The score is the geometric mean, so a collapse on one axis is not averaged away:

$$\text{CES} = \left(\prod_{i=1}^{5} \text{clamp}_{[\varepsilon,1]}(\text{axis}_i) \right)^{1/5}, \quad \varepsilon = 0.01. \tag{5.1}$$

Retention ratio: accuracy on each probe divided by its value before any fine-tuning (Equation (3.2)); the worst probe decides whether a cycle is kept. Table 5.2 summarises the metrics.

Metric	Reported on	Definition
Exact match (EM)	all five benchmarks	label or alias match; TruthfulQA judged by Claude Haiku against the reference answers
Composite hallucination metric (CHM)	all five benchmarks	mean of seven failure rates
Answered, wrong among answered	pooled	delivered view: share not withheld; share of those that are wrong
Status-tag accuracy	pooled, CAEM	share of questions and EM per tag shown to the user
Storage precision π_{store}	per cycle	share of stored answers that are correct
Gate balanced accuracy, Cohen's d	calibration fold	$(\text{TPR} + 1 - \text{FPR})/2$ at τ_{store}; separation of correct and incorrect scores
Composite evaluation score (CES)	pooled	geometric mean of five axes (Equation (5.1))
Retention ratio	probe panel	worst probe accuracy over its pre-fine-tuning value

Table 5.2: Metrics reported in this chapter.

Comparison protocol

The external baselines (Section 2.2.7) are compared with CAEM under one protocol, so that a difference in the results reflects the method rather than the prompt or the scoring.

- **Matched prompts:** every baseline that answers from the question alone receives CAEM's zero-shot question message: the same system instruction, worked example, task line, answer format and forced reasoning prefix. Chain of thought (B2) and five-shot chain of thought (B5) add the step-by-step cue to that message; B5's five demonstrations per benchmark are drawn from the benchmark's training split (for TruthfulQA, which has a single split, from the questions not evaluated), with every evaluated question removed. The two retrieval baselines (B3, B4) use CAEM's retrieval prompt with the same top five passages, vanilla fine-tuning (B6) is evaluated with the prompt it was trained on, which is CAEM's zero-shot message, and FLARE (B7) uses the zero-shot message for its draft and its final answer.
- **One scoring rule:** every system is scored by the rules of Section 5.1.1, applied to its final answer line, and every TruthfulQA answer by the same judge.
- **One hallucination instrument:** CHM needs verifier signals, so each baseline answer is passed through CAEM's verifier with the adapter and composite of cycle three, the same instrument for every baseline. The decision the verifier would take on a baseline answer is only a measurement: the baseline never acted on it. A baseline is therefore never charged with a false refusal or a withheld answer from CAEM's decision rule; its withheld answers are only those it withheld itself (B8).
- **Same questions:** every system answers the same 1,500 evaluation questions, in the same order, checked question by question before scoring.

CAEM's outputs come from the main run. An earlier comparison had given the baselines a shorter prompt than CAEM's and scored some answer formats differently, so after the main run the baselines were re-run under this protocol (B3 and B4 only on TruthfulQA, where a graphics-memory error had left the first batch of 32 questions unanswered; their prompts were already CAEM's). CAEM's own prompts are identical, character for character, in the code of the main run and of the re-run.

Hardware, seed, reproducibility

All phases ran on one rented graphics processor of the 32-gigabyte class, under a single recorded random seed. The main run took cycles zero to five and stopped after cycle five as planned after the cycle-four rollback (Section 3.3.2). The baseline re-run of Section 5.1.1 used a second machine of the same class. For that re-run, the passages for the evaluation questions were retrieved once by exact search over the same index and reused; on a check sample, live search returned the same passages for 50 of 50 questions, and the verifier's passage scores from the main run were reproduced within 0.01 on 98 percent of answers. Signals computed from sampled text (dropout agreement, chain similarity and chain entailment) are random draws by design and differ slightly between runs. The code, the configuration files and the

per-cycle artefacts are listed in Chapter E.

5.1.2 Trajectory of headline metrics

All results in this section use the locked configuration chosen at cycle zero; Section 5.2 explains how it was chosen. The section order follows the BRAC template rather than the order in which the work was done.

The headline in three steps

Table 5.3 separates the three contributions that a single before-and-after comparison would mix together: the architecture (B1, the same generator answering zero-shot with the same prompt, to C0, CAEM before any fine-tuning), the self-improvement loop (C0 to C5) and the whole system (B1 to C5). Significance is read from the paired per-benchmark tests of Section 5.4.1.

System	EM	CHM	Confident confab.	False refusal	Answered	Wrong among answered
B1 (zero-shot)	0.511	0.132	0.208	0.000	100.0%	0.489
CAEM C0	0.517	0.155	0.089	0.276	66.1%	0.405
CAEM C2	0.535	0.127	0.259	0.049	87.7%	0.428
CAEM C5	0.508	0.134	0.305	0.055	83.0%	0.451

Table 5.3: The headline in three steps on the five-benchmark evaluation fold (1,500 questions). B1 to C0 isolates the architecture, C0 to C5 the self-improvement loop, B1 to C5 the whole system.

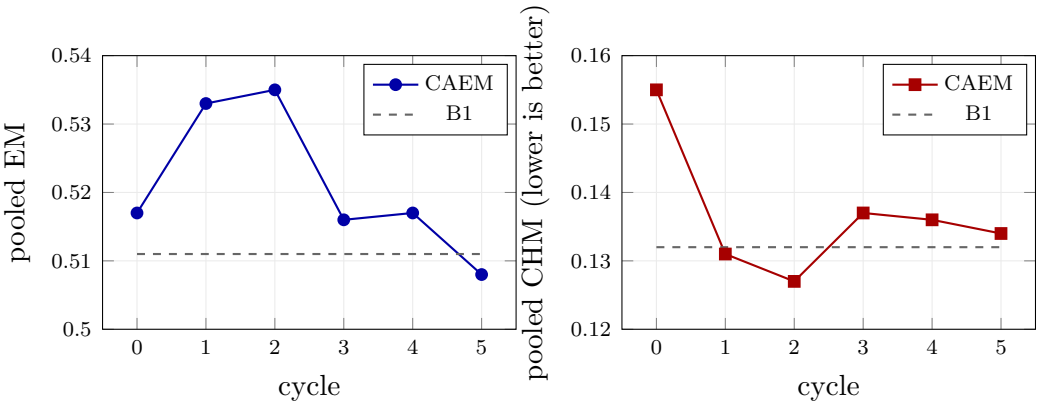


Figure 5.1: Pooled EM (left) and CHM (right) by cycle on the evaluation fold, against B1. EM is level with B1 at C0, peaks at C2 and returns to B1's level by C5; CHM is above B1 at C0, where the system refuses too often, about level with it from C1, and ends just above it.

The architecture adds almost no accuracy: memory, routing, the retrieval-utility classifier and the verifier together move pooled EM from 0.511 to 0.517 before any fine-tuning (Figure 5.1), and from 0.554 to 0.569 on the training panel. No benchmark shows a significant EM difference between B1 and C0.

The architecture alone is over-cautious: at C0 the system abstains on 33.9 percent of evaluation questions and discards another 31.9 percent. False refusal reaches 0.276, which lifts CHM to 0.155, above B1's 0.132, even though confident confabulation falls from 0.208 to 0.089. C0's CHM is significantly higher than B1's on TriviaQA, TruthfulQA and StrategyQA.

Self-improvement helps for two cycles, then erodes: after the first cycle the system abstains on 14.4 percent and discards 9.7 percent, false refusal falls to 0.059 and CHM to 0.131; EM rises to 0.533 at C1 and 0.535 at C2, the best values of the run. From C3 the gains fade: pooled EM ends at 0.508 and CHM at 0.134. On the training panel EM peaks at C1 (0.584, Table 5.5) and ends at 0.549, below B1's 0.554.

Confident confabulation returns: as the refitted gate becomes more permissive, confident confabulation rises to 0.259 at C2 and 0.305 at C5, above B1's 0.208: after self-improvement, the verifier gives a high score to more of the generator's wrong answers than it gave to the base generator's. Because this rate and false refusal are both defined through the verifier's own score, they trade off: their sum is 0.365 at C0, between 0.297 and 0.329 at C1 to C4 and 0.361 at C5 (Figure 5.2). The net CHM change from C0 to C5 (0.155 to 0.134) comes from the other subtypes: fewer wrong answers with unsupported facts (0.381 to 0.327), unsupported reasoning (0.298 to 0.245) or evasive phrasing (0.044 to 0.003).

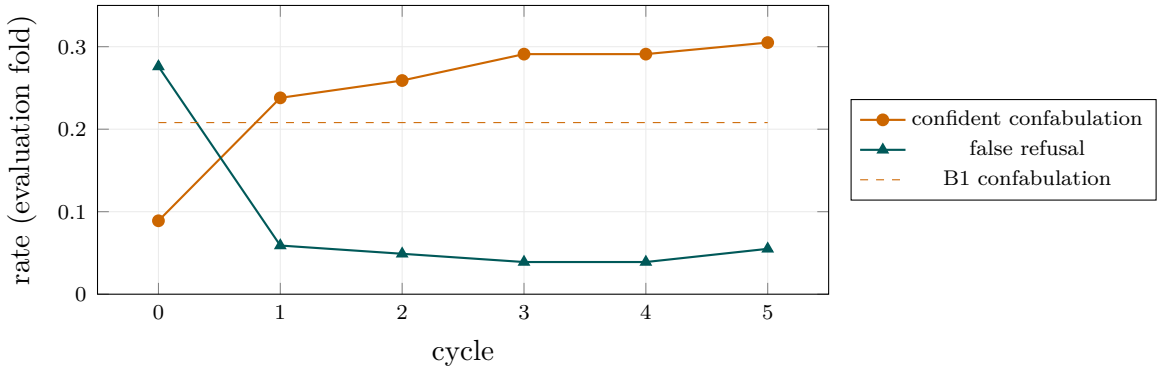


Figure 5.2: Confident confabulation against false refusal by cycle. Both are defined through the verifier's own score, so they move in opposite directions as the refitted gate becomes more permissive; from C2 on, confident confabulation is above B1's rate. B1's false refusal is zero, because a baseline never acts on the verifier's decision.

The whole system ends level with B1: at C5, pooled EM is 0.508 against 0.511 and CHM 0.134 against 0.132; at its best cycle, C2, it is 0.535 and 0.127. None of these differences is significant on any benchmark, at C2 or at C5. In the delivered view (Figure 5.3), CAEM is wrong on 40.5 percent of the questions it answers at C0, where it answers two thirds of them, against 48.9 percent for B1, which answers everything. As self-improvement makes the system answer more (87.7 percent at C2, 83.0 percent at C5), its error among answered rises to 0.428 and 0.451, close to the 0.427 of semantic-entropy abstention (B8), which answers 89.5 percent.

The status tag predicts correctness: the user sees each answer with a status tag (Section 5.1.1), and at every cycle verified answers are more often right than provisional ones, which in turn are more often right than conflicting or insufficient

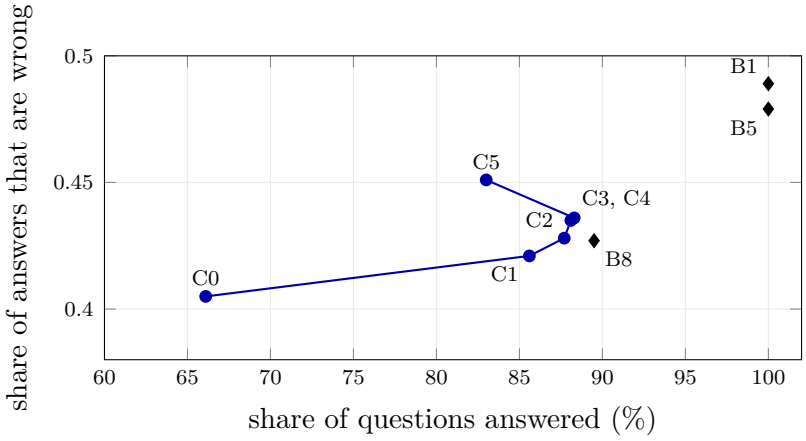


Figure 5.3: Delivered view on the evaluation fold: share of questions answered and share of those answers that are wrong, for each CAEM cycle (abstentions withheld) and for B1 and B5, which answer everything, and B8, which withholds its own abstentions. CAEM's error among answered is lowest at C0, where it answers two thirds of the questions, and approaches B8's as it answers more.

ones (Table 5.4). Answers tagged verified are right 74.6 percent of the time at C0, when a quarter of the questions receive that tag, and 65.5 to 68.9 percent at C1 to C5, when 42 to 56 percent do; provisional answers are right 40 to 60 percent of the time, and conflicting and insufficient ones 15 to 48 percent. B1 is right on 51.1 percent of its answers and gives the user no signal of which ones. The tag is therefore the most direct user-facing benefit of the architecture: from C1 on, a user who acts only on verified answers meets about 30 percent fewer wrong answers than one who trusts every answer of B1 (an error rate of 0.31 to 0.35 against 0.49), at the cost of receiving a verified answer for only about half of the questions.

	Verified		**Provisional**		**Conflicting**		**Insufficient**		
Cycle	**Share**	**EM**	**Share**	**EM**	**Share**	**EM**	**Share**	**EM**	**All**
C0	24.9%	0.746	9.3%	0.597	31.9%	0.477	33.9%	0.365	0.517
C1	44.2%	0.689	31.7%	0.535	9.7%	0.219	14.4%	0.264	0.533
C2	46.8%	0.655	33.6%	0.532	7.3%	0.220	12.3%	0.266	0.535
C3	55.6%	0.668	26.0%	0.403	6.3%	0.326	11.9%	0.151	0.516
C4	55.3%	0.667	26.3%	0.411	6.6%	0.303	11.7%	0.159	0.517
C5	42.3%	0.667	39.1%	0.433	1.5%	0.217	17.0%	0.306	0.508

Table 5.4: Share of evaluation questions and EM for each status tag CAEM shows the user (verified: stored; provisional: deferred; conflicting: discarded, shown with a warning; insufficient: abstained, answer withheld), with EM over all questions for reference. B1, which shows no tag, has EM 0.511.

The retention ratios of every cycle are in Table 5.10: MMLU never comes near the floor (between 0.975 and 1.041); the TriviaQA test probe falls to 0.886 at cycle four and triggers the only rollback. The composite evaluation score (Table C.6 in Section C.6) rises from 0.585 to 0.704, mostly through its verifier-reliability axis, which grows with the calibration-fold separation d (Table 5.13); its accuracy axis

falls over the same cycles, so the rise in CES should not be read as a rise in accuracy.

Per-benchmark trajectory

Table 5.5 and Table 5.6 break the pooled numbers down. On the training panel, FEVER and TriviaQA peak at C1 or C2 (FEVER 0.533 against B1's 0.507; TriviaQA 0.493 against 0.430) and fall back afterwards, FEVER to 0.477, below B1, and TriviaQA to 0.453, still above it. CommonsenseQA is highest at C0 (0.757 against 0.727) and ends at 0.717. TriviaQA is the benchmark where the architecture helps most and where the verifier separates correct from incorrect answers best (Section 5.6.2); even there, the C2 gain over B1 is not significant at 300 questions. On the transfer panel, TruthfulQA stays between 0.230 and 0.277 at every cycle against B1's 0.250, and StrategyQA between 0.610 and 0.653 against 0.643. Across subtypes, factual and logical fabrication fall gently after C0; defensive evasion and over-long output are near zero. Template leakage is zero for every cycle and every baseline, and this is a real zero: the answer prompt shows the reasoning line as a bracketed placeholder, and no system ever copied a placeholder into its output.

Benchmark	**B1**	**C0**	**C1**	**C2**	**C3**	**C4**	**C5**	**Change to C2**	**Change to C5**
Training panel									
FEVER (EM)	0.507	0.487	0.533	0.533	0.503	0.507	0.477	+0.027	−0.030
FEVER (CHM)	0.139	0.159	0.129	0.116	0.137	0.136	0.144	−0.022	+0.005
TriviaQA (EM)	0.430	0.463	0.480	0.493	0.463	0.463	0.453	+0.063	+0.023
TriviaQA (CHM)	0.139	0.179	0.129	0.125	0.138	0.137	0.142	−0.014	+0.003
CSQA (EM)	0.727	0.757	0.740	0.717	0.743	0.743	0.717	−0.010	−0.010
CSQA (CHM)	0.101	0.091	0.098	0.106	0.098	0.097	0.104	+0.005	+0.003
Pooled (EM)	0.554	0.569	0.584	0.581	0.570	0.571	0.549	+0.027	−0.006
Pooled (CHM)	0.126	0.143	0.119	0.116	0.124	0.123	0.130	−0.010	+0.004
Transfer panel									
TruthfulQA (EM)	0.250	0.270	0.267	0.277	0.230	0.230	0.253	+0.027	+0.003
TruthfulQA (CHM)	0.147	0.173	0.170	0.158	0.165	0.167	0.168	+0.011	+0.021
StrategyQA (EM)	0.643	0.610	0.647	0.653	0.640	0.640	0.640	+0.010	−0.003
StrategyQA (CHM)	0.137	0.176	0.129	0.131	0.147	0.145	0.113	−0.006	−0.023
All five (EM)	0.511	0.517	0.533	0.535	0.516	0.517	0.508	+0.023	−0.003
All five (CHM)	0.132	0.155	0.131	0.127	0.137	0.136	0.134	−0.005	+0.002

Table 5.5: EM (upper row) and CHM (lower row) by benchmark and cycle on the evaluation fold (300 questions per benchmark), with B1 and the change from B1 to C2 and to C5. Shaded rows pool the training panel and all five benchmarks.

Subtype	B1 ref	C0	C1	C2	C3	C4	C5
Conf. confab.	0.208	0.089	0.238	0.259	0.291	0.291	0.305
Fact. fabric.	0.392	0.381	0.338	0.323	0.335	0.335	0.327
Logic. fabric.	0.318	0.298	0.272	0.251	0.288	0.284	0.245
Def. evasion	0.009	0.044	0.009	0.006	0.002	0.002	0.003
Templ. leak	0.000	0.000	0.000	0.000	0.000	0.000	0.000
False refusal	0.000	0.276	0.059	0.049	0.039	0.039	0.055
Over-long	0.000	0.000	0.001	0.003	0.005	0.005	0.004
CHM (mean of 7)	**0.132**	0.155	0.131	0.127	0.137	0.136	0.134

Table 5.6: The seven subtypes of the composite hallucination metric (Section 5.1.1) by cycle, pooled over the evaluation fold (1,500 questions), with B1 for reference.

Training versus transfer contrast

On the training panel, whose benchmarks feed the calibration fold and the training pool, CAEM gains a little and then loses it: EM 0.554 (B1) against 0.581 at C2 and 0.549 at C5, and CHM 0.126 against 0.116 and 0.130. The transfer panel, which the system never trains on, shows no gain at any cycle: TruthfulQA EM is 0.253 at C5 against 0.250 for B1, with a higher CHM (0.168 against 0.147), and StrategyQA EM 0.640 against 0.643, with a lower CHM (0.113 against 0.137). Self-improvement therefore did not damage the transfer benchmarks, but it did not carry over to them either.

5.1.3 Four-outcome decision distribution

The three readings in this subsection use three different folds, named in each paragraph; the second and third tables are in Section C.6.

On the calibration fold (Table 5.7), the share stored rises from 42 percent at C1 to 59 percent at C5, deferral from 12 to 29 percent, while discard falls from 25 to 3 percent and abstention from 21 to 10 percent. The outcomes are ordered by correctness at every cycle: stored answers are right 73 to 75 percent of the time, deferred ones less, discarded ones less again, and abstentions least (0.38 at C1, 0.17 at C5). The gap between stored and the next outcome widens from 14 points at C1 to between 24 and 30 points afterwards.

On the stream chunks and the transfer evaluation folds (Table C.7), stored answers are more often correct than deferred ones in every row. The full ordering stored, deferred, discarded holds in half the rows; where it fails, the discard class is small. On TruthfulQA, stored answers are right only 31 to 38 percent of the time at C1 to C5, far below the training panel.

On the evaluation fold (Table C.8), FEVER abstention falls from 64.3 percent at C0 to 5.3 percent at C5; CommonsenseQA almost never abstains, because a multiple-choice prompt forces an answer; TruthfulQA abstains on 23 to 33 percent of questions, a rate that falls slightly over the cycles while its EM stays flat; StrategyQA swings between 2 and 43 percent.

	Store		Defer		Discard		Abstain	
	share	EM	share	EM	share	EM	share	EM
(a) Pooled training panel								
C1	42.1%	0.74	12.0%	0.60	25.0%	0.58	20.9%	0.38
C2	67.2%	0.74	18.7%	0.44	5.4%	0.38	8.7%	0.25
C3	60.4%	0.75	26.0%	0.47	5.5%	0.41	8.1%	0.22
C5	58.6%	0.73	28.5%	0.49	3.4%	0.46	9.5%	0.17
*(b) By benchmark at C3 (*500 *each)*								
FEVER	55.1%	0.76	38.1%	0.47	4.3%	0.52	2.6%	0.38
TriviaQA	40.5%	0.79	28.9%	0.45	10.6%	0.30	20.0%	0.17
CommonsenseQA	85.7%	0.72	11.1%	0.55	1.6%	0.88	1.6%	0.62

Table 5.7: Share of each decision and the EM within it on the training-panel calibration fold: (a) pooled by cycle; (b) per benchmark at cycle three. C0 and the rolled-back C4 have no reading.

5.1.4 Three-tier router efficiency

Per-tier hit rate trajectory and per-tier exact match

The design expects the memory-direct share to grow as the memory fills and the zero-shot share to grow as fine-tuning folds retrieved knowledge into the generator. Neither happened on the evaluation fold (Table 5.9). Tier 1 answered 7 of the 9,000 evaluation queries over six cycles, because every stream chunk and the evaluation fold share no question with the stored episodes; all seven answers were correct, and a direct probe of the router answered 42 of 50 exact repeats and 19 of 50 paraphrases of stored questions, 41 and 19 of them correctly (Table 5.8). Tier 1 is built for deployment, where users repeat questions; this study used fresh questions to measure learning, so the probe is the test of this tier. Because a repeated question receives an answer that has already passed the storage gate, both exact match and the composite hallucination metric are expected to improve in deployment as the share of repeated questions grows. The zero-shot share moved from 48.7 percent at C0 to 50.2 percent at C5, peaking at 55.2 percent at C2.

Within tiers, zero-shot answers are more accurate than retrieval-augmented ones at every cycle (0.565 against 0.472 at C0, 0.519 against 0.495 at C5; Table C.12 in Section C.6). At C0 this holds on four of five benchmarks, with TriviaQA the exception (Table C.9). Part of the difference is selection: the router and the classifier send to Tier 2 the questions they judge the generator can already answer. The zero-shot tier's accuracy also falls over the cycles, which is where the self-improvement loop acts.

Per-tier latency and cost

The mean time per evaluation query, from the batched evaluation log, stays between 13.6 and 14.7 seconds per query (about six hours per cycle for the 1,500 evaluation questions), as expected when the tier shares hardly move. This is about a hundred times slower than B1 (142 milliseconds), because every generated answer passes through the verifier's resampling, retrieval and entailment steps. Per benchmark at

	Exact repeats			Paraphrases		
Source benchmark	Asked	Tier 1	Correct	Asked	Tier 1	Correct
FEVER	17	13	13	17	7	7
TriviaQA	23	22	22	23	12	12
CommonsenseQA	10	7	6	10	0	0
Total	50	42	41	50	19	19

Table 5.8: Router probe of the memory-direct tier at cycle five: 50 stored questions asked again word for word and 50 asked as paraphrases. Tier 1 counts the questions answered from memory; Correct counts those answers that match the gold answer.

	Tier 1 (memory)		Tier 2 (zero-shot)		Tier 3 (RAG)		
	share	EM	share	EM	share	EM	**Pooled EM**
(a) Pooled over the five benchmarks							
C0	0.0%	–	48.7%	0.565	51.3%	0.472	0.517
C1	0.0%	–	49.8%	0.566	50.2%	0.501	0.533
C2	0.1%	1.000	55.2%	0.548	44.7%	0.517	0.535
C3	0.1%	1.000	44.7%	0.532	55.1%	0.502	0.516
C4	0.1%	1.000	45.6%	0.532	54.3%	0.502	0.517
C5	0.1%	1.000	50.2%	0.519	49.7%	0.495	0.508
(b) By benchmark at C5 (300 each)							
FEVER	0.0%	–	43.3%	0.500	56.7%	0.459	0.477
TriviaQA	0.3%	1.000	37.0%	0.423	62.7%	0.468	0.453
CommonsenseQA	0.0%	–	82.3%	0.745	17.7%	0.585	0.717
TruthfulQA	0.3%	1.000	64.3%	0.254	35.3%	0.245	0.253
StrategyQA	0.0%	–	24.0%	0.639	76.0%	0.640	0.640

Table 5.9: Share of evaluation queries sent to each tier and the EM within it: (a) pooled by cycle; (b) per benchmark at C5.

C5, CommonsenseQA is fastest (8.6 seconds) and TruthfulQA slowest (17.8 seconds). The time per tier could not be separated from the batched log, so the deployment-cost model of Equation (3.5) cannot be filled in tier by tier; given the flat tier shares, it would predict a flat cost in any case.

Why the safety override fires

The safety override fired on 63 of the 9,000 evaluation queries (0.7 percent) and on 3 of the 23,500 stream-chunk queries. Of the evaluation firings, 62 were CommonsenseQA questions after that benchmark's floor was refitted from 0.38 to 0.491 at cycle three and 0.569 at cycle five; the other was one StrategyQA question at cycle five. On FEVER and TriviaQA the override never fired, because their pre-routing temperature reached its upper bound and pulled u_{pre} toward 0.5 (Section 4.2.2). The override therefore acted as a rarely used safeguard in this run, not as an active part of routing.

5.1.5 Retention and continual learning

The retention guard reads three probes after every fine-tune (Table 5.10). It kept cycles one, two, three and five, and rolled back cycle four, when the TriviaQA test ratio fell to 0.886; the memory was kept and grew across the rollback. On the benchmarks themselves, CommonsenseQA lost 4.0 points and FEVER 1.0 point from C0 to C5, and both fell further from their peaks (Table 5.5); the probe panel did not catch these losses because the CommonsenseQA probe stayed above its floor and FEVER is not probed. The training set grew from 107 episodes at the first fine-tune to 5,006 at the last, and the final training loss fell from 0.258 to 0.070 over cycles one to four, then rose to 0.091 at cycle five after training restarted from the cycle-three adapter (Table 5.10).

			Retention probe ratio			
Cycle	**Episodes**	**Final loss**	**MMLU**	**TQA-test**	**CSQA-test**	**Outcome**
C1	107	0.258	1.041	1.013	1.006	commit
C2	1 474	0.171	0.975	0.949	0.982	commit
C3	2 966	0.109	0.992	0.987	0.976	commit
C4	3 624	0.070	1.018	**0.886**[†]	0.982	rolled back
C5	5 006	0.091	0.999	1.013	0.958	commit

[†] Cycle-four TriviaQA-test ratio of 0.886 fell below the floor of 0.93, so the cycle was rolled back.

Table 5.10: Episodes used for fine-tuning, final training loss and retention ratios by cycle.

5.1.6 Memory and training-pool dynamics

Memory growth: the memory grew from the 431 gold-filtered cold-start entries to 12,175 at C5, adding between 2,214 and 2,484 entries per cycle, including the rolled-back cycle four; the deferred buffer grew to 4,284 entries (Table 5.11).

Training pool: the training threshold ($u_{\text{stored}} \geq 0.70$) keeps a purer subset of the stored answers (Table 5.12). On the stream chunks, the training pool is 79.8

	Memory		Deferred buffer			Consolidation	
Cycle	**Entries**	**Change**	**Entries**	**Promoted**	**Dropped**	**Merged**	**Removed**
C0	431	—	0	—	—	—	—
C1	2 818	+2 387	1 123	0	0	0	0
C2	5 151	+2 333	2 399	131	0	3	3
C3	7 477	+2 326	2 692	220	924	11	11
C4*	9 961	+2 484	4 068	—	—	—	—
C5	12 175	+2 214	4 284	299	1 169	43	44
Total				650	2 093	57	58

Table 5.11: Memory and deferred-buffer size at the close of each cycle, answers promoted from or dropped by the buffer, and near-duplicate groups merged. *Cycle four was rolled back, so its reconsideration and consolidation passes were skipped.

to 81.8 percent correct, against 72.4 to 75.3 percent for everything stored, so the stricter threshold removes between 56 and 73 percent of the wrong stored answers before training. About a fifth of the training pool is still wrong. Stored scores are well ordered against correctness on the stream chunks at every cycle, but on the evaluation fold only at cycles one and four (Table C.10 in Section C.6).

Cycle	n_{stored}	**Correct**	**Wrong**	π_{store}	**Above** τ_{train}	**Share above**	π_{SIL}	**Used in training**
(a) Evaluation fold								
C0	336	254	82	0.756	58	17.3%	0.793	—
C1	515	374	141	0.726	231	44.9%	0.788	—
C2	534	370	164	0.693	266	49.8%	0.726	—
C3	565	402	163	0.712	279	49.4%	0.735	—
C4*	566	402	164	0.710	272	48.1%	0.735	—
C5	504	353	151	0.700	181	35.9%	0.735	—
(b) Stream chunks								
C1	2 456	1 850	606	0.753	1 309	53.3%	0.818	107
C2	2 494	1 873	621	0.751	1 448	58.1%	0.810	1 474
C3	2 490	1 802	688	0.724	1 337	53.7%	0.798	2 966
C4*	2 522	1 862	660	0.738	1 413	56.0%	0.803	—
C5	2 380	1 724	656	0.724	895	37.6%	0.803	5 006

*Cycle four was rolled back, so its training pool is not reported.

Table 5.12: Stored answers on the training panel, their precision, the share passing the training threshold ($u_{\text{stored}} \geq 0.70$) and the precision of that training pool, on (a) the evaluation fold and (b) the stream chunks.

Re-verification and deferral: reconstructing the memory snapshots identifies 1,094 entries removed by the retroactive pass (the run log counts 1,061); 38.4 percent of them were wrong, against about 22 percent in the pool, so the pass removes wrong entries at about 1.75 times their base rate (Table C.11 in Section C.6). The deferred buffer promoted 650 answers (131, 220 and 299 at cycles two, three and five, Table 5.11); of the 620 promotions recovered from the snapshots, 55.6 percent were correct, which is below the stored pool but means about 345 correct answers

rescued from the deferral band. Consolidation merged 57 groups and removed 58 entries (Table 5.11); its same-answer guard blocked several merges of near-identical questions with different answers.

5.1.7 Theoretical predictions checked against the trajectory

This subsection reads the statements of Section 4.3 against the run. The two proven results are checked against their assumptions; the heuristics and remarks are read as consistency checks. Table 5.15 summarises the readings.

	Pooled training panel						Balanced accuracy		
Cycle	Stored	Precision	TPR	FPR	Bal. acc.	Cohen's d	FEVER	Trivia-QA	CSQA
C0*	560 (37.3%)	0.780	0.653	0.148	0.753	1.468	–	–	–
C1	630 (42.0%)	0.740	0.510	0.280	0.615	0.593	0.665	0.500	0.522
C2	1001 (66.7%)	0.736	0.791	0.465	0.663	0.773	0.672	0.689	0.554
C3	899 (59.9%)	0.746	0.725	0.397	0.664	0.756	0.653	0.719	0.531
C5	868 (57.9%)	0.732	0.705	0.389	0.658	0.842	0.633	0.687	0.550

Table 5.13: Storage gate quality on the training-panel calibration fold (1,500 questions) by cycle. *C0 is in-sample (the composite was fitted on this fold); C1 to C5 score the fold with the new adapter before the composite is refitted. C4 was rolled back and has no reading.

Pool purity

Pool purity floor: the floor of Theorem 4.1 depends on new entries being as often correct as the calibration fold at the same cut-point (assumption (ii)). On the training panel this approximately held: the calibration-fold precision at $\tau_{\text{store}} = 0.60$ was between 0.732 and 0.746 at C1 to C5 (Table 5.14), and the answers actually stored from each stream chunk were 0.753, 0.751, 0.724, 0.738 and 0.724 correct. On the evaluation fold the pooled storage precision was between 0.693 and 0.756; FEVER's own evaluation-fold precision was lower, between 0.614 and 0.634 at C1 to C4 and 0.649 at C5, which shows that the pooled statement does not carry over to every benchmark. On TruthfulQA, where the questions are unlike the calibration fold, the assumption failed: stored answers were right only 31 to 38 percent of the time.

Purification condition (H1): the gate purifies only if its balanced accuracy exceeds one half (Proposition 4.2). Pooled over the training panel it was 0.615, 0.663, 0.664 and 0.658 at C1, C2, C3 and C5 (Table 5.13), measured with the new adapter before each refit; the C0 value of 0.753 is in-sample. The weakest benchmarks were CommonsenseQA (0.52 to 0.55) and TriviaQA at C1, where the gate stored no TriviaQA answer and the balanced accuracy is exactly 0.500.

Monotonicity

Heuristic 4.5 predicts that better separation stores more answers when the correct class is denser at the cut-point. The separation d rose from 0.593 at C1 to 0.773

	Cal-fold pooled		Eval-fold pooled		Eval-fold per-bench π_{store}		
Cycle	n	π_{store}	n	π_{store}	FEVER	TriviaQA	CSQA
C0	560	0.780†	336	0.759	0.661	—	0.779
C1	631	0.740	515	0.726	0.634	0.797	0.736
C2	1 001	0.736	534	0.693	0.630	0.733	0.705
C3	899	0.746	565	0.712	0.617	0.741	0.750
C4*	—	—	566	0.710	0.614	0.733	0.754
C5	868	0.732	504	0.700	0.649	0.704	0.725

† In-sample: the cycle-zero composite was fitted on this fold (re-run sweep, locked configuration).
*Cycle four was rolled back, so the calibration fold was not refitted; its evaluation-fold reading uses the cycle-three composite.

Table 5.14: Storage precision π_{store} at $\tau_{\text{store}} = 0.60$ by cycle: share of stored answers that are correct, on the training-panel calibration fold and on the evaluation fold.

at C2 while the share stored rose from 0.420 to 0.667, as predicted; from C2 to C3, d was flat (0.756) and the share fell to 0.599; from C3 to C5, d rose to 0.842 while the share fell to 0.579 (Table 5.13). Only the first step agrees. The refit of the composite at every cycle, which the heuristic does not model, is the most likely reason; separating the two would need a cycle with the composite frozen (Chapter 6).

Convergence

Convergence envelope (H3): the calibration-fold precision stayed between 0.732 and 0.746 at C1 to C5, a band of width 0.014 (Table 5.14). For about 900 stored answers per cycle, the standard error of one reading is about 0.015, so the band is as narrow as sampling noise allows; six cycles cannot estimate a convergence rate.

Repeated filtering (H2): Remark 4.4 describes purity rising under repeated filtering by a fixed gate. The calibration-fold precision was 0.780 at C0 (in-sample), then 0.740, 0.736, 0.746 and 0.732: it fell at C1, C2 and C5. H2 is therefore not supported as registered; the changes are within sampling noise, and purity stayed above the floor of Theorem 4.1 throughout.

Equilibrium under rollback: parts (i) and (iii) of Heuristic 4.9 hold. The commit pattern over cycles one to five was $1, 1, 1, 0, 1$ (commit rate 0.80), with worst probe ratios 1.006, 0.949, 0.976, 0.886 and 0.958, and the memory grew at every cycle. Part (iv) cannot be tested in five cycles.

Asymptotic hallucination floor

Heuristic 4.7 bounds the error rate of Tier 1 answers by the impurity of the pool. On the evaluation fold Tier 1 gave 7 answers, all correct, which is consistent but too few to estimate a rate; the router probe gives the testable case (Section 5.1.4). Per tier, CHM is 0.121 to 0.129 for zero-shot answers and 0.131 to 0.181 for retrieval-augmented ones (Table C.12 in Section C.6). The decomposition of Remark 4.10 cannot be tested here, because Tier 1 answered at most 0.1 percent of evaluation queries.

Heuristic checks: equilibrium structure and self-correction

Heuristic 4.8 predicts that repeated re-verification removes wrong entries over time. The share of checked entries removed fell from 13.0 percent at C1 to 9.2, 6.6 and 3.7 percent at C2, C3 and C5 (Table C.13 in Section C.6), and removed entries were wrong more often than the pool (Section 5.1.6), both consistent with the heuristic; the falling rate is partly diluted by the growing number of fresh entries. The entry records do not count how many passes each entry survived, so the predicted survival distribution could not be measured. The corpus ceiling of Remark 4.11 was not measured.

Statement	Status	Reading	What the run shows	Evidence
Pool purity floor (Theorem 4.1)	proven	holds on the training panel	New-entry precision on the stream chunks 0.724 to 0.753, close to the calibration-fold precision 0.732 to 0.746; on TruthfulQA, where the like-calibration assumption fails, stored answers are 0.31 to 0.38 correct.	Tables C.7 and 5.14
Purification condition (Proposition 4.2)	proven	precondition met	Pooled gate balanced accuracy 0.615 to 0.664 at C1 to C5; weakest benchmarks CommonsenseQA (0.52 to 0.55) and TriviaQA at C1 (0.500).	Table 5.13
Storage-rate sensitivity (Heuristic 4.5)	heuristic	consistent at one of three steps	C1 to C2: d 0.593 $\rightarrow$ 0.773 and share stored 0.420 $\rightarrow$ 0.667, as predicted; C2 to C3: d flat; C3 to C5: d rises, share falls.	Table 5.13
Purity rises with accuracy (Remark 4.3)	remark	not tested	α changes every cycle and p varies little.	–
Convergence envelope (Heuristic 4.6)	heuristic	consistent (band)	Calibration-fold precision stays within 0.732 to 0.746 (width 0.014) at C1 to C5; a rate cannot be estimated from six cycles.	Table 5.14
Repeated filtering gives purity 1 (Remark 4.4)	remark	not met	Calibration-fold precision 0.780 (C0, in-sample), 0.740, 0.736, 0.746, 0.732: it falls at C1, C2 and C5 (H2).	Table 5.14
Error decomposition (Remark 4.10)	remark	not testable here	Tier 1 answered at most 0.1 percent of evaluation queries.	Table 5.9
Tier 1 error ceiling (Heuristic 4.7)	heuristic	consistent	All 7 Tier 1 answers on the evaluation fold correct; router probe 41 of 42 repeats and 19 of 19 paraphrases correct.	Table C.12
Corpus ceiling (Remark 4.11)	remark	not measured	Retrieved passages were not stored per question.	–
Self-correction (Heuristic 4.8)	heuristic	consistent (direction)	Share of checked entries pruned 13, 9, 7, 4 percent at C1, C2, C3, C5; survival counts not recorded.	Table C.13
Equilibrium under rollback (Heuristic 4.9)	heuristic	(i), (iii) hold; (iv) not testable	Commit pattern $1, 1, 1, 0, 1$; memory grew at every cycle (431 to 12,175).	Tables 5.10 and 5.11

Table 5.15: What the run shows for each formal statement of Section 4.3. Heuristic and remark readings are consistency checks, not tests of a proven claim.

5.2 Final Design Adjustments

5.2.1 Calibration discipline and disjoint folds

The floor of Theorem 4.1 rests on the calibration fold being like the candidate answers, and no sample may appear in both. The six folds of Table 5.1 were therefore built once, checked for overlap by content hash, and never changed; only the training panel enters the calibration fold.

5.2.2 The composite parameter selection sweep

The pre-specified grid

Before the main run, 100 variants were scored: the storage cut-point $\tau_{\text{store}} \in \{0.55, 0.60, 0.65, 0.70\}$ (deferral cut-point 0.45 throughout), the boost-layer regularisation $C \in \{0.010, 0.025, 0.050, 0.100, 1.000\}$ (smaller is stronger) and the shrinkage strength $\alpha_{\text{shrink}} \in \{0.0, 0.4, 0.6, 0.8, 1.0\}$ (0 uses only the pooled curve, 1 only the benchmark's own). The sweep was re-run with the final code for this thesis; the re-run reproduces the locked composite exactly, and all numbers below come from it.

Selection criteria

Four criteria were set in advance, applied in order: (1) pooled calibration-fold Cohen's d of at least 0.20; (2) at least 20 answers stored on the calibration fold; (3) among the survivors, prefer the variant that stores more correct answers at a similar precision; (4) between equivalent variants, prefer stronger regularisation. Of the 100 variants, 84 pass criterion 1 and 83 pass criteria 1 and 2.

Top variants

Table 5.16 shows the locked variant and its neighbours. At $\tau_{\text{store}} = 0.60$ the locked variant stores 560 calibration-fold answers at precision 0.780, against 281 at 0.865 for $\tau_{\text{store}} = 0.65$: the stricter cut stores about half as many answers at higher precision, and 0.60 was chosen for volume. On the held-out validation fold the locked variant stores 558 answers at 0.751. Read literally, criterion 3 would have chosen $\alpha_{\text{shrink}} = 1$, which stores more correct calibration-fold answers (476 against 437); the locked $\alpha_{\text{shrink}} = 0.6$ was chosen instead because pulling small per-benchmark folds toward the pooled fit separates correct from incorrect answers better on the validation fold (pooled d 0.72 against 0.66).

5.2.3 The locked configuration

The locked configuration ($\tau_{\text{store}} = 0.60$, $C = 0.01$, $\alpha_{\text{shrink}} = 0.6$, together with $\tau_{\text{defer}} = 0.45$, the abstention floor 0.20, $\tau_{\text{retro}} = 0.50$ and $\rho_{\min} = 0.93$) was carried unchanged into every cycle (Table B.1 in Chapter B). $C = 0.01$ is the strongest regularisation in the grid, a hundredfold tighter penalty than the library default of 1.0. Table B.4 in Chapter B gives the cycle-zero direction and boost weight of each signal.

			Calibration fold			Validation fold			
τ_{store}	C	α_{shrink}	d	Stored	Precision	d	Stored	Precision	Criteria 1–2
0.55	0.01	0.6	1.47	621	0.767	0.72	610	0.741	yes
0.60	0.01	0.0	1.26	76	0.961	0.85	60	0.817	yes
0.60	0.01	0.4	1.45	393	0.824	0.75	368	0.747	yes
0.60	0.01	0.6	1.47	560	0.780	0.72	558	0.751	yes
0.60	0.01	0.8	1.48	610	0.769	0.69	596	0.742	yes
0.60	0.01	1.0	1.48	620	0.768	0.66	604	0.740	yes
0.65	0.01	0.6	1.47	281	0.865	0.72	260	0.758	yes
0.70	0.01	0.6	1.47	108	0.972	0.72	84	0.798	yes
0.60	1	0.6	0.80	237	0.730	0.73	197	0.772	yes

Table 5.16: Selected variants of the 100-variant cycle-zero sweep (re-run with the locked code; shaded: locked configuration). Calibration fold and held-out validation fold have 500 questions per training benchmark.

5.2.4 Per-cycle composite-refit trajectory

The cut-points stay fixed, but the mapping from signals to probability is refitted at every kept cycle (Table C.14 in Section C.6). The pooled pre-routing temperature starts at its upper bound ($T = 20.086 \approx e^3$), drops to 1.089 at cycle one and stays between 0.74 and 0.87 afterwards; the FEVER and TriviaQA temperatures remain 4.8 to 10.7 times that of CommonsenseQA, and CommonsenseQA's safety floor is the only one refitted (to 0.491 and 0.569). The expected calibration error after each refit stays between 0.041 and 0.049. Of the nine boost weights, two move: the chain-entailment weight rises from 0.067 to 0.285 and the question-match weight falls from 0.491 to about 0.26; the other seven move by at most 0.07.

5.2.5 Calibration-versus-evaluation gap

The earlier split-conformal design was dropped because its guarantees depend on the calibration and evaluation data being alike (Section 4.2.8). The locked fixed-threshold gate makes no such promise, and the gap can be read directly. On the training panel the calibration-fold precision (0.732 to 0.746) and the evaluation-fold precision (0.693 to 0.756) stay within about five points of each other (Table 5.14).

At cycle zero the gap is large per benchmark (Table 5.17). On the held-out validation fold the locked composite stores 93 FEVER answers (0.720 correct), 465 CommonsenseQA answers (93 percent of them, 0.757 correct) and no TriviaQA answer at all. CommonsenseQA's scores barely separate correct from incorrect answers (Section 5.2.7), so almost everything clears the cut-point; TriviaQA's scores separate well but sit below 0.60 on this fold. From cycle one the refitted composite stores TriviaQA answers normally. Storing fewer answers on a benchmark is the gate doing what it is asked: it promises precision at the cut-point, not balance across benchmarks. The deferred buffer and re-verification give rejected but correct answers later chances (Section 5.1.6).

5.2.6 The long-hypothesis judge ablation

A second entailment judge, built on the frozen generator, was meant to handle answers longer than MiniCheck's 512-token input. It was required to agree with MiniCheck with a Pearson correlation of at least 0.70 in logit space on a 500-sample overlap set; it reached 0.584 and was dropped (Section B.4, where only the correlation and the floor were kept). Every answer is therefore judged by MiniCheck, with inputs over 512 tokens truncated; the short-answer prompts of this panel rarely reach that length.

5.2.7 The validation gate verdict

Before the main run, the locked configuration had to pass a set of checks on the held-out cycle-zero validation fold (Table 5.17).

- **Pooled separation:** training-panel composite Cohen's d of at least 0.20. Passed: 0.717.
- **Store-minus-discard gap:** at least 0.05 per benchmark. Computable only on FEVER (0.268, passed): TriviaQA stored nothing and CommonsenseQA discarded nothing.
- **Wrong share among stored:** below 0.50 per benchmark. Passed on FEVER (0.28) and CommonsenseQA (0.243); not computable on TriviaQA.

The registered pooled check therefore passes, but two per-benchmark checks cannot be computed at cycle zero, and CommonsenseQA's own separation is weak ($d = 0.14$). This explains the weak CommonsenseQA gate quality later in the run (0.52 to 0.55, Table 5.13). On the transfer panel, shown for information only, d is 0.38 for TruthfulQA and 0.00 for StrategyQA. Table 5.18 gives the same separation for each signal.

Benchmark	EM	Cohen's d (≥ 0.20)	Stored	Store minus discard (≥ 0.05)	Wrong among stored (< 0.50)
Training panel (registered checks)					
FEVER	0.480	+0.56	93	+0.27	0.28
TriviaQA	0.454	+1.40	0	–	–
CommonsenseQA	0.750	+0.14	465	–	0.24
Pooled		+0.72	558		
Transfer panel (diagnostic only)					
TruthfulQA	0.384	+0.38	3	+0.51	0.00
StrategyQA	0.606	−0.00	64	+0.37	0.36
Pooled		+0.42	67		

Table 5.17: Cycle-zero validation gate re-scored with the locked composite on the held-out validation fold (500 questions per benchmark). A dash marks a check that cannot be computed because the benchmark stored, or discarded, no answers.

Signal	Training panel				Transfer panel		
	FEVER	TriviaQA	CSQA	**Pooled**	TruthfulQA	StrategyQA	**Pooled**
u_{stored}	+0.56	+1.40	+0.14	**+0.72**	+0.38	-0.00	**+0.42**
$p^{\text{mean}}_{\text{ground}}$	+0.50	+1.12	+0.04	**+0.47**	+0.24	+0.29	**+0.11**
$p^{\text{max}}_{\text{ground}}$	+0.58	+1.15	+0.02	**+0.49**	+0.21	+0.33	**+0.09**
$p^{\text{atomic}}_{\text{ground}}$	+0.50	+1.12	+0.04	**+0.47**	+0.24	+0.29	**+0.11**
p_{entail}	+0.15	+1.26	-0.18	**+0.20**	+0.26	+0.18	**+0.19**
q_{match}	-0.37	+0.51	+0.35	**+0.26**	+0.33	+0.19	**+0.42**
u_{internal}	+0.14	+0.34	+0.01	**+0.55**	+0.11	-0.11	**+0.20**
u_{token}	+0.21	-0.09	-0.15	**-0.03**	+0.14	+0.03	**+0.19**
u_{dropout}	-0.10	-0.37	-0.30	**-0.56**	-0.02	+0.11	**-0.19**
s_{avg}	+0.00	+0.58	-0.02	**+0.30**	-0.12	-0.07	**-0.13**

Table 5.18: Per-signal Cohen's d between correct and incorrect answers on the held-out cycle-zero validation fold. Positive values mean the signal is higher on correct answers. The first row is the locked composite.

5.2.8 Failed iterations as evidence of method discipline

The first cycle-zero calibration failed the validation gate because of a bug in the step that writes out the verifier signals: only some of the nine signals were saved, so the composite was fitted on an incomplete signal vector and failed the separation check. The gate caught the bug on the first run; after the fix the calibration was repeated and produced the locked configuration. The failed iteration is reported because it shows the gate doing what it was set up to do.

5.3 Statistical Analysis

5.3.1 Tests used

Every baseline answered the same evaluation questions under the comparison protocol of Section 5.1.1, so each comparison is paired question by question (Section D.1). CAEM is compared with every baseline at C0, C2 and C5.

Exact match: for each benchmark and baseline, a paired McNemar test with continuity correction (Edwards [2]) gives a p-value; the p-values of the baselines within a benchmark are corrected with the Holm–Bonferroni procedure [5]; and a paired bootstrap gives a 95 percent interval on the EM difference (2,000 replicates). A contrast is starred when the Holm-corrected p-value is below 0.05 *and* the absolute EM difference is at least 0.02.

Composite hallucination metric: every question has its own CHM value, the share of the seven failure flags it raises, so this axis is paired in the same way. A Wilcoxon signed-rank test (Wilcoxon [1]) on the per-question differences gives a p-value, with questions that show no difference dropped; the p-values are Holm-corrected within each benchmark; and a paired bootstrap gives the interval. A contrast is starred when the corrected p-value is below 0.05 *and* the absolute CHM difference is at least 0.01. A second CHM test uses only the five subtypes that depend neither on CAEM's composite score nor on its decisions, as a check on the

verifier's role (Section 5.6.4).

The full contingency counts, the Holm sequences and the procedural detail are in Chapter D.

5.4 Comparisons and Relationships

5.4.1 External baseline panel under matched protocol

The eight baselines are zero-shot (B1), chain of thought (B2), retrieval-augmented generation (B3), chain of thought with retrieval (B4), five-shot chain of thought (B5), vanilla fine-tuning (B6), FLARE active retrieval (B7) and semantic-entropy abstention (B8); Section 2.2.7 describes them and Section 5.1.1 the protocol under which they were run. All use the same backbone.

Pooled comparison (Table 5.19, Figure 5.4): the systems fall into three groups. The prompting baselines (B1, B2, B5), semantic-entropy abstention (B8) and CAEM form one cluster, with pooled EM between 0.508 and 0.535 and CHM between 0.127 and 0.137 (CAEM C0, at 0.155, sits above it); CAEM at C2 has the highest EM (0.535) and the lowest CHM (0.127) of the cluster, ahead of five-shot chain of thought (0.521, 0.129), and CAEM at C5 sits inside it (0.508, 0.134). The retrieval baselines (B3, B4, B7) sit apart, with lower EM (0.445 to 0.466) and higher CHM (0.158 to 0.196). Vanilla fine-tuning (B6), trained on gold answers, has an EM inside the cluster (0.512) and the lowest CHM of all systems (0.103). In the delivered view, CAEM at C0 has the lowest error among answered of any system (0.405), at the cost of answering only 66.1 percent of the questions; at C2 and C5 its error among answered (0.428, 0.451) is close to B8's (0.427) and below that of every baseline that answers everything (0.479 to 0.555).

Per-benchmark tests (Table 5.20; every contrast with its interval is in Tables D.5 and D.6): of the 40 contrasts per cycle (8 baselines on 5 benchmarks), EM gives 9 significant wins and 3 defeats at C2 and 5 wins and 3 defeats at C5; CHM gives 13 wins and 5 defeats at C2 and 10 wins and 4 defeats at C5. The wins are almost all against the retrieval baselines and FLARE, and against vanilla fine-tuning on TriviaQA and TruthfulQA. Against the prompting baselines and semantic-entropy abstention (B1, B2, B5, B8), no EM contrast is a significant win at either cycle, and only two CHM contrasts are (B2 and B5 on StrategyQA at C5). The defeats are against vanilla fine-tuning (EM on FEVER and CommonsenseQA; CHM on FEVER, CommonsenseQA and StrategyQA) and against five-shot chain of thought (EM on CommonsenseQA; CHM on TruthfulQA, and at C2 also on CommonsenseQA). At C0, CAEM's CHM is significantly higher than that of most baselines (17 defeats, 6 wins), the cost of its cold-start over-refusal, while its EM already gives 9 wins and 2 defeats.

Each baseline against its expectation:

- **Zero-shot (B1):** the floor; the three-step comparison of Section 5.1.2 is built on it.
- **Chain of thought (B2):** expected to help on multi-step questions; instead it scored the same as B1 on pooled EM (0.511) with a slightly higher CHM

#	System	Model view		Delivered view	
		EM	CHM	Answered	Wrong among answered
	CAEM C0	0.517	0.155	66.1%	0.405
	CAEM C2	0.535	0.127	87.7%	0.428
	CAEM C5 (final)	0.508	0.134	83.0%	0.451
B1	zero-shot	0.511	0.132	100.0%	0.489
B2	chain of thought	0.511	0.137	100.0%	0.489
B5	five-shot chain of thought	0.521	0.129	100.0%	0.479
B8	semantic-entropy abstention	0.519	0.135	89.5%	0.427
B3	retrieval (RAG)	0.445	0.165	100.0%	0.555
B4	chain of thought + RAG	0.452	0.158	100.0%	0.548
B7	FLARE	0.466	0.196	100.0%	0.534
B6	vanilla fine-tune (gold labels)	0.512	0.103	100.0%	0.488

Table 5.19: Every system on the evaluation fold (1,500 questions) under the comparison protocol of Section 5.1.1. Model view: EM and CHM on every raw answer. Delivered view: share of questions answered (CAEM withholds abstentions, B8 its own abstentions) and share of those answers that are wrong. Groups: prompting baselines, retrieval baselines, and the fine-tuned baseline, which trains on gold answers.

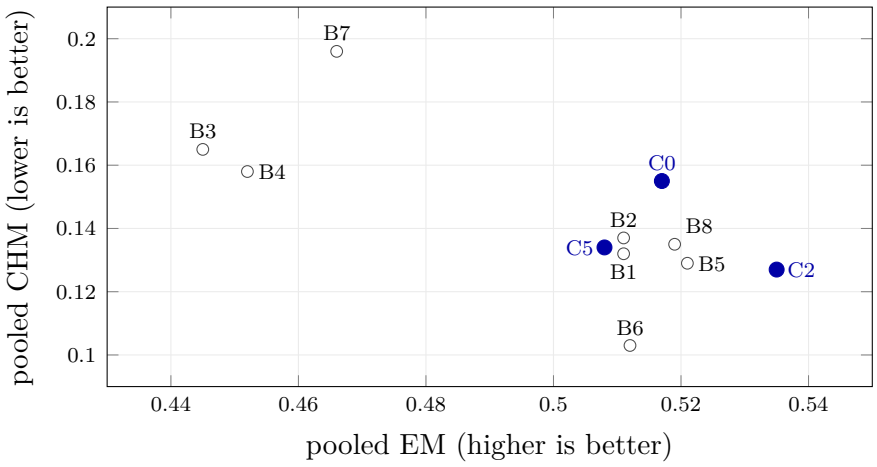


Figure 5.4: CAEM at C0, C2 and C5 (filled) and the eight baselines (open) on pooled EM and CHM in the model view (Table 5.19). The best systems sit at the lower right. The prompting baselines, B8 and CAEM form one cluster; the retrieval baselines sit apart at lower EM and higher CHM; the fine-tuned baseline B6 has the lowest CHM.

	CAEM C0		CAEM C2		CAEM C5	
Baseline	**EM**	**CHM**	**EM**	**CHM**	**EM**	**CHM**
B1 zero-shot	0 / 0	0 / 3	0 / 0	0 / 0	0 / 0	0 / 0
B2 chain-of-thought	0 / 0	0 / 2	0 / 0	0 / 0	0 / 0	1 / 0
B3 RAG	3 / 0	1 / 1	3 / 0	5 / 0	1 / 0	2 / 0
B4 CoT + RAG	3 / 0	1 / 1	3 / 0	3 / 0	1 / 0	2 / 0
B5 five-shot CoT	0 / 1	0 / 3	0 / 1	0 / 2	0 / 1	1 / 1
B6 vanilla fine-tune	2 / 1	0 / 4	2 / 2	1 / 3	2 / 2	0 / 3
B7 FLARE	1 / 0	4 / 0	1 / 0	4 / 0	1 / 0	4 / 0
B8 semantic entropy	0 / 0	0 / 3	0 / 0	0 / 0	0 / 0	0 / 0
Total	9 / 2	6 / 17	9 / 3	13 / 5	5 / 3	10 / 4

Table 5.20: Significant wins / defeats of CAEM against each baseline over the five benchmarks, so each row has 5 contrasts per cell and the total row 40 (paired tests, Holm-corrected within each benchmark; Chapter D). C0 is CAEM before any self-training.

(0.137), in line with Wei et al. [41], who find that the step-by-step cue helps little in models well below 100 billion parameters, and Chung et al. [69] for small instruction-tuned models. CAEM's zero-shot prompt already asks for reasoning before the answer, so B1 and B2 differ only in the added cue.

- **Retrieval (B3) and retrieval with chain of thought (B4):** retrieving for every question lowered pooled EM (0.445 and 0.452, against 0.511 for B1) and raised CHM (0.165 and 0.158), the strongest evidence in the panel that indiscriminate retrieval hurts on this backbone; CAEM's retrieval-utility classifier exists for this reason (Section 5.4.2).
- **Five-shot chain of thought (B5):** the strongest prompting baseline, with pooled EM 0.521 and CHM 0.129; its demonstrations help most on CommonsenseQA (0.793) and TruthfulQA (0.347, the best of any system), while on StrategyQA they hurt (0.560 against 0.643 for B1).
- **Vanilla fine-tuning (B6):** a full fine-tune of the backbone on the same training questions CAEM saw in each cycle (2,000 FEVER, 2,000 TriviaQA and 700 CommonsenseQA questions per cycle, five cycles, three epochs each), with the gold answer as the training target after CAEM's zero-shot prompt, and with no verifier filter, memory or retention guard. Unlike CAEM, it sees the correct answer for every training question, 23,500 in all. It is the most accurate system on FEVER (0.640) and CommonsenseQA (0.807), the least accurate on TriviaQA (0.290) and TruthfulQA (0.163), and ends with pooled EM 0.512, below CAEM at C2 and level with it at C5, and the lowest CHM (0.103). Its retention was not measured.
- **FLARE (B7):** retrieval triggered by low-confidence tokens did not avoid the cost of retrieval on this panel: pooled EM 0.466 and the highest CHM (0.196).
- **Semantic-entropy abstention (B8):** the closest comparator to CAEM's abstention. It withheld 75 TriviaQA and 81 TruthfulQA answers and almost none elsewhere, answering 89.5 percent of all questions with an error among

answered of 0.427; its pooled EM is 0.519 and its CHM 0.135. Because semantic entropy measures disagreement across samples, it cannot flag a misconception the model holds consistently, as Farquhar et al. [71] note, and its TruthfulQA EM (0.243) stays at B1's level.

STaR and Self-RAG were not run: STaR's rationalisation step and binary filter differ too much from CAEM's verifier for a clean comparison, and Self-RAG's critic tokens are tied to a different model family.

5.4.2 Retrieval-utility classifier ablation

The classifier is evaluated by comparing the main run with an earlier trajectory that ran without it, at cycle three, the last cycle both trajectories kept, with the composite fixed at the same cycle-three calibration so that the verifier signals are identical. The classifier-on run also carried some pipeline corrections (prompt fixes and minor bug fixes), so the comparison measures the classifier together with those corrections, not the classifier alone. Two acceptance gates were set in advance: (1) a pooled EM gain with no benchmark losing more than two points of EM, and (2) TruthfulQA EM reaching the B1 level. The B1 level was 0.433 when the gate was set; under the final comparison protocol, with the judge reading only the final answer, it is 0.250, and both runs are scored the same way.

	Exact match			**CHM**			**Tier 3 share**		
Benchmark	**Off**	**On**	**Δ** (pp)	**Off**	**On**	**Δ** (% rel)	**Off**	**On**	**Δ** (pp)
Training panel									
FEVER	0.503	0.503	+0.0	0.133	0.137	+2.5%	92.3%	70.7%	−21.7
TriviaQA	0.437	0.463	+2.7	0.141	0.138	−2.0%	95.7%	68.0%	−27.7
CommonsenseQA	0.603	0.743	+14.0	0.150	0.098	−34.4%	57.7%	20.3%	−37.3
Transfer panel									
StrategyQA	0.617	0.640	+2.3	0.136	0.147	+8.1%	100.0%	76.7%	−23.3
TruthfulQA	0.177	0.230	+5.3	0.160	0.165	+3.3%	99.0%	40.0%	−59.0
Pooled (all five)	0.467	0.516	+4.9	0.144	0.137	−4.8%	88.9%	55.1%	−33.8

Table 5.21: CAEM without (Off) and with (On) the retrieval-utility classifier at cycle three, with the composite fixed at the cycle-three calibration: EM, CHM and retrieval share per benchmark.

Gate 1 passes: pooled EM rises from 0.467 to 0.516 (+4.9 points) with no benchmark losing EM; CommonsenseQA gains 14.0 points, TruthfulQA 5.3, TriviaQA 2.7 and StrategyQA 2.3, and FEVER is unchanged at 0.503 (Table 5.21). Pooled CHM falls from 0.144 to 0.137, mostly on CommonsenseQA (−34.4 percent); on FEVER, StrategyQA and TruthfulQA it rises slightly (+2.5, +8.1 and +3.3 percent).

Gate 2 fails, narrowly: TruthfulQA EM rises from 0.177 to 0.230 but stays below B1's 0.250, and its CHM rises slightly (0.160 to 0.165, against 0.147 for B1).

Mechanism: the retrieval share falls from 88.9 to 55.1 percent pooled, and on every benchmark, by 21.7 points on FEVER, where retrieval genuinely helps, up to

59.0 points on TruthfulQA (99.0 to 40.0 percent), where it amplified misconceptions. Separating the classifier from the accompanying corrections would need a run with the corrections but without the classifier (Chapter 6).

5.5 Summary of Findings

5.5.1 Headline outcome in one paragraph

Under a matched comparison protocol, CAEM performs at the level of strong prompting baselines, clearly better than retrieval baselines, and level on accuracy with a fine-tuned baseline that needs gold answers, but it does not reduce hallucination below the strong baselines. Pooled EM moves from 0.511 (B1) to 0.517 at C0, peaks at 0.535 at C2 and ends at 0.508 at C5; CHM moves from 0.132 to 0.155 at C0, where the system refuses too often, to 0.127 at C2 and 0.134 at C5. No difference between CAEM at C2 or C5 and B1 is significant on any benchmark. Self-improvement helps for one or two cycles and then erodes, and confident confabulation rises above B1's level as the gate becomes more permissive. What the architecture adds is selectivity: CAEM is wrong on 40.5 percent of the questions it answers at C0 (B1: 48.9 percent of all questions), and its status tag predicts correctness, with verified answers right 65 to 75 percent of the time. Its gains are concentrated where the verifier can check answers against retrieved passages, chiefly TriviaQA. The retrieval-utility classifier, with accompanying corrections, adds 4.9 points of pooled EM.

5.5.2 Hypothesis verdicts

Each pre-specified hypothesis of Section 1.4.3 is judged by its registered statistic, using only the three registered verdicts (Table 5.22).

ID	Hypothesis (Section 1.4.3)	Verdict	Deciding statistic and reading
H1	Verifier balanced accuracy exceeds one half on the calibration-matched distribution at every cycle (Proposition 4.2 precondition).	**supported**	Pooled gate balanced accuracy on the calibration fold: 0.615, 0.663, 0.664, 0.658 at C1, C2, C3, C5 (C0 0.753, in-sample; Table 5.13). Weakest: CommonsenseQA 0.52 to 0.55; TriviaQA 0.500 at C1, when no TriviaQA answer was stored.
H2	Pool purity is non-decreasing across cycles whenever the precondition is satisfied (Remark 4.4).	**unsupported**	Calibration-fold precision 0.780 (C0, in-sample), 0.740, 0.736, 0.746, 0.732: it falls at C1, C2 and C5. The changes are within sampling noise and purity stays above the floor of Theorem 4.1 (Table 5.14).
H3	Late-cycle pool-purity increments enter a bounded stationary band on the committed-cycle subsequence (Heuristic 4.6).	**supported**	Precision within 0.732 to 0.746 at C1 to C5, a band of width 0.014, about one standard error of a single reading; a convergence rate cannot be estimated in six cycles.
H4	The composite hallucination metric at the realised cycle horizon approaches a parameter-bounded asymptote (Remark 4.10).	**partially supported**	CHM falls from 0.155 to 0.131 at C1 and stays between 0.127 and 0.137 afterwards (0.134 at C5), consistent with flattening; six cycles are too few to estimate the limit itself.

Table 5.22: Verdicts on the four pre-specified hypotheses.

5.6 Discussion

5.6.1 Headline interpretation

What the matched comparison changed: an earlier comparison gave the baselines a shorter prompt than CAEM's and scored some answer formats differently; under it, CAEM appeared to gain about three points of EM over B1 before any fine-tuning and to lead every baseline on CHM. Giving every system the same question message and the same scoring rule (Section 5.1.1) removed most of that lead. On a three-billion-parameter generator, the prompt format alone moved the baselines' accuracy by several points, which is as large as the effects being measured; comparisons of small models are only as fair as their prompts.

What carries the remaining gain: the architecture's measurable contribution is selectivity rather than accuracy. The verifier's score, turned into a status tag and an abstention, lets CAEM tell the user which answers to trust: verified answers are right about two thirds of the time against about half for B1, and at C0 its answers are wrong less often than any baseline's. The self-improvement loop's main effect was behavioural: after one cycle the generator stopped refusing questions it could answer, which removed the cold-start over-refusal and gave the best EM and CHM of the run at C1 and C2. Its gains lasted one or two cycles, in line with Song et al. [93], who find that self-improvement loops saturate within a few rounds.

Why the theory does not promise more: the purification results of Section 4.3 are about the stored pool, not the model. They held: the stored pool was purer than the generator's raw answers. Whether a model trained on a pool that is still about one-fifth wrong improves is a separate question, and Yi et al. [96] show that training filtered through a verifier converges to the verifier's own knowledge, mistakes included. The fall in zero-shot accuracy after C2 and the rise in confident confabulation are consistent with that, though the retraining on a cleaned pool that would show it was not run.

Label efficiency: CAEM is label-efficient, not label-free. A labelled calibration fold of 1,500 questions fits the verifier's composite and thresholds, and each later cycle re-scores the same fold with the new adapter and refits; the 603 cold-start answers were checked against gold labels once (Section 4.2.3); every later training answer is labelled by the verifier alone, about 2,100 gold labels in all. Vanilla fine-tuning (B6) needs a gold answer for each of its 23,500 training questions. With that difference, CAEM reaches B6's pooled EM (0.535 at C2 and 0.508 at C5, against 0.512), while B6 reaches the lower CHM (0.103).

TruthfulQA: retrieval does not help on questions written to invite a misconception. At C0, retrieval-tier answers on TruthfulQA were right 21.4 percent of the time against 29.9 percent for zero-shot ones (Table C.9); the difference is partly selection, since the router sends the harder questions to retrieval, but it agrees with the classifier ablation, which gained 5.3 points on TruthfulQA mainly by retrieving less (Section 5.4.2). With every system judged on its final answer, CAEM's TruthfulQA EM stays at B1's level throughout (0.230 to 0.277 against 0.250), and five-shot chain of thought is the most accurate system there (0.347).

5.6.2 Where the verifier can check answers

The verifier's ability to separate correct from incorrect answers differs by benchmark. Its area under the ROC curve on the evaluation fold is 0.86, 0.79 and 0.75 on TriviaQA at C0, C2 and C5, 0.64 to 0.67 on FEVER, and 0.51 to 0.62 on CommonsenseQA, StrategyQA and TruthfulQA. The difference follows from what the signals can see. The signals that carry information about correctness on this panel compare the answer with retrieved Wikipedia passages or, through the question-match score, with the question (Section 4.2.6); a trivia answer is either stated in a passage or not, while a commonsense choice, an implicit multi-step yes-or-no answer or a misconception is rarely settled by any single passage. The two internal signals do not make up for this: the token signal scores the whole response rather than the answer (Section 4.2.6), and the dropout and chain signals measure agreement, which a consistently held mistake passes. CAEM's gains follow the same pattern. TriviaQA is where the architecture helps most (EM 0.463 at C0 and 0.493 at C2 against 0.430 for B1), and the benchmarks where the verifier is close to chance are where CAEM is level with, or below, the strong baselines. A verifier that can judge commonsense and reasoning answers, or a system scoped to questions that can be checked against documents, are the two directions this points to (Chapter 6).

5.6.3 A side effect of self-improvement

After the first fine-tune, almost every zero-shot answer began with a reference to "the context", although Tier 2 gives the generator no passages. The share of Tier 2 answers on the evaluation fold that mention a context in their first sentences was 3 percent at C0 (25 of 731), then 93, 98, 97, 97 and 99 percent at C1 to C5 (693 of 747 at C1, 746 of 753 at C5). The cause is the training data: the fine-tune learns from stored reasoning chains, and most of those were written under the retrieval prompt, where citing the context is correct. The answers can still be right, but each such answer cites a source that does not exist, which is a faithfulness error in its own right. Removing or rewriting context references in the training chains before fine-tuning would have prevented it; this preprocessing step was missed and is proposed in Chapter 6.

5.6.4 Threats to validity

Single seed and single backbone

All numbers come from one seed and one three-billion-parameter generator. Repeating the run with other seeds would turn the per-cycle readings into intervals, and repeating it with another generator of similar size would show whether the findings depend on this model.

Benchmark coverage

The panel covers short-answer factual and commonsense questions. Long-form generation, code, mathematics, dialogue and other languages are outside its scope (Section 1.6.1), and the atomic-fact signal and the long-input judge, built for long answers, were not exercised.

Calibration to evaluation gap

The storage precision promised at the cut-point is measured on the calibration fold and holds on new data only as far as that data resembles it. On the training panel the gap stayed within about five points; on TruthfulQA it did not hold at all (Section 5.1.7).

Cycle budget

The run stopped after cycle five as planned (Section 3.3.2). Six cycles are too few to estimate convergence rates or long-run limits; that would need at least fifteen kept cycles.

Storage-tail sample size

Per-benchmark numbers rest on few stored answers in places: at cycle zero the locked gate stored 93 FEVER answers, 465 CommonsenseQA answers and no TriviaQA answer on the validation fold, and Tier 1 gave only 7 answers on the evaluation fold. Pooled readings are more reliable than per-benchmark ones.

Retrieval-utility classifier mis-routing

The classifier's accuracy differs widely across benchmarks (AUROC 0.718 on FEVER to 0.936 on TriviaQA, Section 4.2.5), so questions unlike its training data may be routed the wrong way. It was not tested on a benchmark held out from its training data.

Measuring hallucination with the system's own verifier

Two of the seven CHM subtypes depend on CAEM's own verifier: confident confabulation uses its stored score and false refusal uses its decision. The baselines have no verifier of their own, so their answers were scored with CAEM's verifier and the composite fixed at cycle three, and the verifier's decision on a baseline answer is not charged to the baseline (Section 5.1.1). CHM differences are still partly measured with the instrument CAEM was built around. Two checks limit this concern:

- **Without the two verifier-defined subtypes:** the other five use neither CAEM's composite score nor its decisions. Restricted to them, CAEM at C2 and C5 is lower than every baseline (0.083 pooled at both cycles, against 0.084 for vanilla fine-tuning, the nearest, and 0.103 for B1), and the paired test gives 28 significant wins and 2 defeats at C2 and 31 wins and 3 defeats at C5 out of 40 contrasts. Part of this advantage is built in: the factual-fabrication flag checks answers against passages from the same retrieval that CAEM's retrieval tier writes its answers from.
- **Exact match and the delivered view's error among answered:** both are scored against gold labels and do not depend on the verifier's scores.

5.6.5 Sensitivity analyses

From the re-run sweep (Table 5.16): moving the cut-point from 0.60 to 0.65 halves the number of stored answers (560 to 281 on the calibration fold) and raises their precision (0.780 to 0.865). Raising the regularisation strength C from 0.01 to 0.025 or more stores far fewer answers on the validation fold (313 or fewer, against 558) at a similar precision (0.744 to 0.772, against 0.751). Raising α_{shrink} stores more answers but separates correct from incorrect answers less well on the validation fold (pooled d 0.85 at 0, 0.72 at 0.6, 0.66 at 1). Sensitivity to the safety floor and to the early-exit thresholds was not measured.

5.6.6 Comparison to the strongest prior systems

Conformal factuality: Mohri and Hashimoto [78] raise the correctness of GPT-4 answers from 78 to 93 percent on Natural Questions by removing low-scoring sub-claims at a split-conformal threshold, and Cherian et al. [68] add conditional guarantees and a learned combination of claim scores. CAEM applies its gate when storing an answer rather than when showing it, keeps the gated answers for reuse, and combines its signals in the manner of Valentin et al. [83]; it also found that conformal guarantees broke under its calibration-to-evaluation shift (Section 4.2.8).

Verifier-filtered self-improvement: STaR [43] and V-STaR [73] train on their own correctness-filtered outputs, and Huang et al. [88] analyse self-improvement as sharpening. CAEM differs in using an external multi-signal verifier and in keeping verified answers in memory; like those loops, its gains saturated quickly.

Selective retrieval: Adaptive-RAG of Jeong et al. [75], the popularity-based routing of Mallen et al. [55], the self-knowledge routing of Wang et al. [63] and the probing of Baek et al. [86] all decide per question whether to retrieve. CAEM's classifier belongs to this line, and its ablation (Section 5.4.2) agrees with their finding that retrieving for every question leaves accuracy on the table.

Retrieval and misconceptions: on questions written to invite a familiar wrong answer, retrieved passages tend to share the question's framing rather than refute it, so the retrieval tier amplifies the misconception and the storage gate cannot recover what retrieval has already corrupted. This is why retrieval-tier answers on TruthfulQA are the least accurate of the panel (Section 5.6.1), and the case the classifier was built for.

Chapter G compares each architectural component with plain fine-tuning at the same labelling budget.

5.6.7 Limitations

- **Memory scale:** the memory reached 12,175 entries, so its behaviour at the millions of entries of a deployment was not tested.
- **Pre-routing calibration:** the pre-routing temperature reached its upper bound at cycle zero ($T = 20.086$) on FEVER and TriviaQA, which flattened u_{pre} and left the safety override almost inactive (Section 5.1.4); the storage decision does not depend on this temperature.

- **Deferred-pool horizon:** only three reconsideration passes ran, so the long-run behaviour of the deferred buffer is not known.
- **Question-match signal:** the fourth signal family was meant to catch off-topic answers, but it scores the short answer string, which the cross-encoder cannot relate to the question; it acted as a correctness cue instead, and off-topic answers were not detected (Section 4.2.6).
- **Phantom context:** fine-tuning on retrieval-written chains taught the generator to cite a non-existent context (Section 5.6.3).
- **Verifier scope:** the verifier separates correct from incorrect answers well only where retrieved passages can confirm an answer (TriviaQA), and is close to chance on commonsense, multi-step and misconception questions (Section 5.6.2).
- **Confident confabulation after self-improvement:** the verifier scores more of the fine-tuned generator's wrong answers highly than it did the base generator's (0.305 at C5 against 0.208 for B1), so the gate's selectivity weakens as the generator it judges changes.
- **No repeated questions:** the evaluation uses fresh questions to measure learning, so the memory-direct path, built for repeated questions in deployment, and its cost saving were exercised only by the router probe; the gain in deployment depends on the share of repeats, which this study does not measure.

Limitations identified against the literature: six further limitations follow from the cited literature.

- **Saturation and format learning:** Song et al. [93] report that verifier-filtered self-improvement saturates within two or three rounds and that part of the first-round gain at larger scales comes from better adherence to the answer format. The run is consistent with early saturation, and because the main metric is exact match, part of the early gain may be format learning; a flexible-match re-scoring would separate the two.
- **Optimising toward the verifier:** Gulcehre et al. [48] find reward-model gains rising while human-judged quality peaks after the first round, and Yi et al. [96] show that verifier-filtered retraining converges to the verifier's own knowledge. Exact match against gold labels is independent of CAEM's verifier; CHM is not (Section 5.6.4).
- **Systematic verifier errors:** agreement-based signals do not detect misconceptions the model holds consistently, as Farquhar et al. [71] warn, and repeated verification removes only independent errors (Heuristic 4.8); TruthfulQA's low stored precision is the realised case. The sample-agreement signal is an embedding cosine similarity, a family that Sinha [92] shows can miss plausible hallucinations, although that evidence concerns similarity to a source rather than agreement across samples.
- **Calibration-fold size:** Niculescu-Mizil and Caruana [8] find isotonic regression reliable only from about 1,000 calibration points; CAEM fits per-benchmark curves on 500 points, and the shrinkage toward the pooled fit limits but does not remove the resulting variance.

- **Low chance floor:** Ho et al. [49] note that filtering on the final answer does not ensure correct reasoning, and Zelikman et al. [43] that binary tasks yield many correct answers with poor rationales. StrategyQA and FEVER have chance levels of one half and one third, and FEVER answers enter the training pool.
- **Adapter capacity and drift:** Biderman et al. [65] find that low-rank adaptation learns less of a new target than full fine-tuning and that its forgetting grows with training; the cycle-four rollback and the accuracy decline after cycle two are consistent with this, and the retention guard bounds the drift rather than preventing it.

5.7 Chapter signpost

This chapter reported the main run, the cycle-zero choices behind it, the comparison with eight baselines and the classifier ablation, and judged the four hypotheses: H1 and H3 supported, H4 partially supported, H2 unsupported. Chapter 6 restates the contributions against these results and turns the limitations above into future work.

Chapter 6

Conclusion and Future Work

This chapter states what the thesis built and what its evaluation showed, judges the four hypotheses, collects the limitations, and turns them into future work.

6.1 Conclusion

6.1.1 Contributions

The thesis set out to reduce confident hallucination in factual question answering on a small open generator by changing the system around the model rather than the model alone (Chapter 1). It contributes the following.

- **An architecture:** CAEM routes each question to a verified episodic memory, to zero-shot generation or to retrieval-augmented generation, with a retrieval-utility classifier that decides when retrieval is likely to help (Chapter 4). A verifier of nine signals produces a calibrated probability of correctness; a four-outcome decision stores, defers, abstains on or discards each answer; stored answers are re-checked at every cycle boundary and used to fine-tune a low-rank adapter under a retention guard that rolls the adapter back while keeping the memory.
- **A per-answer reliability signal:** every answer reaches the user with its probability and a status tag (verified, provisional, conflicting or insufficient evidence), and the answer is withheld only when the evidence is insufficient (Section 4.2.9).
- **A formal analysis of the stored pool:** two proven results, that storage makes the pool purer than the generator's raw output exactly when the verifier's balanced accuracy exceeds one half (Proposition 4.2) and that the pool's precision is bounded below by the calibration precision when new questions resemble the calibration data (Theorem 4.1), together with design heuristics whose idealising assumptions are stated and checked against the run (Section 4.3).
- **A matched evaluation:** a six-cycle run on five benchmarks against eight baselines under one comparison protocol (the same questions, prompt, scoring rule and hallucination instrument for every system), reported in a model view and a delivered view, with paired tests corrected for multiple comparisons

and every reported number rebuilt from the saved answers by one command (Section 5.1.1, Chapter E).

- **A diagnosed limit:** a measured account of where verifier-gated self-improvement helps, where it stops helping and why (Section 5.6).

6.1.2 Headline finding

CAEM is a label-efficient system that matches strong baselines, gives users a status tag that genuinely predicts correctness, and shows a measured, diagnosed limit of verifier-gated self-training. The evidence for each part of that statement is as follows.

Accuracy and hallucination: the architecture alone added almost no accuracy: pooled exact match was 0.517 at C0 against 0.511 for the zero-shot baseline B1, and CHM was higher (0.155 against 0.132) because the untrained system refused too often (Table 5.3). One or two cycles of self-improvement removed most of the over-refusal and gave the best readings of the run (0.535 and 0.127 at C2); later cycles eroded both, and the run ended level with B1 (0.508 and 0.134 at C5). No difference between CAEM and B1 is significant on any benchmark at C2 or C5.

Against the baselines: under the matched protocol CAEM is level with the strong prompting baselines and semantic-entropy abstention (B1, B2, B5, B8), with no significant exact-match win against any of them; it is significantly better than the retrieval baselines and FLARE on several benchmarks; and it matches the exact match of vanilla fine-tuning (B6, 0.512), which used a gold answer for each of 23,500 training questions against about 2,100 gold labels for CAEM, while B6 reached the lower CHM (0.103) (Tables 5.19 and 5.20). An earlier comparison that gave the baselines a shorter prompt had shown CAEM ahead on both measures; the matched comparison removed most of that lead.

Selectivity: what the architecture measurably adds is a reliable signal of which answers to trust. Answers tagged verified were right 65 to 75 percent of the time at every cycle, against 51 percent for B1 over all questions, and the tags are ordered by accuracy, verified above provisional above the other two (Table 5.4). At C0 CAEM was wrong on 40.5 percent of the questions it answered, less often than any baseline.

Where it works: the gains follow the verifier's ability to check answers against retrieved passages. On TriviaQA, where that ability is highest (area under the ROC curve 0.75 to 0.86), CAEM is most accurate relative to B1; on CommonsenseQA, StrategyQA and TruthfulQA, where the verifier is close to chance, it is level with or below the strong baselines (Section 5.6.2).

Why self-improvement stopped helping: the stored pool stayed about three-quarters correct, as the theory predicts, but the generator trained on it. Training on answers a verifier has passed converges toward the verifier's own view, as Yi et al. [96] show: confident confabulation rose from 0.208 for B1 to 0.305 at C5, so the verifier scored more of the fine-tuned generator's wrong answers highly. Training on chains written under the retrieval prompt also taught the generator to cite a context it was not given, in 93 to 99 percent of zero-shot answers after the first fine-tune (Section 5.6.3).

Routing: the retrieval-utility classifier, with accompanying corrections, raised pooled

exact match by 4.9 points at cycle three, mainly by retrieving less on questions where retrieval misleads (Section 5.4.2).

6.1.3 Hypothesis verdicts

The four pre-specified hypotheses of Section 1.4.3 are judged in Table 5.22.

- **H1, supported:** the gate's balanced accuracy on the calibration fold stayed above one half at every cycle (0.615 to 0.664 pooled), so the precondition for purification held; CommonsenseQA was the weakest benchmark (0.52 to 0.55).
- **H2, unsupported:** pool purity did not rise from cycle to cycle; calibration-fold precision fell at C1, C2 and C5, within sampling noise.
- **H3, supported:** precision stayed in a band of width 0.014 (0.732 to 0.746) over the kept cycles; a convergence rate cannot be estimated from six cycles.
- **H4, partially supported:** CHM fell after the first cycle and stayed between 0.127 and 0.137, consistent with flattening, but the limit itself cannot be estimated.

The two proven results concern the stored pool, and both held where their assumptions held. None of the formal statements bounds the accuracy of the model trained on the pool, which is where the run fell short.

6.1.4 Limitations

The limitations are discussed with their evidence in Sections 5.6.4 and 5.6.7; the ones that bound the conclusions above are these.

- **One seed, one backbone, five benchmarks:** every result comes from a single run of a three-billion-parameter generator on 300 evaluation questions per benchmark, and only the paired tests account for sampling noise.
- **Six cycles:** the run stopped after cycle five, as planned after the cycle-four rollback, which is too short to estimate convergence rates or long-run limits.
- **Verifier scope:** the verifier separates correct from incorrect answers well only where passages can confirm an answer, and its selectivity weakened as the generator it judged changed.
- **The hallucination instrument:** two of the seven CHM subtypes use CAEM's own verifier; exact match and the delivered view's error among answered do not depend on it.
- **No repeated questions:** the evaluation used fresh questions to measure learning, so the memory-direct tier and its cost saving were tested only by the router probe (Table 5.8).
- **Bundled ablation:** the classifier ablation also carries pipeline corrections, so it measures the classifier together with them.
- **Labels:** CAEM is label-efficient, not label-free; it used about 2,100 gold labels, and a deployment needs a labelled batch at every cycle (Section A.5).

6.1.5 Closing remark

The thesis began from the observation that a user cannot tell a confident wrong answer from a confident right one. CAEM does not make a small generator much more accurate, and its self-improvement loop helped for only one or two cycles. What it does provide is an answer that carries an honest estimate of its own reliability, a pool of stored answers that stayed as pure as the theory's floor predicts, and a measured account of why training on verifier-passed answers stopped helping, consistent with the verifier's blind spots becoming the generator's. The most useful next step is therefore a better verifier rather than more cycles.

6.2 Future Work

The items below follow from the limitations above and from the points that Chapters 4 and 5 left open. The first three address the causes of the limit the run measured; the rest complete measurements this study did not take.

6.2.1 A verifier for answers that passages cannot settle

The verifier is close to chance on commonsense, multi-step and misconception questions because its informative signals compare the answer with passages or with the question (Section 5.6.2). Four changes follow.

- **Answer-level internal signals:** the token signal scores the whole response, so it measures fluency more than confidence in the answer (Section 4.2.6); scoring the answer line alone, and computing semantic entropy over the final answers of sampled responses, the setting in which Farquhar et al. [71] found it effective, would give internal signals that track correctness.
- **Checking the reasoning:** every signal scores the final answer, so a chain with a wrong step and a correct conclusion passes. Applying the atomic-fact check to the reasoning chain, where several facts are asserted, would catch such chains (Section 4.2.6).
- **A learned verifier:** the composite combines fixed signals; a verifier trained on the calibration fold to judge answer and reasoning together, as V-STaR [73] trains its verifier on correct and incorrect self-generated solutions, could learn cues the fixed signals miss. The long-answer judge that was dropped at cycle zero (Section 5.2.6) could be revisited in the same way.
- **Off-topic answers:** the question-match signal acts as a correctness cue on short answers and does not detect off-topic ones (Section 4.2.6); a check on the full response, or a judge asked directly whether the answer addresses the question, would let the off-topic subtype of CHM be measured.

6.2.2 Contradiction-aware verification

The entailment judge returns only whether a passage supports a claim, so an answer that a passage contradicts scores the same as one the passages ignore, and the factual-contradiction subtype of CHM could not be measured (Section 5.1.1). A

three-way judge (support, contradiction, neither) would let the decision discard contradicted answers outright and would complete the metric. The language-model judge tried for the two unmeasured subtypes agreed too weakly with hand labels (Cohen's kappa about 0.35), so a better-validated judge is needed first.

6.2.3 Cleaner self-improvement

The loop's gains faded after two cycles while confident confabulation rose. Four changes target the cause.

- **Remove context references from training chains:** rewriting or removing references to a context in retrieval-written chains before fine-tuning would have prevented the phantom-context citations (Section 5.6.3).
- **Weight training answers by confidence:** about one stored answer in four is wrong, and the wrong ones carry lower scores; weighting each training answer by its calibrated probability, or raising the training threshold as the pool grows, would reduce the noise the generator learns from.
- **Retry when unsure:** answers the verifier is unsure about are now deferred or withheld; regenerating them, with retrieval or with more samples, before deciding would turn some of those into verified answers.
- **Adapter capacity:** Biderman et al. [65] find that low-rank adaptation forgets less but learns less than full fine-tuning; comparing ranks, or full fine-tuning on the verified pool, would show whether the plateau is set by the adapter or by the pool.

6.2.4 Scope to checkable questions

The other direction the verifier result points to is to narrow the system rather than widen the verifier: to deploy CAEM only where answers can be checked against documents, such as trivia, entity and document-grounded questions, and to measure it on a panel of such benchmarks. Because the main run fine-tuned on all three training benchmarks, such a study needs its own run rather than a re-reading of this one. Whether a narrow system or a general verifier is the better path is left open.

6.2.5 Isolating the effects

- **The classifier alone:** a run with the pipeline corrections but without the retrieval-utility classifier would separate the two parts of the measured 4.9-point gain (Section 5.4.2).
- **Multiple seeds:** repeating the run with different seeds would show how much of the cycle-to-cycle movement, of a size similar to the effects measured, is noise.
- **A flexible-match re-scoring:** part of the early gain may be format learning (Section 5.6.7); scoring with a flexible answer match as well as exact match would separate the two.

6.2.6 Testing the formal heuristics

The monotonicity heuristic and hypothesis H2 assume a fixed verifier, while the run refitted the composite every cycle. A cycle in which the composite is frozen and only the generator changes would separate the two effects (Section 4.3). Measuring the gate's true- and false-positive rates and the generator's accuracy on a labelled sample at every cycle would check the purification condition directly rather than through balanced accuracy.

6.2.7 A longer horizon

A convergence rate, the long-run limit of CHM, and the equilibrium behaviour under rollbacks all need at least fifteen kept cycles under a fixed composite (Section 4.3). A longer run would also show whether the deferred buffer, which ran only three reconsideration passes, recovers more answers over time, and whether a rotating calibration fold behaves better than a fixed one over many cycles (Section G.8).

6.2.8 Measurements this study did not take

- **Time per tier:** the batched evaluation log could not separate the time spent in each tier; timing each tier at a batch size of one would fill in the deployment-cost model (Section 5.1.4).
- **The corpus ceiling:** checking, for every evaluation question, whether any retrieved passage contains the gold answer would bound what retrieval can achieve (Remark 4.11).
- **Retention of the fine-tuned baseline:** the retention probes were run only on CAEM; running them on B6 would show whether the retention guard protects anything that plain fine-tuning loses.

6.2.9 Larger backbones and other domains

All results come from one three-billion-parameter generator on English factual question answering. Song et al. [93] find that small models often gain little from verifying their own outputs, so repeating the study on a larger generator, and on other domains and languages, would show how far the findings carry.

6.2.10 Deployment study

The memory-direct tier and the production design (Section A.5) were built for traffic in which users repeat questions, which this study did not have. A deployment with live users would measure the share of repeated questions, the cost saving of the memory-direct tier, how users read the status tag, and the labelling effort a cycle needs in practice.

References

[1] F. Wilcoxon, "Individual comparisons by ranking methods," *Biometrics Bulletin*, vol. 1, no. 6, pp. 80–83, 1945. DOI: 10.2307/3001968

[2] A. L. Edwards, "Note on the "correction for continuity" in testing the significance of the difference between correlated proportions," *Psychometrika*, vol. 13, no. 3, pp. 185–187, 1948. DOI: 10.1007/BF02289261

[3] H. Robbins and S. Monro, "A stochastic approximation method," *The Annals of Mathematical Statistics*, vol. 22, no. 3, pp. 400–407, 1951.

[4] H. Robbins and D. Siegmund, "A convergence theorem for non negative almost supermartingales and some applications," in *Optimizing Methods in Statistics*, J. S. Rustagi, Ed., Academic Press, 1971, pp. 233–257.

[5] S. Holm, "A simple sequentially rejective multiple test procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979. [Online]. Available: https://www.jstor.org/stable/4615733

[6] M. McCloskey and N. J. Cohen, "Catastrophic interference in connectionist networks: The sequential learning problem," in *Psychology of Learning and Motivation*, ser. Psychology of Learning and Motivation, G. H. Bower, Ed., vol. 24, Academic Press, 1989, pp. 109–165. DOI: 10.1016/S0079-7421(08)60536-8

[7] B. T. Polyak, "New stochastic approximation type procedures," *Automation and Remote Control*, vol. 51, no. 7, pp. 937–946, 1990.

[8] A. Niculescu-Mizil and R. Caruana, "Predicting good probabilities with supervised learning," in *Proceedings of the 22nd International Conference on Machine Learning (ICML 2005)*, 2005, pp. 625–632. DOI: 10.1145/1102351.1102430

[9] R. Bar-Haim et al., "The second PASCAL recognising textual entailment challenge," in *Proceedings of the Second PASCAL Challenges Workshop on Recognising Textual Entailment*, Venice, Italy, 2006.

[10] N. Natarajan, I. S. Dhillon, P. K. Ravikumar, and A. Tewari, "Learning with noisy labels," in *Advances in Neural Information Processing Systems*, vol. 26, 2013, pp. 1196–1204. [Online]. Available: https://papers.nips.cc/paper/5073-learning-with-noisy-labels.pdf

[11] A. Graves, G. Wayne, and I. Danihelka, *Neural turing machines*, arXiv preprint arXiv:1410.5401, 2014. DOI: 10.48550/arXiv.1410.5401

[12] M. P. Naeini, G. F. Cooper, and M. Hauskrecht, "Obtaining well calibrated probabilities using Bayesian binning," in *Proceedings of the Twenty-Ninth AAAI Conference on Artificial Intelligence*, 2015, pp. 2901–2907.

[13] S. Sukhbaatar, A. Szlam, J. Weston, and R. Fergus, "End-to-end memory networks," in *Advances in Neural Information Processing Systems 28 (NeurIPS 2015)*, C. Cortes, N. Lawrence, D. Lee, M. Sugiyama, and R. Garnett, Eds., vol. 28, Curran Associates, Inc., 2015. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2015/file/8fb21ee7a2207526da55a679f0332de2-Paper.pdf

[14] J. Weston, S. Chopra, and A. Bordes, "Memory networks," in *International Conference on Learning Representations (ICLR)*, 2015. eprint: https://arxiv.org/abs/1410.3916. [Online]. Available: https://arxiv.org/abs/1410.3916

[15] Y. Gal and Z. Ghahramani, "Dropout as a bayesian approximation: Representing model uncertainty in deep learning," in *Proceedings of The 33rd International Conference on Machine Learning*, M. F. Balcan and K. Q. Weinberger, Eds., ser. Proceedings of Machine Learning Research, vol. 48, New York, New York, USA: PMLR, Jun. 2016, pp. 1050–1059. [Online]. Available: http://proceedings.mlr.press/v48/gal16.pdf

[16] A. Graves et al., "Hybrid computing using a neural network with dynamic external memory," *Nature*, vol. 538, no. 7626, pp. 471–476, 2016. DOI: 10.1038/nature20101

[17] Y. Geifman and R. El-Yaniv, "Selective classification for deep neural networks," *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. [Online]. Available: https://arxiv.org/abs/1705.08500

[18] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *Proceedings of the 34th International Conference on Machine Learning*, D. Precup and Y. W. Teh, Eds., ser. Proceedings of Machine Learning Research, vol. 70, PMLR, Jun. 2017, pp. 1321–1330. [Online]. Available: http://proceedings.mlr.press/v70/guo17a/guo17a.pdf

[19] M. Joshi, E. Choi, D. Weld, and L. Zettlemoyer, "TriviaQA: A large scale distantly supervised challenge dataset for reading comprehension," in *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Vancouver, Canada: Association for Computational Linguistics, Jul. 2017, pp. 1601–1611. DOI: 10.18653/v1/P17-1147

[20] J. Kirkpatrick et al., "Overcoming catastrophic forgetting in neural networks," *Proceedings of the National Academy of Sciences*, vol. 114, no. 13, pp. 3521–3526, 2017. DOI: 10.1073/pnas.1611835114 eprint: https://www.pnas.org/doi/pdf/10.1073/pnas.1611835114.

[21] A. Vaswani et al., "Attention is all you need," in *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*, I. Guyon et al., Eds., vol. 30, Curran Associates, Inc., 2017. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf

[22] J. Thorne, A. Vlachos, C. Christodoulopoulos, and A. Mittal, "FEVER: A large-scale dataset for fact extraction and VERification," in *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*, M. Walker, H. Ji, and A. Stent, Eds., New Orleans, Louisiana: Association for Computational Linguistics, Jun. 2018, pp. 809–819. DOI: 10.18653/v1/N18-1074

[23] Z. Yang et al., "HotpotQA: A dataset for diverse, explainable multi-hop question answering," in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, E. Riloff, D. Chiang, J. Hockenmaier, and J. Tsujii, Eds., Brussels, Belgium: Association for Computational Linguistics, Oct. 2018, pp. 2369–2380. DOI: 10.18653/v1/D18-1259

[24] T. Kwiatkowski et al., "Natural questions: A benchmark for question answering research," *Transactions of the Association for Computational Linguistics*, vol. 7, pp. 452–466, 2019. DOI: 10.1162/tacl_a_00276

[25] Y. Liu et al., *RoBERTa: A robustly optimized BERT pretraining approach*, arXiv preprint arXiv:1907.11692, 2019. DOI: 10.48550/arXiv.1907.11692

[26] Y. Ovadia et al., "Can you trust your model's uncertainty? Evaluating predictive uncertainty under dataset shift," in *Advances in Neural Information Processing Systems 32 (NeurIPS 2019)*, 2019. [Online]. Available: https://papers.nips.cc/paper_files/paper/2019/hash/8558cb408c1d76621371888657d2eb1d-Abstract.html

[27] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, K. Inui, J. Jiang, V. Ng, and X. Wan, Eds., Hong Kong, China: Association for Computational Linguistics, Nov. 2019, pp. 3982–3992. DOI: 10.18653/v1/D19-1410

[28] E. Strubell, A. Ganesh, and A. McCallum, "Energy and policy considerations for deep learning in NLP," in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL 2019)*, 2019, pp. 3645–3650. DOI: 10.18653/v1/P19-1355

[29] A. Talmor, J. Herzig, N. Lourie, and J. Berant, "CommonsenseQA: A question answering challenge targeting commonsense knowledge," in *Proceedings of NAACL-HLT 2019*, Association for Computational Linguistics, 2019, pp. 4149–4158. DOI: 10.18653/v1/N19-1421

[30] S. Gururangan et al., "Don't stop pretraining: Adapt language models to domains and tasks," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, D. Jurafsky, J. Chai, N. Schluter, and J. Tetreault, Eds., Online: Association for Computational Linguistics, Jul. 2020, pp. 8342–8360. DOI: 10.18653/v1/2020.acl-main.740

[31] A. Kamath, R. Jia, and P. Liang, "Selective question answering under domain shift," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL 2020)*, 2020, pp. 5684–5696. DOI: 10.18653/v1/2020.acl-main.503

[32] V. Karpukhin et al., "Dense passage retrieval for open-domain question answering," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Online: Association for Computational Linguistics, Nov. 2020, pp. 6769–6781. DOI: 10.18653/v1/2020.emnlp-main.550

[33] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive nlp tasks," in *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, and H. Lin, Eds., vol. 33, Curran Associates, Inc., 2020, pp. 9459–9474. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2020/file/6b493230205f780e1bc26945df7481e5-Paper.pdf

[34] M. Geva, D. Khashabi, E. Segal, T. Khot, D. Roth, and J. Berant, "Did aristotle use a laptop? a question answering benchmark with implicit reasoning strategies," *Transactions of the Association for Computational Linguistics*, vol. 9, pp. 346–361, Apr. 2021. DOI: 10.1162/tacl_a_00370 eprint: https://direct.mit.edu/tacl/article-pdf/doi/10.1162/tacl_a_00370/1924104/tacl_a_00370.pdf.

[35] D. Hendrycks et al., "Measuring massive multitask language understanding," in *International Conference on Learning Representations (ICLR)*, 2021. [Online]. Available: https://openreview.net/forum?id=d7KBjmI3GmQ

[36] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with gpus," *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2021. DOI: 10.1109/TBDATA.2019.2921572

[37] E. J. Hu et al., "LoRA: Low-rank adaptation of large language models," in *International Conference on Learning Representations (ICLR)*, 2022. [Online]. Available: https://arxiv.org/abs/2106.09685

[38] S. Kadavath et al., *Language models (mostly) know what they know*, arXiv preprint arXiv:2207.05221, Nov. 2022. DOI: 10.48550/arXiv.2207.05221

[39] T. Kojima, S. S. Gu, M. Reid, Y. Matsuo, and Y. Iwasawa, "Large language models are zero-shot reasoners," in *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*, 2022, pp. 22 199–22 213. [Online]. Available: https://arxiv.org/abs/2205.11916

[40] S. Lin, J. Hilton, and O. Evans, "TruthfulQA: Measuring how models mimic human falsehoods," in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, S. Muresan, P. Nakov, and A. Villavicencio, Eds., Dublin, Ireland: Association for Computational Linguistics, May 2022, pp. 3214–3252. DOI: 10.18653/v1/2022.acl-long.229

[41] J. Wei et al., "Chain-of-thought prompting elicits reasoning in large language models," in *Advances in Neural Information Processing Systems 35 (NeurIPS)*, 2022, pp. 24 824–24 837.

[42] T. Wu, M. Caccia, Z. Li, Y.-F. Li, G. Qi, and G. Haffari, "Pretrained language model in continual learning: A comparative study," in *International Conference on Learning Representations (ICLR)*, 2022.

[43] E. Zelikman, Y. Wu, J. Mu, and N. Goodman, "Star: Bootstrapping reasoning with reasoning," in *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, Eds., vol. 35, Curran Associates, Inc., 2022, pp. 15 476–15 488. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2022/file/639a9a172c044fbb64175b5fad42e9a5-Paper-Conference.pdf

[44] H. Alkaissi and S. I. McFarlane, "Artificial hallucinations in ChatGPT: Implications in scientific writing," *Cureus*, vol. 15, no. 2, e35179, 2023. DOI: 10.7759/cureus.35179

[45] R. Anil et al., *PaLM 2 technical report*, arXiv preprint arXiv:2305.10403, 2023. DOI: 10.48550/arXiv.2305.10403

[46] Anthropic, *Claude 2*, Blog post, Jul. 2023. [Online]. Available: https://www.anthropic.com/news/claude-2

[47] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," in *Advances in Neural Information Processing Systems 36 (NeurIPS)*, 2023. [Online]. Available: https://arxiv.org/abs/2305.14314

[48] C. Gulcehre et al., *Reinforced self-training (ReST) for language modeling*, arXiv preprint arXiv:2308.08998, 2023.

[49] N. Ho, L. Schmid, and S.-Y. Yun, "Large language models are reasoning teachers," in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Association for Computational Linguistics, 2023, pp. 14 852–14 882. DOI: 10.18653/v1/2023.acl-long.830

[50] Z. Ji et al., "Survey of hallucination in natural language generation," *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023. DOI: 10.1145/3571730

[51] Z. Jiang et al., "Active retrieval augmented generation," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Singapore: Association for Computational Linguistics, Dec. 2023, pp. 7969–7992. DOI: 10.18653/v1/2023.emnlp-main.495

[52] D. Kalajdzievski, *A rank stabilization scaling factor for fine-tuning with LoRA*, arXiv preprint arXiv:2312.03732, 2023. [Online]. Available: https://arxiv.org/abs/2312.03732

[53] L. Kuhn, Y. Gal, and S. Farquhar, "Semantic uncertainty: Linguistic invariances for uncertainty estimation in natural language generation," in *International Conference on Learning Representations (ICLR)*, 2023.

[54] J. Li, X. Cheng, W. X. Zhao, J.-Y. Nie, and J.-R. Wen, "HaluEval: A large-scale hallucination evaluation benchmark for large language models," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, Singapore: Association for Computational Linguistics, Dec. 2023, pp. 6449–6464. DOI: 10.18653/v1/2023.emnlp-main.397

[55] A. Mallen, A. Asai, V. Zhong, R. Das, D. Khashabi, and H. Hajishirzi, "When not to trust language models: Investigating effectiveness of parametric and non-parametric memories," in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL 2023)*, Association for Computational Linguistics, 2023, pp. 9802–9822. DOI: 10.18653/v1/2023.acl-long.546

[56] P. Manakul, A. Liusie, and M. Gales, "SelfCheckGPT: Zero-resource black-box hallucination detection for generative large language models," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, H. Bouamor, J. Pino, and K. Bali, Eds., Singapore: Association for Computational Linguistics, Dec. 2023, pp. 9004–9017. DOI: 10.18653/v1/2023.emnlp-main.557

[57] S. Mehta, D. Patil, S. Chandar, and E. Strubell, "An empirical investigation of the role of pre-training in lifelong learning," *Journal of Machine Learning Research*, 2023.

[58] S. Min et al., "FActScore: Fine-grained atomic evaluation of factual precision in long form text generation," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, H. Bouamor, J. Pino, and K. Bali, Eds., Singapore: Association for Computational Linguistics, Dec. 2023, pp. 12 076–12 100. DOI: 10.18653/v1/2023.emnlp-main.741

[59] OpenAI, *GPT-4 technical report*, arXiv preprint arXiv:2303.08774, 2023. DOI: 10.48550/arXiv.2303.08774

[60] K. Tian et al., "Just ask for calibration: Strategies for eliciting calibrated confidence scores from language models fine-tuned with human feedback," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023, pp. 5433–5442. [Online]. Available: https://arxiv.org/abs/2305.14975

[61] X. Wang et al., "Orthogonal subspace learning for language model continual learning," in *Findings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP Findings)*, 2023.

[62] X. Wang et al., "Self-consistency improves chain of thought reasoning in language models," in *International Conference on Learning Representations (ICLR 2023)*, 2023. [Online]. Available: https://openreview.net/forum?id=1PL1NIMMrw

[63] Y. Wang, P. Li, M. Sun, and Y. Liu, "Self-knowledge guided retrieval augmentation for large language models," in *Findings of EMNLP 2023*, Association for Computational Linguistics, 2023, pp. 10 303–10 315. DOI: 10.18653/v1/2023.findings-emnlp.691

[64] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-RAG: Learning to retrieve, generate, and critique through self-reflection," in *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.

[65] D. Biderman et al., "LoRA learns less and forgets less," *Transactions on Machine Learning Research*, 2024.

[66] M. Chelli et al., "Hallucination rates and reference accuracy of ChatGPT and bard for systematic reviews: Comparative analysis," *J Med Internet Res*, vol. 26, e53164, May 2024. DOI: 10.2196/53164

[67] C. Chen et al., "INSIDE: LLMs' internal states retain the power of hallucination detection," in *International Conference on Learning Representations (ICLR 2024)*, 2024. [Online]. Available: https://arxiv.org/abs/2402.03744

[68] J. J. Cherian, I. Gibbs, and E. J. Candès, "Large language model validity via enhanced conformal prediction methods," in *Advances in Neural Information Processing Systems (NeurIPS 2024)*, 2024.

[69] H. W. Chung et al., "Scaling instruction-finetuned language models," *Journal of Machine Learning Research*, vol. 25, no. 70, pp. 1–53, 2024. [Online]. Available: http://jmlr.org/papers/v25/23-0870.html

[70] P. Das et al., "Larimar: Large language models with episodic memory control," in *Proceedings of the 41st International Conference on Machine Learning (ICML 2024)*, ser. Proceedings of Machine Learning Research, vol. 235, 2024. [Online]. Available: https://arxiv.org/abs/2403.11901

[71] S. Farquhar, J. Kossen, L. Kuhn, and Y. Gal, "Detecting hallucinations in large language models using semantic entropy," *Nature*, vol. 630, no. 8017, pp. 625–630, 2024. DOI: 10.1038/s41586-024-07421-0

[72] J. Geng, F. Cai, Y. Wang, H. Koeppl, P. Nakov, and I. Gurevych, "A survey of confidence estimation and calibration in large language models," in *Proceedings of the 2024 Annual Conference of the North American Chapter of the Association for Computational Linguistics (NAACL 2024)*, 2024. [Online]. Available: https://aclanthology.org/2024.naacl-long.366/

[73] A. Hosseini, X. Yuan, N. Malkin, A. Courville, A. Sordoni, and R. Agarwal, "V-STaR: Training verifiers for self-taught reasoners," in *Conference on Language Modeling (COLM 2024)*, 2024. [Online]. Available: https://arxiv.org/abs/2402.06457

[74] J. Huang et al., "Large language models cannot self-correct reasoning yet," in *International Conference on Learning Representations (ICLR 2024)*, 2024. [Online]. Available: https://arxiv.org/abs/2310.01798

[75] S. Jeong, J. Baek, S. Cho, S. J. Hwang, and J. C. Park, "Adaptive-RAG: Learning to adapt retrieval-augmented large language models through question complexity," in *Proceedings of the 2024 Annual Conference of the North American Chapter of the Association for Computational Linguistics (NAACL 2024)*, 2024. [Online]. Available: https://arxiv.org/abs/2403.14403

[76] R. Kamoi, Y. Zhang, N. Zhang, J. Han, and R. Zhang, "When can LLMs actually correct their own mistakes? a critical survey of self-correction of LLMs," *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 1417–1440, 2024. DOI: 10.1162/tacl_a_00713

[77] H. Lang, D. Sontag, and A. Vijayaraghavan, "Theoretical analysis of weak-to-strong generalization," in *Advances in Neural Information Processing Systems (NeurIPS 2024)*, 2024. [Online]. Available: https://arxiv.org/abs/2405.16043

[78] C. Mohri and T. Hashimoto, "Language models with conformal factuality guarantees," in *Proceedings of the 41st International Conference on Machine Learning (ICML 2024)*, 2024. [Online]. Available: https://arxiv.org/abs/2402.10978

[79] A. Nikitin, J. Kossen, Y. Gal, and P. Marttinen, "Kernel language entropy: Fine-grained uncertainty quantification for LLMs from semantic similarities," in *Advances in Neural Information Processing Systems (NeurIPS 2024)*, 2024. [Online]. Available: https://arxiv.org/abs/2405.20003

[80] V. Quach et al., "Conformal language modeling," in *International Conference on Learning Representations (ICLR 2024)*, 2024. [Online]. Available: https://arxiv.org/abs/2306.10193

[81] Qwen Team, *Qwen2.5: A party of foundation models*, arXiv preprint arXiv:2412.15115, 2024. DOI: 10.48550/arXiv.2412.15115

[82] L. Tang, P. Laban, and G. Durrett, "MiniCheck: Efficient fact-checking of LLMs on grounding documents," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2024, pp. 8818–8847.

[83] S. Valentin, J. Fu, G. Detommaso, S. Xu, G. Zappella, and B. Wang, *Cost-effective hallucination detection for LLMs*, arXiv preprint arXiv:2407.21424, 2024.

[84] M. Xiong et al., "Can LLMs express their uncertainty? an empirical evaluation of confidence elicitation in LLMs," in *International Conference on Learning Representations (ICLR 2024)*, 2024. [Online]. Available: https://arxiv.org/abs/2306.13063

[85] Y. A. Yadkori et al., *Mitigating LLM hallucinations via conformal abstention*, arXiv preprint arXiv:2405.01563, 2024. [Online]. Available: https://arxiv.org/abs/2405.01563

[86] I. Baek, H. Chang, B. Kim, J. Lee, and H. Lee, "Probing-RAG: Self-probing to guide language models in selective document retrieval," in *Findings of the Association for Computational Linguistics: NAACL 2025*, 2025, pp. 3287–3304.

[87] R. Das et al., "Retraining with predicted hard labels provably increases model accuracy," in *Proceedings of the 42nd International Conference on Machine Learning (ICML 2025)*, ser. Proceedings of Machine Learning Research, vol. 267, 2025, pp. 12 509–12 538. [Online]. Available: https://arxiv.org/abs/2406.11206

[88] A. Huang et al., "Self-improvement in language models: The sharpening mechanism," in *International Conference on Learning Representations (ICLR 2025)*, 2025. [Online]. Available: https://arxiv.org/abs/2412.01951

[89] L. Huang et al., "A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions," *ACM Transactions on Information Systems*, vol. 43, no. 2, 2025. DOI: 10.1145/3703155

[90] M. Marina et al., "LLM-independent adaptive RAG: Let the question speak for itself," in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, 2025, pp. 8697–8709. [Online]. Available: https://arxiv.org/abs/2505.04253

[91] M. Nandakishor, *Confidence-aware routing for large language model reliability enhancement: A multi-signal approach to pre-generation hallucination mitigation*, arXiv preprint arXiv:2510.01237, 2025.

[92] D. Sinha, *The semantic illusion: Certified limits of embedding-based hallucination detection in RAG systems*, arXiv preprint arXiv:2512.15068, 2025. [Online]. Available: https://arxiv.org/abs/2512.15068

[93] Y. Song, H. Zhang, C. Eisenach, S. Kakade, D. Foster, and U. Ghai, "Mind the gap: Examining the self-improvement capabilities of large language models," in *International Conference on Learning Representations (ICLR 2025)*, 2025. [Online]. Available: https://arxiv.org/abs/2412.02674

[94] R. Vashurin et al., "Benchmarking uncertainty quantification methods for large language models with LM-Polygraph," *Transactions of the Association for Computational Linguistics*, vol. 13, pp. 220–248, 2025. DOI: 10.1162/tacl_a_00737

[95] W. Xu, Z. Liang, K. Mei, H. Gao, J. Tan, and Y. Zhang, *A-MEM: Agentic memory for LLM agents*, arXiv preprint arXiv:2502.12110, 2025. [Online]. Available: https://arxiv.org/abs/2502.12110

[96] B. Yi, Q. Liu, Y. Cheng, and H. Xu, "Escaping model collapse via synthetic data verification: Near-term improvements and long-term convergence," in *International Conference on Learning Representations (ICLR 2026)*, 2026. [Online]. Available: https://arxiv.org/abs/2510.16657

Appendix A

Implementation reference

A.1 Code and data availability

The full implementation of the architecture, the cycle runner, the calibration scripts, the baseline and ablation drivers, the verifier ensemble, the retrieval-utility classifier, and the figure and table aggregators is released as the open-source repository at https://github.com/aksaN000/caem-thesis. The empirical artefact bundle for the realised six-cycle trajectory, the cycle-zero validation gate, the eight-baseline panel, the retrieval-utility-classifier on/off ablation, the per-cycle logs, and the post-trajectory analysis is released as the Google Drive folder at https://drive.google.com/drive/folders/1xErhsRDYe_qO0ZgUBn0WXK81eHc5jZCa, organised under the subfolder `v2_1_phase1e_main/` (the realised trajectory) alongside `pre_main_snapshot/` (the cycle-zero anchor for snapshot recovery) and the earlier-phase archives. The sixty-four-gigabyte Wikipedia passage FAISS index and the per-commit pre-main snapshot bundles are additionally mirrored on the Hugging Face dataset repository `aksaN000/caem-passage-index-21m` (private; access on request), as documented in Section E.2. The three publication endpoints are intentionally complementary: the GitHub repository carries the executable code and the locked configuration JSONs, the Google Drive folder carries the realised empirical artefacts and the per-cycle JSON/CSV logs that the thesis reads, and the Hugging Face dataset host carries the bulky retrieval substrate and the snapshot bundles that anchor the credit-burnout recovery sequence of Section E.2.

A.2 Repository layout and module map

The repository root is `caem/` at the top of the Vast workspace. Body prose throughout the thesis refers to the architectural components by concept name; the inventory below maps every concept to the file that implements it. Paths are relative to the repository root.

Table A.1: Repository layout. Body prose throughout the thesis refers to architectural components by their concept names (memory store, three-tier router, verifier ensemble, retrieval-utility classifier, self-improvement loop, retention guard); this table maps each concept to the file that implements it.

Path	Role	Description
Top-level directories		
`caem/`	package root	Core Python package: pipeline, memory, routing, verification, training.
`scripts/`	runners	Cycle runner, calibration scripts, baseline runners, rescore utilities.
`eval/`	harness	Shared per-benchmark eval scaffold (EM, em_llm_judged, CHM, CES).
`outputs/`	artefacts	Per-cycle JSON, CSV, PNG, and log evidence (cycle_0/, full_run/, baselines/).
`data/`	external	Passage index, calibration pairs, alias dictionary, retention and RUC datasets.
`tests/`	test suite	Pytest suite covering storage gate, deferred buffer, retention probe, SIL, verifier.
`docs/`	runbook	PRODUCTION_RUNBOOK.md: deployment-mode spec and cycle-trigger policies.
`thesis_report/`	thesis	LaTeX chapters, bibliography, figures, main.pdf build target.
Pipeline entry points		
`caem/pipeline.py`	per-query	Pipeline class: serial driver used in production and demo paths.
`caem/pipeline_batch.py`	per-cycle	BatchPipeline class: batched cycle driver used by the experiment runner.
`scripts/run_experiment.py`	runner	Canonical cycle driver: invokes BatchPipeline, SIL, retention probe, retroverify.
Configuration		
`caem/config.py`	registry	Central CAEMConfig dataclass; every threshold, weight, structural toggle.
`caem/ruc/v2/feature_spec.json`	RUC config	RUC threshold, regulariser, feature order, model paths.
Memory module		
`caem/memory/store.py`	storage gate	EpisodicMemoryStore: fixed-threshold admission and FAISS-backed retrieval.
`caem/memory/deferred.py`	deferred buffer	DeferredBuffer and DeferredEntry: TTL-bounded reconsider queue.
`caem/memory/encoder.py`	embeddings	SBERT-side encoder for memory keys and Tier 1 lookup.
Routing		
`caem/routing/router.py`	tier dispatch	Tier 1/2/3 router with safety override and combined-score thresholds.
`caem/routing/ruc.py`	RUC head	LRRetrievalUtilityClassifier: binary head for Tier 2 versus Tier 3 routing.
Verification		
`caem/verification/verifier.py`	verifier ensemble	UnifiedVerifier: nine-signal composite producing u_{pre} and u_{stored}.
`caem/verification/adaptive_nli_judge.py`	NLI dispatch	AdaptiveNLIJudge: length-gated dispatcher; in the deployed configuration every hypothesis routes to MiniCheck under a 512-token truncation contract because the long-hypothesis Qwen-3B backend was ablated (Section 5.2.6).
`caem/verification/cal_prob_composite.py`	calibration	Per-signal isotonic curves plus per-benchmark shrinkage and boost layer.
`caem/retrieval/rag.py`	retrieval	FAISS-backed passage index reader for Tier 3 grounded generation.
Training		
`caem/training/self_improvement.py`	SIL trainer	SelfImprovementLoop: LoRA fine-tune and retroverify hook.

Path	Role	Description
`caem/training/retention_probe.py`	retention guard	Per-cycle multi-modal probe panel and ratio-floor check.
`caem/training/pool_reweighting.py`	pool balancer	Per-benchmark temperature-mixed reweighting and share ceiling.
`caem/training/loop_filter.py`	SIL admission	Per-pair filter at $\tau_{\text{train}} = 0.70$ above the storage cut-point.
Evaluation		
`eval/harness.py`	eval scaffold	Per-benchmark EM, F1, latency, CHM, and CES.
`eval/benchmarks.py`	benchmark loaders	TriviaQA, FEVER, CommonsenseQA, TruthfulQA, StrategyQA, MMLU loaders.
`caem/eval/equilibrium.py`	SE metric	Stochastic-equilibrium computation for the convergence-check table.

A.3 Configuration registry

Every constant that governs runtime behaviour lives in the `CAEMConfig` dataclass at `caem/config.py`. The exception is the retrieval-utility classifier, whose configuration lives in `caem/ruc/v2/feature_spec.json` at deployment time (the classifier is loaded once at pipeline construction and is not refit at cycle boundaries; see Section 4.2.5). The registry below groups every field by concept, records the deployed value, and tags the provenance as *literature* (anchored to a cited paper), *design* (architectural decision documented before any run), *calibrated* (selected empirically at cycle zero), or *pre-specified* (chosen before any data is observed).

Table A.2: Configuration registry. Provenance tags: *literature* (cited paper), *design* (architectural choice documented before any run), *calibrated* (chosen empirically at cycle zero or refit at a cycle boundary), *pre-specified* (registered horizon before any data is observed).

Field	Value	Provenance	Source / rationale
Routing thresholds (Section 4.2.4)			
`routing_lambda`	0.70	design	Combined-score weight on similarity.
`tier1_combined_threshold`	0.90	design	Tier 1 dispatch floor on combined score.
`tier2_similarity_threshold`	0.75	design	Tier 2 dispatch floor on similarity alone.
`safety_u_pre_min`	0.38	calibrated	Re-tuned at cycle one from the registered 0.60; safety-override floor.
`u_pre_token_weight`	0.60	design	Token-probability weight in u_{pre}.
`u_pre_cconv_weight`	0.40	design	Encoder-convergence weight in u_{pre}.
Storage gate thresholds (Section 4.2.9)			
`store_threshold`	0.60	design	STORE if $u_{\text{stored}} \geq$ this.
`defer_threshold`	0.45	design	DEFER if `defer_threshold` $\leq u_{\text{stored}} <$ `store_threshold`.
`abstain_pground_ceiling`	0.20	design	ABSTAIN if $u_{\text{stored}} <$ `defer_threshold` and $p_{\text{ground,max}} <$ this.
`early_exit_u_internal`	0.70	design	Confabulation early-exit on internal-uncertainty floor.
`early_exit_p_ground_max`	0.20	design	Confabulation early-exit on grounding ceiling.
Calibration parameters (Section 4.2.7)			

Field	Value	Provenance	Source / rationale
composite_mode	"auto"	design	Auto-switch to calibrated probability composite when the calibration JSON loads.
composite_shrinkage_alpha	0.6	calibrated	Per-benchmark child curve shrinkage toward pooled fit.
adaptive_thresholds_per_cycle	true	design	Refit temperature, isotonic curves, and boost weights at every successful cycle.
adaptive_thresholds_ema_alpha	0.7	design	Exponential-moving-average smoothing on the per-cycle temperature refit.
Retention guard (Section 5.1.5)			
forgetting_tolerance	0.93	design	Worst-probe ratio floor; trigger asymmetric rollback if any probe drops below.
retention_probes	3-probe panel	design	Multi-modal probe panel (mmlu, triviaqa_test, commonsense_qa_test); replaces the single-MMLU probe of v1.
retention_probe_n	200	design	Samples per probe at every cycle.
Deferred buffer (Section 4.2.10)			
deferred_buffer_max_size	10 000	design	Bounded FIFO capacity for deferred entries.
deferred_buffer_ttl_cycles	2	design	Reconsider-invocation TTL; entries dropped after two passes.
Retroverify and consolidation (Section 4.2.10)			
retroverify_prune_threshold	0.50	design	Prune from memory if recalibrated u_{stored} drops below.
hit_counter_force_retroverify	10	design	Per-entry retrieval-hit threshold forcing reverification next cycle.
enable_consolidation	true	design	Cycle-boundary SBERT-similarity clustering pass.
consolidation_similarity_threshold	0.92	design	Minimum cosine similarity for cluster admission.
consolidation_search_k	20	design	Per-entry FAISS search depth for neighbours.
consolidation_max_u_spread	0.15	design	Spread guardrail; reject clusters whose u_{stored} range exceeds this.
Self-improvement training (Section 4.2.10)			
use_lora_training	true	design	LoRA SIL primary path; replaces the v1 full-fine-tune fallback.
lora_r	32	literature	Rank, per Biderman et al. 2024 audit on Qwen-class backbones.
lora_alpha	64	literature	Standard $\alpha = 2r$ scaling rule.
lora_dropout	0.05	literature	Hu et al. 2021 default.
lora_target_modules	(q,k,v,o, gate,up,down)_proj	literature	All linear projections of the transformer block.
lora_learning_rate	2×10^{-4}	literature	Standard LoRA learning rate.
use_8bit_adamw	true	design	Applies only to the full fine-tuning fallback; the LoRA path overrides it and uses 32-bit AdamW.
batch_size	4	calibrated	Reduced from 16 at cycle one after GPU memory pressure.
grad_accum_steps	4	design	Holds the effective batch at 16.
epochs_per_cycle	3	design	SIL training duration per successful cycle.

Field	Value	Provenance	Source / rationale
warmup_steps	500	design	Linear warmup before the standard schedule.
min_u_stored_for_training	0.70	calibrated	τ_{train}; reduced from 0.75 at cycle one.
pool_reweighting_enabled	true	design	Temperature-mixed reweighting across training benchmarks.
pool_reweighting_temperature	2.0	literature	Mixture temperature; Du et al. 2022.
pool_reweighting_upsample_cap	3.0	calibrated	Upper bound on per-benchmark upsampling factor.
pool_reweighting_max_share	0.40	design	Per-benchmark hard share ceiling.
loop_distinct4_threshold	0.25	calibrated	SIL-pool loop filter on token-4-gram distinct ratio.
Verifier ensemble (Section 4.2.6)			
verifier_backend	"minicheck"	literature	Short-hypothesis backend; Tang et al. 2024.
minicheck_model	lytang/MiniCheck-Flan-T5-Large	literature	Verifier checkpoint.
cross_encoder_model	BAAI/bge-reranker-v2-m3	design	Shared reranker; consumed by grounding and question-answer match.
verifier_retrieve_k	20	design	Top-k passages retrieved before reranking.
verifier_rerank_k	3	design	Top-N retained after reranking.
atomic_min_tokens	12	calibrated	Length gate below which atomic decomposition is bypassed.
mc_dropout_k, mc_dropout_rate	5, 0.1	literature	Gal and Ghahramani 2016.
sc_chains_m	3	literature	Wang et al. 2022 self-consistency.
se_samples_k, se_temperature	10, 1.0	literature	Farquhar et al. 2024 semantic-entropy protocol.
disable_h_norm	true	calibrated	Retired at cycle zero on cal-fold Cohen's d audit.
disable_p_contra	true	design	Structurally zero under the MiniCheck binary judge.
Generation and retrieval			
base_model_name	Qwen/Qwen2.5-3B-Instruct	design	Apache-licensed three-billion-parameter instruction-tuned backbone.
rag_max_context_tokens	384	design	Cap on retrieved context tokens.
rag_max_new_tokens	512	design	Generation budget under retrieval-augmented prompt.
rag_do_sample	false	design	Greedy decoding for reproducibility.
rag_index_type	"ivf_pq"	design	FAISS index backend.
rag_faiss_nlist, nprobe_tier3, nprobe_tier2	32768, 64, 16	design	IVF-PQ density and adaptive probe budget.
rag_top_k	5	literature	Karpukhin et al. 2020 DPR standard.
Memory store (Section 4.2.3)			

Field	Value	Provenance	Source / rationale
`sbert_model`	`sentence-transformers/all-mpnet-base-v2`	literature	768-dimensional sentence encoder.
`embedding_dim`	768	literature	Must match the encoder output dimension.
`max_memory_size`	1 000 000	design	Capacity for thesis-scale and production-scale runs.
`memory_index_type`	`"ivf_pq"`	design	FAISS index backend.
`novelty_threshold`	0.95	design	Skip storage if nearest-neighbour cosine similarity exceeds this.
`pruning_trigger`	0.95	design	Trigger pruning when memory reaches this fraction of capacity.
`pruning_amount`	0.20	design	Remove the bottom fraction by value score.
`value_importance_weight`, `value_recency_weight`	0.70, 0.30	design	Value-score decomposition.
Experiment scope			
`num_cycles`	10	pre-specified	Registered horizon; realised trajectory closed at cycle five under the equilibrium-saturation gate.
`calibration_set_size`	500	design	Cycle-zero calibration-fold size per training benchmark.
`purity_validation_set_size`	500	design	Cycle-zero purity-validation slice per training benchmark.
`questions_per_cycle`	5 000	design	Stream-chunk question budget per cycle.

The retrieval-utility classifier (Section 4.2.5) has no fields in `caem/config.py`. Its operating point lives in `caem/ruc/v2/feature_spec.json`: the threshold $\tau_{\text{RUC}} = 0.375$, the regularisation strength $C = 0.5$, the thirty-six-feature ordering, the standard-scaler and logistic-head joblib paths, and the model-type guard `"logistic_regression_scaled"`. The classifier is loaded once at pipeline construction from the on-disk specification and is not refit at any cycle boundary; the architectural rationale is documented in Section 4.2.5.

A.4 Hyperparameter listing

The consolidated hyperparameter table below pairs every quantity that the thesis discusses by symbol with its deployed value, its source file, and the chapter subsection that introduces the symbol. The rightmost column tags whether the cycle-boundary refit moves the quantity (*refit*), whether it stays architecturally frozen across the trajectory (*frozen*), or whether it was calibrated empirically at cycle zero and held constant thereafter (*C0-cal frozen*).

Table A.3: Consolidated hyperparameter table. The four per-cycle refit quantities are the pooled temperature scalar, the per-benchmark temperature vector, the nine per-signal isotonic curves, and the boost-layer weights plus intercept. Every other quantity is architecturally frozen across the trajectory.

Symbol (thesis)	**Value**	**Source**	**Cycle role**
λ (router blend)	0.70	`config.py: routing_lambda`	frozen
θ_1 (Tier 1 floor)	0.90	`tier1_combined_threshold`	frozen
θ_{sim} (Tier 2 floor)	0.75	`tier2_similarity_threshold`	frozen
ϕ_{safe} (safety floor)	0.38	`safety_u_pre_min`	C0-cal frozen
τ_{store}	0.60	`store_threshold`	frozen
τ_{defer}	0.45	`defer_threshold`	frozen
τ_{train}	0.70	`min_u_stored_for_training`	C0-cal frozen
τ_{retro}	0.50	`retroverify_prune_threshold`	frozen
τ_{RUC}	0.375	`ruc/v2/feature_spec.json`	frozen
$\rho_{\min}$ (retention floor)	0.93	`forgetting_tolerance`	frozen
T (pooled temperature)	$20.09 \rightarrow 0.74$ band	per-cycle `calibrated_config_cycle*.json`	**refit**
T_b (per-benchmark)	per-bench tables	per-cycle `calibrated_config_cycle*.json`	**refit**
α_{shrink}	0.6	`composite_shrinkage_alpha`	frozen
C (boost L2)	0.01	boost layer fit in composite refit	frozen
Per-signal isotonic curves	non-parametric step functions	`composite_calibration.json`	**refit**
Boost-layer weights (nine signals)	per-cycle vectors	`composite_calibration.json`	**refit**
LoRA r, α, dropout	32, 64, 0.05	`lora_*`	frozen
LoRA learning rate	2×10^{-4}	`lora_learning_rate`	frozen
LoRA target modules	all-linear seven projections	`lora_target_modules`	frozen
SIL batch $\times$ accum	$4 \times 4 = 16$	`batch_size`, `grad_accum_steps`	frozen
SIL epochs per cycle	3	`epochs_per_cycle`	frozen
$\alpha_{\text{T-EMA}}$	0.7	`adaptive_thresholds_ema_alpha`	frozen
Pool reweighting temperature	2.0	`pool_reweighting_temperature`	frozen
Pool reweighting upsample cap	3.0	`pool_reweighting_upsample_cap`	frozen
Pool reweighting share ceiling	0.40	`pool_reweighting_max_share`	frozen
Deferred-buffer TTL	2 reconsider invocations	`deferred_buffer_ttl_cycles`	frozen
Consolidation similarity floor	0.92	`consolidation_similarity_threshold`	frozen

Symbol (thesis)	Value	Source	Cycle role
Consolidation FAISS-k	20	`consolidation_search_k`	frozen
Consolidation u-spread guard	0.15	`consolidation_max_u_spread`	frozen
MC-dropout K, p	5, 0.1	`mc_dropout_k`, `mc_dropout_rate`	frozen
Self-consistency chains M	3	`sc_chains_m`	frozen
SE samples K, temperature	10, 1.0	`se_samples_k`, `se_temperature`	frozen
Verifier retrieve-k, rerank-N	20, 3	`verifier_retrieve_k`, `verifier_rerank_k`	frozen
Atomic length gate (tokens)	12	`atomic_min_tokens`	C0-cal frozen
RAG context tokens, new tokens	384, 512	`rag_max_*`	frozen
SBERT embedding dimension	768	`embedding_dim`	frozen
Memory FAISS (nlist, nprobe, m, nbits)	4096, 32, 64, 8	`faiss_*`	frozen
RAG FAISS (nlist, nprobe$_{T3}$, $T2$)	32768, 64, 16	`rag_faiss_*`	frozen

A.5 Runbook step graph

The end-to-end execution sequence partitions into a pre-run setup block, a six-step main cycle loop, and a post-run analysis block. The pre-run block runs once before the first cycle; the main cycle loop repeats per cycle; the post-run block runs once after the final cycle closes. Authoritative paths: `run_phase1a.sh` (the top-level driver), `scripts/run_experiment.py` (the in-cycle implementation), and `docs/PRODUCTION_RUNBOOK.md` Section 5 (the production mirror).

Pre-run setup (Steps 1–6):

1. **Prompt smoke test** (`step_5_9_prompt_smoke`). Renders five samples per benchmark against the registered prompt templates and asserts the answer-line contract. Gate: every sample renders with a non-empty answer line.
2. **Alias dictionary build** (`step_5_9_5_alias_dict`). Builds `data/alias_dict.pkl` from a local Wikidata dump. The alias signal is retired in the deployed composite; the dictionary is preserved for ablation only.
3. **Cold-start seeding** (`step_6_reseed` and `step_6_5_em_prune`). Seeds the episodic memory with verified entries from the training benchmarks' dev splits under a cold-start storage threshold of 0.50; the realised cold-start store holds 431 entries (the seeding plan targeted approximately three thousand candidates before deduplication and hygiene). Gate: deduplication ratio sane; the EM-prune step removes fewer than ten percent of seeded entries.
4. **Cycle-zero baseline evaluation and cal-fold scoring** (`step_7_0_cycle0`).

Runs the pristine generator against the held-out evaluation fold and the calibration fold (five hundred samples per training benchmark, fifteen hundred pooled). Writes per-sample nine-signal vectors and ground-truth labels. Gate: every JSON has the expected sample count and no NaN signals.

5. **Composite calibration and validation gate** (`step_7_0_calibrate`, `step_7_0_2_5_rescore_eval`, `step_7_0_3_validate_weights`). Fits the per-signal isotonic curves, the per-benchmark shrinkage-blended children, the boost-layer logistic regression, the per-benchmark temperature scalars, and the per-benchmark safety floors. Runs the cycle-zero validation gate (Section A.9). Gate: cycle-zero canonical composite locked at `outputs/cycle_0/composite_calibration.json`; never overwritten by later cycles.

6. **Retention baseline freeze and snapshot upload** (`step_19_2_slice` and `step_gdrive_upload_pre_main`). Records the pristine-model probe accuracies on MMLU, TriviaQA-test, and CommonsenseQA-test as the cycle-zero anchor for the 0.93 retention floor. Pushes the snapshot to Hugging Face for credit-burnout recovery (Section A.6).

Main cycle loop (Steps 1–6, repeating per cycle):

1. **Self-improvement fine-tune:** Draws the SIL training pool from memory entries above $\tau_{\text{train}} = 0.70$, applies the pool-reweighting layer, runs the LoRA fine-tune for three epochs, and probes the retention panel. Gate: every probe ratio against the cycle-zero baseline clears 0.93. On failure, the adapter is rolled back to the previous cycle's checkpoint, the memory is preserved, and Steps 2.1–2.5 are skipped under the asymmetric-rollback contract.

2. **Cycle-boundary recalibration:** Re-scores the calibration fold under the post-fine-tune model, refits the per-benchmark temperature scalars, refits the per-signal isotonic curves under the shrinkage prior, and runs the retroactive re-verification pass against the stored memory under the recalibrated verifier. Skipped on aborted cycles.

3. **Stream-chunk pre-routing pass:** Runs the per-query pipeline on the cycle's stream chunk (two thousand FEVER, two thousand TriviaQA, seven hundred CommonsenseQA samples), accumulating storage admissions, deferred entries, abstentions, and discards.

4. **Stream-chunk verification and storage:** Applies the calibrated probability composite, the four-outcome decision tree, and the storage admission. Writes `memory_store_cycle_N.{faiss,meta}` and `deferred_buffer_cycle_N.pkl`.

5. **Held-out evaluation:** Runs the per-query pipeline on the held-out evaluation fold (three hundred samples per benchmark across the five-benchmark panel) under `store_to_memory=False` so the eval fold does not contaminate the memory pool.

6. **Checkpoint and halt-trigger evaluation:** Writes the cycle's coverage diagnostic, the per-tier hit-rate roll-up, and the per-cycle composite calibration

snapshot. Evaluates the halt trigger (storage saturation, retention floor failures, or the equilibrium-saturation gate).

Post-run analysis:

1. Re-run the baseline panel under the comparison protocol of Section 5.1.1 (`scripts/rental/final_version.sh`): exact-search retrieval of the evaluation passages, the baselines, the evaluation of the vanilla fine-tune, and the verifier pass over every baseline answer with the cycle-three adapter and composite.
2. Judge every TruthfulQA answer, CAEM's and every baseline's, with the language-model judge on the final answer line (`scripts/rescore_truthfulqa.py`).
3. Rebuild every reported number with one command (`scripts/make_final_results.sh`): the final correctness label, CHM in the model and delivered views, the status-tag accuracy, the paired tests (McNemar on exact match, Wilcoxon on CHM, bootstrap intervals, Holm within each benchmark), and every table and figure of Chapter 5.
4. Push the post-trajectory artefact bundle to the shared Drive folder (Section A.1).

Asymmetric-rollback contract: Triggered when any retention probe drops below 0.93 of its cycle-zero baseline during Step 1 of the main cycle loop. The adapter reverts to the previous successful cycle's checkpoint, but the memory state does not revert; storage admissions from the failed cycle's stream-chunk pass are preserved. Steps 2.1–2.5 are skipped on the aborted cycle (the cycle-boundary recalibration, retroactive re-verification, and deferred-buffer reconsider all become provable no-ops under unchanged weights). The retroverify log records `"skipped": "aborted_cycle"` so downstream readers distinguish the skip from a genuine zero-delta reverify. Steps 3–6 of the main cycle loop continue to run on aborted cycles: the stream-chunk grows memory unconditionally, the held-out evaluation runs under the rolled-back model so the empirical chapter can compare the aborted cycle's exact match to the previous successful cycle's, and the checkpoint and halt-trigger step records the abort cleanly. The empirical receipt for the contract is the cycle-four firing on the realised trajectory: the TriviaQA-test probe ratio of 0.886 crossed below the floor and triggered the rollback, the cycle-five reconsider pass recovered the C4-pushed deferred entries that the skipped cycle-four pass had preserved, and the realised commit fraction closed at $\pi_{\text{commit}} = 4/5$.

A.6 Snapshot reference

The pre-main snapshot is hosted at the private Hugging Face dataset repository `aksaN000/caem-passage-index-21m`. The repository holds both the sixty-four-gigabyte Wikipedia passage FAISS index at the repository root (`passages.{faiss,meta}`) and the per-commit pre-main artefact bundles under `pre_main_snapshot/<git-commit>/` subtrees. The canonical pre-Step-7 anchor is the commit tag `c5b8066/`; the later RUC-era anchor is `700b9e3/`. The mani-

fest `pre_main_snapshot/c5b8066/MANIFEST.json` records the upload timestamp, source host, git commit, purpose tag, and recovery hint; it is written by the `step_gdrive_upload_pre_main` stage.

The recovery sequence for a re-implementer is the following three commands run from a fresh repository clone with the Hugging Face CLI authenticated to the private repository:

```
hf download aksaN000/caem-passage-index-21m \
    --include 'passages.*' 'pre_main_snapshot/*'
cp -r pre_main_snapshot/c5b8066/* outputs/
bash run_phase1a.sh
```

The runner picks up at the highest-cycle artefact set on disk.

The snapshot contains the cold-start memory store (431 seeded episodes), the cycle-zero memory and deferred-buffer snapshots, the cycle-zero calibration fold (per-sample nine-signal vectors and gold labels), the cycle-zero composite calibration JSON (per-signal isotonic curves, per-benchmark shrinkage children, boost-layer weights), the calibrated configuration JSON (per-benchmark temperature scalars and safety floors), the cycle-zero validation-gate verdict, the cycle-zero evaluation fold (pristine model), the post-calibration rescored evaluation fold, the dataset-splits manifest, the MiniCheck-versus-RoBERTa head-to-head calibration record, the long-hypothesis judge Platt-ablation diagnostic, the cycle-zero retention slice baseline, the phase-four artefact bundle, and the run logs. Items that must be regenerated rather than pulled from the snapshot are the Qwen-2.5-3B-Instruct base weights (recovered from the Hugging Face model hub at deployment time), the per-cycle LoRA adapters at cycles one through five, the per-cycle SIL training artefacts, and the post-cycle-zero composite refits.

A.7 Hardware and software profile

Production hardware: The trajectory ran on a Vast.ai-rented RTX 5090 with thirty-two gigabytes of video memory (approximately thirty-one gigabytes usable after framework overhead), one hundred and fifty gigabytes of disk, and approximately ninety gigabytes of host random-access memory. The rental rate sat at approximately seventy cents per on-demand GPU-hour. The realised six-cycle trajectory plus cycle-zero calibration plus post-run analysis spans approximately forty calendar days of elapsed time as cycles, ablations, and rescore passes run sequentially on the single-card envelope, for approximately nine hundred and sixty GPU-hours at a total billed cost of approximately six hundred and seventy-two United States dollars (forty days at twenty-four hours per day at the on-demand rate; Vast.ai billing records). The verifier-side scoring path (nine-signal extraction, composite refit, retroactive re-verification, and the matched-protocol Haiku-judged TruthfulQA rescore across the baseline panel) is the dominant wall-clock consumer over the run, exceeding the per-cycle low-rank-adapter fine-tune itself, because every entry on every cycle and every baseline-arm answer passes through the full verifier ensemble while the adapter touches only the strict training-pool subset.

Backbone model: The base generator is Qwen2.5-3B-Instruct (`Qwen/Qwen2.5-3B-Instruct` on the Hugging Face model hub), a three-billion-parameter decoder-only transformer with instruction tuning. The model is held in bfloat16 precision throughout the trajectory. The LoRA adapter sits on top with rank thirty-two, alpha sixty-four, dropout zero point zero five, all seven linear projections targeted (query, key, value, output, gate, up, down), at learning rate two times ten to the minus four under a 32-bit AdamW optimiser (the eight-bit AdamW flag applies only to the full fine-tuning fallback path).

Framework versions: The Python environment is pinned at version 3.12. PyTorch is version 2.10 with CUDA 13.0 and CUDA toolkit 13.1; the GPU compute capability target is `sm_120`. The transformers library is version 4.57.6. The remaining dependencies are the peft library for LoRA, the accelerate library for distributed-training scaffolding, the datasets library for benchmark loaders, the faiss-cpu library for the passage and memory indices, the sentence-transformers library for SBERT embeddings, the scikit-learn library for isotonic regression and logistic boost layer, the scipy library for statistical tests, the bitsandbytes library for the eight-bit optimiser, and the Anthropic SDK for the Haiku-judged TruthfulQA correctness rule.

Determinism: A single seed (`seed=42`) propagates through the generator sampling, the FAISS index construction, and the MC-Dropout passes. The seed-everything routine in `scripts/run_cyclic_ablation.py` sets the Python `random` module, the `PYTHONHASHSEED` environment variable, the NumPy random state, the PyTorch CPU random state, and the PyTorch CUDA random state on every device. The flag `torch.use_deterministic_algorithms` is set to false deliberately because the cuDNN attention path is not bitwise-deterministic; determinism is enforced at the metric level (exact match and F1) rather than at the bit level.

Writeup envelope: The thesis itself was written on a Windows Subsystem for Linux 2 environment running Linux kernel 6.6, with twelve gigabytes of video memory on an RTX 3060 and sixteen gigabytes of host random-access memory. The twenty-one-million-passage FAISS index is sixty-four gigabytes on disk and requires approximately ninety gigabytes of resident page cache to mmap; the writeup hardware cannot host the index, so all retrieval-side numbers in the thesis are read from the Vast-side evaluation JSONs rather than recomputed locally.

A.8 Calibration script reference

The three calibration entry points are run in sequence at cycle zero and the second and third are rerun at every successful cycle boundary as part of Step 2 of the main cycle loop.

Temperature scaling and per-benchmark calibration (`scripts/run_calibration.py`). Fits the pooled temperature scalar T and the per-benchmark temperature vector T_b under a limited-memory BFGS optimiser against the expected-calibration-error objective on the cycle's calibration fold. The fold loads through the `calib_ids` field of `outputs/che`

ckpoints/dataset_splits.json; the default size is five hundred samples per training benchmark, which gives the canonical fifteen-hundred-sample pooled fold across FEVER, TriviaQA, and CommonsenseQA. Within-benchmark content-hash deduplication runs through the _dedupe_by_content_id helper, and cross-pool disjointness against the SIL-train and eval folds is asserted by assert_disjoint_calibration. The script writes outputs/calibration/calibrated_config.json and the per-cycle calibrated_config_cycle*.json, plus the per-sample dump calibration_fold_samples.json that the composite fitter consumes. Approximate runtime is one hour standalone for a cold cycle-zero fit and three and a half hours when invoked as part of the per-cycle composite-refit chain.

Composite fit (scripts/fit_composite_calibration.py). Consumes the nine verifier signals and the exact-match labels from calibration_fold_samples.json and fits the per-benchmark calibrated-probability composite. The fit chain is per-signal isotonic regression, per-benchmark shrinkage blend toward the pooled fit under $\alpha_{\text{shrink}} = 0.6$, and an L2-regularised logistic boost layer under --boost_C 0.01. The fit aborts if fewer than two hundred labelled samples are available. Cross-benchmark content-hash deduplication is inherited from the upstream calibration-fold writer. Approximate runtime is two minutes on CPU. The script writes outputs/cycle_0/composite_calibration.json at cycle zero and is invoked again at every cycle boundary against the cycle's own labelled fold to write outputs/cycle_N/composite_calibration.json.

Per-cycle composite refit: The cycle-boundary refit is implemented inside scripts/run_experiment.py as the run_per_cycle_composite_refit helper. The helper shells out to fit_composite_calibration.py under the same --cherian_boost --boost_C 0.01 --shrinkage_alpha 0.6 flags as the cycle-zero fit, but against the cycle's own fold under outputs/cycle_{N}/calibration/*.json. Deduplication is inherited from _dedup_inplace inside the runner. The canonical cycle-zero composite calibration path is intentionally not overwritten by the per-cycle refit; later-cycle composites live at their own per-cycle paths and the cycle-three composite is what the post-run baseline rescore reads. Approximate runtime is two minutes per cycle.

A.9 Validation gate reference

The cycle-zero validation gate licenses the deployed configuration to enter the main run. The gate is implemented at scripts/validate_composite_weights.py and runs automatically as the step_7_0_3_validate_weights stage of the pre-run setup block. The gate is a conjunction of four pre-specified checks; the verdict is recorded at outputs/cycle_0/weight_validation.json.

```
python scripts/validate_composite_weights.py \
    --eval_dir outputs/cycle_0/eval \
    --output  outputs/cycle_0/weight_validation.json \
    --cohen_d_threshold 0.20 \
    --store_discard_gap 0.05 \
    --max_poisoning_rate 0.50
```

Check 1: composite Cohen's d on the training-pooled evaluation fold.

Rationale. Audits the precondition of Proposition 4.2 that the verifier discriminates correct from incorrect outputs strictly above chance.

Threshold. $d_{\text{id}} \geq 0.20$.

Realised. $d_{\text{id}} = +0.635$, clearing the floor by more than three times the registered margin. The transfer-pooled Cohen's d reads $+0.173$ and is reported alongside as an out-of-distribution diagnostic; it is not gated.

Check 2: per-benchmark store-versus-discard mean-composite gap.

Rationale. Audits that the storage rule behaves consistently across training benchmarks rather than being driven by a single dominating subset.

Threshold. $\Delta \geq 0.05$ on every training benchmark.

Realised. 0.282 on FEVER, 0.320 on TriviaQA, 0.258 on CommonsenseQA.

Check 3: per-benchmark memory-poisoning rate.

Rationale. Audits that the storage gate does not admit wrong-confident entries at a rate that would compromise the verified pool. Defined as the share of stored entries whose stored answer is wrong against the gold label.

Threshold. rate ≤ 0.50 on every training benchmark.

Realised. 0.206 on FEVER, 0.179 on TriviaQA, 0.247 on CommonsenseQA.

Check 4: no strong-signal inversion.

Rationale. Audits that no signal carries a strongly negative Cohen's d against its registered direction.

Threshold. no signal (except the registered-inverse u_{dropout}) reports $d < -0.20$ on the cal fold.

Realised. no inversion observed.

The overall verdict is PASS: the conjunction holds and the deployed configuration is licensed to enter the main run. The verdict is reported in Section 5.2.7 with the same numbers; the canonical v2.1-panel artefact path is `outputs/analysis/weight_validation_v2_1.json`, whose store-versus-discard gap is keyed against the three training benchmarks of the realised panel (FEVER, TriviaQA, CommonsenseQA). The legacy pre-v2.1 artefact at `outputs/cycle_0/weight_validation.json` reads a different benchmark trio (FEVER, StrategyQA, TruthfulQA) under the earlier panel registration and is preserved for audit but is superseded by the v2.1-panel reading.

Appendix B

Cycle 0 evidence catalog

B.1 The one-hundred-variant composite parameter sweep

The locked configuration selected from the sweep, with the rationale for each parameter, is given in Table B.1; the locked variant and its neighbours are compared in Table 5.16. The full sweep over the pre-specified $\tau_{\text{store}} \times C \times \alpha_{\text{shrink}}$ grid ($4 \times 5 \times 5 = 100$ variants) is recorded in the cycle-zero sweep archive with one record per variant; no per-variant comparison table is reproduced in the thesis. Per-benchmark sweep diagnostics for any variant in the grid are recoverable by reading the corresponding JSON entry.

Parameter	**Locked value**	**Selection rationale**
Storage cut-point τ_{store}	0.60	Stores about twice as many answers as 0.65 at acceptable precision (Table 5.16)
Deferral cut-point τ_{defer}	0.45	Pre-specified 0.15 below storage cut
Abstention floor ϕ_{pg}	0.20	Early-exit rule Equation (4.1)
Boost regularisation C	0.010	Strongest regularisation in the grid; a hundredfold tighter penalty than the library default $C = 1.0$
Shrinkage strength α_{shrink}	0.6	Pulls small per-benchmark fits toward the pooled fit; $\alpha = 1$ stores more correct answers but separates less well on the validation fold (Table 5.16)
Retroactive prune τ_{retro}	0.50	0.10 below storage cut by registration
Retention floor $\rho_{\min}$	0.93	Multi-modal probe panel rollback threshold

Table B.1: Locked configuration carried into every cycle of the main run, with the reason for each value.

B.2 The calibration-to-evaluation gap analysis

The pooled gap reading at the locked configuration is reported in Section 5.2.5. The per-benchmark decomposition is reported in Table 5.17, which is auto-generated from the cycle-zero evaluation outputs. The narrative analysis behind the per-benchmark

Table B.2: Empty-answer and refusal rates per benchmark, before versus after the registered prompt-compliance and over-abstention fixes.

benchmark	pre empty	post empty	pre IDK	post IDK
FEVER	43.6%	44.2%	42.2%	36.0%
TriviaQA	43.0%	48.8%	40.4%	29.8%
NQ	44.4%	43.6%	38.8%	35.6%
TruthfulQA	53.0%	47.2%	37.0%	31.2%
StrategyQA	17.4%	18.8%	69.8%	66.6%
ARC-C	12.0%	10.8%	55.0%	55.8%

gap is recorded in the calibration-to-evaluation gap note within the cycle-zero sweep archive.

B.3 The first-iteration failure trace

The first iteration of the cycle-zero calibration failed the pre-specified validation gate due to a JSONL signal-dump bug; the narrative is recorded in Section 5.2.8. The diagnostic record is preserved in the first-iteration archival snapshot dated to the failure, together with the degenerate composite calibration record whose conformal-threshold collapse triggered the corrective patch.

B.4 The long-hypothesis judge ablation diagnostic

The Platt fit diagnostic that licensed the ablation of the long-hypothesis judge is recorded in the long-hypothesis-judge Platt-ablation calibration record. The two persisted headline statistics are the observed Pearson $\rho = 0.5843$ in logit space and the pre-specified floor of $\rho_{\min} = 0.70$ against which the observed correlation is audited. The verdict is therefore that the long-hypothesis judge fails the registered correlation floor; the architectural consequence is the ablation of the long-hypothesis backend from the deployed configuration, with the unified verifier dispatching every hypothesis through the short-hypothesis backend under a documented five-hundred-and-twelve-token truncation contract. Section 5.2.6 reports the architectural consequence and the future-work item that would re-fit the judge under task-specific training. The companion long-hypothesis judge Platt diagnostic log records the per-sample diagnostic trace whose summary statistics produced the headline correlation reading.

B.5 Pre and post prompt-fix compliance

The per-benchmark empty-answer and abstention rates before and after the registered prompt-design fixes are auto-generated from the cycle-zero evaluation outputs.

Table B.3: Atomic decomposition scope per benchmark. Atomic falls back to $p_{\text{ground}}^{\text{mean}}$ on short answers (length-gate at 12 tokens); only long-form answers get genuine per-fact NLI. Aligns with FActScore's documented scope (Min et al. EMNLP 2023, §3).

benchmark	n	fallback (count)	fallback rate	long-form rate
FEVER	500	500	100.0%	0.0%
TriviaQA	500	500	100.0%	0.0%
NQ	500	500	100.0%	0.0%
TruthfulQA	500	500	100.0%	0.2%
StrategyQA	500	500	100.0%	0.0%
ARC-C	500	500	100.0%	0.0%

B.6 Atomic decomposition coverage

The per-benchmark coverage of the atomic-fact decomposition signal $p_{\text{ground,atomic}}$, conditioned on the registered length gate of twelve tokens, is auto-generated below.

B.7 Composite weight table

The cycle-zero direction and boost weight of each signal are given in Table B.4. The per-cycle re-calibration of the composite (the per-benchmark temperature scalars, the per-benchmark safety floors, and the pooled boost-layer weights on the nine deployed signals) across the realised cycle-one through cycle-five trajectory is reported in Table C.14 (Section C.6), with the cycle-four entry omitted under the asymmetric-rollback contract that skips the cal-fold refit on aborted cycles. The per-signal isotonic curves themselves are non-parametric step functions on the calibration fold and are not enumerated cell-by-cell across cycles; they are recoverable from the per-cycle composite calibration records, with the cycle-zero canonical record held immutable across the trajectory under the lock-cycle-zero patch. The storage and deferral cut-points stay frozen at the cycle-zero registered values throughout, so the per-cycle composite refit moves the signal-to-probability mapping the cut-points operate on rather than the cut-points themselves.

B.8 Per-benchmark precision-cliff curves

The threshold-versus-precision tradeoff at the locked storage cut-point $\tau_{\text{store}} = 0.60$, which licenses the rate-precision frontier discussed in Section 5.6.5, is auto-generated below: Figure B.1 renders the curve and Table B.5 reports the underlying threshold and precision values.

B.9 Production-envelope sample renderings

The production-envelope rendering script presents two representative serving samples per outcome tag (store, defer, abstain, discard) with the verifier signal block and the

Signal	Pearson ρ	Direction	Boost weight
$p_{\text{ground}}^{\text{mean}}$	+0.101	increasing	+0.184
$p_{\text{ground}}^{\text{max}}$	+0.140	increasing	+0.379
$p_{\text{ground}}^{\text{atomic}}$	+0.101	increasing	+0.184
p_{entail}	-0.156	decreasing	+0.067
q_{match}	+0.299	increasing	+0.491
u_{internal}	+0.425	increasing	+0.334
u_{token}	+0.119	increasing	+0.126
u_{dropout}	-0.411	decreasing	+0.190
s_{avg}	+0.098	increasing	+0.175
		Boost intercept	+0.265

Table B.4: Direction of each signal's calibration curve (from its correlation with correctness) and its boost weight in the cycle-zero composite, fitted on the 1,500-question calibration fold.

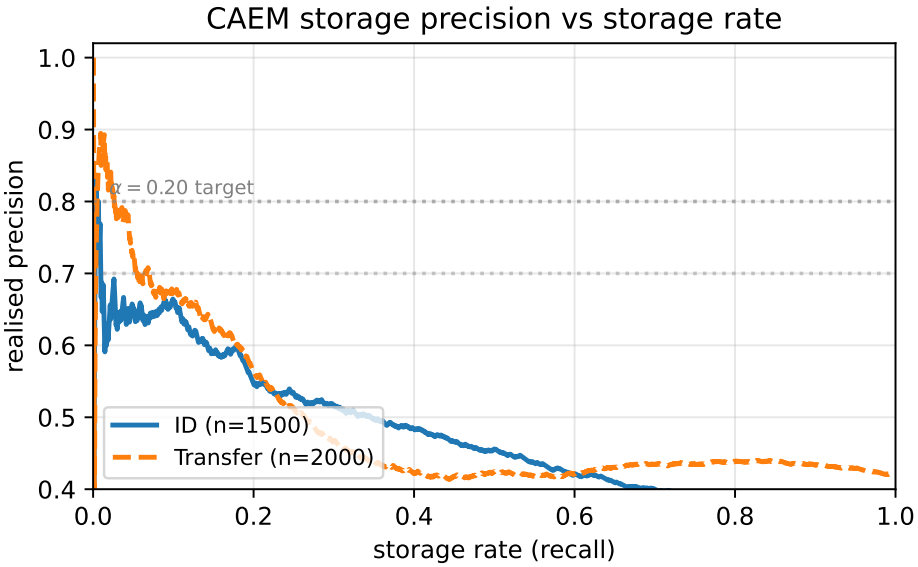


Figure B.1: Per-benchmark precision-cliff curves at the locked configuration. Auto-generated from the precision-cliff diagnostic; the underlying threshold-versus-precision data are in Table B.5.

Table B.5: Storage precision at fixed targets, in-distribution vs transfer. Lower storage rate = higher operating-point threshold.

target precision	ID storage rate	transfer storage rate
$\geq 95\%$	unreach	unreach
$\geq 90\%$	unreach	unreach
$\geq 85\%$	unreach	1.6%
$\geq 80\%$	unreach	2.6%
$\geq 75\%$	unreach	4.5%
$\geq 70\%$	unreach	5.3%

Table B.6: Prompt-design ablation: Cycle-0 metrics under the legacy per-task minimal-CoT prompts (OLD) versus the uniform few-shot scaffolded CoT prompt with forced decoder prefix (NEW, the design adopted in the rest of this thesis; see Chapter 4 §4.2.1, *Generator and retrieval substrate*). Both runs use identical cold-start memory, identical calibration-fold quantile thresholds, and identical random seeds. Δ columns are NEW − OLD. *Mean pred. words* is the average number of whitespace-separated tokens in the model's generated answer (before scaffold stripping).

	EM			**Mean u_{stored}**			**Mean pred. words**	
Benchmark	Old	New	Δ	Old	New	Δ	Old	New
FEVER	0.428	0.456	+0.028	0.429	0.503	+0.074	49.3	37.4
TriviaQA	0.272	0.374	+0.102	0.362	0.144	-0.218	38.0	27.2
Natural Questions	0.156	0.172	+0.016	0.366	0.164	-0.203	36.8	25.4
TruthfulQA	0.378	0.334	-0.044	0.357	0.102	-0.254	37.7	26.2
StrategyQA	0.588	0.602	+0.014	0.354	0.259	-0.095	43.5	32.8
ARC-Challenge	0.736	0.732	-0.004	0.390	0.369	-0.021	50.4	32.7

served answer alongside, so that a reader can audit the four-outcome decision tree on real entries.

Placeholder. The production-envelope sample table is pending the final main run; the generator script will overwrite this stub with the four-outcome rendering (store, defer, abstain, discard) at compile time.

B.10 Prompt-design ablation

The pre-specified prompt-design ablation isolates the contribution of two prompt fixes (the empty-answer compliance fix and the open-QA over-abstention fix) by re-running the cycle-zero evaluation under the pre-fix prompt template family and comparing against the post-fix template family on the identical evaluation fold.

Appendix C

Per-cycle artifact dictionary

The five tables in the first five sections are the per-cycle index against the on-disk JSON snapshots that the empirical chapter reads. The dictionary covers calibration snapshots in Table C.1, the threshold history in Table C.2, the memory-store trajectory in Table C.3, the per-cycle outcomes in Table C.4, and the fine-tuning diagnostics in Table C.5. Cycle zero is the cold-start calibration cycle; cycles one through five are the main-run cycles; cycle four is aborted under the retention guard with the asymmetric-rollback contract that skips the cal-fold refit and the retroactive re-verification pass on the aborted cycle. The realised commit fraction is $\pi_{\text{commit}} = 4/5$; the run ended after cycle five as planned.

C.1 Per-cycle calibration snapshots

Table C.1 reports the per-cycle pooled temperature scalar, the expected calibration error before and after the per-cycle refit, the boost-layer intercept, the calibration-fold sample size, and the boost-layer L2 regularisation strength. The cycle-zero temperature lands at the upper bound of the optimisation envelope (the principled cap $T \approx e^3$); the cycle-one boundary refit drops the scalar by more than an order of magnitude under the post-fine-tune distribution, and the scalar stabilises in the band $[0.74, 0.87]$ from cycle two onward. The expected-calibration-error column shows the cycle-zero refit moves expected calibration error from 0.276 to 0.066; from cycle one onward the post-refit expected calibration error stays inside the tight band $[0.041, 0.049]$, the empirical receipt that the per-cycle distribution drift is absorbed into the signal-to-probability mapping rather than into the expected-calibration-error budget.

C.2 Per-cycle threshold history

The four-way storage decision tree of Section 4.2.9 reads against the architecturally frozen cut-points $\tau_{\text{store}} = 0.60$ and $\tau_{\text{defer}} = 0.45$, which do not move across the trajectory; the threshold history table below records the per-cycle quantities that do move under the cycle-boundary refit. Table C.2 reports the pooled temperature scalar, the three per-benchmark temperature scalars on the training panel, and the three per-benchmark safety floors at each successful cycle. The training-benchmark

Cycle	T (post-refit)	ECE before	ECE after	Boost intercept
C0	20.086	0.2761	0.0662	0.2649
C1	1.089	0.0508	0.0440	−0.6130
C2	0.836	0.0509	0.0494	−0.6338
C3	0.740	0.0451	0.0407	−0.5655
C4*	—	—	—	—
C5	0.869	0.0551	0.0427	−0.5204

Table C.1: Per-cycle calibration snapshot summary. The cal-fold sample size is pinned at $n_{\text{cal}} = 1\,500$ and the boost-layer L2 regularisation is pinned at $C = 0.01$ across cycles by the per-cycle refit invocation. The asterisk marks cycle four, which was aborted under the retention guard; the cal-fold refit was skipped under the asymmetric-rollback contract. Sources: the cycle-zero calibrated-configuration JSON for the cold-start values and the per-cycle calibrated-configuration JSONs for cycles one, two, three, and five; the boost-layer intercept is read from the pooled-fit field of each cycle's composite-calibration record.

temperature pattern is heterogeneous by construction: the FEVER and TriviaQA per-benchmark scalars sit an order of magnitude above the CommonsenseQA scalar at every cycle, the empirical receipt that the underlying token-probability distributions on open-text factual claim verification and open-domain trivia are systematically more overconfident than on the five-option multiple-choice surface. The CommonsenseQA per-benchmark safety floor ratchets up from the registered default $\phi_{\text{safe}} = 0.380$ to 0.491 at cycle three and to 0.569 at cycle five; the FEVER and TriviaQA floors stay at the architectural minimum throughout.

Cycle	T (pooled)	T_{FEVER}	T_{TQA}	T_{CSQA}	$\phi_{\text{safe,FEVER}}$	$\phi_{\text{safe,TQA}}$	$\phi_{\text{safe,CSQA}}$
C0	20.086	20.086	20.086	1.427	0.380	0.380	0.380
C1	1.089	14.092	14.123	1.342	0.380	0.380	0.380
C2	0.836	9.899	10.007	1.209	0.380	0.380	0.380
C3	0.740	6.986	7.163	1.083	0.380	0.380	0.491
C4*	—	—	—	—	—	—	—
C5	0.869	4.972	11.040	1.028	0.380	0.380	0.569

Table C.2: Per-cycle threshold history. The architecturally frozen storage and deferral cut-points do not move; what moves under the cycle-boundary refit is the per-benchmark temperature vector and the per-benchmark safety floor. The CommonsenseQA per-benchmark safety floor ratchets up from 0.380 to 0.491 at cycle three and to 0.569 at cycle five in response to the structurally lower pre-routing confidence on the five-option multiple-choice surface. The asterisk marks cycle four, where the cal-fold refit was skipped under the asymmetric-rollback contract. The architectural cut-points held fixed across cycles are registered in the Chapter 4 threshold registry. Sources: the per-cycle calibrated-configuration JSONs under the experiment-run calibration directory.

C.3 Per-cycle memory size and storage rate

Table C.3 reports the memory pool size at each cycle close together with the four mechanisms that move the pool size across the cycle boundary (stream-chunk admissions through the storage gate, retroactive re-verification prunes against the stored pool under the recalibrated verifier, deferred-buffer promotions from the bounded reconsider queue, and consolidation removals of near-duplicate episodes) and the cycle's net change in pool size. The cycle-four row records no retroverify prunes, deferred-buffer promotions, or consolidation removals under the asymmetric-rollback contract that skips these passes on the aborted cycle; the cycle-four stream-chunk admissions and the cycle-four pool growth proceed unchanged because the fresh-data side of the architecture runs unconditionally.

Cycle	Pool size at close	Stream-chunk admissions	Retroverify prunes	Deferred-buffer promotions	Consolidation removals	Net change
C0	431	431	0	0	0	+431
C1	2 818	2 443	56	0	0	+2 387
C2	5 151	2 477	271	131	3	+2 333
C3	7 477	2 469	352	220	11	+2 326
C4*	9 961	2 484	—	—	—	+2 484
C5	12 175	2 343	382	299	44	+2 214

Table C.3: Per-cycle memory store trajectory. The pool grows from the four-hundred-and-thirty-one-entry cold-start seed at cycle zero to twelve thousand one hundred and seventy-five entries at the cycle-five close, a twenty-eight-fold increase across the realised trajectory. The cycle-zero admissions row records the cold-start seed rather than a stream-chunk pass. The asterisk marks cycle four, which was aborted under the retention guard; the retroverify, deferred-reconsider, and consolidation passes are skipped under the asymmetric-rollback contract and rendered as em-dashes. The per-cycle stream-chunk admission count stays in the band $[2\,343, 2\,484]$ across cycles one through five against a stream-chunk candidate budget of 4 700 samples per cycle (two thousand FEVER, two thousand TriviaQA, seven hundred CommonsenseQA), so the storage gate admits approximately half of every candidate stream chunk. The admissions count the entries actually added to the store; they sit slightly below the gate-pass counts n_{stored} of Table 5.12 because the memory store's novelty filter rejects near-duplicates of existing entries between the gate and the insert. The cycle-five admission count is the cycle-close pool minus the post-re-verification pool $(12\,175 - 9\,832)$. The consolidation-removal column is the episodes-removed count of Table 5.11. Admissions minus prunes plus promotions minus consolidation removals reproduces the net change at every cycle except a residual of one entry at cycle two and two entries at cycle five; the residual is promoted entries dropped as duplicates of entries already in the store (the cycle-two consolidation pass sees 2 948 entries against $2\,818 + 131 = 2\,949$). Sources: the per-cycle memory-store metadata side-files supply the cycle-close pool sizes (the length of the metadata list inside the pickled snapshot); the per-cycle retroverification records supply the per-cycle prune counts; per-cycle deferred-buffer promotion counts are read from the deferred-reconsideration log emissions in the experiment run log.

C.4 Per-cycle equilibrium fit

Table C.4 reports the outcome of each cycle together with the worst-probe retention ratio, the post-training MMLU probe accuracy, and the pooled evaluation-fold exact match on each cycle (the series the equilibrium fit reads). The equilibrium-saturation gate is the disjunction of three trigger signals (storage saturation under repeated zero-admission cycles, MMLU-floor breach against the cycle-zero pristine baseline, and composite-evaluation-score plateau across the late-cycle subsequence); the gate fires when at least two of the three signals fire simultaneously. Cycles one to three were committed; cycle four was rolled back by the retention guard (TriviaQA-test ratio 0.886 below the registered floor of 0.93); cycle five was committed, and the run ended there as planned.

Cycle	Outcome	Worst probe	Worst ratio	MMLU acc.	Pooled EM*
C1	committed	CommonsenseQA-test	1.006	0.635	0.557
C2	committed	TriviaQA-test	0.949	0.595	0.567
C3	committed	CommonsenseQA-test	0.976	0.605	0.547
C4	**rolled back**	TriviaQA-test	**0.886**	0.621	0.548
C5	committed (last)	CommonsenseQA-test	0.958	0.609	0.556

Table C.4: Per-cycle outcome and retention probes. At cycle four the TriviaQA-test ratio of 0.886 fell below the floor $\rho_{\min} = 0.93$, so the weights were rolled back to the cycle-three checkpoint with memory kept. The MMLU column is the post-training MMLU probe accuracy against the pristine anchor of 0.610; the pooled-EM column is the pooled evaluation-fold exact match, the series stored as the history array of the cycle-five equilibrium-fit record.

*Five-benchmark pooled exact match with string-match TruthfulQA scoring, the series the runner's equilibrium fit stores in its composite-evaluation-score field. It is neither the composite epistemic score (CES) of Equation (5.1), whose trajectory is reported in Table C.6, nor the judged pooled exact match of Figure 5.1.

The equilibrium-saturation fit at the cycle-five close lives at the cycle-five equilibrium-fit record and reports the saturating-exponential parameters $c_\infty = 0.548$, $a = -0.018$, $k = 0.350$, $R^2 = 0.207$ over the five-cycle pooled evaluation-fold exact-match history (the pooled-EM column of Table C.4, which the record stores as its composite-evaluation-score history); the low coefficient of determination shows that five points do not support a saturating-exponential fit. Per-cycle coverage diagnostics covering admission rates, candidate counts, pool entropy, and adapter singular-value spectra are persisted as a single artefact at the cycle-five close under the cycle-five coverage diagnostic record. The pre-cycle-five admission behaviour is reconstructable from the composite-evaluation-score and storage-rate history arrays inside the equilibrium-fit record and from the per-cycle commit verdicts in the experiment-run stochastic-equilibrium trajectory CSV.

C.5 Per-cycle fine-tuning diagnostics

Table C.5 reports the per-cycle self-improvement training trajectory: the size of the training pool drawn from cumulative memory above $\tau_{\text{train}} = 0.70$, the per-epoch final loss, the worst-probe retention ratio at the cycle close, and whether

the cycle committed or rolled back. The training-pool size grows monotonically across the trajectory from one hundred and seven entries at the first fine-tune to five thousand and six entries at the cycle-five fine-tune, consistent with the cumulative memory growth reported in Table C.3. The final epoch-three loss declines almost monotonically across the successful cycle subsequence ($0.258 \rightarrow 0.171 \rightarrow 0.109$) and reaches its lowest in-flight value at cycle four (0.070); the cycle-five restart resumes at the slightly higher value 0.091 because the cycle re-initialises from the cycle-three adapter and trains over a larger and more diverse pool. The retention guard fires exactly once on the realised trajectory, at cycle four, on the TriviaQA-test probe: post-training accuracy 0.350 against the pristine baseline of 0.395 yields the worst-probe ratio of 0.886, below the registered floor of 0.93. The adapter is restored to the cycle-three checkpoint before evaluation runs under the rolled-back weights.

Cycle	n_{used}	**Loss epoch 1**	**Loss epoch 2**	**Loss epoch 3**	**Rollback**
C1	107	1.387	0.595	0.258	—
C2	1 474	0.728	0.367	0.171	—
C3	2 966	0.535	0.230	0.109	—
C4	3 624	0.424	0.147	0.070	**yes**
C5	5 006	0.482	0.194	0.091	—

Table C.5: Per-cycle fine-tuning diagnostics. The retention guard binds on the TriviaQA-test probe across the realised trajectory: the worst non-abort ratio occurs at cycle two (0.949 on TriviaQA-test) and the cycle-four abort fires on the same probe at ratio 0.886 against the registered floor $\rho_{\min} = 0.93$. The MMLU probe ratio stays in the band $[0.975, 1.041]$ across the trajectory, confirming that TriviaQA-test is the binding retention constraint for the realised self-improvement trajectory and that the cycle-four rollback is a probe-specific regression rather than a pool-wide one. Pristine retention anchors held fixed across cycles: MMLU 0.610, TriviaQA-test 0.395, CommonsenseQA-test 0.825. Sources: the per-cycle retroverification JSONs supply the training-batch size and the final-loss fields; the per-cycle post-training probe lines in the experiment-run log supply the per-probe retention ratios; the pristine probe-baseline record supplies the cycle-zero anchors.

C.6 Additional per-cycle tables from the evaluation

The tables in this section support statements made in Chapter 5; each is referred to from the paragraph that uses it.

Cycle	ACC	EPI	RET	CAL	VER	CES
C0	0.517	0.845	1.000	0.898	0.174	0.585
C1	0.533	0.869	1.000	0.912	0.325	0.672
C2	0.535	0.873	0.949	0.901	0.415	0.698
C3	0.516	0.863	0.976	0.919	0.407	0.695
C4*	0.517	0.864	**0.886**	0.919	0.407	**0.682**
C5	0.508	0.866	0.958	0.915	**0.448**	**0.704**

* Cycle four was aborted under the retention guard ($\rho_{\min} = 0.93$); CAL and VER reuse cycle three.

Table C.6: Composite evaluation score (Equation (5.1)) and its five axes by cycle, for CAEM only: three of the axes do not exist for single-pass baselines.

Cycle	Benchmark	Store ✓	Store ×	Store π	Defer ✓	Defer ×	Discard ✓	Discard ×
Cycle two (the within-trajectory best cycle)								
C2	FEVER	803	240	0.770	344	419	58	45
C2	TriviaQA	605	209	0.743	262	329	61	163
C2	CommonsenseQA	465	172	0.730	34	19	1	0
C2	TruthfulQA	33	58	0.363	22	49	8	41
C2	StrategyQA	57	20	0.740	125	73	1	0
Cycle five (trajectory-end close)								
C5	FEVER	764	210	0.784	474	432	10	2
C5	TriviaQA	532	269	0.664	284	430	8	13
C5	CommonsenseQA	428	177	0.707	32	32	11	6
C5	TruthfulQA	25	50	0.333	36	103	3	13
C5	StrategyQA	45	10	0.818	100	53	0	1

Table C.7: Correct and wrong answers in each decision at C2 and C5, with the precision of the stored class. Training-panel rows come from the stream chunks, transfer-panel rows from the evaluation fold.

Cycle	FEVER	TriviaQA	CSQA	TruthfulQA	StrategyQA
C0	64.3%	29.0%	0.0%	33.3%	43.0%
C1	24.3%	17.7%	0.3%	27.3%	2.3%
C2	5.7%	17.3%	1.0%	29.3%	8.0%
C3	10.7%	20.0%	0.0%	25.0%	4.0%
C4	10.0%	21.0%	0.0%	25.3%	2.3%
C5	5.3%	26.0%	0.3%	23.0%	30.3%

Table C.8: Share of evaluation-fold questions on which the system abstained (300 per benchmark).

Cycle	Benchmark	T1 EM	T1 n	T2 EM	T2 n	T3 EM	T3 n	Pooled EM
Cycle zero (no Tier 1 answers)								
C0	FEVER	—	0	55.3	132	43.5	168	48.7
C0	TriviaQA	—	0	46.1	89	46.4	211	46.3
C0	CommonsenseQA	—	0	77.8	275	52.0	25	75.7
C0	TruthfulQA	—	0	29.9	197	21.4	103	27.0
C0	StrategyQA	—	0	68.4	38	59.9	262	61.0
Cycle two								
C2	FEVER	—	0	51.6	122	54.5	178	53.3
C2	TriviaQA	—	0	51.0	96	48.5	204	49.3
C2	CommonsenseQA	—	0	72.4	275	64.0	25	71.7
C2	TruthfulQA	100.0	1	27.5	204	27.4	95	27.7
C2	StrategyQA	—	0	66.4	131	64.5	169	65.3
Cycle five (end of the run)								
C5	FEVER	—	0	50.0	130	45.9	170	47.7
C5	TriviaQA	100.0	1	42.3	111	46.8	188	45.3
C5	CommonsenseQA	—	0	74.5	247	58.5	53	71.7
C5	TruthfulQA	100.0	1	25.4	193	24.5	106	25.3
C5	StrategyQA	—	0	63.9	72	64.0	228	64.0

Table C.9: EM (percent) and number of queries per tier and benchmark at C0, C2 and C5.

Cycle	[0.60, 0.65)		[0.65, 0.70)		[0.70, 0.80)		[0.80, 0.90)		[0.90, 1.00]	
	n	EM	n	EM	n	EM	n	EM	n	EM
(a) Evaluation fold										
C0	174	0.759	104	0.731	46	0.804	10	0.700	2	1.000
C1	139	0.655	145	0.697	170	0.776	59	0.814	2	1.000
C2	113	0.566	155	0.729	225	0.716	35	0.771	6	0.833
C3	85	0.600	201	0.726	244	0.738	34	0.706	1	1.000
C4	91	0.648	203	0.704	231	0.732	40	0.750	1	1.000
C5	164	0.677	159	0.686	164	0.720	14	0.929	3	0.667
(b) SIL stream chunks										
C1	596	0.666	551	0.693	832	0.798	419	0.845	58	0.914
C2	512	0.635	534	0.702	1 043	0.785	356	0.871	49	0.898
C3	423	0.572	730	0.675	1 012	0.768	292	0.880	33	1.000
C4	427	0.578	682	0.705	1 060	0.776	316	0.870	37	0.973
C5	732	0.637	753	0.716	752	0.793	127	0.858	16	0.875

Table C.10: Stored training-panel answers by score band and the EM within each band, on (a) the evaluation fold and (b) the stream chunks.

	Retroverify prunes				Deferred-buffer promotions			
Cycle	Pruned	em-wrong	em-correct	Wrong-caught	Promoted	em-correct	em-wrong	Correct-recovered
C1	56	0	56	0.0%	0	0	0	—
C2	261	114	147	43.7%	117	65	52	55.6%
C3	355	122	233	34.4%	210	119	91	56.7%
C4*	—	—	—	—	—	—	—	—
C5	422	184	238	43.6%	293	161	132	54.9%
Pooled	**1 094**	**420**	**674**	**38.4%**	**620**	**345**	**275**	**55.6%**

Table C.11: Share of wrong entries among those removed by re-verification, and of correct entries among those promoted from the deferred buffer, reconstructed from the memory snapshots.

Cycle	Tier 1 (memory)		Tier 2 (zero-shot)		Tier 3 (RAG)		Pooled	
	EM	CHM	EM	CHM	EM	CHM	EM	CHM
C0	–	–	0.565	0.128	0.472	0.181	0.517	0.155
C1	–	–	0.566	0.121	0.501	0.141	0.533	0.131
C2	1.000	0.000	0.548	0.124	0.517	0.131	0.535	0.127
C3	1.000	0.000	0.532	0.128	0.502	0.145	0.516	0.137
C4	1.000	0.000	0.532	0.129	0.502	0.143	0.517	0.136
C5	1.000	0.000	0.519	0.129	0.495	0.140	0.508	0.134

Table C.12: EM and CHM of the answers given by each tier, by cycle, on the evaluation fold.

Cycle	Pool size at pass	Updated	Pruned	Prune rate	Survivor share
C1	431	375	56	13.0%	87.0%
C2	2,945	2,674	271	9.2%	90.8%
C3	5,358	5,006	352	6.6%	93.4%
C5	10,214	9,832	382	3.7%	96.3%
Cycle 4: retroactive pass skipped under the cycle-four retention abort (skip-retroverify-on-abort patch).					

Table C.13: Entries checked, kept and removed by the retroactive pass at each kept cycle; the pass was skipped at the rolled-back cycle four.

Parameter	C0	C1	C2	C3	C5	$\lvert\Delta_{C0\to C5}\rvert$
(a) Pooled calibration discipline						
Temperature scalar T (pooled, post-refit)	20.086	1.089	0.836	0.740	0.869	19.217
Expected calibration error ECE after refit	—	0.044	0.049	0.041	0.043	—
Boost-layer intercept b_0	+0.265	−0.613	−0.634	−0.565	−0.520	0.785
(b) Per-benchmark temperature scalar (post-refit)						
T_{FEVER}	—	14.092	9.899	6.986	4.972	—
T_{TriviaQA}	—	14.123	10.007	7.163	11.040	—
$T_{\text{CommonsenseQA}}$	—	1.342	1.209	1.083	1.028	—
(c) Per-benchmark safety floor (drift observed only on CommonsenseQA)						
$\phi_{\text{safe}}^{\text{CommonsenseQA}}$	0.380	0.380	0.380	0.491	0.569	0.189
(d) Pooled boost-layer weights on the nine deployed signals						
u_{token}	+0.126	+0.211	+0.177	+0.060	+0.110	0.016
u_{dropout}	+0.190	+0.132	+0.225	+0.235	+0.204	0.014
u_{internal}	+0.334	+0.257	+0.297	+0.257	+0.321	0.013
s_{avg}	+0.175	+0.220	+0.177	+0.264	+0.242	0.067
p_{entail}	+0.067	+0.368	+0.324	+0.359	+0.285	**0.218**
$p_{\text{ground}}^{\text{max}}$	+0.379	+0.296	+0.333	+0.288	+0.356	0.023
$p_{\text{ground}}^{\text{mean}}$	+0.184	+0.262	+0.238	+0.223	+0.230	0.046
$p_{\text{ground}}^{\text{atomic}}$	+0.184	+0.262	+0.238	+0.223	+0.230	0.046
q_{match}	+0.491	+0.360	+0.259	+0.264	+0.265	**0.226**

Table C.14: What the refit changes at each kept cycle: (a) pooled temperature, calibration error and boost intercept; (b) per-benchmark temperature; (c) per-benchmark safety floor; (d) boost weights.

Appendix D

Significance test details

This appendix gives the procedure behind the paired comparisons of Section 5.3: how CAEM's and each baseline's answers are paired, the exact-match and hallucination tests, the bootstrap intervals, the Holm correction and the rule that turns a test into a win or a defeat. Every number below is read from the final test record, which is rebuilt by the results command of Chapter E.

D.1 Pairing rule and correctness label

Pairing: each contrast compares CAEM at one cycle (C0, C2 or C5) with one baseline on one benchmark. The answers of the two systems are matched on the question identifier, restricted to the evaluation-fold identifiers of the dataset-split record, so a contrast uses only questions both systems answered. Under the comparison protocol of Section 5.1.1 every system answered the same 300 questions per benchmark, and every contrast in this appendix has $n = 300$ pairs.

Correctness label: both sides are scored with the single rule of Section 5.1.1: the uniform scorer on FEVER, TriviaQA, CommonsenseQA and StrategyQA, and on TruthfulQA the language-model judge applied to the final answer line, the same judge for every system. The earlier ROUGE-based TruthfulQA field is not read.

Hallucination value: the per-question CHM is the share of the seven active failure flags a question raises (Section 5.1.1). Each baseline answer carries the verifier signals of the comparison protocol, so B8 is tested on this axis as well, and no baseline is charged with a decision it did not take.

D.2 Contingency tables

For exact match, each contrast forms the paired two-by-two table of correct and wrong answers. The two off-diagonal cells are the discordant pairs: b counts questions CAEM answers correctly and the baseline does not, and c the reverse. The McNemar test reads only these two cells, through the continuity-corrected statistic of Edwards [2]

with one degree of freedom,

$$\chi^2 \;=\; \frac{\big(|b-c|-1\big)^2}{b+c},$$

with $p = 1$ when $b + c = 0$. Table D.1 shows four contrasts at C5: a tie with the zero-shot baseline, CAEM's largest win and its largest defeat, both against vanilla fine-tuning, and a defeat by five-shot chain of thought.

Contrast at C5	**EM CAEM**	**EM base**	$\boldsymbol{\Delta}_{\mathrm{EM}}$	$\boldsymbol{b}$	$\boldsymbol{c}$	$\boldsymbol{\chi^2}$	$\boldsymbol{p}_{\mathrm{raw}}$
B1 zero-shot · FEVER	0.477	0.507	−0.030	71	80	0.42	0.515
B6 vanilla fine-tune · TriviaQA	0.453	0.290	+0.163	72	23	24.25	8.5×10^{-7}
B6 vanilla fine-tune · FEVER	0.477	0.640	−0.163	30	79	21.14	4.3×10^{-6}
B5 five-shot CoT · CommonsenseQA	0.717	0.793	−0.077	24	47	6.82	0.009

Table D.1: Four exact-match contrasts at C5 ($n = 300$ each): b counts questions only CAEM answers correctly, c questions only the baseline answers correctly. The two systems disagree on 151 FEVER questions against B1 but split them almost evenly, so the difference is not significant.

For the composite hallucination metric, the per-question differences (CAEM minus baseline) are tested with the Wilcoxon signed-rank test [1], two-sided, with questions of zero difference dropped.

D.3 Bootstrap protocol

The interval on each difference is the percentile bootstrap over paired questions: 2,000 resamples of the 300 pairs with replacement, from a generator seeded with 42, with the 95 percent interval read from the 2.5 and 97.5 percentiles of the resampled means. The same resampling gives the interval on the CHM difference. The percentile method is used rather than the bias-corrected and accelerated method because the per-question exact-match difference takes only the values −1, 0 and +1, on which the acceleration estimate is unstable at this sample size.

The interval describes the size of a difference; whether a contrast counts as significant is decided by the Holm-corrected test of the next section. Table D.2 shows why the two are reported separately: the interval of the retrieval baseline on TriviaQA excludes zero, but the contrast does not survive the correction for eight comparisons.

Contrast at C5	$\boldsymbol{\Delta}_{\mathrm{EM}}$	**95% interval**	**Holm p, verdict**
B1 zero-shot · FEVER	−0.030	[−0.117, +0.050]	1.000, not significant
B3 RAG · TriviaQA	+0.053	[+0.003, +0.107]	0.367, not significant
B6 vanilla fine-tune · TriviaQA	+0.163	[+0.100, +0.220]	< 0.001, CAEM win

Table D.2: Three bootstrap intervals on the exact-match difference at C5, with the Holm-corrected p-value and the verdict.

D.4 Holm sequence

The Holm–Bonferroni correction [5] is applied within each benchmark, separately on each axis, over the $m = 8$ baselines. The raw p-values are sorted in ascending order, the k-th smallest is multiplied by $m - k + 1$, a running maximum keeps the adjusted values in order, and values above one are set to one. The family-wise error rate is held at 0.05 within each benchmark. Tables D.3 and D.4 give the exact-match sequences at C5 for TriviaQA, where CAEM's two wins lie, and for CommonsenseQA, the one benchmark with both wins and defeats.

Rank	Baseline	Δ_{EM}	Raw p	$m - k + 1$	Holm p	Verdict
1	B6 vanilla fine-tune	+0.163	8.5×10^{-7}	8	6.8×10^{-6}	CAEM win
2	B7 FLARE	+0.083	0.0039	7	0.027	CAEM win
3	B4 CoT + RAG	+0.057	0.061	6	0.367	–
4	B3 RAG	+0.053	0.065	5	0.367	–
5	B2 chain of thought	+0.033	0.308	4	1.000	–
6	B1 zero-shot	+0.023	0.494	3	1.000	–
7	B8 semantic entropy	+0.020	0.556	2	1.000	–
8	B5 five-shot CoT	+0.013	0.737	1	1.000	–

Table D.3: Holm sequence for exact match on TriviaQA at C5. CAEM is ahead of every baseline on this benchmark, but only the two largest differences survive the correction. Rank four takes the adjusted value of rank three through the running maximum.

Rank	Baseline	Δ_{EM}	Raw p	$m - k + 1$	Holm p	Verdict
1	B3 RAG	+0.117	8.6×10^{-5}	8	6.9×10^{-4}	CAEM win
2	B6 vanilla fine-tune	−0.090	0.0034	7	0.024	baseline win
3	B4 CoT + RAG	+0.087	0.0037	6	0.024	CAEM win
4	B5 five-shot CoT	−0.077	0.0090	5	0.045	baseline win
5	B8 semantic entropy	−0.050	0.068	4	0.273	–
6	B2 chain of thought	−0.043	0.137	3	0.410	–
7	B1 zero-shot	−0.010	0.807	2	1.000	–
8	B7 FLARE	+0.007	0.903	1	1.000	–

Table D.4: Holm sequence for exact match on CommonsenseQA at C5. CAEM beats the two retrieval baselines and loses to vanilla fine-tuning and five-shot chain of thought; rank three takes the adjusted value of rank two through the running maximum.

Tables D.5 and D.6 list every contrast at C5 and C2 with its interval; $*$ marks a CAEM win and (B) a baseline win under the rule of the next section.

D.5 Win and defeat rule

A contrast counts as a win or a defeat only when two conditions hold together: the Holm-corrected p-value is below 0.05, and the absolute difference is at least 0.02 in

Benchmark	**Baseline**	**ΔEM** [95% CI]		**ΔCHM** [95% CI]	
FEVER	B1	−0.030 [−0.12, +0.05]	-	+0.005 [−0.02, +0.03]	-
	B2	−0.037 [−0.12, +0.05]	-	+0.006 [−0.02, +0.03]	-
	B3	+0.040 [−0.05, +0.13]	-	−0.029 [−0.06, +0.00]	-
	B4	+0.040 [−0.05, +0.13]	-	−0.017 [−0.05, +0.01]	-
	B5	+0.010 [−0.07, +0.09]	-	+0.012 [−0.01, +0.03]	-
	B6	−0.163 [−0.23, −0.10]	(B)	+0.068 [+0.05, +0.09]	(B)
	B7	+0.060 [−0.03, +0.16]	-	−0.065 [−0.10, −0.03]	*
	B8	−0.037 [−0.11, +0.04]	-	+0.010 [−0.01, +0.03]	-
TriviaQA	B1	+0.023 [−0.03, +0.08]	-	+0.003 [−0.01, +0.02]	-
	B2	+0.033 [−0.02, +0.09]	-	−0.000 [−0.02, +0.02]	-
	B3	+0.053 [+0.00, +0.11]	-	−0.008 [−0.02, +0.01]	-
	B4	+0.057 [+0.00, +0.11]	-	−0.002 [−0.02, +0.01]	-
	B5	+0.013 [−0.04, +0.07]	-	+0.000 [−0.02, +0.02]	-
	B6	+0.163 [+0.10, +0.22]	*	−0.024 [−0.04, −0.01]	-
	B7	+0.083 [+0.03, +0.14]	*	−0.075 [−0.10, −0.06]	*
	B8	+0.020 [−0.03, +0.08]	-	−0.000 [−0.02, +0.02]	-
CommonsenseQA	B1	−0.010 [−0.07, +0.04]	-	+0.003 [−0.02, +0.02]	-
	B2	−0.043 [−0.10, +0.01]	-	+0.012 [−0.01, +0.03]	-
	B3	+0.117 [+0.07, +0.17]	*	−0.048 [−0.07, −0.03]	*
	B4	+0.087 [+0.03, +0.14]	*	−0.037 [−0.06, −0.01]	*
	B5	−0.077 [−0.13, −0.03]	(B)	+0.028 [+0.01, +0.05]	-
	B6	−0.090 [−0.15, −0.04]	(B)	+0.056 [+0.03, +0.08]	(B)
	B7	+0.007 [−0.05, +0.06]	-	−0.020 [−0.04, +0.00]	-
	B8	−0.050 [−0.10, −0.00]	-	+0.017 [−0.00, +0.04]	-
TruthfulQA	B1	+0.003 [−0.06, +0.07]	-	+0.021 [+0.00, +0.04]	-
	B2	−0.027 [−0.09, +0.03]	-	+0.020 [+0.00, +0.04]	-
	B3	+0.073 [+0.01, +0.13]	-	−0.015 [−0.03, +0.00]	-
	B4	+0.060 [+0.00, +0.12]	-	−0.010 [−0.03, +0.01]	-
	B5	−0.093 [−0.16, −0.03]	-	+0.033 [+0.02, +0.05]	(B)
	B6	+0.090 [+0.03, +0.15]	*	+0.022 [+0.01, +0.04]	-
	B7	+0.020 [−0.05, +0.08]	-	−0.087 [−0.11, −0.06]	*
	B8	+0.010 [−0.05, +0.08]	-	−0.005 [−0.02, +0.01]	-
StrategyQA	B1	−0.003 [−0.08, +0.07]	-	−0.023 [−0.05, +0.00]	-
	B2	+0.060 [−0.02, +0.14]	-	−0.051 [−0.08, −0.02]	*
	B3	+0.030 [−0.05, +0.11]	-	−0.056 [−0.08, −0.03]	*
	B4	+0.037 [−0.04, +0.12]	-	−0.053 [−0.08, −0.03]	*
	B5	+0.080 [−0.01, +0.16]	-	−0.048 [−0.07, −0.02]	*
	B6	−0.020 [−0.09, +0.05]	-	+0.035 [+0.02, +0.05]	(B)
	B7	+0.040 [−0.05, +0.12]	-	−0.060 [−0.09, −0.03]	*
	B8	+0.000 [−0.07, +0.07]	-	−0.025 [−0.05, +0.00]	-

Table D.5: Per-benchmark paired tests of CAEM at C5 against each baseline (300 questions per benchmark). Baselines are numbered as in Section 5.4.1. A positive ΔEM and a negative ΔCHM favour CAEM; $*$ marks a significant CAEM win and (B) a significant baseline win.

Benchmark	**Baseline**	**ΔEM** [95% CI]		**ΔCHM** [95% CI]	
FEVER	B1	+0.027 [−0.05, +0.10]	-	−0.022 [−0.05, +0.00]	-
	B2	+0.020 [−0.06, +0.10]	-	−0.022 [−0.05, +0.00]	-
	B3	+0.097 [+0.01, +0.18]	-	−0.056 [−0.08, −0.03]	*
	B4	+0.097 [+0.01, +0.18]	-	−0.045 [−0.07, −0.02]	*
	B5	+0.067 [−0.01, +0.15]	-	−0.016 [−0.04, +0.01]	-
	B6	−0.107 [−0.17, −0.04]	(B)	+0.040 [+0.02, +0.06]	(B)
	B7	+0.117 [+0.03, +0.20]	-	−0.092 [−0.12, −0.06]	*
	B8	+0.020 [−0.06, +0.10]	-	−0.018 [−0.04, +0.01]	-
TriviaQA	B1	+0.063 [+0.01, +0.12]	-	−0.014 [−0.03, +0.00]	-
	B2	+0.073 [+0.01, +0.13]	-	−0.018 [−0.03, −0.00]	-
	B3	+0.093 [+0.05, +0.14]	*	−0.026 [−0.04, −0.01]	*
	B4	+0.097 [+0.05, +0.14]	*	−0.020 [−0.04, −0.00]	*
	B5	+0.053 [−0.00, +0.11]	-	−0.017 [−0.03, −0.00]	-
	B6	+0.203 [+0.14, +0.26]	*	−0.041 [−0.06, −0.03]	*
	B7	+0.123 [+0.07, +0.18]	*	−0.093 [−0.11, −0.07]	*
	B8	+0.060 [+0.01, +0.11]	-	−0.018 [−0.03, −0.00]	-
CommonsenseQA	B1	−0.010 [−0.05, +0.03]	-	+0.005 [−0.01, +0.02]	-
	B2	−0.043 [−0.09, +0.01]	-	+0.015 [−0.01, +0.03]	-
	B3	+0.117 [+0.06, +0.18]	*	−0.046 [−0.07, −0.02]	*
	B4	+0.087 [+0.03, +0.14]	*	−0.034 [−0.06, −0.01]	*
	B5	−0.077 [−0.12, −0.03]	(B)	+0.030 [+0.01, +0.05]	(B)
	B6	−0.090 [−0.14, −0.04]	(B)	+0.059 [+0.04, +0.08]	(B)
	B7	+0.007 [−0.04, +0.06]	-	−0.018 [−0.04, +0.01]	-
	B8	−0.050 [−0.09, −0.01]	-	+0.020 [+0.00, +0.04]	-
TruthfulQA	B1	+0.027 [−0.03, +0.09]	-	+0.011 [−0.00, +0.03]	-
	B2	−0.003 [−0.06, +0.06]	-	+0.010 [−0.01, +0.02]	-
	B3	+0.097 [+0.04, +0.15]	*	−0.025 [−0.04, −0.01]	*
	B4	+0.083 [+0.03, +0.14]	*	−0.020 [−0.04, −0.00]	-
	B5	−0.070 [−0.13, −0.00]	-	+0.023 [+0.01, +0.04]	(B)
	B6	+0.113 [+0.06, +0.17]	*	+0.012 [−0.00, +0.03]	-
	B7	+0.043 [−0.02, +0.10]	-	−0.097 [−0.12, −0.07]	*
	B8	+0.033 [−0.03, +0.10]	-	−0.015 [−0.03, +0.00]	-
StrategyQA	B1	+0.010 [−0.06, +0.07]	-	−0.006 [−0.03, +0.02]	-
	B2	+0.073 [+0.00, +0.15]	-	−0.033 [−0.06, −0.01]	-
	B3	+0.043 [−0.02, +0.11]	-	−0.039 [−0.07, −0.01]	*
	B4	+0.050 [−0.01, +0.11]	-	−0.035 [−0.06, −0.01]	-
	B5	+0.093 [+0.02, +0.16]	-	−0.030 [−0.06, −0.00]	-
	B6	−0.007 [−0.07, +0.06]	-	+0.052 [+0.03, +0.07]	(B)
	B7	+0.053 [−0.02, +0.12]	-	−0.042 [−0.07, −0.01]	*
	B8	+0.013 [−0.05, +0.08]	-	−0.007 [−0.03, +0.02]	-

Table D.6: Per-benchmark paired tests of CAEM at C2 against each baseline (300 questions per benchmark). Baselines are numbered as in Section 5.4.1. A positive ΔEM and a negative ΔCHM favour CAEM; $*$ marks a significant CAEM win and (B) a significant baseline win.

exact match or 0.01 in CHM. The direction of the difference decides whether it is a CAEM win or a baseline win (for CHM, lower is better). The second condition matters because with 300 paired questions a test can detect differences too small to matter; the floors keep such contrasts out of the counts. The floors are the same for every contrast, cycle and baseline.

Counts: over the 40 contrasts per cycle (8 baselines on 5 benchmarks), exact match gives 9 CAEM wins and 2 defeats at C0, 9 and 3 at C2, and 5 and 3 at C5; CHM gives 6 wins and 17 defeats at C0, 13 and 5 at C2, and 10 and 4 at C5 (Table 5.20). Against the prompting baselines and semantic-entropy abstention (B1, B2, B5, B8), no exact-match contrast is a CAEM win at any cycle. The exact-match defeats at C2 and C5 are against vanilla fine-tuning on FEVER and CommonsenseQA and five-shot chain of thought on CommonsenseQA.

Verifier-free check: the same tests on the CHM restricted to the five subtypes that use neither CAEM's composite score nor its decisions give 15 wins and 4 defeats at C0, 28 and 2 at C2, and 31 and 3 at C5. CAEM fares better on this measure than on the full CHM, partly by construction, as discussed in Section 5.6.4.

Appendix E

Reproducibility recipe

E.1 Data preparation

The five benchmark loaders that feed the realised evaluation panel are registered in `eval/benchmarks.py` against the Hugging Face dataset hub identifiers. FEVER reads through `lucadiliello/fever` under the development and training splits; TriviaQA reads through `trivia_qa` under the reading-comprehension-no-context configuration; CommonsenseQA reads through `commonsense_qa`; TruthfulQA reads through `truthful_qa` under the generation configuration; StrategyQA reads through `ChilleD/StrategyQA` with documented fallbacks to `wics/strategy-qa` and the Eladsegal train-JSON mirror. Earlier loader registrations for Natural Questions and HotpotQA are preserved in the module for reproducibility of the cycle-zero diagnostics but do not feed the realised five-benchmark panel.

The six-way disjoint partition is constructed once at pool-builder time by `c aem/benchmark_splits.py::build_all_benchmark_pools`, which is invoked by the experiment runner and writes the per-benchmark identifier manifest to `outputs/full_run/dataset_splits.json`. Each benchmark contributes a one-thousand-sample seed pool for cold-start memory population, a five-hundred-sample calibration fold, a five-hundred-sample purity-validation slice, ten per-cycle stream-chunk slices (FEVER and TriviaQA at two thousand samples per cycle, CommonsenseQA at seven hundred samples per cycle to fit the smaller available train pool of approximately ten thousand samples after deduplication), a three-hundred-sample evaluation fold, and a test fold of up to five hundred samples (clamped to the available remainder where the source split admits fewer than the registered budget). The shuffle is seeded at forty-two on the training side and forty-three on the evaluation side, so the partitions are stable across re-invocations on the same source dataset.

Within-benchmark deduplication runs through the `_dedupe_by_content_id` helper, which keys on the first five hundred characters of the question text under a sixteen-character SHA-256 hash. Cross-benchmark disjointness is verified by the `assert_ cross_benchmark_disjoint` assertion. Both checks run at pool-builder time; the runner re-stamps the same checks at every successful cycle boundary through `_dedu p_inplace` and `assert_disjoint_calibration`, so any drift in the partition would be caught at the cycle close rather than at the metric-computation step.

The realised evaluation fold reads three hundred samples per benchmark across the five-benchmark panel, for a total of one thousand five hundred evaluation contrasts per cycle. The StrategyQA test fold reads three hundred and eighty-seven samples rather than the registered five hundred because the source split the loader reads (the ChilleD StrategyQA test split) admits only six hundred and eighty-seven labelled samples, of which three hundred feed the evaluation fold and the remaining three hundred and eighty-seven feed the test fold under the auto-clamp branch in the pool builder.

E.2 Snapshot recovery

Two complementary publication endpoints anchor the snapshot-recovery sequence. The Google Drive folder at https://drive.google.com/drive/folders/1xErhsRDYe_qO0ZgUBn0WXK81eHc5jZCa carries the realised empirical artefacts under the `v2_1_phase1e_main/` subtree (cycle-zero through cycle-five outputs, baseline panels, the classifier on/off ablation, and the per-cycle logs the thesis reads), and the `pre_main_snapshot/` subtree that mirrors the per-commit pre-main snapshot bundles for fresh-clone recovery. The Hugging Face dataset repository `aksaN000/caem-passage-index-21m` (private; access on request) holds the sixty-four-gigabyte Wikipedia passage FAISS index at the repository root and the per-commit pre-main snapshot bundles under `pre_main_snapshot/<git-commit>/` subtrees. The pre-Step-7 anchor is the commit tag `c5b8066`; the retrieval-utility-classifier-era anchor is `700b9e3`. The manifest at `pre_main_snapshot/<commit>/MANIFEST.json` records the upload timestamp, the source host, the git commit, the purpose tag, and the recovery hint; the integrity guarantee rides on Hugging Face's per-blob SHA-256 checksums plus the manifest's git-commit anchor against the live code tree.

The three-command recovery sequence runs from a fresh clone with the Hugging Face command-line interface authenticated to the private repository:

```
# 1. Fetch the index and the pre-main snapshot bundles
hf download aksaN000/caem-passage-index-21m \
    --include 'passages.*' 'pre_main_snapshot/*'

# 2. Stage the chosen anchor into outputs/
cp -r pre_main_snapshot/c5b8066/* outputs/

# 3. Pin the code to the matching commit
git checkout c5b8066    # or 700b9e3 for the RUC-era resume point
```

After the third command the working tree holds the cold-start memory store, the cycle-zero memory and deferred-buffer snapshots, the cycle-zero calibration fold (per-sample nine-signal vectors and gold labels), the cycle-zero composite calibration JSON, the calibrated configuration JSON, the cycle-zero validation-gate verdict, the cycle-zero evaluation fold under the pristine generator, the post-calibration rescored evaluation fold, the dataset-splits manifest, and the run logs. The four artefacts that must be regenerated rather than pulled from the snapshot are the Qwen-2.5-3B-Instruct base weights (recovered from the Hugging Face model hub at deployment time), the per-cycle LoRA adapters at cycles one through five, the

per-cycle self-improvement training artefacts, and the post-cycle-zero composite refits.

E.3 Step-by-step run sequence

The end-to-end sequence below is the runner's view of the trajectory from a fresh clone to the closed cycle-five artefact bundle. Per-step commands are the actual invocations from the project runner; expected wall-times are reported on the production envelope of Section E.5.

Pre-run setup:

Prompt smoke test.
Approximately fifteen minutes on the production GPU. Writes `outputs/prompt_smoke/results.json`. Gate: the FEVER not-enough-information rate stays inside the registered tolerance band and no template-leak strings appear in the rendered samples.

```
python scripts/prompt_smoke_test.py \
    --benchmarks fever triviaqa commonsense_qa
```

Alias-dictionary build.
Approximately five minutes on CPU. Writes `data/alias_dict.json`. The deployed verifier does not consume the alias signal in the locked composite; the dictionary is built for the ablation registry only.

```
python scripts/build_alias_dict_from_benchmarks.py
```

Cold-start seeding.
Approximately two hours on the production GPU. Writes the cold-start memory store under `outputs/cold_start_memory/`. Gate: the total seeded count clears the registered floor; the EM-prune branch removes fewer than ten percent of the seeded entries.

```
python -m scripts.seed_cold_start
```

Cycle-zero baseline evaluation and calibration fold scoring.
Approximately six hours on the production GPU. Writes per-sample nine-signal vectors and ground-truth labels for the cycle-zero calibration fold and the held-out evaluation fold. Gate: every per-bench JSON has the expected sample count and no NaN signals.

```
python -m scripts.run_experiment \
    --output_dir outputs/cycle_0 \
    --num_cycles 0
```

Composite calibration fit and validation gate.
Approximately fifteen minutes on CPU. Writes the cycle-zero composite calibration record and the validation-gate verdict at `outputs/cycle_0/composite_calibration.json` and `outputs/cycle_0/weight_validation.json`. Gate: the four-check conjunction of Section A.9 passes.

```
python scripts/fit_composite_calibration.py
python scripts/validate_composite_weights.py
```

Retention baseline freeze and snapshot upload.
Approximately one hour on the production GPU. Writes the pristine probe accuracies and pushes the pre-main snapshot to the Hugging Face repository.

```
python -m scripts.run_retention_baseline
bash scripts/upload_pre_main_snapshot.sh
```

Main cycle loop: A single invocation drives all six per-cycle sub-steps and resumes from the highest-cycle artefact set on disk:

```
python -m scripts.run_experiment \
    --output_dir outputs/full_run \
    --num_cycles 10 \
    --n_questions 5000 \
    --n_eval_questions 300 \
    --benchmarks fever triviaqa commonsense_qa truthfulqa strategyqa \
    --passage_index data/passage_index \
    --cold_start_memory outputs/cold_start_memory/memory_store \
    --verifier_backend minicheck \
    --resume_from_cycle <N>
```

The runner internally sequences the self-improvement fine-tune, the cycle-boundary recalibration, the retroactive re-verification pass, the deferred-buffer reconsider pass, the stream-chunk admission pass, the held-out evaluation pass, and the cycle-checkpoint plus halt-trigger evaluation. Per-cycle wall-time on the production GPU grows with the cumulative memory size, from roughly six hours at cycle one to more than twenty-two hours at cycle five (see the per-cycle wall-clock budget below). Cycle close writes the per-cycle memory and deferred-buffer snapshots, the per-cycle composite calibration record, the per-cycle calibrated configuration, the per-cycle evaluation JSON, and the per-cycle coverage diagnostic. The retention guard licenses advancement to the next cycle; failure triggers the asymmetric rollback contract documented at Section A.5.

Post-run analysis: The baselines and every reported number are produced by two commands.

Baseline panel under the comparison protocol.
The baseline re-run of Section 5.1.1 on one rented 32-gigabyte graphics processor: retrieval of the evaluation passages by exact search, B1, B2, B5 and B8 on all five benchmarks, B3 and B4 on TruthfulQA, FLARE, the evaluation of the vanilla fine-tune, and the verifier pass over every baseline answer with the cycle-three adapter and composite. Each stage can be run on its own.

```
bash scripts/rental/final_version.sh all
```

Final numbers, tables and figures.
On CPU in about two minutes: writes the final correctness label into every answer file, recomputes CHM in the model and delivered views and the status-tag accuracy,

runs the paired tests, and writes every chapter-five table and figure and the result tables of the appendices. The option `-judge` first re-judges every TruthfulQA answer with the language-model judge (requires `ANTHROPIC_API_KEY`); the option `-check` fails if any committed output changes.

```
bash scripts/make_final_results.sh --check
```

E.4 Version pinning

The realised environment runs against the version pins below. The project does not ship a `requirements.txt` or `pyproject.toml`; the pins are reconstructed from `README.md`, the per-version diagnostics emitted by `scripts/check_env.py`, the import set exercised by the unit-test suite, and the entries in `branch_C_log.md`. The canonical environment is the virtual environment at `/venv/main` on the Vast production host.

Package	Version	Role
Python	3.12	interpreter
CUDA toolkit	13.0 (driver 13.1)	RTX 5090 Blackwell compute capability `sm_120`
torch	2.10.0+cu130	tensor backend, bf16 forward path, MC-dropout sampler
transformers	4.57.6	Qwen-2.5-3B-Instruct loader, generate, MiniCheck encoder
peft	0.19+	LoRA wrapper for the self-improvement fine-tune
accelerate	0.34+	device placement during the fine-tune step
datasets	5.0+	benchmark loaders and retention-probe shuffles
faiss-cpu	1.8+	passage index and episodic memory store
sentence-transformers	4.1.0	SBERT memory encoder and cross-encoder reranker
scipy	1.11+	saturating-exponential equilibrium fit, Wilcoxon test
scikit-learn	1.4+	per-signal isotonic regression and boost-layer logistic
bitsandbytes	0.43+	eight-bit AdamW optimiser path
anthropic	< 1	TruthfulQA judge client
numpy, pandas, tqdm	latest stable	arithmetic, table aggregation, progress bars
biber + biblatex	TeX Live 2024+	thesis bibliography compile

Table E.1: Framework version pinning. Versions are reconstructed from the diagnostic emitters and live import set; the canonical environment is the virtual environment on the production host. Any reproducer who pins to the listed versions exactly will produce numbers within the determinism tolerance of Section E.6.

The determinism switches that produce identical numbers from a fresh checkout are recorded in the seed-everything helper inside the runner. The Python pseudo-random module, the NumPy random state, the PyTorch CPU and CUDA random states, and the `PYTHONHASHSEED` environment variable are all set to forty-two. The FAISS index build, the MC-dropout passes, and the generator sampling all consume the same seed through their respective subsystems. The flag `torch.use_deterministic_algorithms` is set to false intentionally because the cuDNN attention path is not bitwise-deterministic; the `torch.backends.cudnn.deterministic` flag is set to true and the `torch.backends.cudnn.benchmark` flag is set to false. TensorFloat-32 matrix multiplication is enabled on the Blackwell compute capability, and the bfloat16 forward path is the default.

The two required environment variables are `ANTHROPIC_API_KEY` for the Haiku rescore of the TruthfulQA correctness rule and `HF_TOKEN` (equivalently `HUGGINGFACE_HUB_TOKEN`) for the pull from the private pre-main snapshot repository. Optional environment variables `HUGGINGFACE_HUB_CACHE`, `CAEM_DEVICE`, and `CUDA_VISIBLE_DEVICES` override the cache location, the compute device, and the GPU index respectively.

E.5 Hardware envelope

Production envelope: The realised trajectory runs on a Vast.ai-rented RTX 5090 with thirty-two gigabytes of video memory (approximately thirty-one gigabytes usable after framework overhead), one hundred and fifty gigabytes of NVMe disk, and approximately ninety gigabytes of host random-access memory at approximately seventy cents per on-demand GPU-hour. The trajectory closes across approximately forty calendar days of elapsed time over the cold-start seeding, the cycle-zero calibration and validation gate, the six-cycle main run, the eight-baseline panel, and the post-run analysis, for approximately nine hundred and sixty GPU-hours at a total billed cost of approximately six hundred and seventy-two United States dollars (from the Vast.ai billing records; forty days at twenty-four hours per day at the on-demand rate). The verifier-side scoring path (nine-signal extraction across every served answer, the per-cycle composite refit, the cycle-boundary retroactive re-verification pass, and the matched-protocol Haiku-judged TruthfulQA rescore across the baseline panel) is the dominant wall-clock consumer over the run, exceeding the per-cycle low-rank-adapter fine-tune itself, because every entry on every cycle and every baseline-arm answer passes through the full verifier ensemble while the adapter touches only the strict training-pool subset. The baseline re-run of Section 5.1.1 ran later on a second rented machine of the same class and is not included in these totals.

Per-cycle wall-clock budget: The self-improvement fine-tune step consumes approximately thirty-five to forty-five minutes per cycle under the bounded LoRA envelope. The cycle-boundary recalibration step consumes approximately fifteen to twenty minutes including the per-signal isotonic fit, the per-benchmark shrinkage blend, and the boost-layer refit. The retroactive re-verification pass consumes approximately twenty to thirty minutes depending on the cumulative memory size. The stream-chunk admission pass consumes approximately three and a half to four

and a half hours at four thousand seven hundred candidate samples per cycle through the full per-query pipeline including the retrieval-augmented tier. The run log does not time the held-out evaluation pass on its own; the measured window from the end of the fine-tune to the end of the retroactive re-verification pass, which holds the calibration-fold scoring, the held-out evaluation pass, and the re-verification pass, is five hours forty-five minutes at cycle one, nine hours forty-four minutes at cycle two, thirteen hours fifty-four minutes at cycle three, and twenty-one hours fifty minutes at cycle five. A full cycle therefore takes at least these measured windows plus the fine-tune and stream-chunk admission steps, so the end-to-end cycle time grows from roughly six hours at cycle one to more than twenty-two hours at cycle five.

Downgraded reproduction envelope: A consumer sixteen-gigabyte video-memory GPU reproduces the trajectory under five parameter changes registered in the runner: the SIL batch size drops from four to two, the gradient accumulation step count rises from four to sixteen (an effective batch size of thirty-two, twice the production value of sixteen), the precision drops from bfloat16 to float16 with dynamic loss scaling (consumer-tier Ampere bfloat16 paths are slower than the equivalent float16 path), the generation new-token cap drops from five hundred and twelve to one hundred and ninety-two to fit the smaller key-value cache, and activation checkpointing is enabled. The LoRA rank and alpha stay unchanged. The expected per-cycle wall-clock penalty is approximately two-point-two to two-point-six times the production envelope, so the full trajectory on a downgraded GPU takes correspondingly longer than on the production envelope, by the same factor.

Writeup envelope: The thesis is written on a Windows Subsystem for Linux 2 environment with an RTX 3060 (twelve gigabytes of video memory) and sixteen gigabytes of host random-access memory. The sixty-four-gigabyte Wikipedia FAISS passage index exceeds the host random-access memory by a factor of four and cannot be loaded on the writeup envelope; all retrieval-side numbers in the thesis are read from cached evaluation JSON files under `outputs/full_run/` rather than recomputed locally. The local writeup envelope hosts the thesis source, the table and figure compilation, and the signal-level audits over saved verifier traces, but does not host the live retrieval substrate.

E.6 Verification of reproduced numbers

A reproducer who has executed the full trajectory uses the ten-item checklist below to confirm that the headline numbers match the thesis within the stated tolerance. The per-benchmark exact match on 300 questions carries a stochastic floor of about ± 0.5 percentage points from the non-deterministic attention path; the pooled exact match over the five benchmarks tightens to about ± 0.3 points; the composite hallucination metric and the storage precision tolerate about ± 1.0 point; integer counts (commit pattern, win and defeat tallies) must match exactly when the results command is run on the same answer files. Items one to three, eight and ten are rebuilt from the saved answer files by the results command above, which reproduces them exactly.

Pooled CAEM exact match.

- Receipt: the final correctness label of every

answer in `outputs/full_run/eval/`, pooled over the five benchmarks (Table 5.5).

- Expected: 0.517 at C0, 0.535 at C2, 0.508 at C5; B1 0.511.
- Tolerance: ±0.3 percentage points.

Pooled composite hallucination metric.
- Receipt: `outputs/analysis/chm_judged.json`, model view (mean of the seven active failure flags).
- Expected: 0.155 at C0, 0.127 at C2, 0.134 at C5; B1 0.132.
- Tolerance: ±1.0 percentage point.

Per-benchmark exact match and CHM by cycle.
- Receipt: the same record, per benchmark.
- Expected: each cell of Table 5.5.
- Tolerance: ±0.5 percentage points.

Calibration-fold pooled storage precision band.
- Receipt: `outputs/full_run/pi_store_trajectory.csv`, column `cal_fold_pi_store` across the committed-cycle rows.
- Expected: band $[0.732, 0.746]$.
- Tolerance: ±1.0 percentage point on each endpoint.

Cycle-zero validation-gate composite Cohen's d.
- Receipt: `outputs/analysis/weight_validation_v2_1.json`, field `composite_cohen_d_id.value`.
- Expected: +0.635.
- Tolerance: ±0.02.

Memory pool size at cycle five close.
- Receipt: `outputs/full_run/memory_store_cycle_5.meta`, entry count.
- Expected: 12 175 entries.
- Tolerance: ±5 for content-hash collision drift.

Realised commit pattern and abort cycle.
- Receipt: `outputs/full_run/stochastic_equilibrium_trajectory.csv`, columns `verdict` and `committed`.
- Expected: commit pattern $(1, 1, 1, 0, 1)$ across C1 to C5, with cycle four aborted by the TriviaQA-test retention probe.
- Tolerance: exact match required.

Retrieval-utility classifier ablation.
- Receipt: the classifier-off answers at cycle three in `outputs/phase1d_eval/` against the main run at cycle three (Table 5.21).
- Expected: pooled exact match 0.467 off and 0.516 on, a lift of 4.9 percentage points.

- Tolerance: ±0.5 percentage points.

Status-tag accuracy.
- Receipt: `outputs/analysis/chm_judged.json`, field `tags` of each CAEM cycle (Table 5.4).
- Expected: verified answers correct 0.746, 0.655 and 0.667 of the time at C0, C2 and C5.
- Tolerance: ±1.0 percentage point.

Significance win and defeat tallies.
- Receipt: `outputs/analysis/paired_baseline_tests_final.json` (Table 5.20).
- Expected: exact match 9 wins and 2 defeats at C0, 9 and 3 at C2, 5 and 3 at C5; CHM 6 and 17 at C0, 13 and 5 at C2, 10 and 4 at C5, each out of 40 contrasts.
- Tolerance: exact match required.

A reproducer whose readings clear every item within the stated tolerance has matched the thesis to within the determinism floor of the compute envelope. An item that fails by a larger margin points to a determinism gap the runner did not absorb (the seed-everything helper is the first place to inspect), a version mismatch against Table E.1, a hardware change not registered in the runner's overrides, or a genuine correction that the empirical chapter should adopt.

Appendix F

Glossary

The glossary collects every concept word that recurs in body prose and whose meaning a reader who has read Chapter 1 alone might not infer. Each entry pairs the term with a one-sentence concept-word definition and a cross-reference to the chapter subsection that introduces the term. Acronyms used in body prose are listed together with their expansions; concept terms are listed in alphabetical order. Implementation identifiers (module paths, class names, script names) do not appear here; they live in Chapter A.

F.1 Concept terms

Asymmetric rollback.
The retention guard's response on a cycle whose worst-probe ratio falls below the registered floor: the adapter weights revert to the previous successful cycle's checkpoint, but the memory store does not revert, so the storage admissions accumulated during the failed cycle's stream-chunk pass are preserved. The retroactive re-verification pass and the deferred-buffer reconsider pass are skipped on the aborted cycle because both would be provable no-ops under unchanged weights. See Section 5.1.5.

Boost layer.
The L2-regularised logistic regression that aggregates the nine per-signal calibrated probabilities produced by the per-signal isotonic regression stage into the final calibrated probability composite. Introduced at Section 4.2.7.

Calibration fold.
The five-hundred-sample-per-training-benchmark labelled slice that fits the per-signal isotonic regression, the per-benchmark shrinkage prior, and the boost-layer logistic regression at every successful cycle boundary. Held content-hash disjoint from the seed pool, the purity-validation slice, the stream chunks, the evaluation fold, and the test fold. See Section 5.2.1.

Calibrated-probability composite.
The two-stage scoring head that maps the nine verifier signals into a single calibrated correctness probability $u_{\text{stored}} \in [0, 1]$ through per-signal isotonic regression followed by the boost layer. The fixed-threshold storage gate reads against this output. See Section 4.2.7.

Combined dispatch score.
The router's weighted blend of the cosine similarity to the nearest stored entry and the stored confidence of that neighbour, against which the memory-direct tier threshold is read. See Section 4.2.4.

Committed cycle.
A cycle whose self-improvement fine-tune cleared the retention guard and whose calibration-fold refit was applied at the cycle boundary. The realised trajectory records four committed cycles (C1, C2, C3, C5) against the cycle-four abort. See Section 5.1.5.

Composite evaluation score (CES).
The five-axis geometric-mean roll-up that combines accuracy, episodic contribution, retention, calibration, and verifier reliability into a single per-cycle scalar. The CES peak across the trajectory falls at cycle five. See Section 5.1.2.

Composite hallucination metric (CHM).
The seven-axis equal-weighted mean over the post-generation hallucination subtypes (confident confabulation, false refusal, factual fabrication, logical fabrication, template leak, defensive evasion, over-long generation); factual contradiction and off-topic answers are recorded but cannot fire in this run. The architecture-vs-baseline CHM contrast is the headline hallucination-reduction axis. See Section 5.1.2.

Cycle.
A single iteration of the self-improvement loop: a stream-chunk pre-routing pass that accumulates storage admissions, a cycle-boundary recalibration that refits the per-signal isotonic curves and the per-benchmark shrinkage and boost-layer weights, a retroactive re-verification pass that updates the stored confidences against the recalibrated verifier, and a self-improvement fine-tune that distils verified retrieval-augmented reasoning into the generator's parametric capacity. See Section 4.1.2.

Deferred buffer.
The bounded-capacity reconsider queue that holds outputs whose calibrated probability composite cleared the deferral threshold but fell short of the storage threshold. Each entry carries an age counter, is rescored under the recalibrated verifier at every successful cycle boundary, and is dropped from the buffer when the counter reaches the registered time-to-live of two reconsider invocations without a promotion. See Section 4.2.10.

Empirical-precision anchor.
The realised storage precision on the calibration fold at the locked storage cut-point, audited once per cycle and reported as the headline diagnostic that licences the calibration-consistent precision floor of the pool-purity theorem. See Section 5.2.5.

Equilibrium-saturation gate.
The two-of-three composite stop trigger that closes the trajectory when at least two of the three signals fire simultaneously: storage saturation under repeated zero-admission cycles, MMLU-floor breach against the cycle-zero pristine baseline, and composite-evaluation-score plateau across the late-cycle subsequence. The realised trajectory closes at cycle five under the storage-saturation and MMLU-floor signals. See Section 5.1.7.

Evaluation fold.
The three-hundred-sample-per-benchmark held-out slice on which every per-cycle headline metric is computed. Held content-hash disjoint from the calibration fold, the seed pool, the purity-validation slice, and the stream chunks. See Section 5.1.1.

Fixed-threshold storage gate.
The architecturally frozen storage admission rule that admits a generated output into the memory pool when its calibrated probability composite clears the registered cut-point $\tau_{\text{store}} = 0.60$. The cut-point does not move across the trajectory; the per-cycle composite refit moves the signal-to-probability mapping the cut-point operates on. See Section 4.2.9.

Four-outcome decision tree.
The per-query post-verification routing rule that maps each generated output to one of the four outcomes STORE, DEFER, ABSTAIN, or DISCARD on the basis of the calibrated probability composite and the grounding signal. See Section 4.2.9.

Matched-protocol contract.
The pairing rule that every per-benchmark contrast between the architecture and an external baseline is computed question by question on the same evaluation-fold questions, with CAEM's question message, the same scoring rule and the same hallucination instrument for every system, and with no baseline charged for a decision it did not take. See Section 5.1.1.

Memory-direct tier (Tier 1).
The serving tier that returns the nearest stored entry's answer directly when the combined dispatch score clears the registered Tier 1 threshold. The cheapest of the three tiers per query because it neither runs the generator nor retrieves passages. See Section 4.2.4.

Memory-poisoning rate.
The share of stored entries whose stored answer is wrong against the ground-truth label, audited at every cycle boundary as the complement of the storage precision. The validation-gate rule registers this rate at a fifty-percent ceiling on every training benchmark at cycle zero. See Section 5.2.7.

Per-cycle composite refit.
The cycle-boundary recalibration of the per-signal isotonic curves, the per-benchmark shrinkage prior, the per-benchmark temperature scalar, the per-benchmark safety floor, and the boost-layer weights and intercept. The cut-points themselves are architecturally frozen; the refit moves the signal-to-probability mapping under which the cut-points operate. See Section 5.2.4.

Pool purity.
The empirical correctness rate of the stored memory pool against the ground-truth label, reported as π_{store} on the calibration fold (the theorem-anchored reading) and on the evaluation fold (the deployment-realised reading). The calibration-fold band across the committed-cycle subsequence is $[0.732, 0.746]$. See Section 5.1.7.

Pre-routing confidence (u_{pre}).
A cheap confidence score computed before an answer is generated: it combines the token confidence of a 32-token greedy decode with a decoder layer-convergence ratio from one forward pass, calibrated by temperature, and drives the safety override.

See Section 4.2.2.

Production envelope.
The per-query four-outcome rendering on the deployment surface: every served answer carries the calibrated correctness probability as a continuous percentage alongside the categorical decision tag (store-and-serve, defer, abstain, retrieve-and-serve). See Section 3.1.1.

Purity-validation slice.
The five-hundred-sample-per-training-benchmark slice held disjoint from the calibration fold and the evaluation fold, against which the empirical-precision audit is run once per cycle to license the calibration-consistent precision floor. See Section 5.2.1.

Retention guard.
The cycle-boundary mechanism that measures the post-fine-tune generator on the three-probe multi-modal panel (MMLU, TriviaQA-test, CommonsenseQA-test) and triggers the asymmetric rollback when the worst-probe ratio against the cycle-zero pristine baseline falls below the registered floor of 0.93. See Section 5.1.5.

Retrieval-augmented tier (Tier 3).
The serving tier that retrieves the top-k passages from the dense passage index, reranks them through a cross-encoder, and runs a grounded generation against the reranked context. The architectural fallback when neither the memory-direct nor the zero-shot tier earns the right to answer. See Section 4.2.4.

Retrieval-utility classifier (RUC).
The binary classifier that intercepts the safety-veto fall-through and the score-formula fall-through of the three-tier router and decides whether to dispatch the query to the retrieval-augmented tier or to the zero-shot tier on the basis of a learned per-query retrieval-utility prediction. Closes the retrieval-amplification regression on misconception-bait benchmarks. See Section 4.2.5.

Retroactive re-verification.
The cycle-boundary pass that re-scores every stored entry under the recalibrated verifier, prunes entries whose recalibrated stored confidence drops below the retention threshold $\tau_{\text{retro}} = 0.50$, and writes the recalibrated confidence back to the memory store. Skipped on aborted cycles under the asymmetric-rollback contract. See Section 4.2.10.

Safety override.
The router rule that dispatches the query off the score-formula path whenever the pre-routing confidence falls below the registered safety floor $\phi_{\text{safe}} = 0.38$. Post-classifier, the retrieval-utility classifier intercepts the safety-veto fall-through and chooses between the zero-shot and retrieval-augmented tiers. See Section 4.2.4.

Seed pool.
The one-thousand-sample-per-training-benchmark slice used to populate the cold-start memory through the cold-start seeding pass at the registered cold-start storage threshold of 0.50. See Section 5.1.1.

Self-improvement loop.
The cycle-boundary fine-tune that draws its training pool from stored memory entries whose calibrated probability composite clears the SIL admission threshold

$\tau_{\text{train}} = 0.70$, applies the parameter-efficient update through the bounded low-rank adapter envelope on the frozen backbone, and re-tests the post-fine-tune model against the retention guard. See Section 4.2.10.

Stochastic equilibrium.
The calendar-timeline refinement of the bounded-band stationarity result that reads the realised commit fraction π_{commit} off the per-cycle commit pattern. The realised pattern $(1, 1, 1, 0, 1)$ yields $\pi_{\text{commit}} = 4/5$. See Heuristic 4.9.

Stored confidence (u_{stored}).
The calibrated probability composite's output for a generated answer: the per-signal isotonic curves and the boost-layer logistic regression aggregate the nine verifier signals into a single scalar in the unit interval, against which the storage cut-point is read. See Section 4.2.7.

Stream chunk.
The per-cycle pre-routing pass over a fresh slice of training-benchmark samples (two thousand FEVER, two thousand TriviaQA, seven hundred CommonsenseQA per cycle) that accumulates the per-cycle storage admissions, deferred entries, abstentions, and discards. See Section 5.1.1.

Test fold.
The held-out slice reserved for the post-trajectory comparative panel. Distinct from the evaluation fold against which the per-cycle headline metrics are reported. See Section 5.1.1.

Training-pool admission threshold (τ_{train}).
The second operating point above the storage cut-point at which a stored memory entry is admitted into the self-improvement training pool. Set at 0.70 and held strictly greater than the storage cut-point $\tau_{\text{store}} = 0.60$ so the SIL pool is a strict refinement of the storage pool. See Section 4.2.10.

Verifier-balanced-accuracy precondition.
The pre-specified floor on the verifier's balanced accuracy at the locked storage cut-point: the precondition of the purification condition (Proposition 4.2) on which the calibration-consistent precision floor of Theorem 4.1 relies, audited at cycle zero through the composite Cohen's d gate at the registered floor of 0.20. See Section 5.2.7.

Verifier ensemble.
The nine-signal post-generation block that converts a generated answer into the four-family signal vector (internal calibration, sample-set agreement, external grounding, question-answer match) feeding the calibrated probability composite. See Section 4.2.6.

Zero-shot tier (Tier 2).
The serving tier that runs a generation on the fine-tuned backbone without retrieving passages, dispatched when the similarity to the nearest stored entry clears the registered Tier 2 threshold but the combined dispatch score does not clear the Tier 1 threshold. See Section 4.2.4.

F.2 Acronyms

Bootstrap CI.
Bootstrap confidence interval. The percentile-method confidence interval on the per-sample exact-match-difference vector at the matched-paired sample size of three hundred. See Section 5.3.1.

CES.
Composite evaluation score. See the concept entry above.

CHM.
Composite hallucination metric. See the concept entry above.

Cohen's d.
An effect-size measure for the standardised mean difference between two distributions. The cycle-zero validation gate reads the composite Cohen's d between exact-match-correct and exact-match-incorrect outputs against the registered precondition floor of 0.20. See Section 5.2.7.

EM.
Exact match. The correctness metric, read from the final answer line of each response by one rule for every system: the leading label on FEVER and StrategyQA, the single option letter on CommonsenseQA, an alias match on TriviaQA, and the Haiku-judged rule on TruthfulQA. See Section 5.1.1.

FAISS.
The dense vector similarity-search index used to host the memory store, the Wikipedia passage substrate, and the cross-encoder pair index. See Section 4.2.3.

Haiku-judged.
The Claude Haiku 4.5 large-language-model judge used as the truthfulness adjudicator on the TruthfulQA evaluation fold. The judge reads the question, the model's answer, the correct reference answers, and the incorrect reference answers, and returns a binary truthful-or-not verdict. The uniform Haiku-judged rule is applied to every TruthfulQA exact-match cell in the empirical chapter, including every baseline, and reads the final answer line only. See Section 5.1.1.

Holm-Bonferroni.
The step-down multiple-comparison correction applied within each benchmark family of size $m = 8$ on each metric axis. Sorts the raw p-values ascending and multiplies the k-th smallest by $m - k + 1$ under a running-maximum monotonicity constraint, clipping at one. See Section 5.3.1.

LoRA.
Low-rank adaptation. The parameter-efficient fine-tuning technique that injects trainable low-rank matrices into the seven linear projections of every transformer block of the frozen base generator at rank 32 and alpha 64. See Section 4.2.10.

McNemar test.
The paired significance test for binary outcomes, applied on the exact-match axis. Reads only the two discordant cells of the matched-pair contingency table under Edwards' continuity correction. See Section 5.3.1.

MMLU.
Massive multitask language understanding. The fifty-seven-subject multiple-choice

benchmark used as one of the three probes in the multi-modal retention probe panel. See Section 5.1.5.

NLI.
Natural language inference. The canonical three-class textual-entailment task (entailment, contradiction, neutral) in the literature; the verifier's grounding signal family consumes the binary supported-versus-unsupported MiniCheck variant of this task, with the contradiction class structurally unavailable under the deployed backend. See Section 4.2.6.

RUC.
Retrieval-utility classifier. See the concept entry above.

SBERT.
Sentence-BERT. The siamese-network-fine-tuned sentence encoder that produces the 768-dimensional embeddings consumed by the memory-store lookup, the cycle-boundary consolidation pass, and the cold-start seeding pass. See Section 4.2.3.

SIL.
Self-improvement loop. See the concept entry above.

Wilcoxon signed-rank.
The paired significance test for continuous outcomes, applied on the composite-hallucination-metric axis. Reads the signed-rank sum of the per-sample architecture-minus-baseline composite-hallucination-metric difference vector. See Section 5.3.1.

Appendix G

Architecture positioning, frequently asked questions

The body chapters specify what CAEM does and how. This appendix collects the comparative-positioning and design-justification questions a reviewer in an adjacent area would ask, and answers each by naming the specific mechanism CAEM contributes that the alternative would lack. The first part answers *why CAEM rather than the simpler alternative of plain exact-match-labelled fine-tuning at the same labelling budget* along the six load-bearing differentiators. The second part answers *why the calibration discipline and cycle-boundary ordering chosen, rather than the straightforward alternatives.*

Part I: Positioning against plain fine-tuning

G.1 What does the per-query precision contract give a user that plain fine-tuning cannot?

Plain fine-tuning at the same labelling budget produces a model with an average exact-match accuracy on the held-out evaluation set; the deployed system serves whatever the model generates, with no per-query confidence axis. CAEM's calibrated-probability composite paired with the fixed-threshold storage gate produces a different deliverable: at every query the system outputs a calibrated probability u_{stored} that the served answer is correct, and at the storage threshold $\tau_{\text{store}} = 0.60$ the calibration fold's empirical precision carries over to the stored answers when new questions resemble the calibration fold (Theorem 4.1; the assumption held on the training panel and failed on TruthfulQA); the per-cycle composite refit absorbs distribution drift into the signal-to-probability mapping so that the same threshold continues to address the same precision target as the generator improves.

The user-facing consequence is the status tag and the abstain class. Every answer is shown with its probability and a tag, and a query whose composite is below the deferral threshold and whose grounding is below the abstention floor returns an insufficient-evidence message instead of a guessed answer. On the evaluation fold the tag predicts correctness: answers tagged verified were right 0.65 to 0.75 of the time at every cycle, against 0.51 for B1 over all questions (Table 5.4). A plain fine-tune serves every answer with no such signal; abstention needs a separate uncertainty estimate, such as the semantic-entropy baseline B8, which withholds 10.5 percent of answers. For deployments where confident but wrong answers cost more than refusals (clinical question answering, legal document review, customer-facing knowledge serving), this per-answer signal is the deliverable that matters. Chatbots have been documented fabricating every reference offered for a clinical explanation [44], up to 91.4% of chatbot-generated references in systematic-review searches are hallucinated [66], and TruthfulQA's inverse-scaling result [40] shows that scale alone does not close this regime.

G.2 The memory-direct tier saves inference cost; does that saving survive at small deployment scale?

A memory-direct query serves an answer through a sentence-embedding cosine search over the FAISS-backed episodic memory and returns the stored answer at retrieval-only cost. A zero-shot generation through the same generator pays the full forward-pass cost. A retrieval-augmented generation pays the FAISS lookup cost plus the cross-encoder reranker cost plus the forward-pass cost over a longer context. The time per tier could not be separated in this run (Section 5.1.4), but a memory-direct answer skips generation and retrieval, the two most expensive steps of the other tiers.

The saving depends on how often questions repeat. In this study every evaluation question was new, so the memory-direct tier answered 7 of 9,000 evaluation queries over six cycles; on a probe that re-asked stored questions it answered 42 of 50 exact

repeats and 19 of 50 paraphrases, 41 and 19 of them correctly (Table 5.8). At the deployment scale projected in Section 3.8.3 the saving grows with traffic, but only in proportion to the share of repeated questions, which this study does not measure. Plain fine-tuning has no equivalent mechanism: every query pays the full generation cost however often the question has been seen, because the knowledge sits in the weights rather than in a separable episode.

G.3 What does asymmetric rollback give the operator that a manual checkpoint backup does not?

The retention guard at every cycle boundary measures the post-fine-tune model on a three-probe multi-modal panel (MMLU, TriviaQA-test, CommonsenseQA-test) and computes the per-probe retention ratio $\rho_k = \text{score}_{k,\text{post}}/\text{score}_{k,\text{pre}}$ for each probe k. When any worst-probe ratio falls below the registered floor of 0.93 the cycle is declared aborted: the weights revert to the previous cycle's parameters, and the new memory written during the cycle is preserved unchanged. The architectural property is that the model is replaceable but the memory is not.

Plain fine-tuning has no equivalent guarantee even when the operator manually keeps the previous checkpoint. The reason is that plain fine-tuning at any cycle commits both the weights and the gradient signal that produced them; rolling the weights back rolls back the cycle's accumulated training experience entirely. CAEM's separation of weights from memory means a failed cycle costs only the cycle's fine-tune compute but does not lose the verified episodes that the cycle accumulated. The next cycle inherits the previous cycle's weights together with the larger memory, and the retroactive re-verification pass at the next cycle boundary refreshes the stored composites under the rolled-back generator. This asymmetric-rollback property is the architectural anchor for the memory-growth invariant established in Heuristic 4.9: memory accumulates strictly across cycles even when weights do not. A weights-only architecture has no monotonicity property to anchor.

G.4 If both architectures need labelled data, how many fewer labels does CAEM use?

Plain fine-tuning uses a gold label for every training example, because the label is the training target. CAEM uses labels for a different purpose: the labelled calibration fold fits the verifier's composite and checks the precision at the storage cut-point, and the answers the generator trains on are labelled by the verifier. In the research run CAEM used about 2,100 gold labels: the 1,500 calibration questions, re-scored at every cycle, and the 603 cold-start answers checked once (Section 4.2.3). The vanilla fine-tuning baseline (B6) used a gold answer for each of its 23,500 training questions, about eleven times as many.

With that difference, CAEM matched B6's pooled exact match (0.535 at C2 and 0.508 at C5, against 0.512), while B6 reached a lower composite hallucination metric (0.103 against 0.134 at C5) and was the most accurate system on FEVER and

CommonsenseQA (Table 5.19). The label saving is therefore real but not free: the verifier's labels are noisier than gold labels, about one stored answer in four is wrong, and the gains of the self-improvement loop lasted one or two cycles (Section 5.6.1). The production design (Section A.5) draws about 3,000 labelled answers per cycle from the stored traffic, so the ratio in deployment depends on the size of the fine-tune it replaces.

G.5 What does the composite catch at inference time that a fine-tune cannot fix?

A fine-tune updates the generator's weights so that the model is more likely to produce the correct answer on questions it has seen. The fine-tune does not alter the model's behaviour on a specific in-flight query that is hallucinating because of a property the training data did not cover: a fresh trivia question on a recently-changed world fact, an off-topic-but-grounded answer that drifts from the question's intent, a confidently-wrong output where the model has a strong prior toward a misconception. These failure modes are not closeable by parametric updates because they are first-order properties of the model's prior, not properties the training data instances reveal.

The calibrated-probability composite catches each of these failure modes at inference time through a different signal family. Confidently wrong outputs with no passage support trip the early-exit conjunction $(u_{\text{internal}} \geq 0.70) \wedge (p_{\text{ground,max}} \leq 0.20)$ and are discarded, so the answer is shown only with the conflicting tag and a warning. Off-topic-but-grounded outputs were meant to be caught by the question-answer match signal q_{match}; on short answers it cannot detect them and acts as a correctness cue instead (Section 4.2.6), so detecting them is future work. Common-misconception outputs that match the model's prior are the hardest case: the sample-set agreement family (the chain-to-answer entailment p_{entail}, the pairwise similarity s_{avg} over $M = 3$ resampled chains, and the dropout dispersion u_{dropout} over $K = 5$ passes) measures agreement, which a consistently held misconception passes, and the verifier was close to chance on TruthfulQA (Section 5.6.2). The semantic-entropy signal h_{norm} was dropped before the main run because its learned weight in the cycle-zero fit was effectively zero while its ten extra samples were the verifier's largest cost. The verifier's value at inference time is separate from what fine-tuning provides: every answer is still checked before it is shown, and answers tagged verified stayed about two-thirds correct at every cycle. The check weakens as the generator changes, however: the share of wrong answers the verifier scored highly rose from 0.208 for B1 to 0.305 at C5 (Table 5.3). [84] report that verbalised confidence distinguishes correct from incorrect outputs only weakly (an area under the curve of about 0.6 for GPT-4) and that no single confidence-elicitation method is consistently best across tasks; the composite therefore aggregates nine signals across four families through per-signal isotonic regression and a regularised logistic boost layer to produce a calibrated correctness probability that does not depend on any one signal.

G.6 What was the zero-shot tier expected to gain from the cycle loop, and did it?

A plain fine-tune produces one checkpoint. The cycle loop produces a sequence of checkpoints trained on stored answers, most of them written by the retrieval-augmented tier and certified by the verifier, so the design expects the fine-tune to fold retrieved knowledge into the generator: later cycles would answer from the zero-shot tier some questions that earlier cycles needed retrieval for, and the retrieval share, and with it the cost per query, would fall.

The run did not show this. The zero-shot share of evaluation queries moved from 48.7 percent at C0 to 50.2 percent at C5, and the zero-shot tier's accuracy fell from 0.565 to 0.519 (Table 5.9). Training on retrieval-written chains also taught the generator to cite a context it was not given (Section 5.6.3). The change in the retrieval share that the run does show comes from the retrieval-utility classifier (Section 5.4.2), not from the self-improvement loop.

G.7 When would CAEM not be worth deploying?

The architecture is not a free contribution. Each of the six differentiators above corresponds to an architectural component whose cost is non-zero: the verifier ensemble pays forward passes through MiniCheck, the cross-encoder, and the sentence encoder at every query; the labelled cycle-boundary batch requires ongoing labelling effort; the cycle policy requires operational discipline. The architecture is worth deploying exactly when at least one of the six differentiators is binding for the deployment, and is not worth deploying when all six relax simultaneously:

No abstain class. Deployments with no requirement for an abstain class.

No scale compounding. Deployments where memory-direct cost savings do not compound (very low traffic, or a product requirement that every query traverse the same path for uniformity).

Distillation traction absent. Deployments where the zero-shot tier's per-cycle improvement does not bind because the deployed workload distribution is too narrow for the SIL-driven distillation to gain traction, or too broad for a 3B-class generator to absorb at any cycle horizon.

Manual checkpoint sufficient. Deployments where a manual previous-checkpoint backup is sufficient and an external snapshot pipeline already solves the catastrophic-forgetting recovery problem.

Labelling abundant. Deployments with a labelling budget large enough to support frequent direct fine-tuning, where the verifier-as-sample-efficiency-multiplier value collapses because labelling cost is not the binding constraint.

Below parametric ceiling. Deployments below the parametric-capacity ceiling of the underlying generator, where direct fine-tuning still produces meaningful accuracy gains so the inference-time-error-catching value of the composite is dominated by what fine-tuning would also provide.

A deployment that satisfies all six conditions is one where plain fine-tuning would deliver comparable quality with a simpler implementation. On the panel of this thesis, fine-tuning on gold labels (B6) matched CAEM's exact match and reached a lower composite hallucination metric; CAEM's measured advantages were the status tag and abstention, and a gold-label budget about eleven times smaller (Chapter 5). A deployment that needs neither a per-answer reliability signal nor a small labelling budget is better served by plain fine-tuning.

Part II: Design-justification: calibration discipline and cycle-boundary ordering

The architecture makes several non-obvious design choices in how the calibration fold is constructed, how the cycle boundary is sequenced, and how the deduplication discipline guards the disjointness of the data partition. Part II answers each as a direct "is this correct, and why not the simpler alternative" question.

G.8 Why use the same calibration fold across all cycles of the registered horizon rather than rotating folds?

The calibration fold is a fixed thousand-five-hundred-sample partition (five hundred samples drawn from each of the three training benchmarks) constructed once at pool-builder time and reused at every cycle boundary. The natural alternative would be to rotate the fold across cycles, drawing a fresh fifteen-hundred-sample slice each cycle from a larger reservoir. The same-fold protocol is preferred for four reasons.

First, the cycle-boundary recalibration is itself a measurement. The labelled fold is the reference distribution against which the verifier's calibration is read; if the reference distribution varies between cycles, then a cycle-to-cycle change in the fitted curves entangles two sources of variation (the verifier improving under self-improvement, and the reference set drifting) and the trajectory analysis cannot decompose them. A fixed reference fold is the variance-isolation lever that lets the per-cycle isotonic and composite-calibration readings be attributed cleanly to verifier change.

Second, the gold labels on the fold are constant across cycles. The model's verifier signals on a given fold question shift cycle-to-cycle (because the post-fine-tune model produces different token probabilities, dropout variances, chain agreement, and so on), but the question's correct answer does not. Re-fitting the per-signal isotonic curves on the same (variable signals, constant labels) data keeps the calibrator matched to the current model, which is the regime in which the post-hoc calibration literature finds such calibration reliable; calibration fitted before a model or data shift degrades after it [26]. Rotating the fold would replace the constant-labels property with a moving-target labelling distribution that confounds the fit.

Third, the fixed-threshold empirical-precision anchor holds against the same-fold protocol. The anchor requires that the calibration-fold composite distribution at the registered cut-point projects forward as a one-sided lower bound on the cycle-$t+1$ candidate pool's storage precision under exchangeability between those two distributions. The protocol does *not* require exchangeability between cycle N's calibration fold and cycle $N+1$'s calibration fold; the same-fold choice across cycles is therefore not a contract violation, because the per-cycle composite refit absorbs any drift in the underlying signal distribution into the signal-to-probability mapping rather than into the threshold.

Fourth, the storage cut-points themselves are held constant across cycles at $\tau_{\text{store}} = 0.60$ and $\tau_{\text{defer}} = 0.45$. With the same fold every cycle and the same cut-points across cycles, any cycle-to-cycle variation in the realised storage precision at the cut-point is attributable to the per-cycle composite refit absorbing distribution drift, rather than to fold-resampling variance or to the cut-points themselves moving.

A defensible alternative for a substantially longer trajectory (say fifty or more cycles) would be a rotating-fold protocol drawn from a five-thousand-sample reservoir partitioned into ten overlapping subsets, where each cycle's fold differs but is still an honest sample from the labelled population. The variance-isolation argument weakens at long horizons because the same-fold protocol risks calibration over-fitting to fold-specific noise. The realised six-cycle horizon (the cycle-zero calibration plus cycles one through five, with cycle four aborted under the retention guard and the run stopping after cycle five, as planned after the cycle-four rollback) is short enough that variance isolation dominates, and the rotating-fold extension is registered as a future-work item should the trajectory horizon extend.

G.9 In production each calibration split would necessarily differ; how is this reconciled with the same-fold research protocol?

In production there is no pre-built calibration partition. The labelled batch at every production cycle boundary is drawn from accumulated serving traffic via stratified sampling (per the production runbook), so each cycle's fold is a fresh sample of three thousand or so labelled queries with content that varies cycle-to-cycle. This is a different operational mode from the research protocol but architecturally compatible with it.

Three points reconcile the two modes. First, the fitting procedure is fold-agnostic. The per-signal isotonic regression is a deterministic monotone fit on whatever (signal, label) pairs are supplied; sklearn's isotonic regression does not care whether the questions were the same as in a previous fit or different. The cycle-zero cut-point selection walks the labelled fold sorted by composite and picks the threshold at the registered precision target; this is run once at cycle zero and the resulting cut-points are held constant across cycles. Both procedures work identically on a fixed fold (research) or a rotating fold (production).

Second, production loses one variance-isolation property and gains a different fidelity property. The research protocol uses a fixed fold to isolate verifier-distribution change as the only source of cycle-to-cycle calibration variation. Production cannot use the fixed fold because the fold material does not exist. Instead, production gains a different property: the calibration fold matches the live deployment distribution exactly, because it is drawn from live deployment traffic. Research's fixed fold is a proxy for deployment (drawn from training-benchmark dev splits); production's rotating fold is the real thing. Each protocol is correct under its own labelling-availability constraint.

Third, the storage and deferral cut-points are fixed in both modes, so a single cycle's traffic anomaly (an unusual spike of out-of-domain queries, for example) cannot move them; it can only change the refitted composite, which the next cycle's labelled batch refits again. The one smoothed quantity is the pre-routing temperature, which blends each cycle's fit with the previous value at weight $\alpha_{\text{ema}} = 0.7$.

The practical consequence: an operator who wants production calibration to behave as close as possible to research-mode discipline can stratify the production sample

to be representative of the five-benchmark workload composition and can hold the labelled batch size near the research-mode fifteen-hundred-sample point. An operator whose live distribution differs sharply from research can let the production fold reflect the live distribution and accept that the calibration trajectory will diverge from the research trajectory by exactly the distribution-shift magnitude. Both behaviours are correct; the choice is between distribution fidelity and protocol parity.

G.10 Why apply the cycle-boundary recalibration before retroactive re-verification rather than after?

The cycle-boundary order is: self-improvement fine-tune, then score the calibration fold under the now-updated generator, then re-fit the temperature scalar and per-signal isotonic curves and per-benchmark composite boost weights on the freshly-scored fold (the storage cut-points themselves are held constant across cycles), then reload the verifier from the new calibration files, then run retroactive re-verification against the now-recalibrated verifier. The natural alternative would be to run retroactive re-verification first under the previous cycle's calibration and recalibrate afterwards. The current order is the architecturally correct one for two reasons.

First, the post-fine-tune signal distribution falls outside the previous cycle's isotonic fit envelope on a non-trivial fraction of stored episodes. The fine-tune saturates token probability and chain agreement on chains the model trained on; the previous cycle's isotonic curves were fitted on the previous cycle's signal distribution and do not extrapolate cleanly past the upper edge of their training-input range. Running retroactive re-verification under the previous cycle's calibration produces calibrated probabilities on memorised chains that are artefactually low, and the resulting composite drops below the prune threshold for many EM-correct stored entries. The empirical evidence for this artefact is documented in the original cycle-one prune of a large share of the cold-seed memory under the stale-calibration ordering; the corrected order has the recalibration absorb the post-fine-tune signal-distribution shift before the prune rule fires.

Second, the corrected order preserves the architectural intent of retroactive re-verification. The re-verification pass is meant to upgrade the stored composite of episodes whose underlying correctness has improved under the new generator and to prune episodes whose correctness has genuinely degraded. The pass cannot do either if the calibration it reads through has not yet been updated to the new model. Under the corrected order, retroactive re-verification reads through fresh per-signal isotonic curves and fresh per-benchmark boost weights at the registered fixed cut-points; an entry whose correctness genuinely improves under the new model maps to a higher calibrated probability and stays in memory with an upgraded u_{stored}, an entry whose correctness genuinely degrades maps to a lower probability and gets pruned. The order is therefore not an implementation detail but a precondition on the theorem-licensing receipt for pool purity in Theorem 4.1.

G.11 Why disable the confabulation early-exit during retroactive re-verification when it remains active at query time?

The early-exit rule fires when the internal-confidence composite is high (above $\eta_u = 0.70$) and the top-passage entailment is low (below $\eta_g = 0.20$). At query time this conjunction is the architectural defence against confabulation: a model that is internally confident on a fresh question but has no supporting passage is generating a confident hallucination, and the rule routes the answer to retrieval-augmented regeneration rather than serving the hallucination. At retroactive re-verification time the same conjunction reads differently. The fine-tune just memorised the stored chains, so the internal-confidence composite is automatically near one for every entry the self-improvement loop touched; the first conjunct is no longer informative. Letting the rule fire under this regime collapses the conjunction into "low top-passage entailment, regardless of confidence," which is a much wider catchment than the registered confabulation rule and over-prunes EM-correct stored entries whose chain happened to have thin passage support.

The architectural fix is a query-time-only flag on the rule. The verifier's verify method takes an is_query_time parameter that defaults to true (preserving query-time inference and deferred-buffer reconsideration behaviour) and is set to false in the retroactive re-verification closure. Under the false setting the early-exit conjunction is short-circuited and the verifier falls through to the standard composite-and-threshold path, which still prunes any entry whose recalibrated u_{stored} falls below $\tau_{\text{retro}} = 0.50$. Confidently-wrong stored entries are still removed; EM-correct stored entries with thin passage support are no longer falsely pruned. The rule's intended semantics (catching query-time confabulation) is preserved; its inheritance into retroverify-time is corrected.

G.12 Why is the training threshold strictly higher than the storage threshold rather than equal?

The architecture registers two different thresholds on the calibrated composite. The storage threshold τ_{store} governs admission to the user-facing memory store and to the retrieval tier. The training threshold τ_{train} governs admission to the next cycle's self-improvement fine-tune pool; only entries whose composite clears τ_{train} enter the gradient signal. The convention is $\tau_{\text{train}} > \tau_{\text{store}}$, with the locked configuration setting $\tau_{\text{store}} = 0.60$ and $\tau_{\text{train}} = 0.70$ (held constant across cycles; the training-pool threshold was lowered from 0.75 to 0.70 on the trajectory-launch date to restore a non-empty verified-episode share while preserving the strictly-stricter invariant). The asymmetry is deliberate.

The reason is that the training pool contributes to all subsequent cycles through the generator's weight update, while the storage pool contributes only to the user-facing retrieval tier at the current cycle. A noisy entry in the storage pool serves at most a single Tier-1 retrieval and is re-scored at the next cycle's retroactive re-verification;

the architectural cost of a borderline storage admission is bounded. A noisy entry in the training pool propagates through the gradient signal into every subsequent cycle's generator and can compound the verifier's bias over the trajectory. The noise-tolerant-learning literature [10] bounds the corruption rate at which training under noisy labels still improves a model rather than degrading it; the architecture spends the strict-training-threshold budget to keep the training pool's empirical purity above that rate by construction.

A defensible alternative would be a single threshold $\tau = \tau_{\text{store}} = \tau_{\text{train}}$ that admits the same entries to both purposes. The simpler alternative loses the asymmetry between storage-class consequences (bounded, single-cycle) and training-class consequences (compounding, multi-cycle), and the noise-tolerant-learning bound on training-pool corruption is no longer enforced architecturally. The two-threshold protocol is the registered defence; the empirical purity of the resulting training pool is reported in Chapter 5.

G.13 Why deduplicate the calibration fold at three layers rather than relying on the dataset's native unique identifiers?

The deduplication discipline runs at three layers in the pool-builder. The first layer hashes every sample's question text into a sixteen-character content identifier and dedupes within each loaded dataset split, dropping ten thousand or more duplicate records on average per training-benchmark train split. The second layer dedupes within each individual partition slice (calibration, purity, training, evaluation) to catch any residual within-slice duplicates. The third layer cross-checks the calibration and purity slices against the evaluation slice and drops any sample whose content hash collides with an evaluation-slice sample. The natural alternative would be to rely on the dataset's native sample identifiers and assume disjointness by construction.

The dataset's native identifiers are unreliable for two reasons. The lucadiliello FEVER dump reuses integer claim identifiers across the train and dev splits, so a sample with id 13957 in the train split is content-distinct from a different sample with id 13957 in the dev split, and a join on native id produces spurious overlap warnings. The TriviaQA dump reuses sfq-prefixed and qw-prefixed and qz-prefixed identifiers within a single split, so two content-identical questions can both appear in the same split with the same id. Relying on native identifiers therefore both over-flags overlap (false positives on different-content same-id pairs) and under-flags overlap (false negatives on same-content different-id pairs). Content-hashing on the question text resolves both regimes by definition: identical questions hash to the same identifier regardless of native id; different questions hash to different identifiers regardless of native id collision.

The three-layer structure is the minimum sufficient cover. A single dataset-level dedup pass leaves intra-slice duplicates untouched. A single intra-slice dedup pass leaves cross-slice content collisions untouched (a calibration question that happens to also appear in the evaluation split). A single cross-slice pass relies on the per-slice slice having been itself dedup-clean; without the within-slice dedup, the cross-slice

check sees inflated counts and miscomputes the disjointness invariant. The empirical evidence for the three-layer requirement is reported in the loading-time logs of every cycle: each layer drops a non-trivial count, and the fold size after all three layers matches the registered fifteen-hundred-sample target rather than a stale-protocol nineteen-hundred-or-more sample inflated count.